

Deep Learning

Deep Learning

- **Course Code** -CSA3007
- **Course Type** LTP
- **Credits** 4

- **Course Objectives:**
 - Gain knowledge in Machine Learning Basics
 - Understand and apply Optimization on Deep Models and Networks
 - Understand and analyze Recurrent and Recursive Networks
 - Understand the representation of neural networks in machine learning

Course Outcomes:

- Explore the fundamentals of Machine Learning Algorithms
- Analyze Deep learning Mathematical Models
- Elucidate the Deep Feedforward Networks
- Apply knowledge for Optimization on Deep Models and Convolutional Networks
- Elucidate the Recurrent and Recursive Networks and Natural language Processing

Text Book(s)& References Book(s)

Text Book(s):

1. Ian Goodfellow, Yoshua Bengio, Aaron Courville, Deep Learning, MIT Press, 2016.
2. Michael Nielsen, Neural Networks and Deep Learning, Determination Press, 2015.

Reference Book(s):

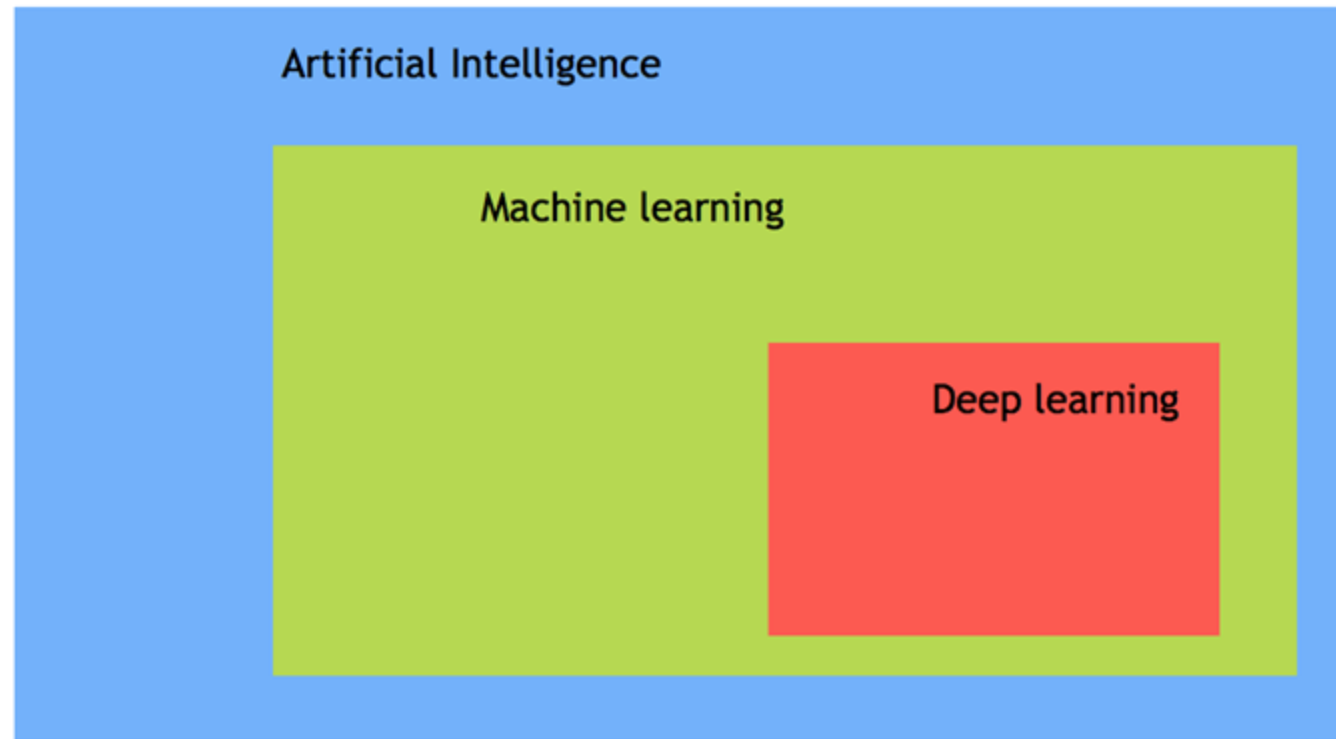
1. Deng & Yu, Deep Learning: Methods and Applications, Now Publishers, 2013.
2. Russell, S. and Norvig, N. Artificial Intelligence: A Modern Approach. Prentice-Hall Series in Artificial Intelligence. 2003.
3. Bishop, C. M. Neural Networks for Pattern Recognition. Oxford University Press. 1995.

- **Introduction: Historical Trends in Deep Learning - Linear Algebra: Scalars - Vectors - Matrices - Tensors - Matrices - Norms – Eigen decomposition - Probability and Information Theory: Random variable and distributed Probability - Bayes Rule -Information Theory and structured probabilistic models.**
- **Machine Learning:** Numerical Computation: Overflow and Underflow – Gradient-based Optimization – Constrained Optimization - Learning Algorithms: Capacity - Overfitting – Underfitting - Bayesian Classification - Supervised - unsupervised algorithms - Building machine learning algorithm

Historical Trends in Deep Learning

- Deep learning has had a long and rich history, but has gone by many names reflecting different philosophical viewpoints, and has waxed and waned in popularity.
- Deep learning has become more useful as the amount of available training data has increased.
- Deep learning models have grown in size over time as computer infrastructure (both hardware and software) for deep learning has improved.
- Deep learning has solved increasingly complicated applications with increasing accuracy over time.

Subfields of Artificial Intelligence



Difference Between Machine Learning, Artificial Intelligence and Deep Learning

Concept

Definition

Artificial intelligence

The field of computer science aims to create intelligent machines that can think and function like humans.

Machine learning

A subfield of artificial intelligence that focuses on developing algorithms and models that can learn from data rather than being explicitly programmed.

Deep learning

A subfield of machine learning that uses multi-layered artificial neural networks to learn complex patterns in data.

Artificial Intelligence



Artificial Intelligence

- Artificial Intelligence is composed of two words **Artificial** and **Intelligence**, where Artificial defines "*man-made*," and intelligence defines "*thinking power*", hence AI means "*a man-made thinking power*."
- It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decisions.

Artificial Intelligence

- Artificial Intelligence exists when a machine can have human based skills such as **learning, reasoning, and solving problems**
- With Artificial Intelligence you do not need to preprogram a machine to do some work, despite that you can create a machine with programmed algorithms which can work with own intelligence, and that is the awesomeness of AI.
- It is believed that AI is not a new technology, and some people says that as per Greek myth, there were Mechanical men in early days which can work and behave like humans.

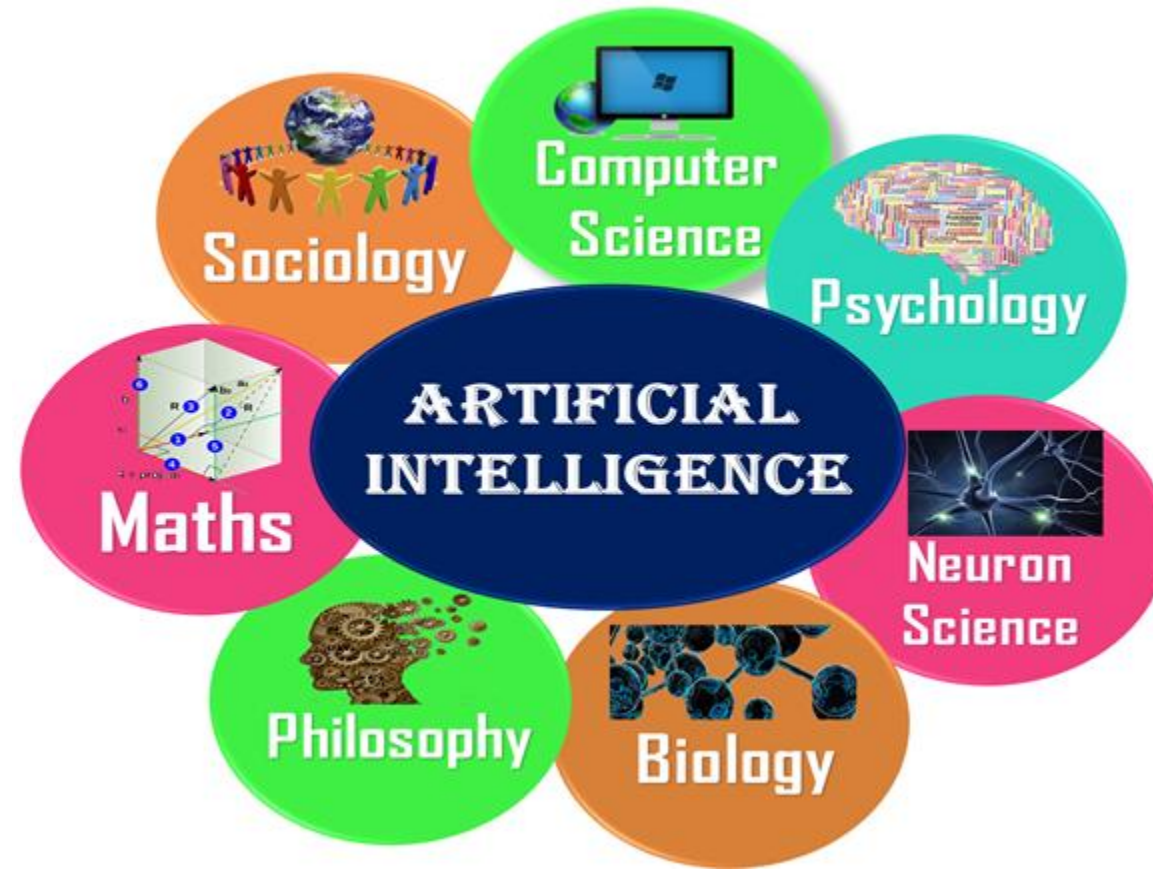
Why Artificial Intelligence?

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

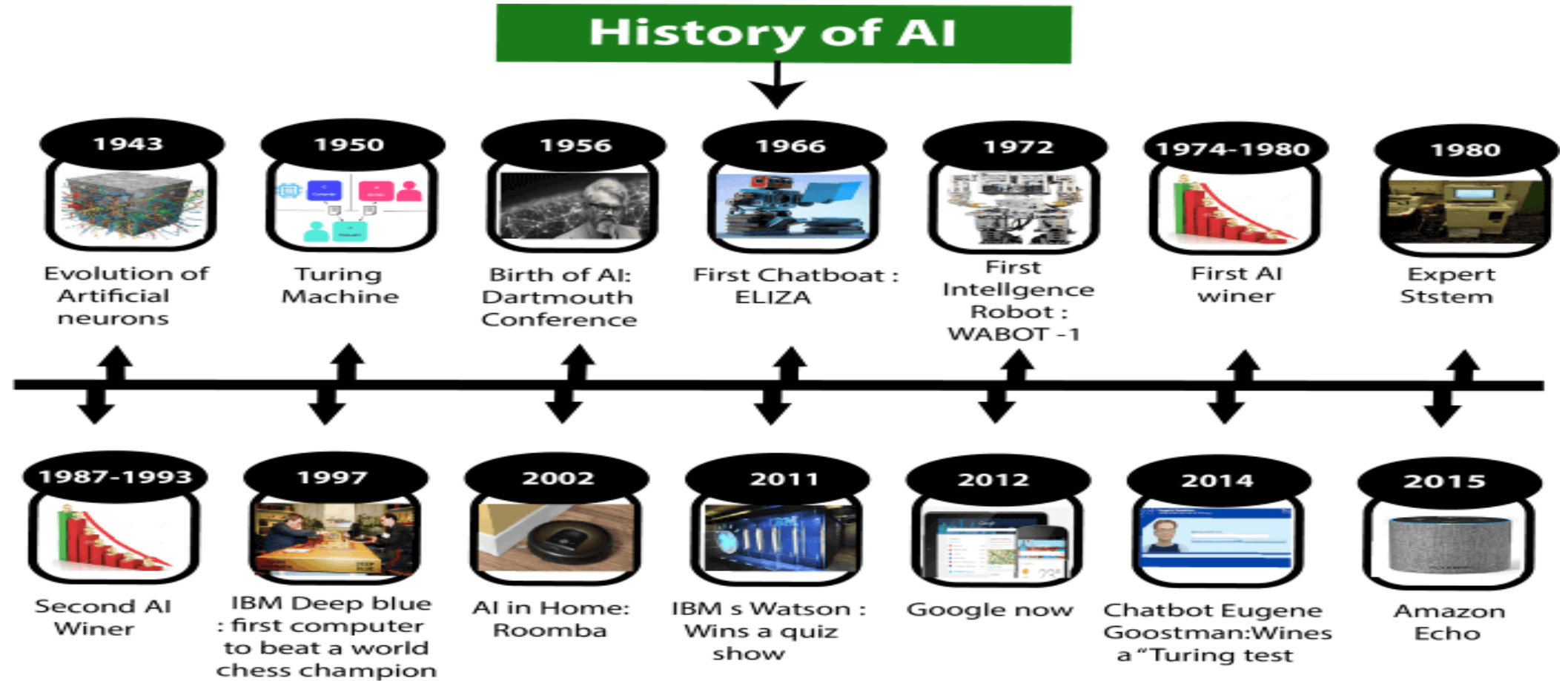
Goals of Artificial Intelligence

1. Replicate human intelligence
2. Solve Knowledge-intensive tasks
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence such as:
 1. Proving a theorem
 2. Playing chess
 3. Plan some surgical operation
 4. Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior, learn new things by itself, demonstrate, explain, and can advise to its user.

What Comprises to Artificial Intelligence?



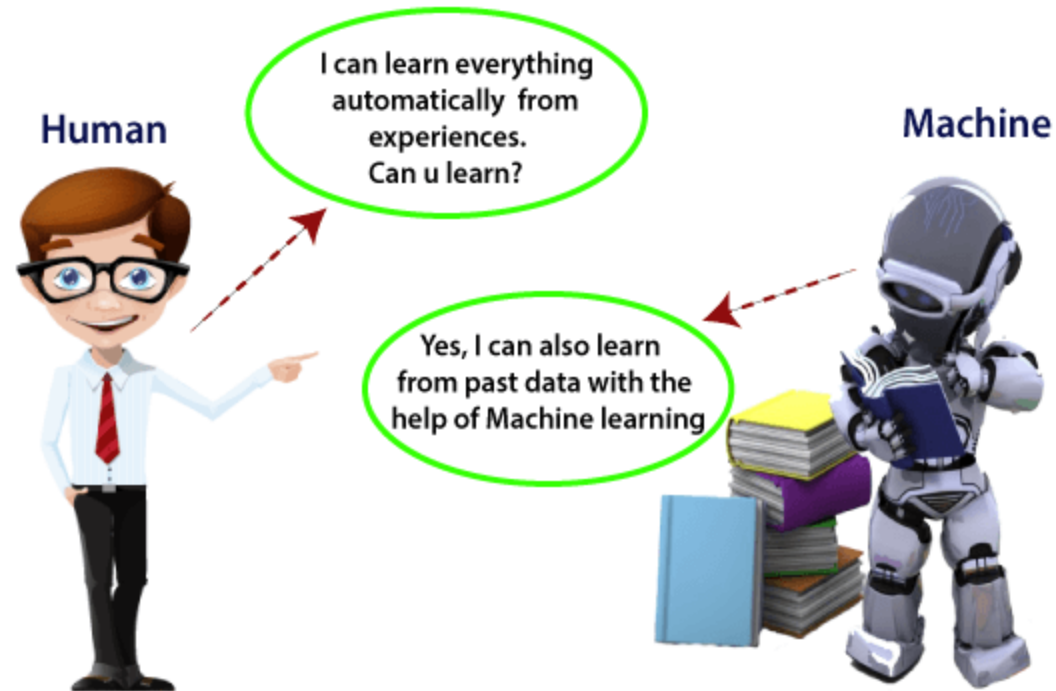
History of Artificial Intelligence



Year	Milestone / Innovation
1923	Karel Čapek plays named “Rossum’s Universal Robots, the first use of the word “robot” in English.
1943	Foundations for neural networks laid.
1945	Isaac Asimov, a Columbia University alumni, use the term Robotics.
1956	John McCarthy first used the term Artificial Intelligence. Demonstration of the first running AI program at Carnegie Mellon University.
1964	Danny Bobrow’s dissertation at MIT showed how computers could understand natural language.
1969	Scientists at Stanford Research Institute Developed Shakey. A robot equipped with locomotion and problem-solving.

1979	The world's first computer-controlled autonomous vehicle, Stanford Cart, was built.
1990	Significant demonstrations in machine learning
1997	The Deep Blue Chess Program beat the then world chess champion, Garry Kasparov.
2000	Interactive robot pets have become commercially available. MIT displays Kismet, a robot with a face that expresses emotions.
2006	AI came into the Business world in the year 2006. Companies like Facebook, Netflix, Twitter started using AI.
2012	Google has launched an Android app feature called "Google now", which provides the user with a prediction.
2018	The "Project Debater" from IBM debated complex topics with two master debaters and performed exceptionally well.

What is Machine Learning

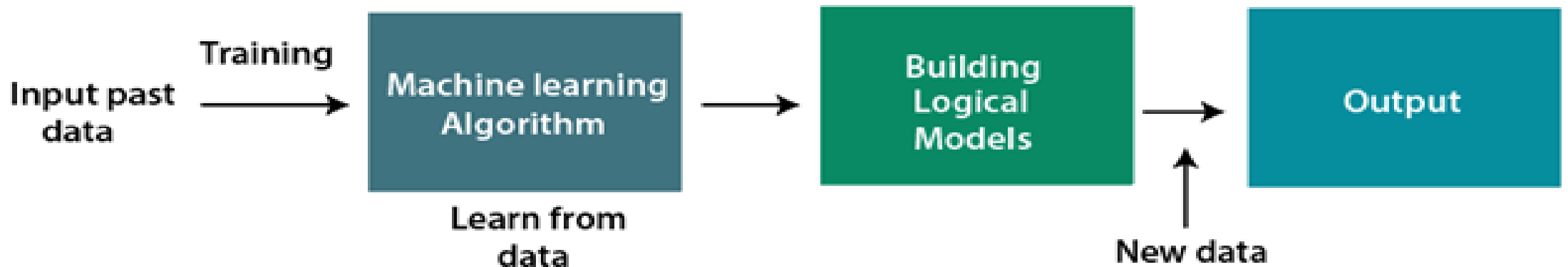


Machine Learning def

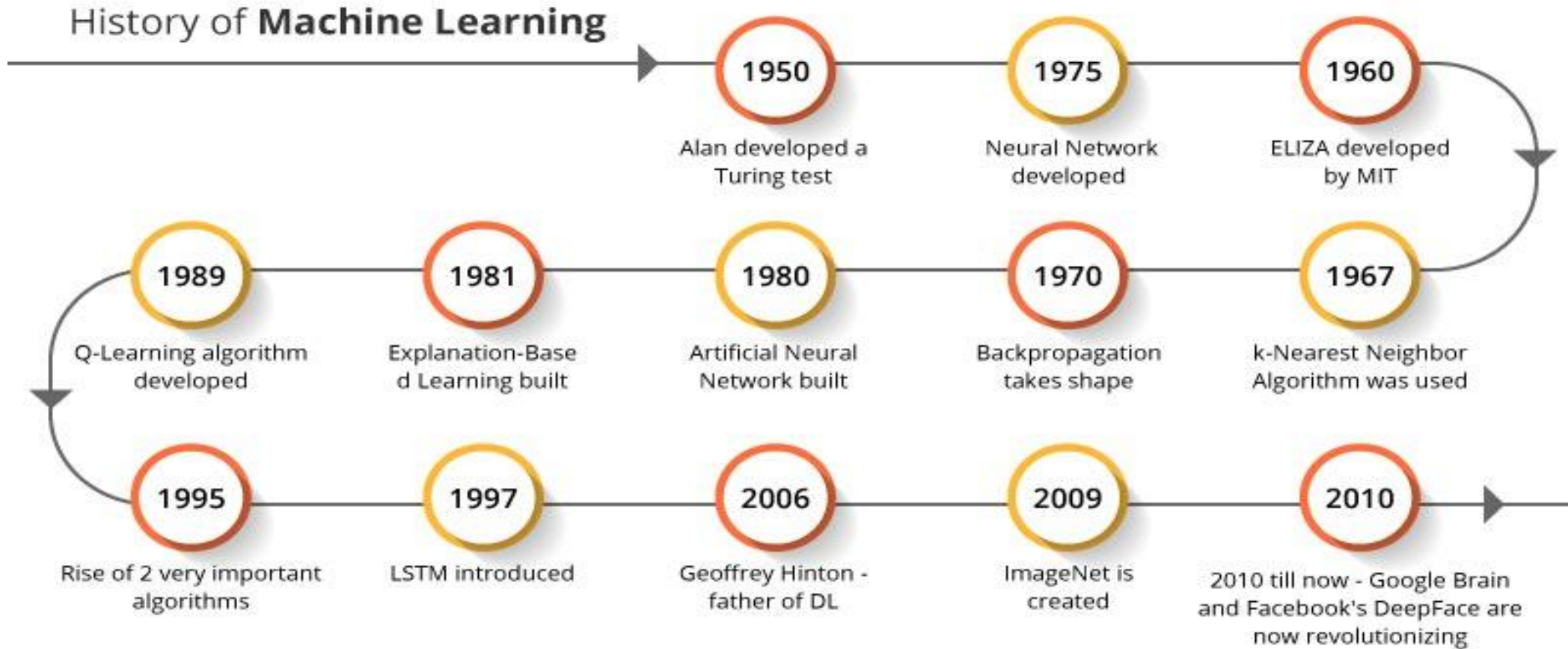
- Machine learning enables a machine to automatically learn from data, improve performance from experiences, and predict things without being explicitly programmed.
- Machine Learning is said as a subset of **artificial intelligence** that is mainly concerned with the development of algorithms which allow a computer to learn from the data and past experiences on their own.
- The term machine learning was first introduced by **Arthur Samuel** in **1959**.

How does Machine Learning work

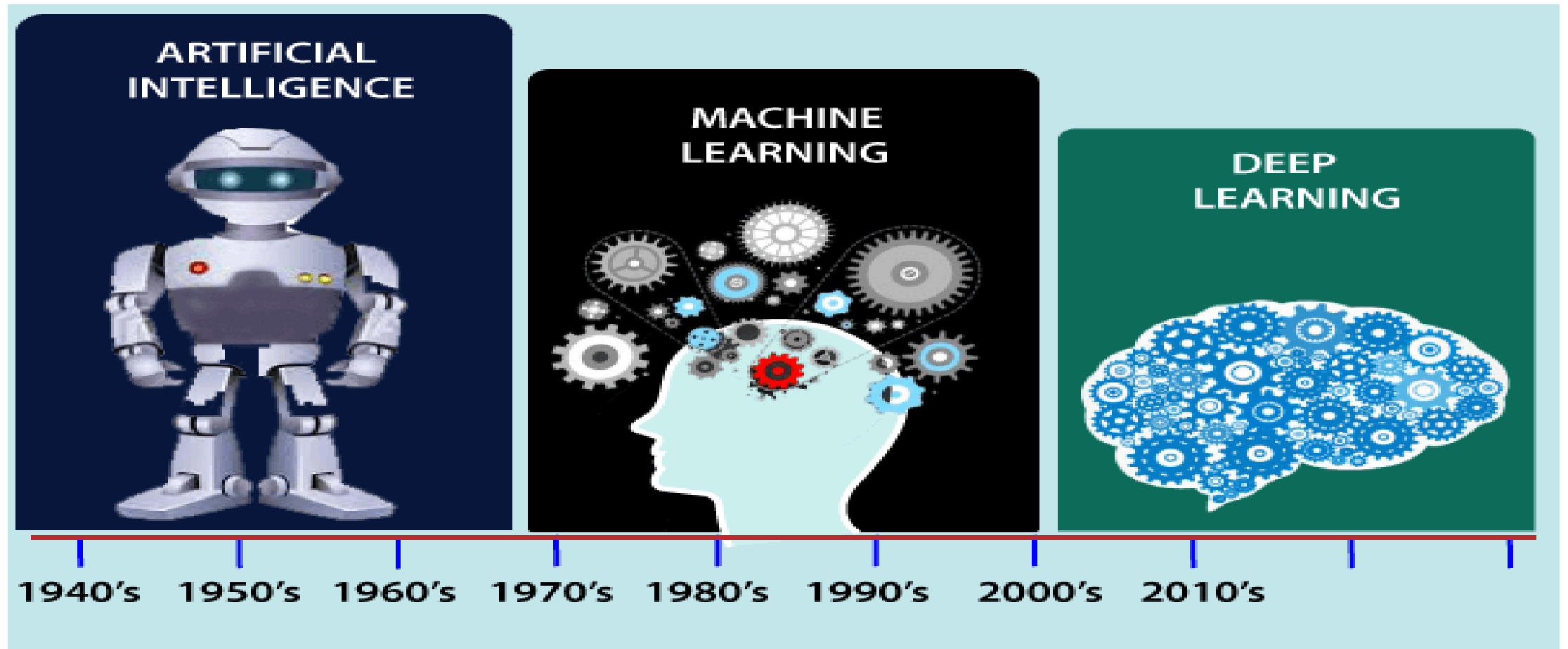
- A Machine Learning system **learns from historical data, builds the prediction models, and whenever it receives new data, predicts the output for it.** The accuracy of predicted output depends upon the amount of data, as the huge amount of data helps to build a better model which predicts the output more accurately.



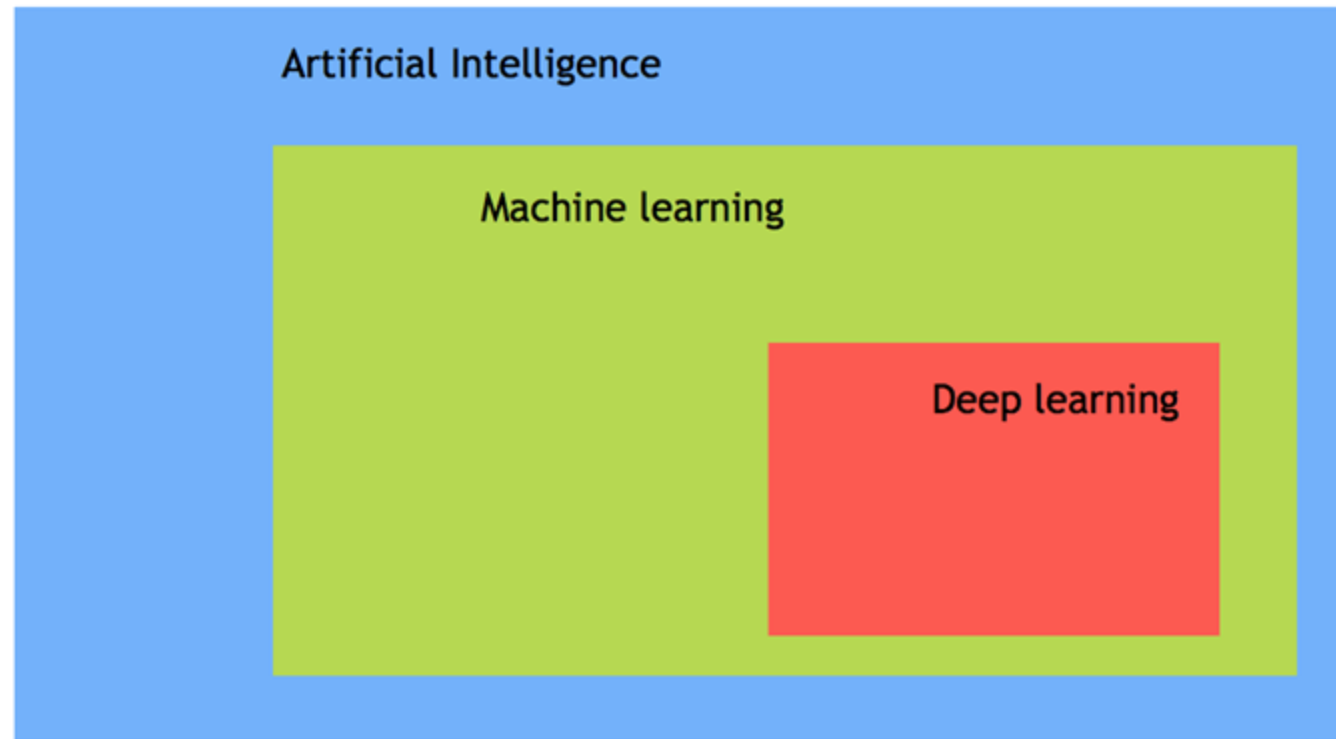
History of Machine Learning



History of Machine Learning



Subfields of Artificial Intelligence



Subfields of Artificial Intelligence

- **Machine Learning:**
- Machine learning is the art of studying algorithms that learn from examples and experiences.
- Machine learning is based on the idea that some patterns in the data were identified and used for future predictions.
- The difference from hardcoding rules is that the machine learns to find such rules.
- **Deep Learning:**
- Deep learning is a sub-field of machine learning.
- Deep learning does not mean the machine learns more in-depth knowledge; it uses different layers to learn from the data.
- The depth of the model is represented by the number of layers in the model. For instance, the Google LeNet model for image recognition counts 22 layers.

Subfields of Artificial Intelligence

- **Natural Language Processing:**
 - A neural network is a group of connected I/O units where each connection has a weight associated with its computer programs. It helps you to build predictive models from large databases.
 - This model builds upon the human nervous system.
 - You can use this model to conduct image understanding, human learning, computer speech, etc.
- **Expert Systems:**
 - An expert system is an interactive and reliable computer-based decision-making system that uses facts and heuristics to solve complex decision-making problems.
 - It is also considered at the highest level of human intelligence.
 - The main goal of an expert system is to solve the most complex issues in a specific domain.
- **Fuzzy Logic:**
 - Fuzzy Logic is defined as a many-valued logic form that may have truth values of variables in any real number between 0 and 1.
 - It is the handle concept of partial truth. In real life, we may encounter a situation where we can't decide whether the statement is true or false.

AI has three different levels

- 1) **Narrow AI:** An artificial intelligence is said to be narrow when the machine can perform a specific task better than a human. The current research of AI is here now
- 2) **General AI:** An artificial intelligence reaches the general state when it can perform any intellectual task with the same accuracy level as a human would
- 3) **Active AI:** An AI is active when it can beat humans in many tasks

Artificial Intelligence

Includes Machine Learning.

Artificial Intelligence combines large amounts of data through iterative processing and intelligent algorithms to help computers learn automatically.

Some of the popular tools that AI uses are-

1. TensorFlow
2. Scikit Learn
3. Keras

Artificial Intelligence uses logic and decision trees.

Chatbots, and Voice assistants are popular applications of AI.

Machine Learning

Subset of Artificial Intelligence.

Machine Learning uses efficient programs that can use data without being explicitly told to do so.

The popular tools that Machine Learning makes use of are-

1. Amazon Lex
2. IBM Watson Studio
3. Microsoft Azure ML Studio

Machine Learning uses statistical models.

Recommendation Systems such as Spotify, and Facial Recognition are popular examples.

Data Science

Includes various Data Operations.

Data Science works by sourcing, cleaning, and processing data to extract meaning out of it for analytical purposes.

Some of the popular tools used by Data Science are-

1. SAS
2. Tableau
3. Apache Spark
4. MATLAB

Data Science deals with structured and unstructured data.

Fraud Detection and Healthcare analysis are popular examples of Data Science.

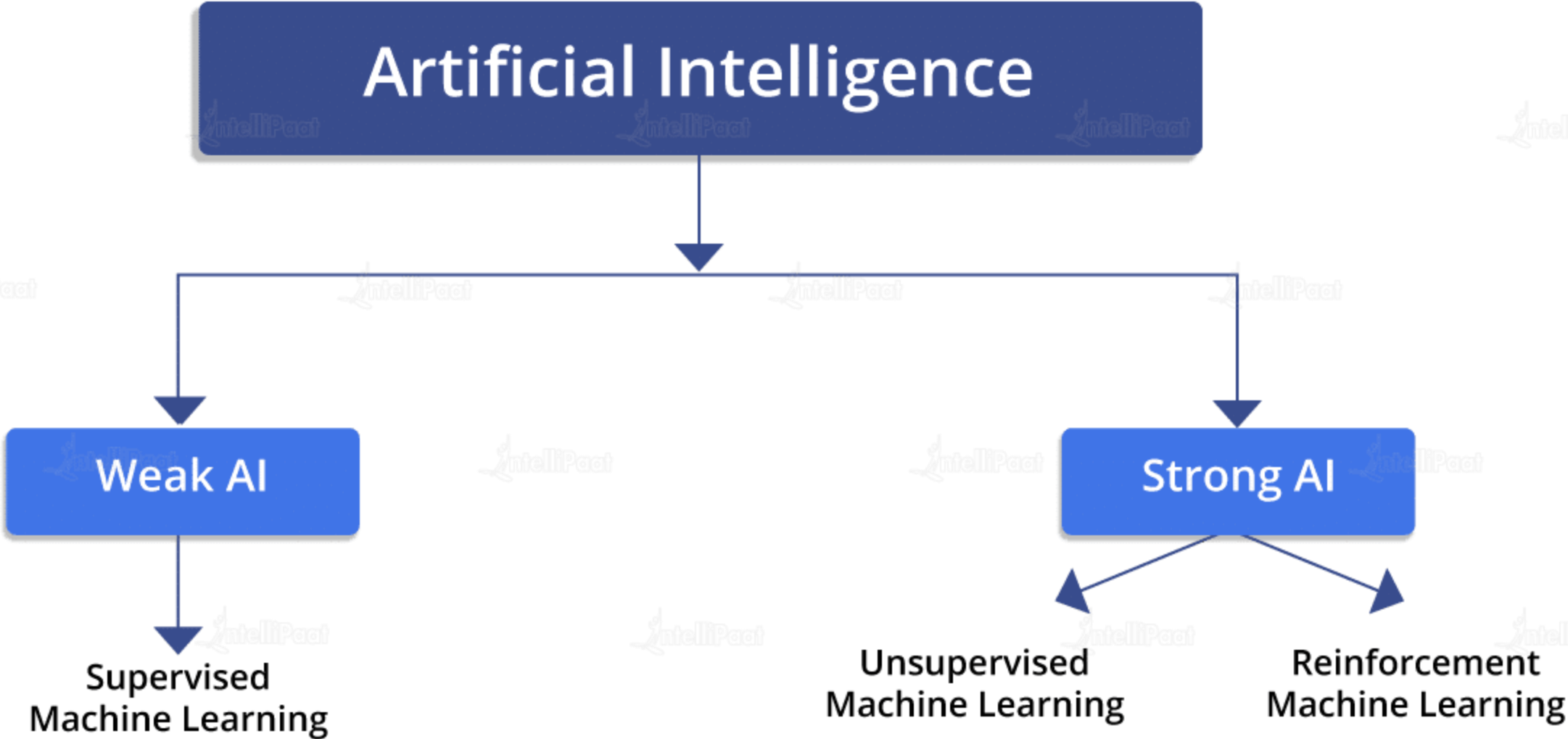
Difference between Machine Learning and Deep Learning

Parameter	Machine Learning	Deep Learning
Data Dependencies	Excellent performances on a small/medium dataset	Excellent performance on a big dataset
Hardware dependencies	Work on a low-end machine.	Requires powerful machine, preferably with GPU: DL performs a significant amount of matrix multiplication
Feature engineering	Need to understand the features that represent the data	No need to understand the best feature that represents the data
Execution time	From few minutes to hours	Up to weeks. Neural Network needs to compute a significant number of weights
Interpretability	Some algorithms are easy to interpret (logistic, decision tree), some are almost impossible (SVM, XGBoost)	Difficult to impossible

When to use ML or DL?

Parameter	Machine Learning	Deep Learning
Training dataset	Small	Large
Choose features	Yes	No
Number of algorithms	Many	Few
Training time	Short	Long

Artificial Intelligence



```
graph TD; AI[Artificial Intelligence] --> WeakAI[Weak AI]; AI --> StrongAI[Strong AI]; WeakAI --> SML[Supervised Machine Learning]; StrongAI --> UML[Unsupervised Machine Learning]; StrongAI --> RML[Reinforcement Machine Learning];
```

Weak AI

**Supervised
Machine Learning**

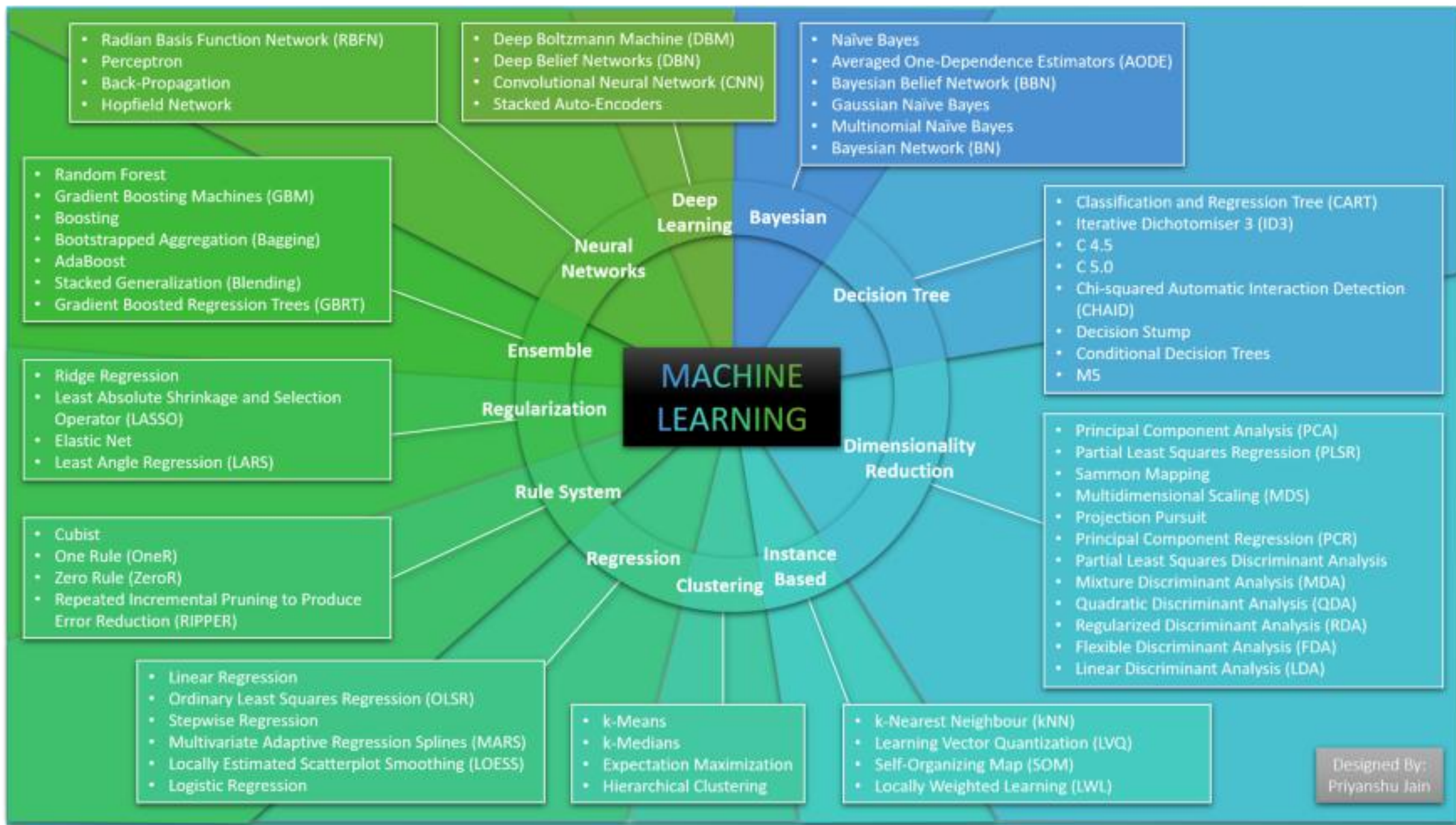
Strong AI

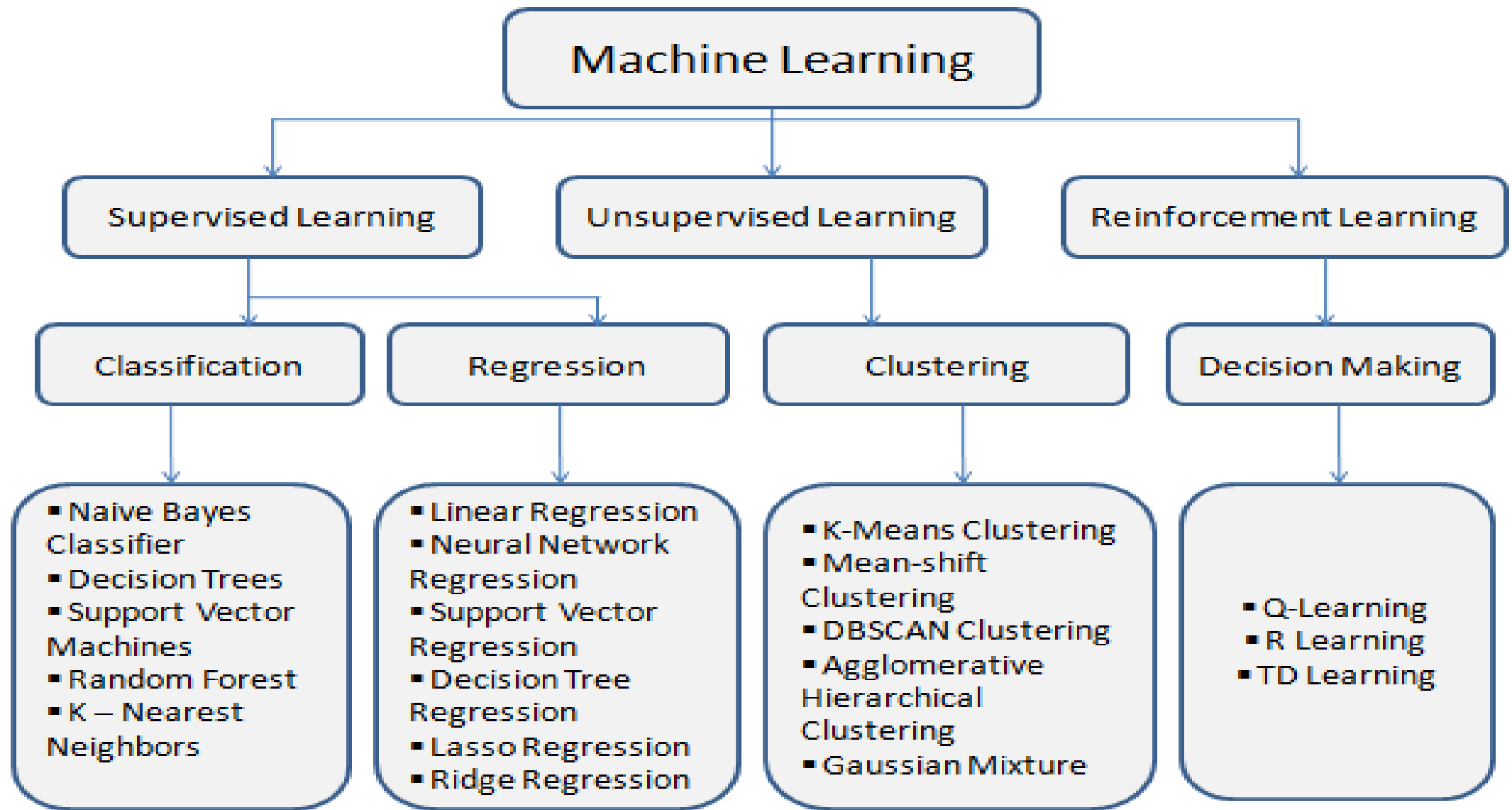
**Unsupervised
Machine Learning**

**Reinforcement
Machine Learning**

Criteria	Supervised ML	Unsupervised ML	Reinforcement ML
Definition	Learns by using labelled data	Trained using unlabelled data without any guidance.	Works on interacting with the environment
Type of data	Labelled data	Unlabelled data	No – predefined data
Type of problems	Regression and classification	Association and Clustering	Exploitation or Exploration
Supervision	Extra supervision	No supervision	No supervision
Algorithms	Linear Regression, Logistic Regression, SVM, KNN etc.	K – Means, C – Means, Apriori	Q – Learning, SARSA
Aim	Calculate outcomes	Discover underlying patterns	Learn a series of action
Application	Risk Evaluation, Forecast Sales	Recommendation System, Anomaly Detection	Self Driving Cars, Gaming, Healthcare

Artificial Intelligence	Machine learning
Artificial intelligence is a technology which enables a machine to simulate human behavior.	Machine learning is a subset of AI which allows a machine to automatically learn from past data without programming explicitly.
The goal of AI is to make a smart computer system like humans to solve complex problems.	The goal of ML is to allow machines to learn from data so that they can give accurate output.
In AI, we make intelligent systems to perform any task like a human.	In ML, we teach machines with data to perform a particular task and give an accurate result.
Machine learning and deep learning are the two main subsets of AI.	Deep learning is a main subset of machine learning.
AI has a very wide range of scope.	Machine learning has a limited scope.
AI is working to create an intelligent system which can perform various complex tasks.	Machine learning is working to create machines that can perform only those specific tasks for which they are trained.
AI system is concerned about maximizing the chances of success.	Machine learning is mainly concerned about accuracy and patterns.
The main applications of AI are Siri, customer support using chatbots , Expert System, Online game playing, intelligent humanoid robot, etc.	The main applications of machine learning are Online recommender system, Google search algorithms, Facebook auto friend tagging suggestions , etc.
On the basis of capabilities, AI can be divided into three types, which are, Weak AI, General AI , and Strong AI .	Machine learning can also be divided into mainly three types that are Supervised learning, Unsupervised learning , and Reinforcement learning .
It includes learning, reasoning, and self-correction.	It includes learning and self-correction when introduced with new data.
AI completely deals with Structured, semi-structured, and unstructured data.	Machine learning deals with Structured and semi-structured data.





Classification of Deep Learning Methods

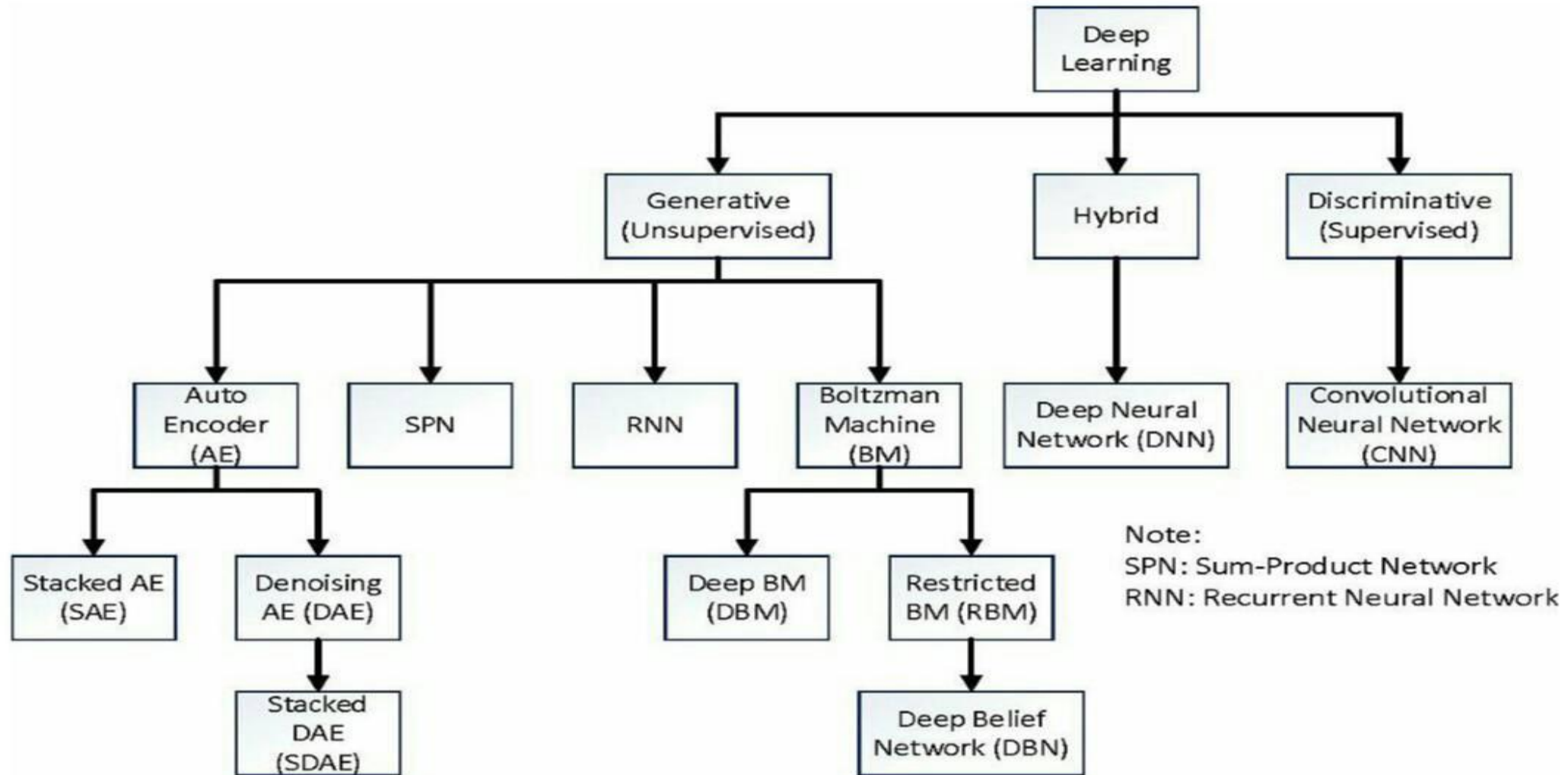
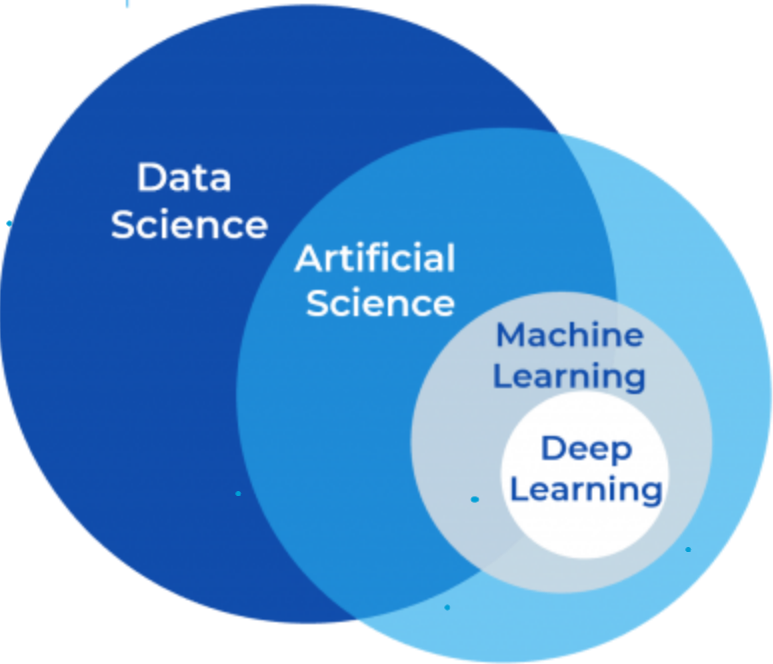


Chart
1



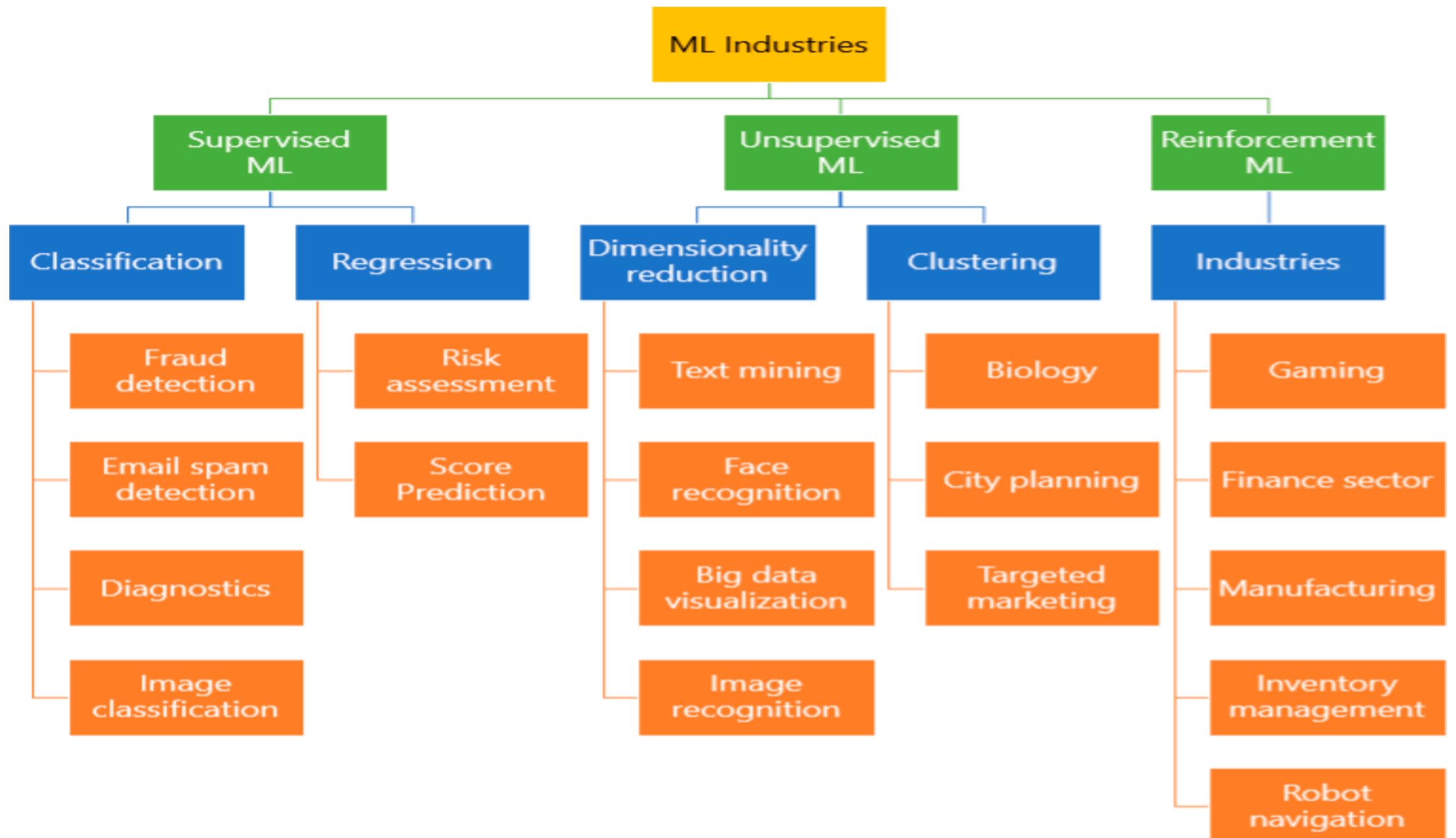
Source: Cambridge Finsights

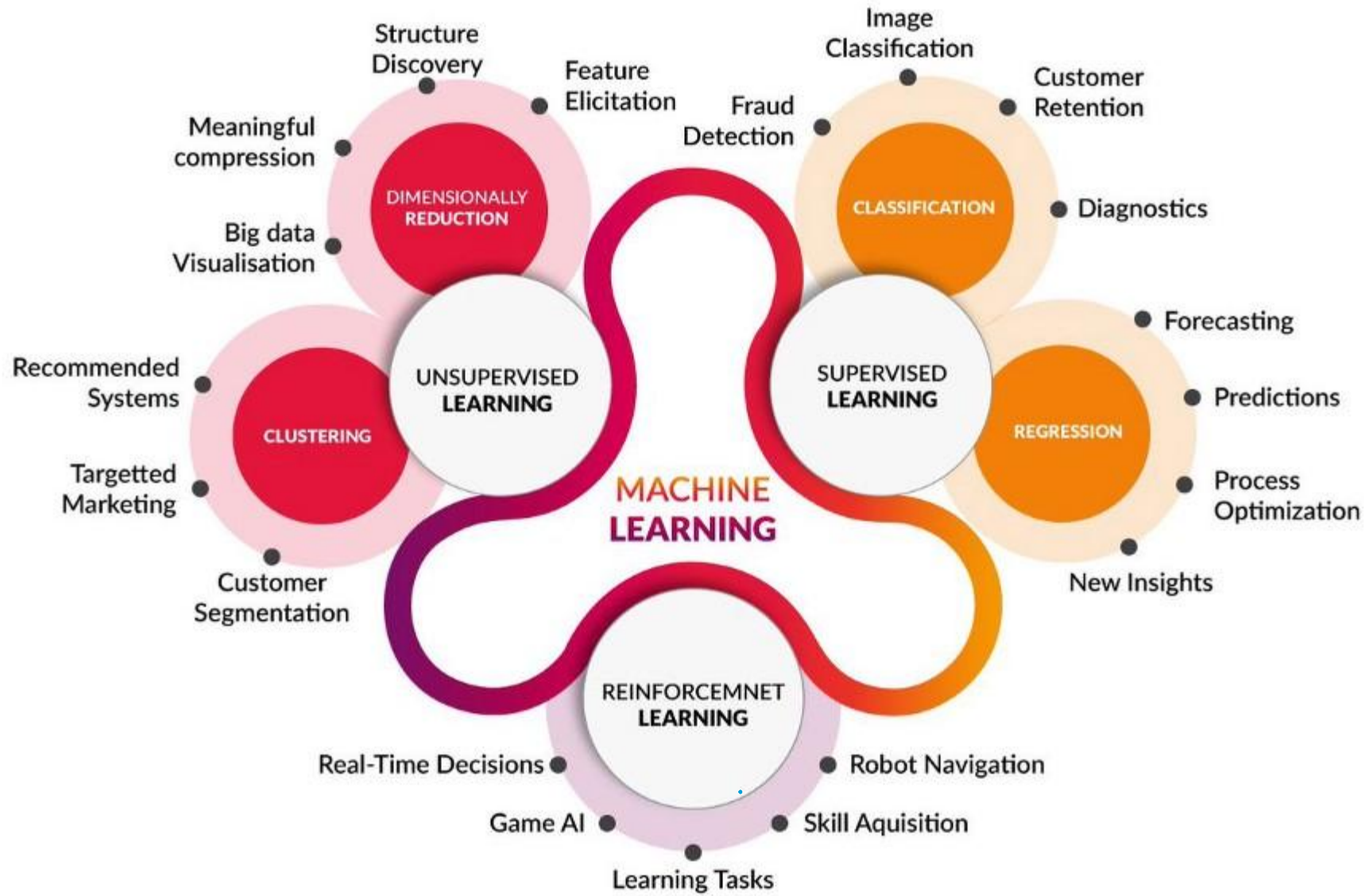
Table
1

Machine Learning Models

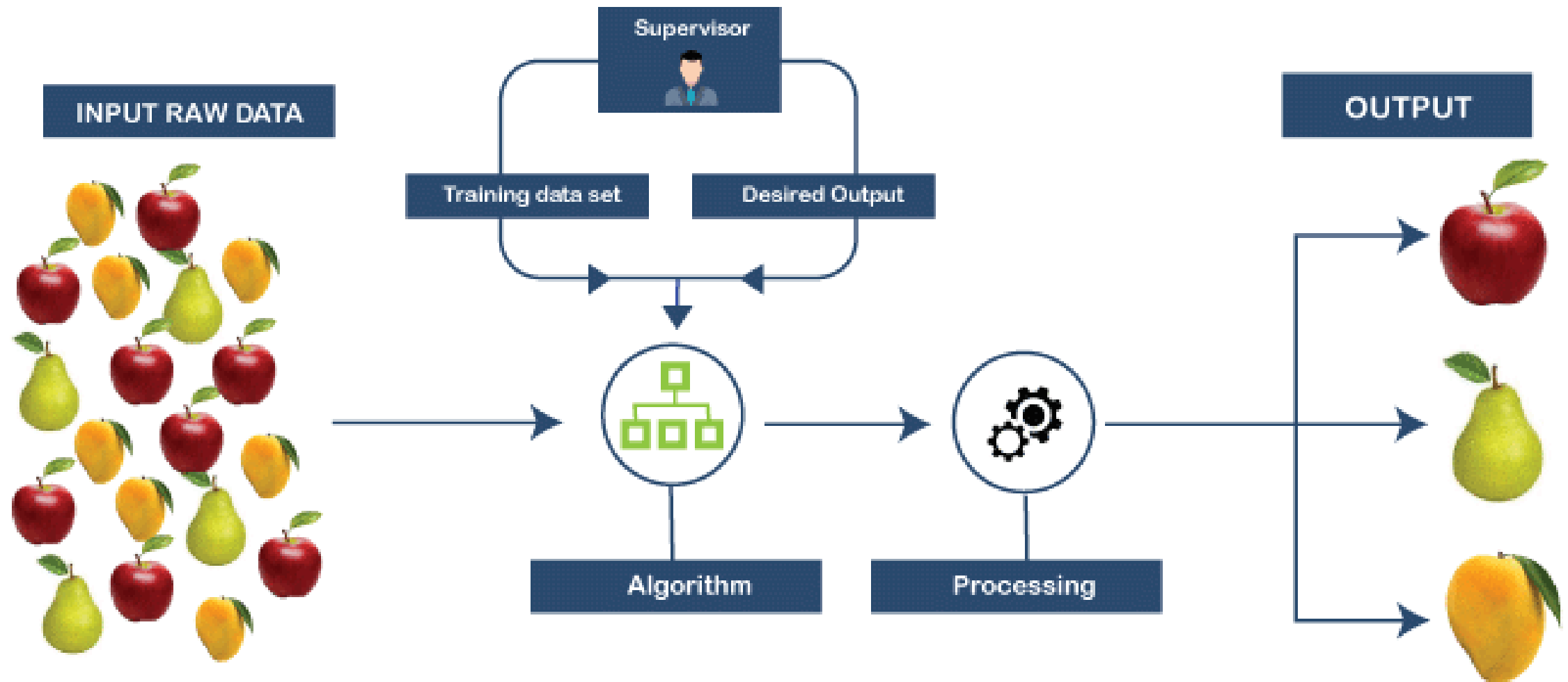
ML Algorithm Type		
Variables	Supervised (Target Variable)	Unsupervised (No Target Variable)
Continuous	Regression - Linear; Penalized Regression/LSSO - Logistic - Classification and Regression Tree (CART) - Random Forest	Dimensionality Reduction - Principal Components Analysis (PCA) Clustering - K-Means - Hierarchical
	Categorical	Dimensionality Reduction - Principal Components Analysis (PCA) Clustering - K-Means - Hierarchical
Continuous or Categorical	Neural Networks Deep Learning Reinforcement Learning	Neural Networks Deep Learning Reinforcement Learning

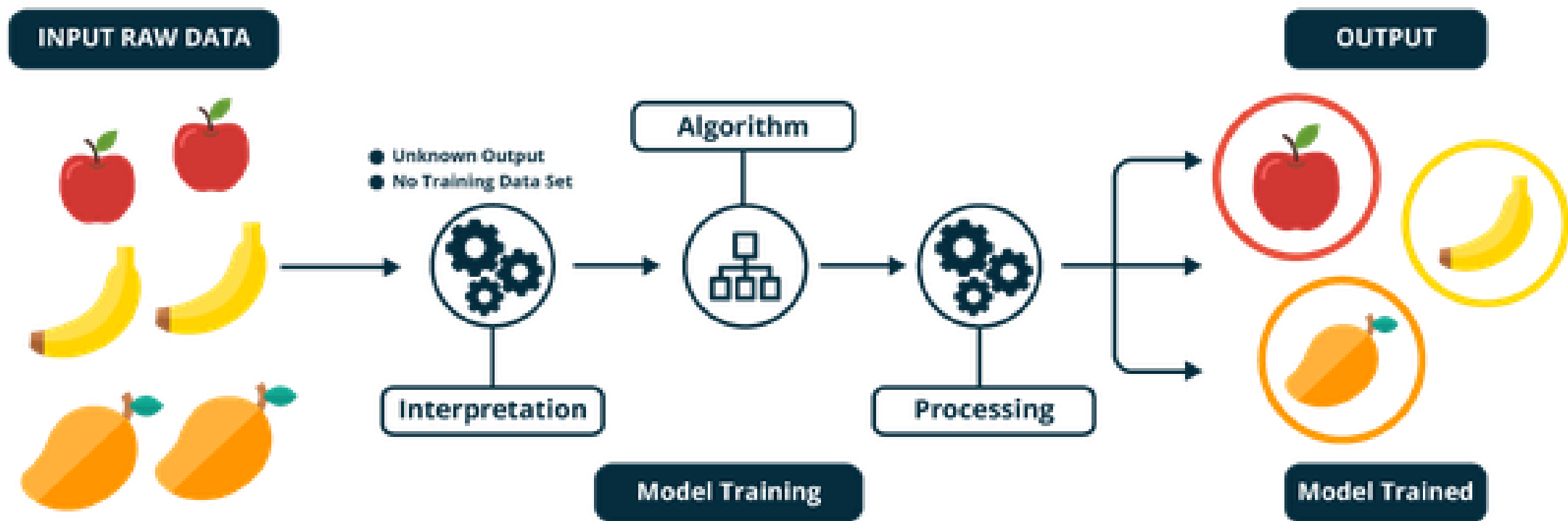
Source: Kathleen DeRose, CFA and Christophe Le Lannou. "Quantitative Methods, Machine Learning", CFA Institute, 2019.



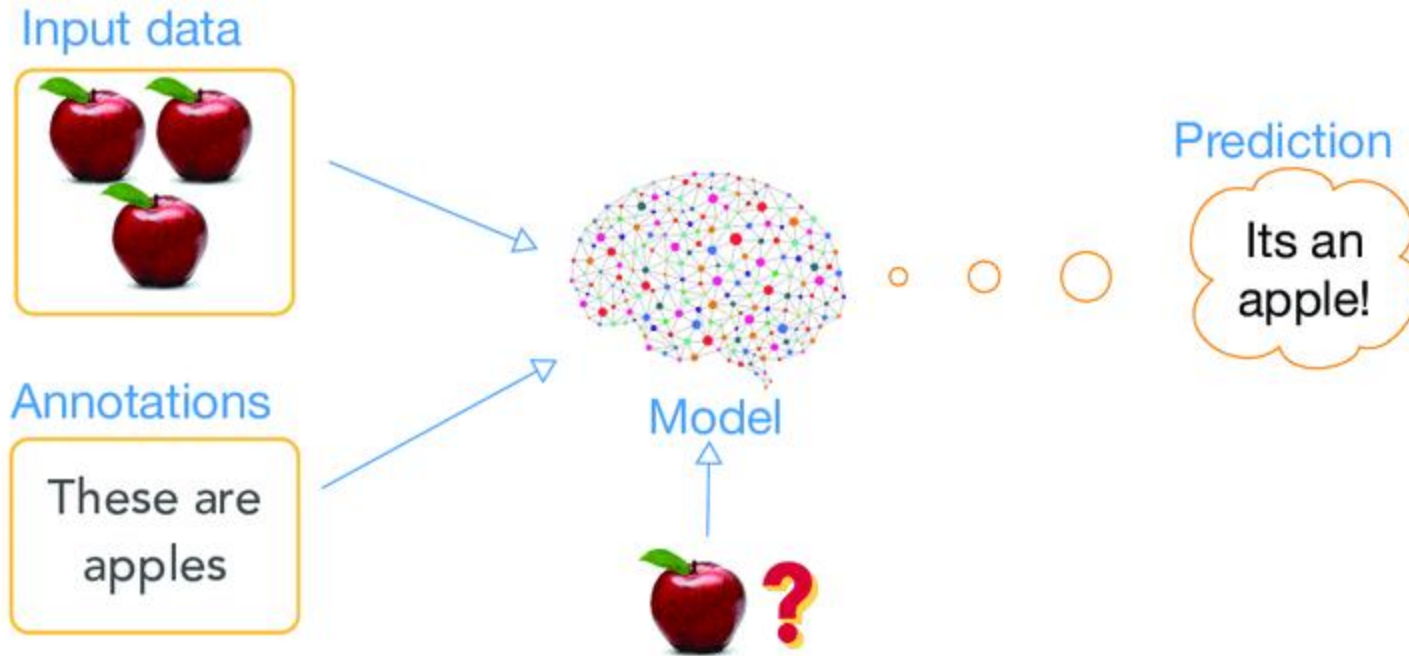


SUPERVISED LEARNING

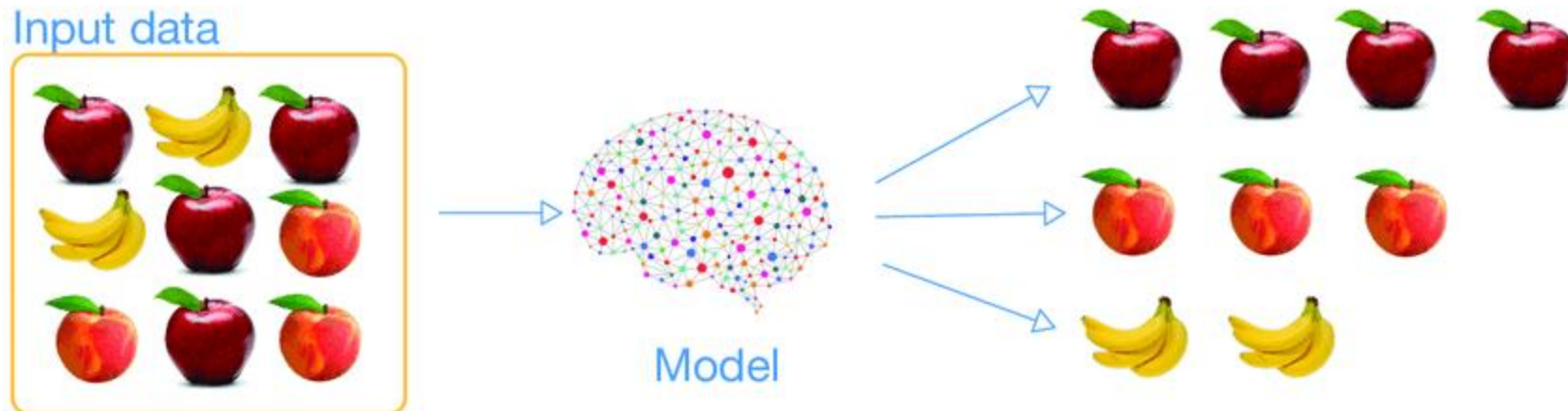




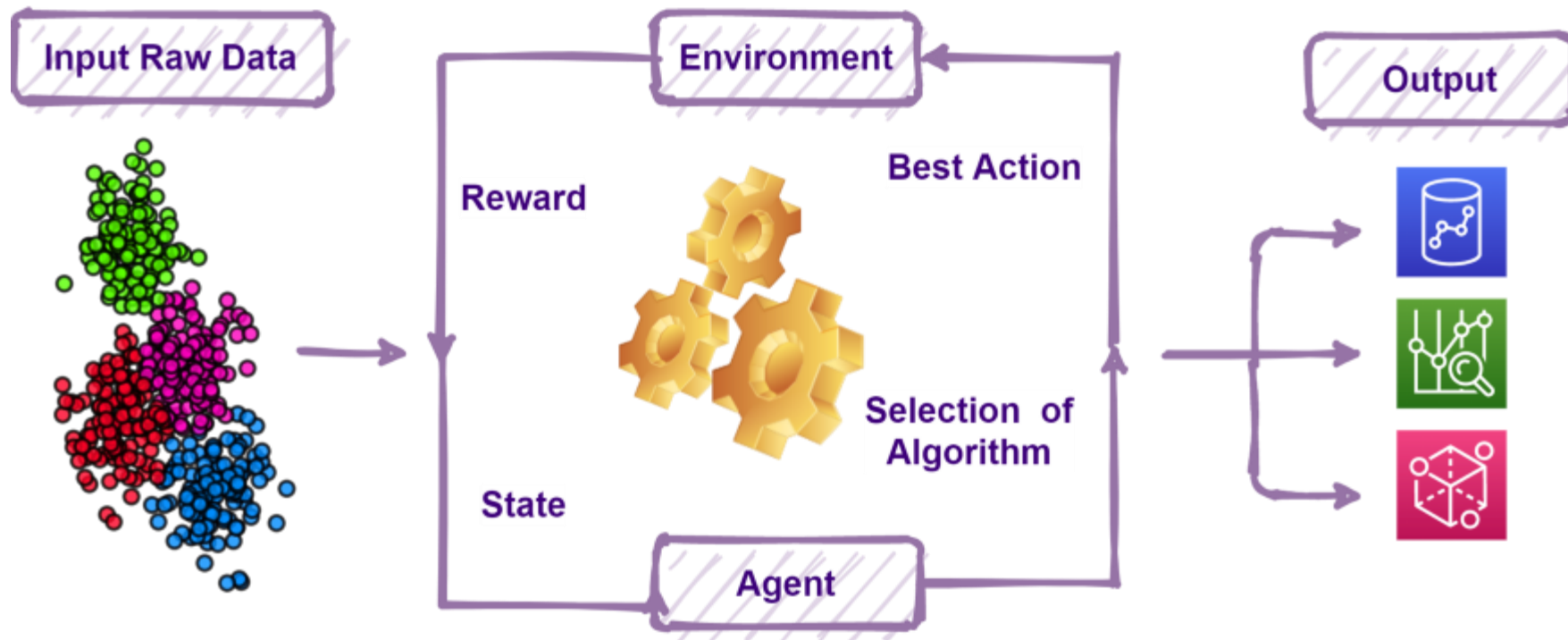
supervised learning



unsupervised learning



Reinforcement Learning



7 steps of Machine Learning



How to get datasets for Machine Learning

- The key to success in the field of machine learning or to become a great data scientist is to practice with different types of datasets.
- But discovering a suitable dataset for each kind of machine learning project is a difficult task.
- So, in this topic, we will provide the detail of the sources from where you can easily get the dataset according to your project.

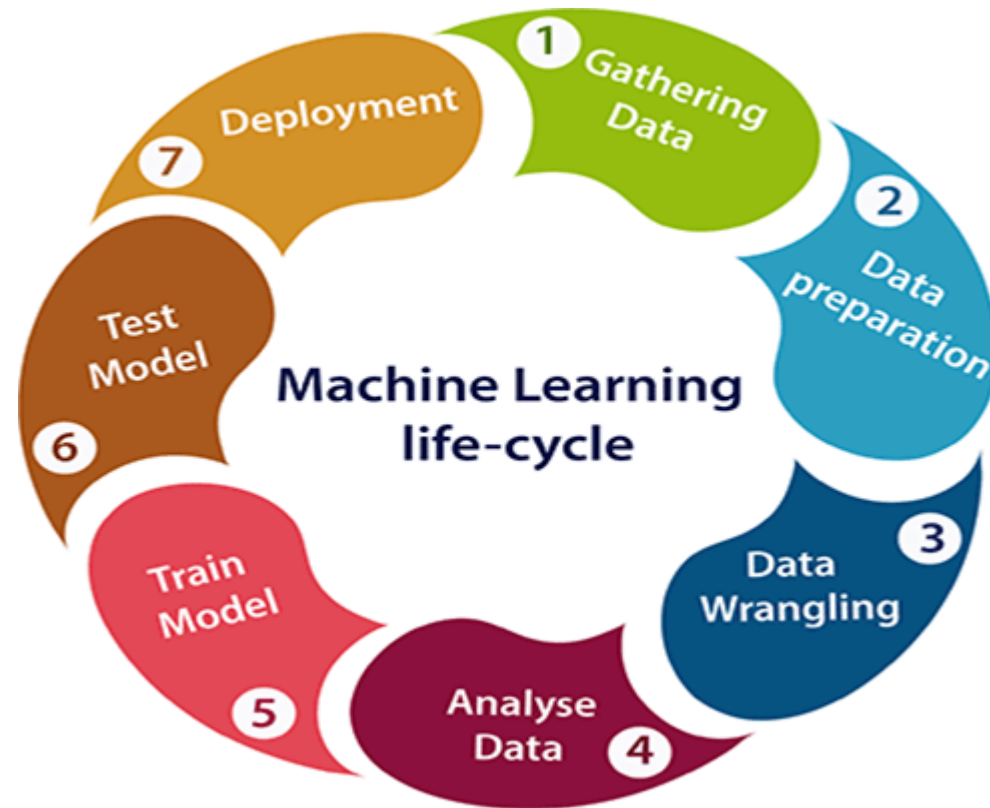
What is a dataset?

- **A dataset** is a collection of data in which data is arranged in some order.
- A dataset can contain any data from a series of an array to a database table.
- **Types of data in datasets**
 - **Numerical data:** Such as house price, temperature, etc.
 - **Categorical data:** Such as Yes/No, True/False, Blue/green, etc.
 - **Ordinal data:** These data are similar to categorical data but can be measured on the basis of comparison.

Need of Dataset

- To work with machine learning projects, we need a huge amount of data, because, without the data, one cannot train ML/AI models.
- Collecting and preparing the dataset is one of the most crucial parts while creating an ML/AI project.
- The technology applied behind any ML projects cannot work properly if the dataset is not well prepared and pre-processed.
- During the development of the ML project, the developers completely rely on the datasets.
- In building ML applications, datasets are divided into two parts:
 - **Training dataset:**
 - **Test Dataset**

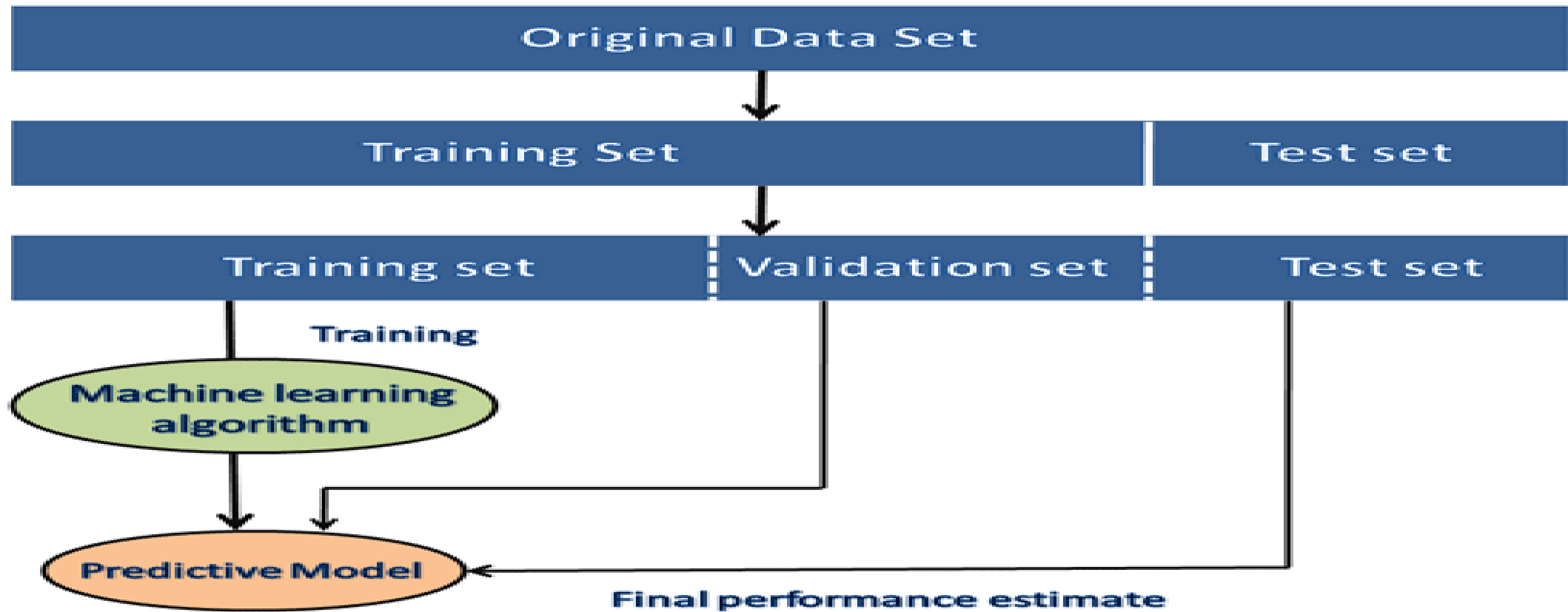
Machine learning Life cycle



Steps Involved in Supervised Learning

- First Determine the type of training dataset
- Collect/Gather the labelled training data.
- Split the training dataset into training **dataset**, **test dataset**, and **validation dataset**.
- Determine the input features of the training dataset, which should have enough knowledge so that the model can accurately predict the output.
- Determine the suitable algorithm for the model, such as support vector machine, decision tree, etc.
- Execute the algorithm on the training dataset. Sometimes we need validation sets as the control parameters, which are the subset of training datasets.
- Evaluate the accuracy of the model by providing the test set. If the model predicts the correct output, which means our model is accurate.

Dataset



Why do we need Data Preprocessing?

- A real-world data generally contains noises, missing values, and maybe in an unusable format which cannot be directly used for machine learning models.
- Data preprocessing is required tasks for cleaning the data and making it suitable for a machine learning model which also increases the accuracy and efficiency of a machine learning model.
- It involves below steps:
 - **Getting the dataset**
 - **Importing libraries**
 - **Importing datasets**
 - **Finding Missing Data**
 - **Encoding Categorical Data**
 - **Splitting dataset into training and test set**
 - **Feature scaling**

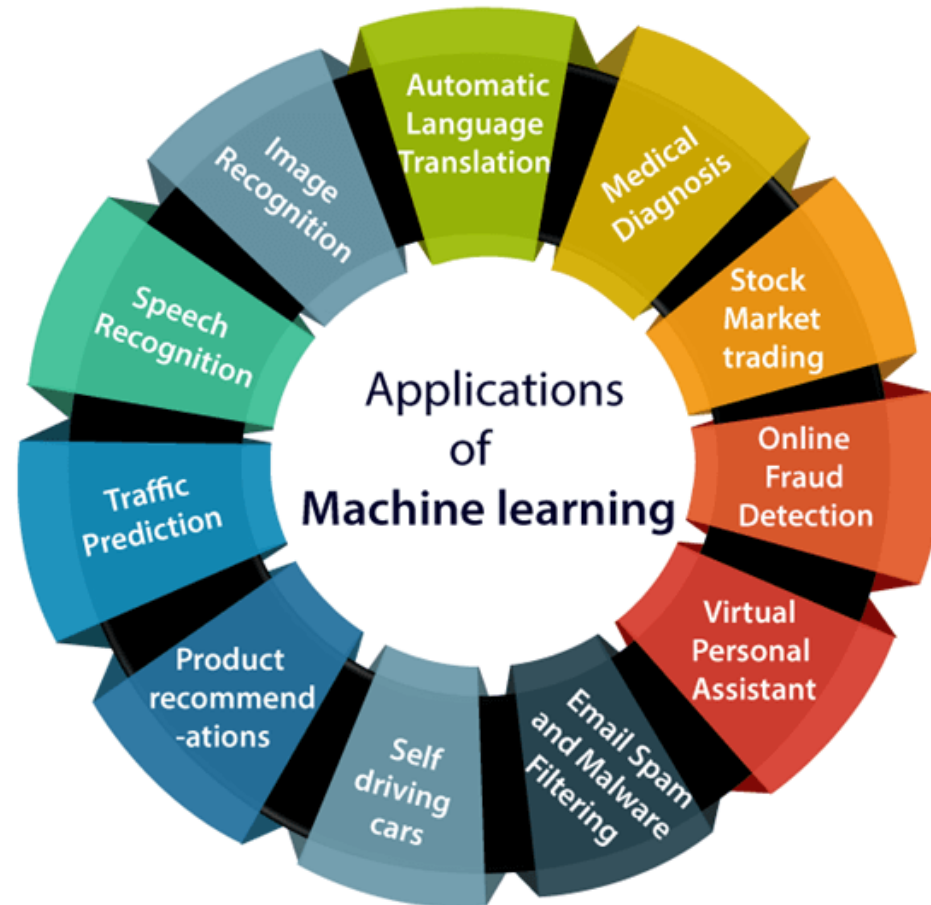
Get the Dataset

- To create a machine learning model, the first thing we required is a dataset as a machine learning model completely works on data.
- The collected data for a particular problem in a proper format is known as the **dataset**.
- Dataset may be of different formats for different purposes, such as, if we want to create a machine learning model for business purpose, then dataset will be different with the dataset required for a liver patient.
- So each dataset is different from another dataset.
- To use the dataset in our code, we usually put it into a **CSV file**. However, sometimes, we may also need to use an HTML or xlsx file.
- **What is a CSV File?**
 - CSV stands for "**Comma-Separated Values**" files; it is a file format which allows us to save the tabular data, such as spreadsheets. It is useful for huge datasets and can use these datasets in programs.

Popular sources for Machine Learning datasets

- [Kaggle Datasets](#)
- [UCI Machine Learning Repository](#)
- [Datasets via AWS](#)
- [Google's Dataset Search Engine](#)
- [Microsoft Datasets](#)
- [Awesome Public Dataset Collection](#)
- [Government Datasets](#)
- [Computer Vision Datasets](#)
- [Scikit-learn dataset](#)

Applications of Machine learning



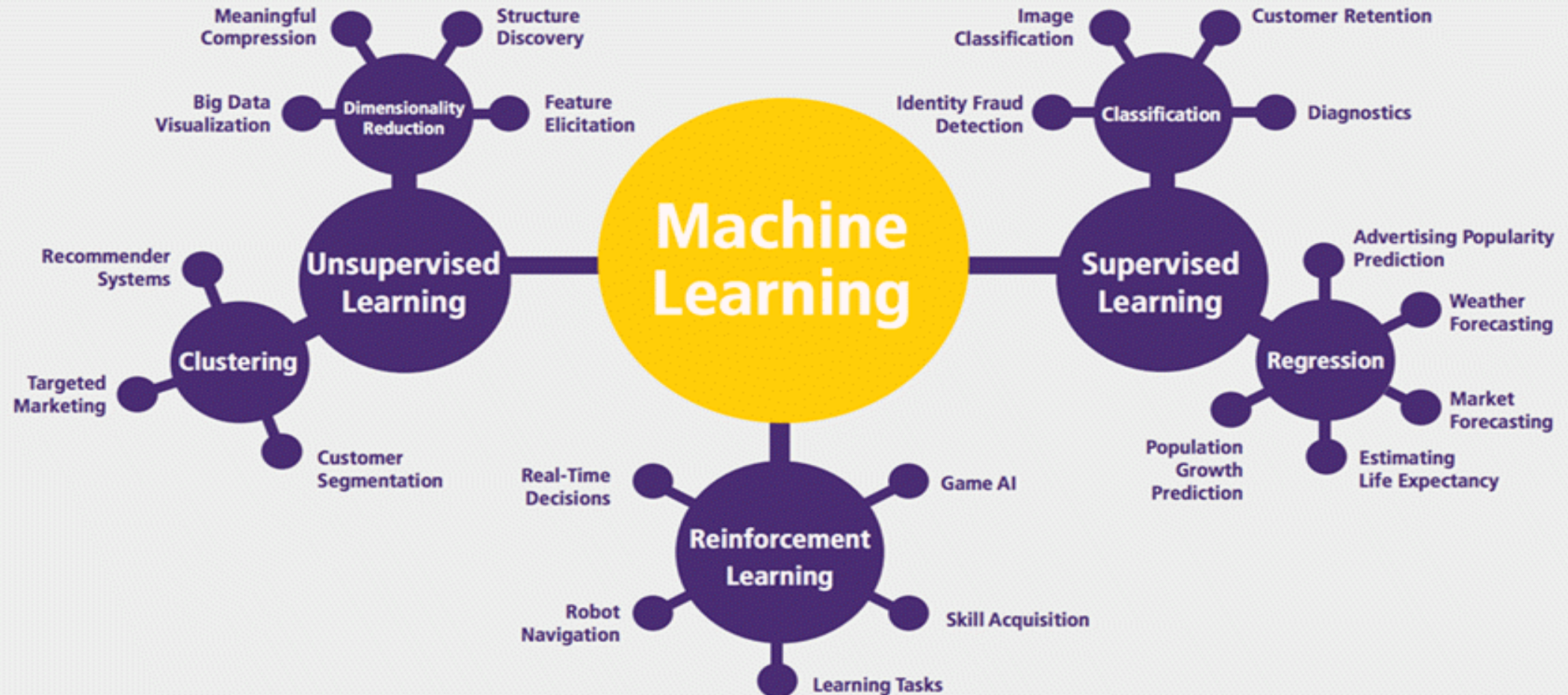
Applications of Machines Learning

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping.

List of Common Machine Learning Algorithms

1. Linear Regression
2. Logistic Regression
3. Decision Tree
4. SVM
5. Naive Bayes
6. kNN
7. K-Means
8. Random Forest
9. Dimensionality Reduction Algorithms
10. Gradient Boosting algorithms
 1. GBM
 2. XGBoost
 3. LightGBM
 4. CatBoost

Machine Learning Algorithms and Where they are Used?



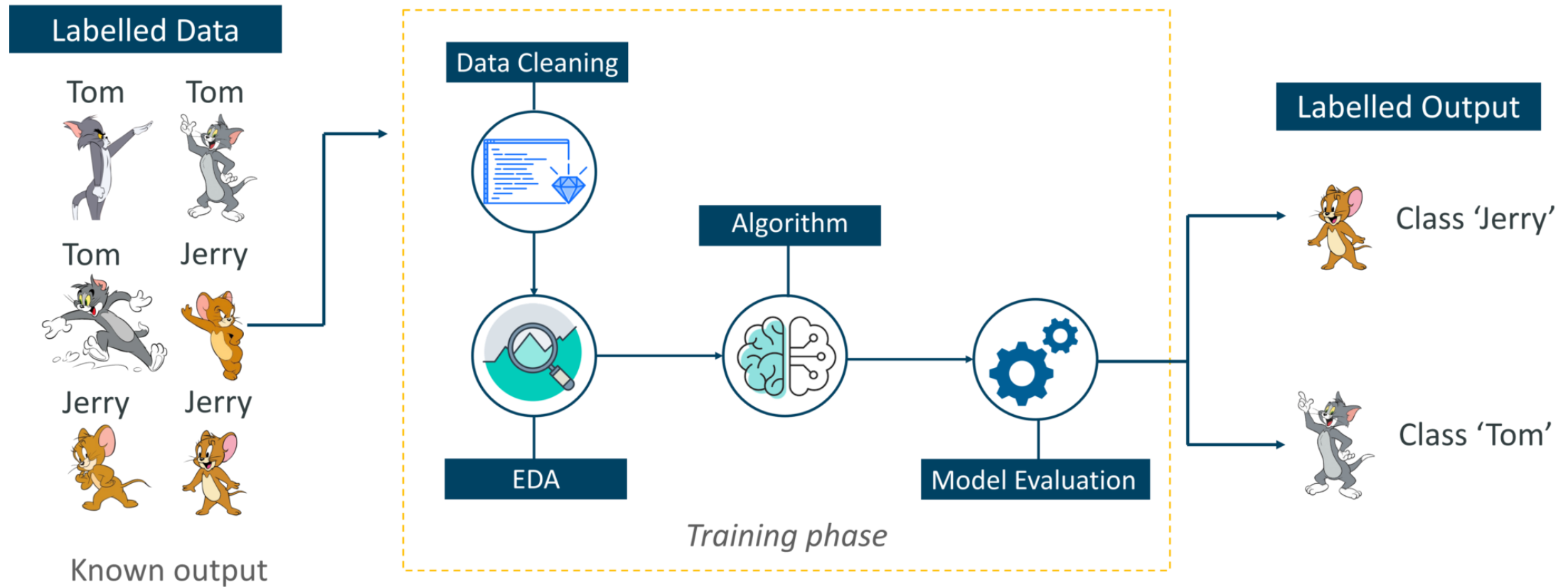
Supervised Learning Algorithms:

- Supervised learning is the types of machine learning in which machines are trained using well "labelled" training data, and on basis of that data, machines predict the output.
- The labelled data means some input data is already tagged with the correct output.
- In the real-world, supervised learning can be used for **Risk Assessment, Image classification, Fraud Detection, spam filtering, etc..**

Supervised Learning Algorithms:

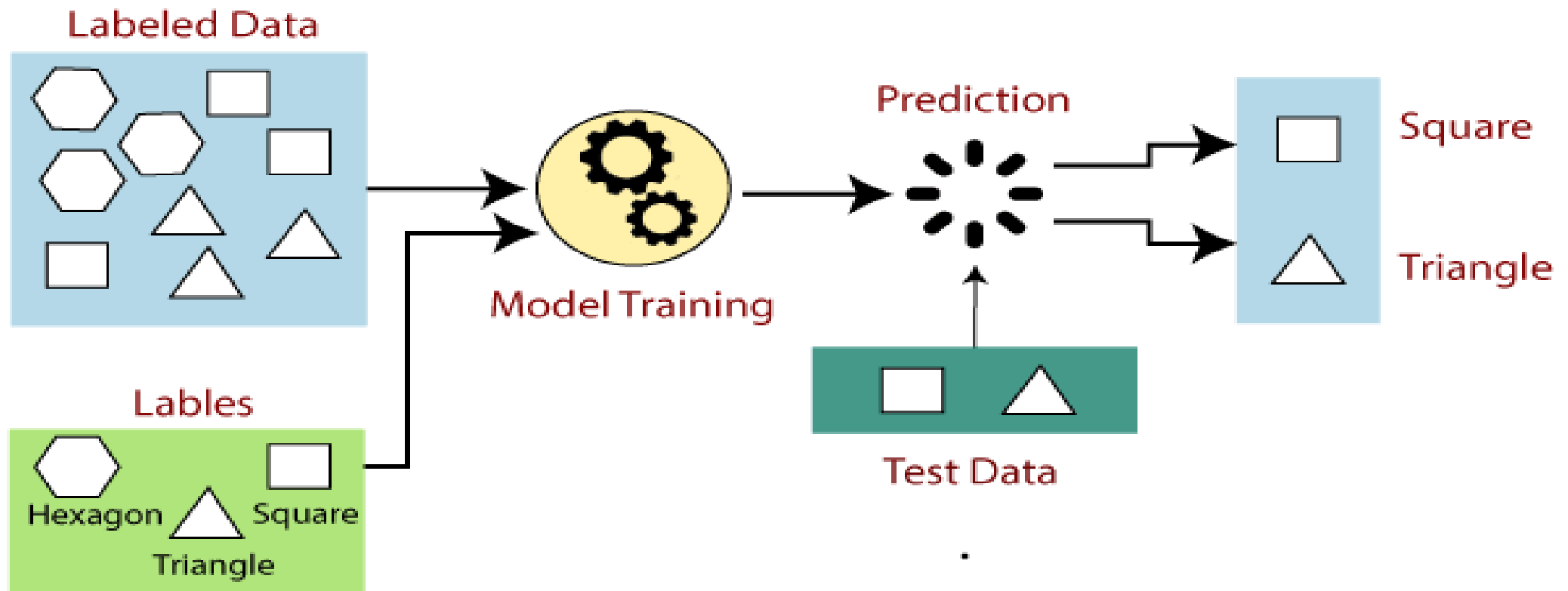
- Supervised learning uses labeled training data to learn the mapping function that turns input variables (X) into the output variable (Y). In other words, it solves for f in the following equation: $Y = f(X)$
- This allows us to accurately generate outputs when given new inputs.

Supervised Learning – Introduction To Machine Learning



EDA or Exploratory Data Analysis

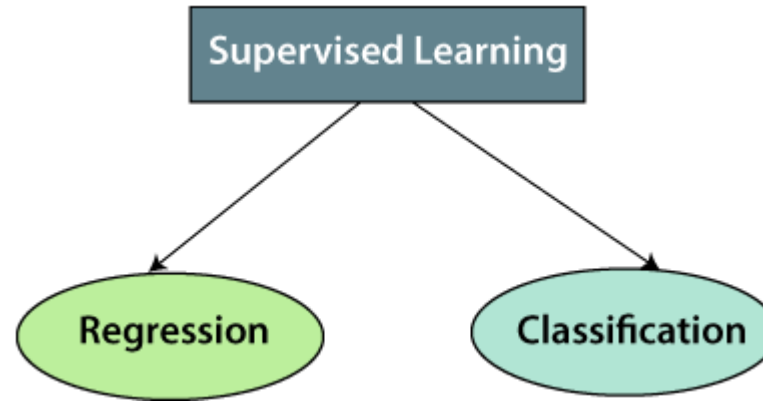
How Supervised Learning Works?



How Supervised Learning Works?

- Suppose we have a dataset of different types of shapes which includes square, rectangle, triangle, and Polygon. Now the first step is that we need to train the model for each shape.
 - If the given shape has four sides, and all the sides are equal, then it will be labelled as a **Square**.
 - If the given shape has three sides, then it will be labelled as a **triangle**.
 - If the given shape has six equal sides then it will be labelled as **hexagon**.

Types of supervised Machine learning Algorithms



1. Regression

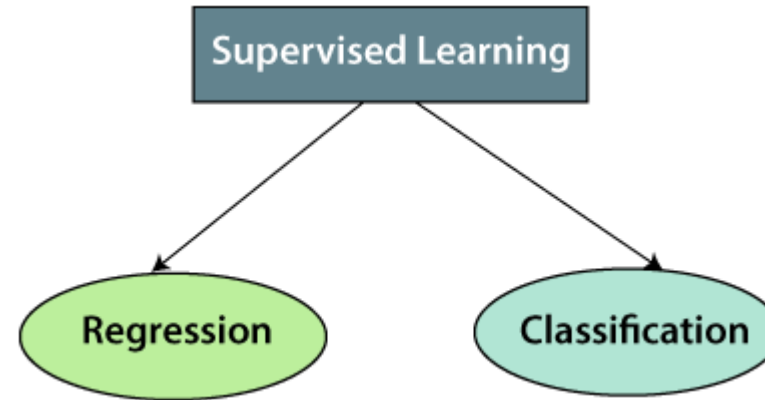
Regression algorithms are used if there is a relationship between the **input variable and the output variable**.

It is used for the prediction of continuous variables, such as Weather forecasting, Market Trends, etc.

Below are some popular Regression algorithms which come under supervised learning:

- Linear Regression
- Regression Trees
- Non-Linear Regression
- Bayesian Linear Regression
- Polynomial Regression

Types of supervised Machine learning Algorithms



2. Classification

Classification algorithms are used when the output variable is categorical, which means there are two classes such as Yes-No, Male-Female, True-false, etc.

Spam Filtering,

- Random Forest
- Decision Trees
- Logistic Regression
- Support vector Machines

Advantages/Disadvantages of Supervised learning:

- **Advantages of Supervised learning:**

- With the help of supervised learning, the model can predict the output on the basis of prior experiences.
- In supervised learning, we can have an exact idea about the classes of objects.
- Supervised learning model helps us to solve various real-world problems such as **fraud detection, spam filtering**, etc.

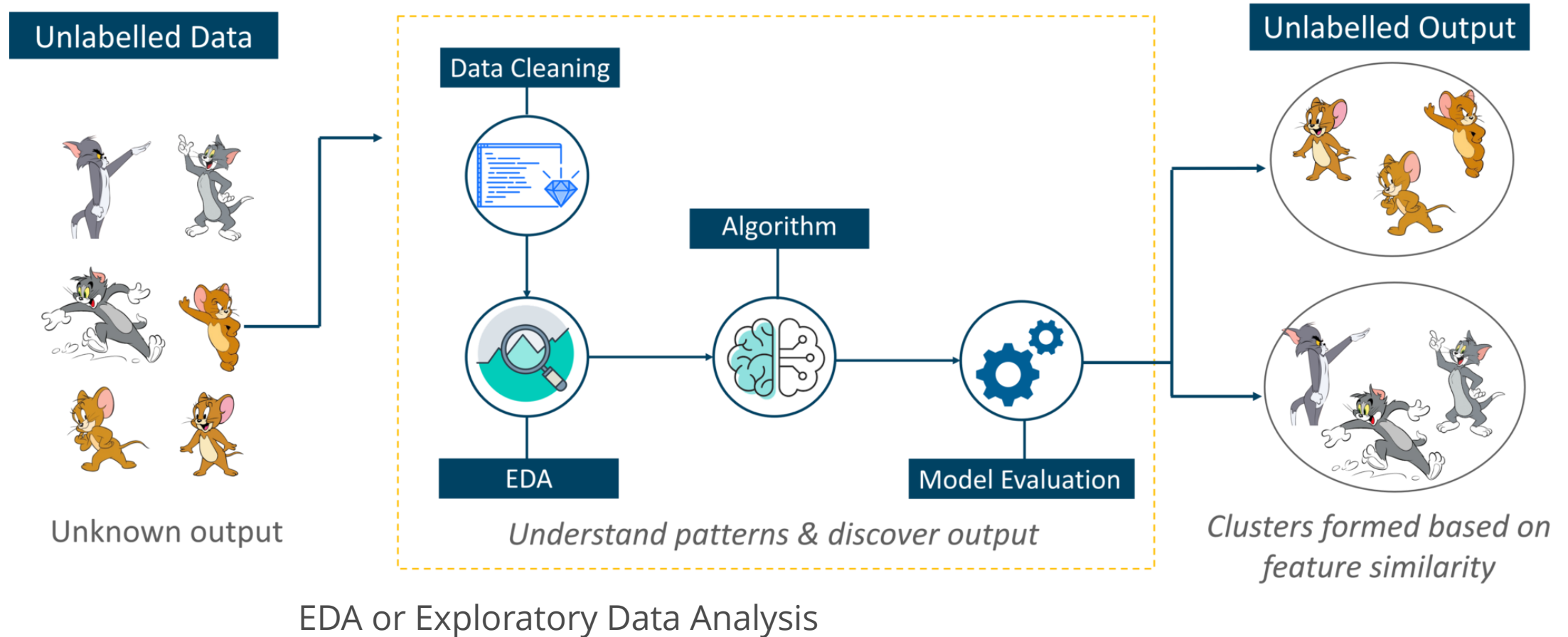
- **Disadvantages of supervised learning:**

- Supervised learning models are not suitable for handling the complex tasks.
- Supervised learning cannot predict the correct output if the test data is different from the training dataset.
- Training required lots of computation times.
- In supervised learning, we need enough knowledge about the classes of object.

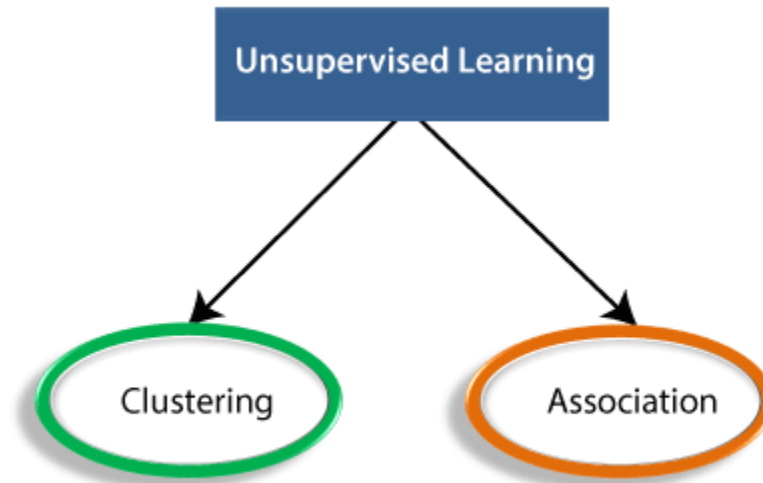
What is Unsupervised Learning?

- *Unsupervised learning is a type of machine learning in which models are trained using unlabeled dataset and are allowed to act on that data without any supervision.*

Unsupervised Learning



Types of Unsupervised Learning Algorithm:

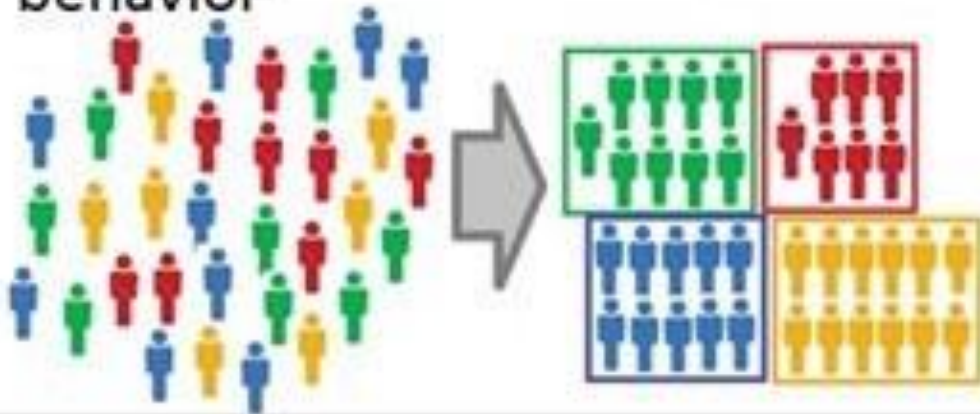


Types of Unsupervised Learning Algorithm:

Unsupervised Learning

Clustering

Grouping customers by purchasing behavior



Association

People that buy X tend to buy Y

People that buy A+B tend to buy C



Analysis.

Un

- **Clusterin**

Clusterin
clusters
remains
with the
Cluster a
data obje
and abse



sample



Cluster/group

Unsupervised Learning algorithms:

- K-means clustering
- KNN (k-nearest neighbors)
- Hierarchical clustering
- Anomaly detection
- Neural Networks
- Principle Component Analysis
- Independent Component Analysis
- Apriori algorithm
- Singular value decomposition

Advantages /Disadvantages of Unsupervised Learning

- **Advantages of Unsupervised Learning**

- Unsupervised learning is used for more complex tasks as compared to supervised learning because, in unsupervised learning, we don't have labeled input data.
- Unsupervised learning is preferable as it is easy to get unlabeled data in comparison to labeled data.

- **Disadvantages of Unsupervised Learning**

- Unsupervised learning is intrinsically more difficult than supervised learning as it does not have corresponding output.
- The result of the unsupervised learning algorithm might be less accurate as input data is not labeled, and algorithms do not know the exact output in advance.

Supervised Learning	Unsupervised Learning
Supervised learning algorithms are trained using labeled data.	Unsupervised learning algorithms are trained using unlabeled data.
Supervised learning model takes direct feedback to check if it is predicting correct output or not.	Unsupervised learning model does not take any feedback.
Supervised learning model predicts the output.	Unsupervised learning model finds the hidden patterns in data.
In supervised learning, input data is provided to the model along with the output.	In unsupervised learning, only input data is provided to the model.
The goal of supervised learning is to train the model so that it can predict the output when it is given new data.	The goal of unsupervised learning is to find the hidden patterns and useful insights from the unknown dataset.
Supervised learning needs supervision to train the model.	Unsupervised learning does not need any supervision to train the model.
Supervised learning can be categorized in Classification and Regression problems.	Unsupervised Learning can be classified in Clustering and Associations problems.
Supervised learning can be used for those cases where we know the input as well as corresponding outputs.	Unsupervised learning can be used for those cases where we have only input data and no corresponding output data.
Supervised learning model produces an accurate result.	Unsupervised learning model may give less accurate result as compared to supervised learning.
Supervised learning is not close to true Artificial intelligence as in this, we first train the model for each data, and then only it can predict the correct output.	Unsupervised learning is more close to the true Artificial Intelligence as it learns similarly as a child learns daily routine things by his experiences.
It includes various algorithms such as Linear Regression, Logistic Regression, Support Vector Machine, Multi-class Classification, Decision tree, Bayesian Logic, etc.	It includes various algorithms such as Clustering, KNN, and Apriori algorithm.

Common issues in Machine Learning

Inadequate Training Data

- ❑ The major issue that comes while using machine learning algorithms is the lack of quality as well as quantity of data.
- ❑ Although data plays a vital role in the processing of machine learning algorithms, many data scientists claim that inadequate data, noisy data, and unclean data are extremely exhausting the machine learning algorithms.
- ❑ For example, a simple task requires thousands of sample data, and an advanced task such as speech or image recognition needs millions of sample data examples.
- ❑ Further, data quality is also important for the algorithms to work ideally, but the absence of data quality is also found in Machine Learning applications.

- **Noisy Data-** It is responsible for an inaccurate prediction that affects the decision as well as accuracy in classification tasks.
- **Incorrect data-** It is also responsible for faulty programming and results obtained in machine learning models. Hence, incorrect data may affect the accuracy of the results also.
- **Generalizing of output data-** Sometimes, it is also found that generalizing output data becomes complex, which results in comparatively poor future actions.

Poor quality of data

- Data plays a significant role in machine learning, and it must be of good quality as well.
- Noisy data, incomplete data, inaccurate data, and unclean data lead to less accuracy in classification and low-quality results.
- Hence, data quality can also be considered as a major common problem while processing machine learning algorithms.

Non-representative training data

- To make sure our training model is generalized well or not, we have to ensure that sample training data must be representative of new cases that we need to generalize.
- The training data must cover all cases that are already occurred as well as occurring.
- Further, if we are using non-representative training data in the model, it results in less accurate predictions.
- A machine learning model is said to be ideal if it predicts well for generalized cases and provides accurate decisions.
- If there is less training data, then there will be a sampling noise in the model, called the non-representative training set. It won't be accurate in predictions.
- To overcome this, it will be biased against one class or a group.
- Hence, we should use representative data in training to protect against being biased and make accurate predictions without any drift.

Overfitting

- Overfitting is one of the most common issues faced by Machine Learning engineers and data scientists.
- Whenever a machine learning model is trained with a huge amount of data, it starts capturing noise and inaccurate data into the training data set.
- It negatively affects the performance of the model.

- Let's understand with a simple example where we have a few training data sets such as 1000 mangoes, 1000 apples, 1000 bananas, and 5000 papayas.
- Then there is a considerable probability of identification of an apple as papaya because we have a massive amount of biased data in the training data set; hence prediction got negatively affected.
- The main reason behind overfitting is using non-linear methods used in machine learning algorithms as they build non-realistic data models

Methods to reduce overfitting:

- Increase training data in a dataset.
- Reduce model complexity by simplifying the model by selecting one with fewer parameters
- Ridge Regularization and Lasso Regularization
- Early stopping during the training phase
- Reduce the noise
- Reduce the number of attributes in training data.
- Constraining the model.

Underfitting

- Underfitting is just the opposite of overfitting.
- Whenever a machine learning model is trained with fewer amounts of data, and as a result, it provides incomplete and inaccurate data and destroys the accuracy of the machine learning model.
- Underfitting occurs when our model is too simple to understand the base structure of the data, just like an **undersized pant**.
- This generally happens when we have limited data into the data set, and we try to build a linear model with non-linear data.
- In such scenarios, the complexity of the model destroys, and rules of the machine learning model become too easy to be applied on this data set, and the model starts doing wrong predictions as well.

Methods to reduce Underfitting

- Increase model complexity
- Remove noise from the data
- Trained on increased and better features
- Reduce the constraints
- Increase the number of epochs to get better results.

Monitoring and maintenance

- Generalized output data is mandatory for any machine learning model; hence, regular monitoring and maintenance become compulsory for the same.
- Different results for different actions require data change; hence editing of codes as well as resources for monitoring them also become necessary.

Getting bad recommendations

- A machine learning model operates under a specific context which results in bad recommendations and concept drift in the model.
- Let's understand with an example where at a specific time customer is looking for some gadgets, but now customer requirement changed over time but still machine learning model showing same recommendations to the customer while customer expectation has been changed.
- This incident is called a **Data Drift**.
- It generally occurs when new data is introduced or interpretation of data changes.
- However, we can overcome this by regularly updating and monitoring data according to the expectations.

Lack of skilled resources

- Machine Learning and Artificial Intelligence are continuously growing in the market, still these industries are fresher in comparison to others.
- The absence of skilled resources in the form of manpower is also an issue. Hence, we need manpower having in-depth knowledge of mathematics, science, and technologies for developing and managing scientific substances for machine learning.

Customer Segmentation

- Customer segmentation is also an important issue while developing a machine learning algorithm.
- To identify the customers who paid for the recommendations shown by the model and who don't even check them.
- Hence, an algorithm is necessary to recognize the customer behavior and trigger a relevant recommendation for the user based on past experience.

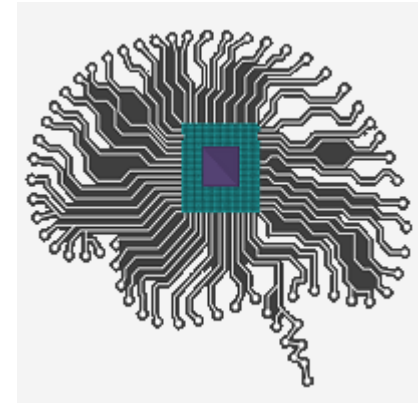
Process Complexity of Machine Learning

- The machine learning process is very complex, which is also another major issue faced by machine learning engineers and data scientists.
- However, Machine Learning and Artificial Intelligence are very new technologies but are still in an experimental phase and continuously being changing over time.
- There is the majority of hits and trial experiments; hence the probability of error is higher than expected.
- Further, it also includes analyzing the data, removing data bias, training data, applying complex mathematical calculations, etc., making the procedure more complicated and quite tedious.

Data Bias

- Data Biasing is also found a big challenge in Machine Learning.
- These errors exist when certain elements of the dataset are heavily weighted or need more importance than others.
- Biased data leads to inaccurate results, skewed outcomes, and other analytical errors.
- However, we can resolve this error by determining where data is actually biased in the dataset.
- Further, take necessary steps to reduce it.

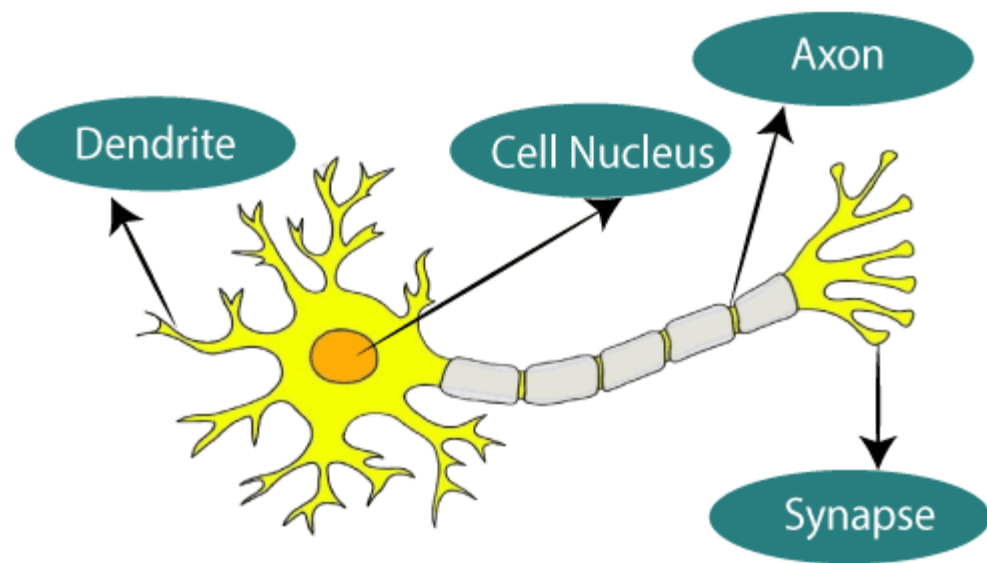
Artificial Neural Network



- The term "**Artificial Neural Network**" is derived from Biological neural networks that develop the structure of a human brain.
- Similar to the human brain that has neurons interconnected to one another, artificial neural networks also have neurons that are interconnected to one another in various layers of the networks.
- These neurons are known as nodes.

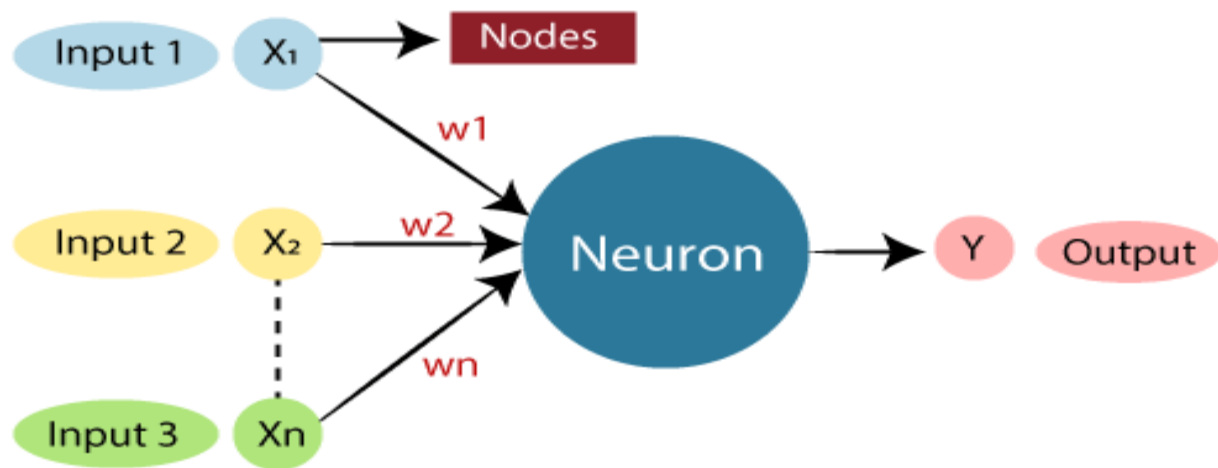
Artificial Neural Network

- A neural network is a very powerful machine learning mechanism which basically mimics how a human brain learns.
- The brain receives the stimulus from the outside world, does the processing on the input, and then generates the output.
- The task gets complicated, multiple neurons form a complex network, passing information among themselves.
- An Artificial Neural Network tries to mimic a similar behavior.
- The network you see below is a neural network made of interconnected neurons.
- Each neuron is characterized by its weight, bias and activation function.
- The input is fed to the input layer, the neurons perform a linear transformation on this input using the weights and biases.



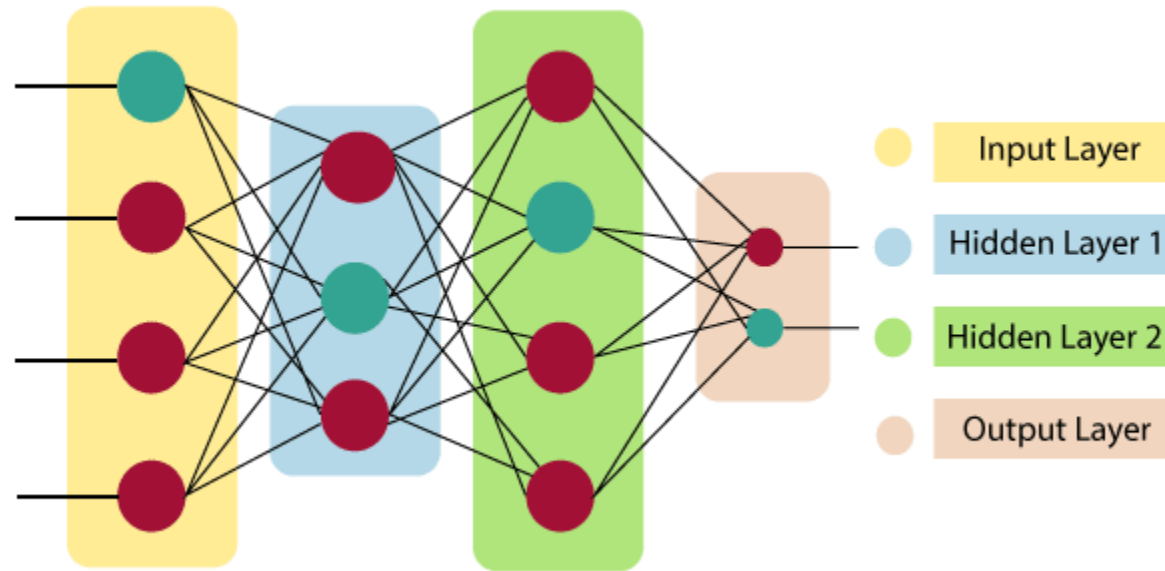
Biological Neural Network

The typical Artificial Neural Network looks something like the given figure below



Biological Neural Network	Artificial Neural Network
Dendrites	Inputs
Cell nucleus	Nodes
Synapse	Weights
Axon	Output

The architecture of an artificial neural network:



Elements of a Neural Network

- **Input Layer:**

- This layer accepts input features.
- It provides information from the outside world to the network, no computation is performed at this layer, nodes here just pass on the information(features) to the hidden layer.

- **Hidden Layer:**

- Nodes of this layer are not exposed to the outer world, they are part of the abstraction provided by any neural network.
- The hidden layer performs all sorts of computation on the features entered through the input layer and transfers the result to the output layer.

- **Output Layer:**

- This layer bring up the information learned by the network to the outer world

What is Activation Function?

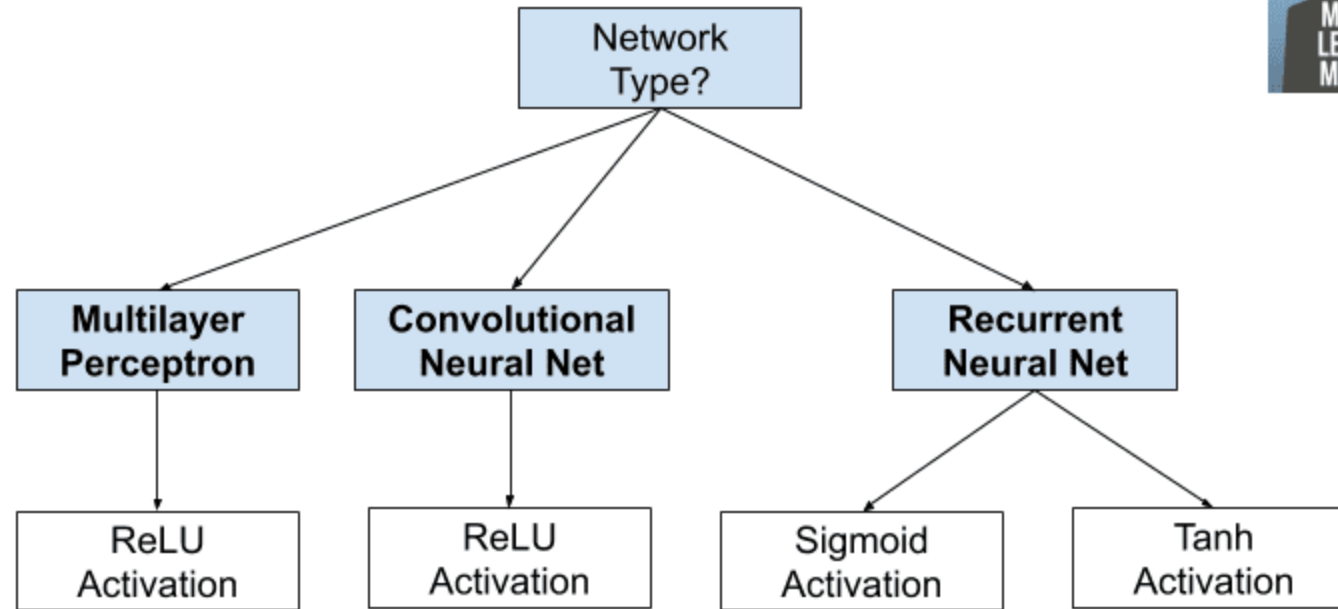
- It's just a thing function that you use to get the output of node. It is also known as Transfer Function.
- Why we use Activation functions with Neural Networks?
 - It is used to determine the output of neural network like yes or no.
 - It maps the resulting values in between 0 to 1 or -1 to 1 etc. (depending upon the function).
- The Activation Functions can be basically divided into 2 types-
 - Linear Activation Function
 - Non-linear Activation Functions

Forward /back propagation

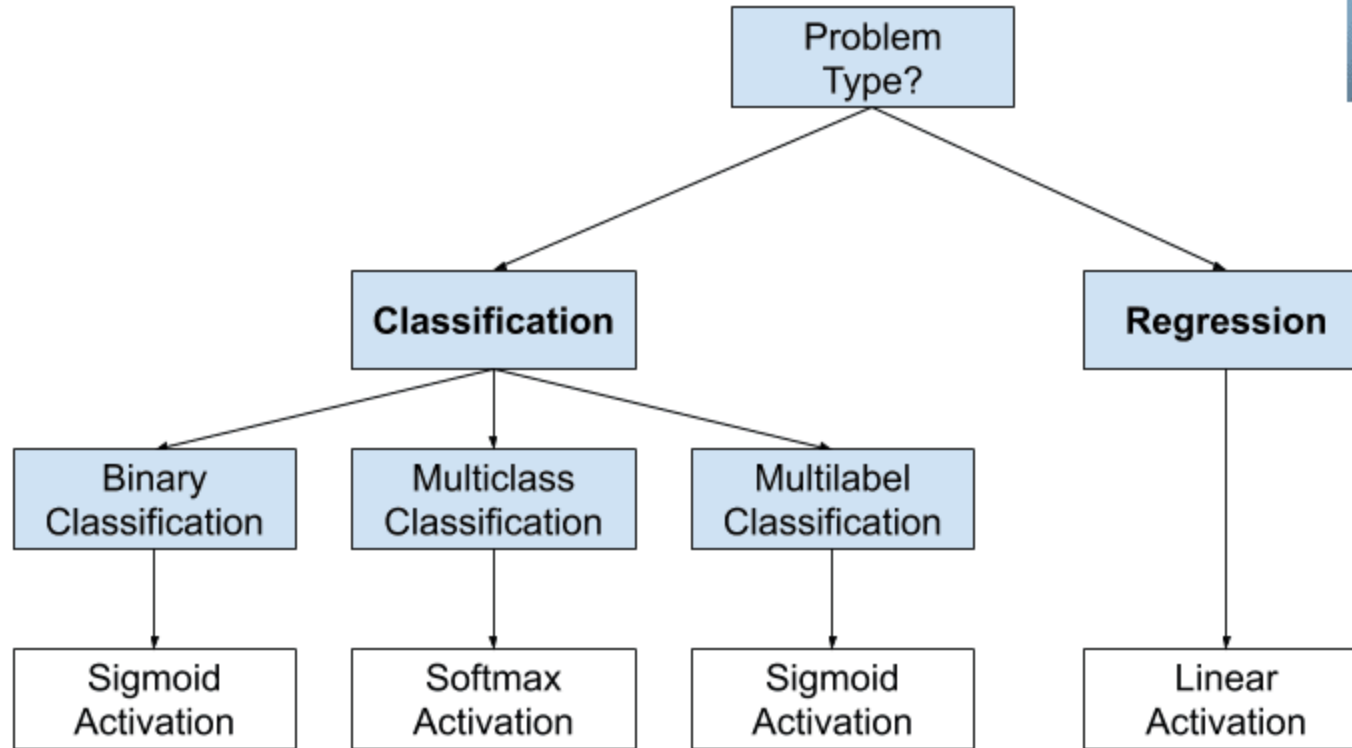
- The output from the activation function moves to the next hidden layer and the same process is repeated. This forward movement of information is known as the *forward propagation*.
- Using the output from the forward propagation, error is calculated.
- Based on this error value, the weights and biases of the neurons are updated. This process is known as *back-propagation*.
- $x = (\text{weight} * \text{input}) + \text{bias}$

$$Y = \text{Activation}(\Sigma(\text{weight} * \text{input}) + \text{bias})$$

How to Choose an Hidden Layer Activation Function



How to Choose an Output Layer Activation Function



Popular types of activation functions

1. Binary Step
2. Linear
3. Sigmoid
4. Tanh
5. ReLU
6. Leaky ReLU
7. Parameterised ReLU
8. Exponential Linear Unit
9. Swish
10. Softmax

<i>Function Type</i>	<i>Equation</i>	<i>Derivative</i>
Linear	$f(x) = ax + c$	$f'(x) = a$
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x) (1 - f(x))$
TanH	$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$
ReLU	$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
Parametric ReLU	$f(x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$
ELU	$f(x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} f(x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$

Binary Step Function

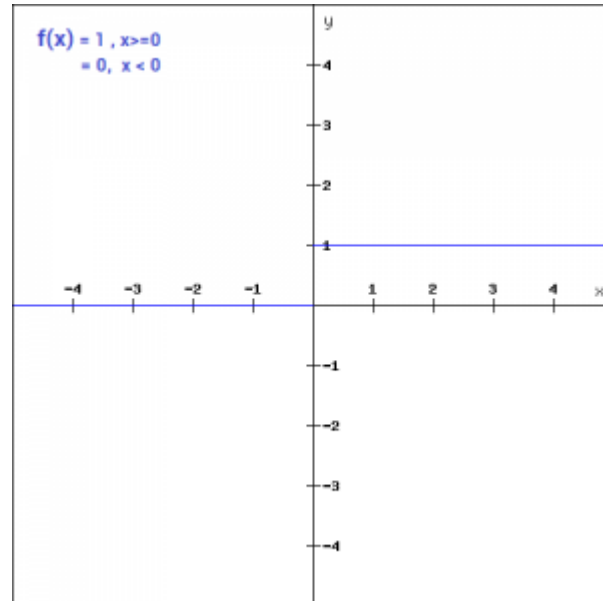
- The first thing that comes to our mind when we have an activation function would be a threshold based classifier i.e. whether or not the neuron should be activated based on the value from the linear transformation.
- In other words, if the input to the activation function is greater than a threshold, then the neuron is activated, else it is deactivated, i.e. its output is not considered for the next hidden layer. Let us look at it mathematically
- $f(x) = 1, x \geq 0$
- $= 0, x < 0$

Binary Step Function

- $f(x) = 1, x \geq 0$
- $= 0, x < 0$
- `def binary_step(x):`
- `if x < 0:`
- `return 0`
- `else:`
- `return 1`
- I/P : `binary_step(5), binary_step(-1)`

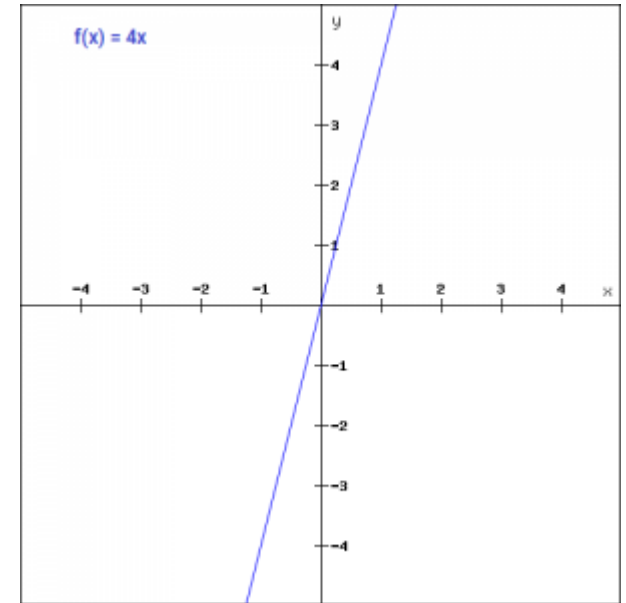
- **Output:**

- (5,0)



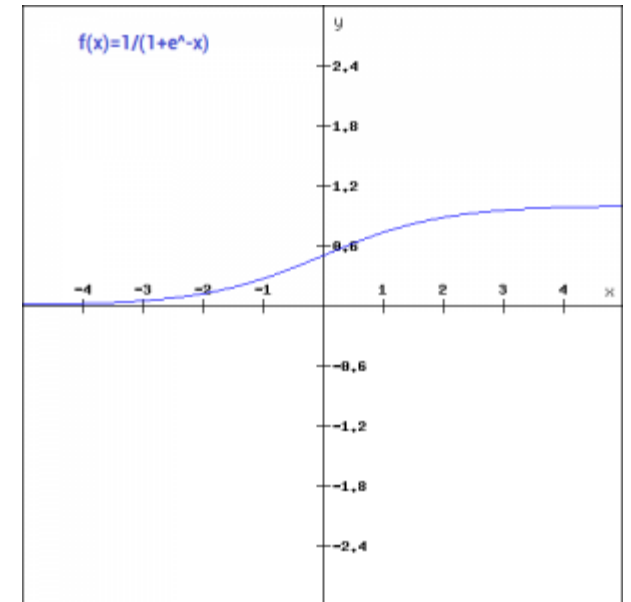
Linear Function

- **Equation :**
- Linear function has the equation similar to as of a straight line i.e. $y = x$
- No matter how many layers we have, if all are linear in nature, the final activation function of last layer is nothing but just a linear function of the input of first layer.
- **Range :** $-\infty$ to $+\infty$
- **Uses : Linear activation function** is used at just one place i.e. output layer.
- **Issues :** If we will differentiate linear function to bring non-linearity, result will no more depend on *input* “ x ” and function will become constant, it won’t introduce any ground-breaking behavior to our algorithm.



Sigmoid

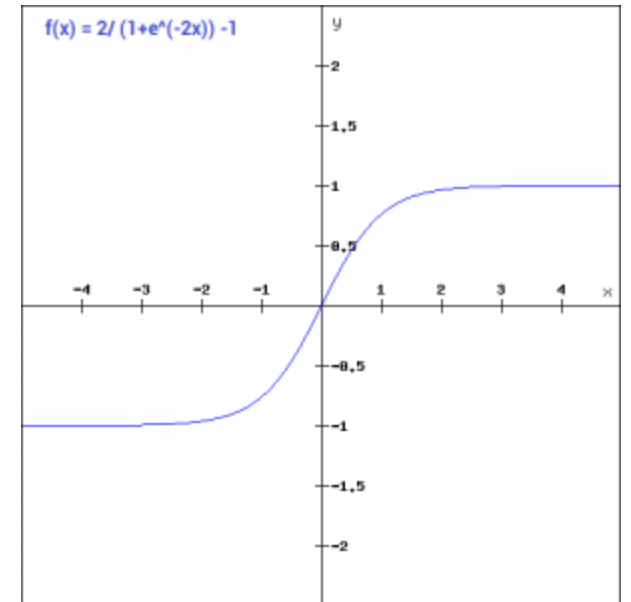
- It is a function which is plotted as ‘S’ shaped graph.
- **Equation** : $A = 1/(1 + e^{-x})$
- **Nature** : Non-linear. Notice that X values lies between -2 to 2, Y values are very steep. This means, small changes in x would also bring about large changes in the value of Y.
- **Value Range** : 0 to 1
- **Uses** : Usually used in output layer of a binary classification, where result is either 0 or 1, as value for sigmoid function lies between 0 and 1 only so, result can be predicted easily to be **1** if value is greater than **0.5** and **0** otherwise.



Tanh Function

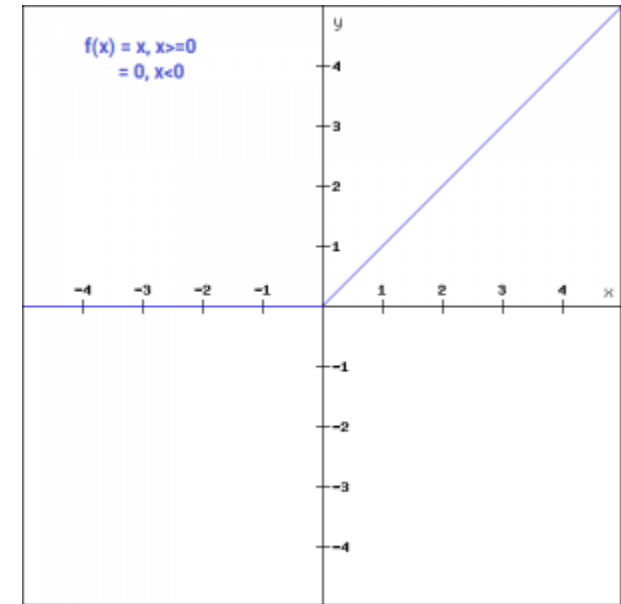
- The activation that works almost always better than sigmoid function is Tanh function also known as **Tangent Hyperbolic function**. It's actually mathematically shifted version of the sigmoid function. Both are similar and can be derived from each other.
- **Value Range :-** -1 to +1
- **Nature :-** non-linear
- **Uses :-** Usually used in hidden layers of a neural network as its values lie between **-1 to 1** hence the mean for the hidden layer comes out to be 0 or very close to it, hence helps in *centering the data* by bringing mean close to 0. This makes learning for the next layer much easier.

- $$f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$$



ReLU

- It Stands for Rectified linear unit. It is the most widely used activation function. Chiefly implemented in hidden layers of Neural network.
- **Equation** :- $A(x) = \max(0, x)$. It gives an output x if x is positive and 0 otherwise.
- **Value Range** :- $[0 \text{ to } \infty]$
- **Nature** :- non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
- **Uses** :- ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.
- In simple words, RELU learns much faster than sigmoid and Tanh function.



Softmax Function

- The softmax function is also a type of sigmoid function but is handy when we are trying to handle multi- class classification problems.
- **Nature :-** non-linear
- **Uses :-** Usually used when trying to handle multiple classes. the softmax function was commonly found in the output layer of image classification problems. The softmax function would squeeze the outputs for each class between 0 and 1 and would also divide by the sum of the outputs.
- **Output:-** The softmax function is ideally used in the output layer of the classifier where we are actually trying to attain the probabilities to define the class of each input.
- The basic rule of thumb is if you really don't know what activation function to use, then simply use RELU as it is a general activation function in hidden layers and is used in most cases these days.
- If your output is for binary classification then, sigmoid function is very natural choice for output layer.
- If your output is for multi-class classification then, Softmax is very useful to predict the probabilities of each classes.

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

How to choose the right Activation Function?

1. ReLU activation function should only be used in the hidden layers.
2. Sigmoid/Logistic and Tanh functions should not be used in hidden layers as they make the model more susceptible to problems during training (due to vanishing gradients).
3. Swish function is used in neural networks having a depth greater than 40 layers.

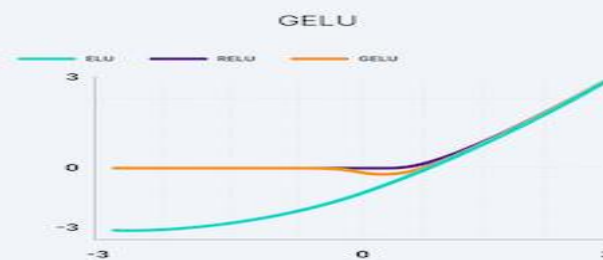
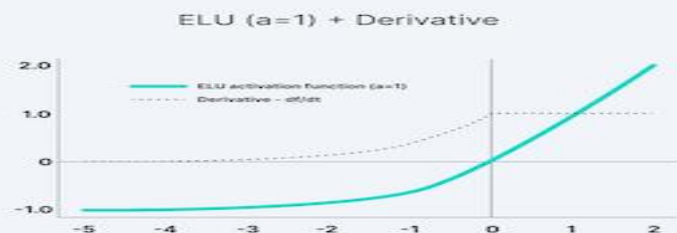
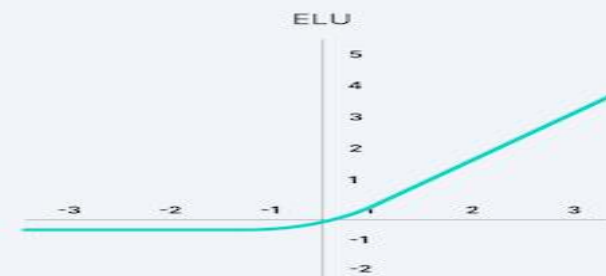
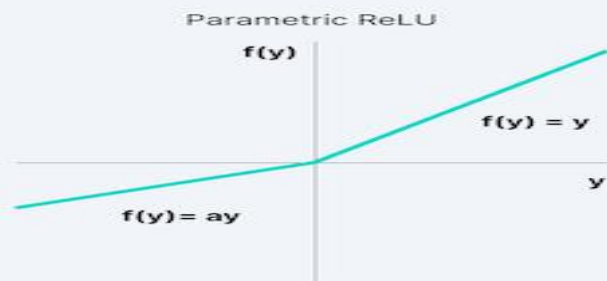
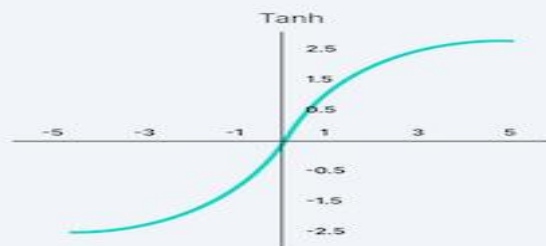
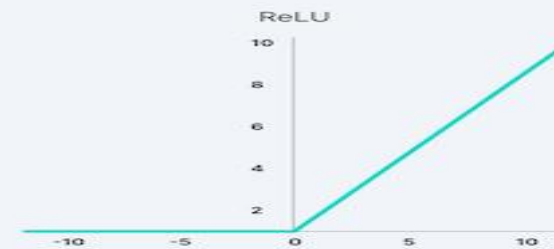
Few rules for choosing the activation function for the output layer

- 1. Regression** - Linear Activation Function
- 2. Binary Classification**—Sigmoid/Logistic Activation Function
- 3. Multiclass Classification**—Softmax
- 4. Multilabel Classification**—Sigmoid

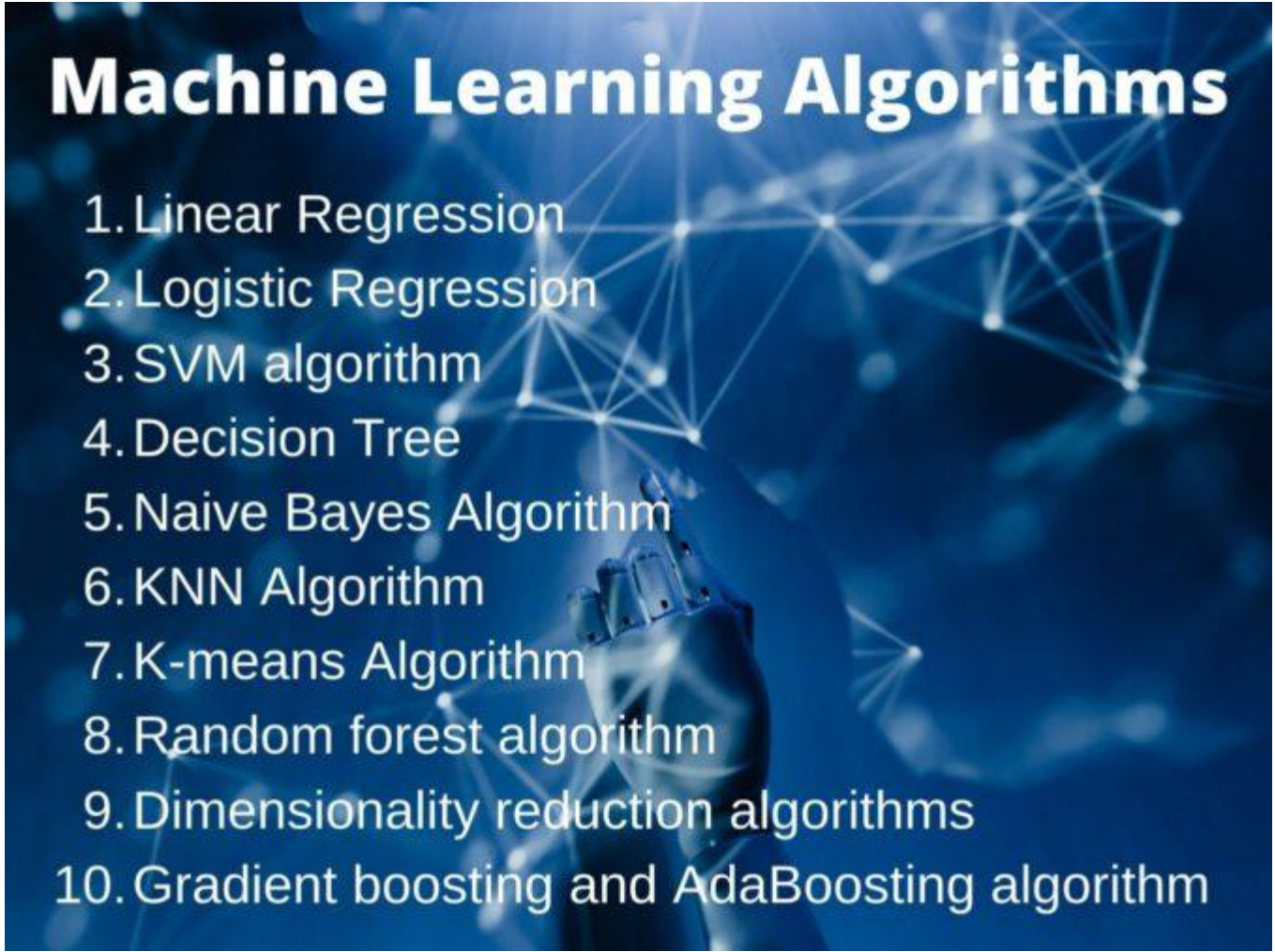
The activation function used in hidden layers

- Convolutional Neural Network (CNN): ReLU activation function.
- Recurrent Neural Network: Tanh and/or Sigmoid activation function.

Neural Network Activation Functions



Machine Learning Algorithms



1. Linear Regression
2. Logistic Regression
3. SVM algorithm
4. Decision Tree
5. Naive Bayes Algorithm
6. KNN Algorithm
7. K-means Algorithm
8. Random forest algorithm
9. Dimensionality reduction algorithms
10. Gradient boosting and AdaBoosting algorithm

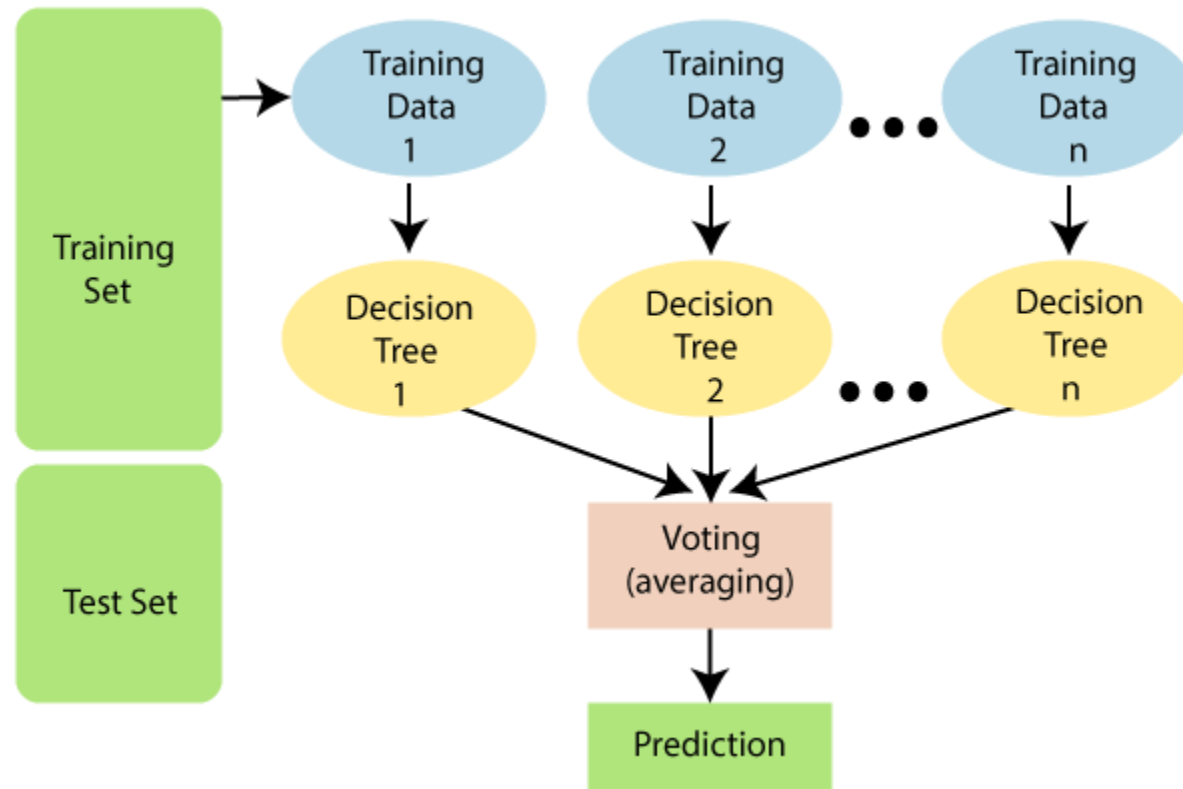
Random Forest Algorithm

- Random Forest is a popular machine learning algorithm that belongs to the supervised learning technique.
- It can be used for both Classification and Regression problems in ML.
- It is based on the concept of **ensemble learning**, which is a process of *combining multiple classifiers to solve a complex problem and to improve the performance of the model*.

Random Forest Algorithm

- Random Forest is a classifier that contains a number of decision trees on various subsets of the given dataset and takes the average to improve the predictive accuracy of that dataset.
- Instead of relying on one decision tree, the random forest takes the prediction from each tree and based on the majority votes of predictions, and it predicts the final output.
- The greater number of trees in the forest leads to higher accuracy and prevents the problem of overfitting.

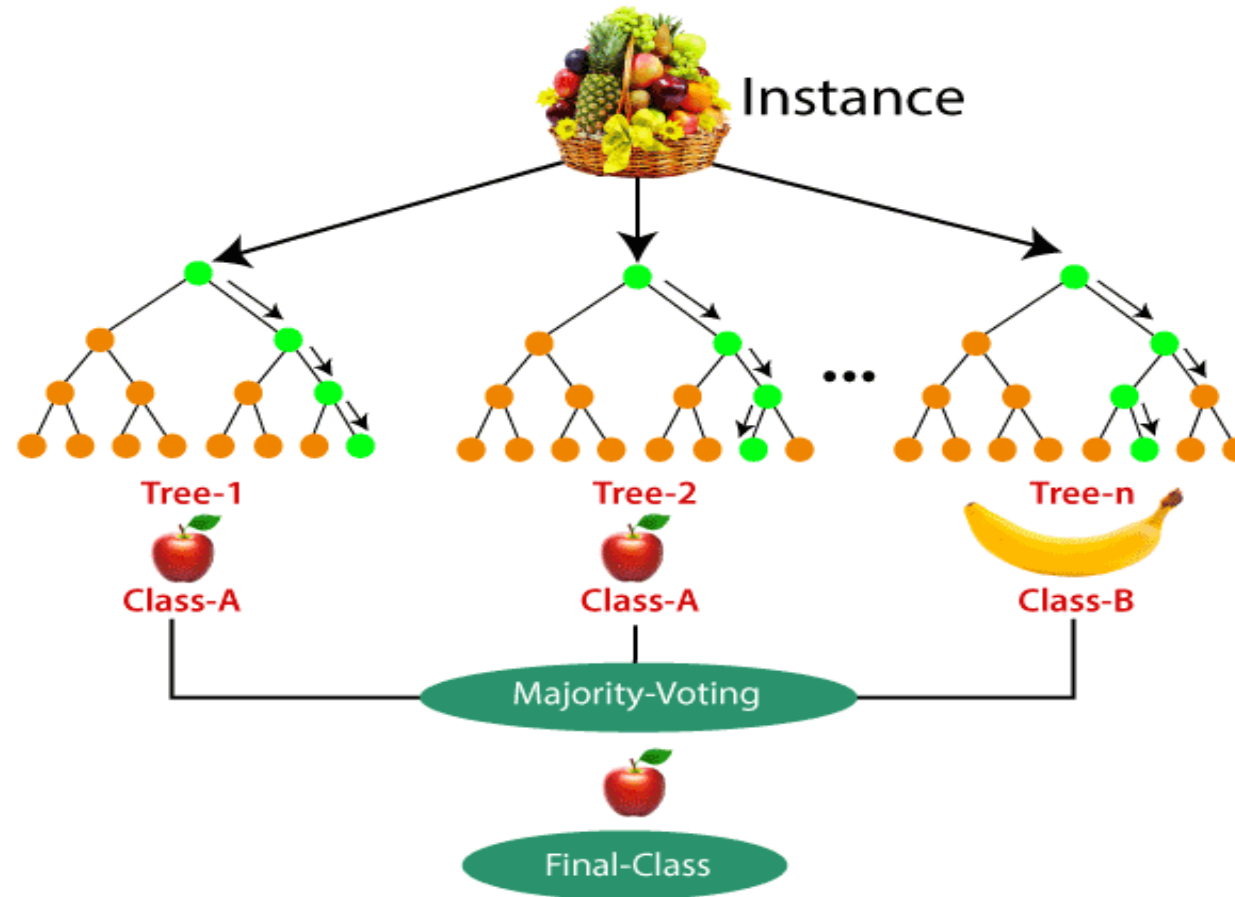
Random Forest algorithm



How does Random Forest algorithm work?

- **Step-1:** Select random K data points from the training set.
- **Step-2:** Build the decision trees associated with the selected data points (Subsets).
- **Step-3:** Choose the number N for decision trees that you want to build.
- **Step-4:** Repeat Step 1 & 2.
- **Step-5:** For new data points, find the predictions of each decision tree, and assign the new data points to the category that wins the majority votes.

Random Forest algorithm



Why use Random Forest?

- It takes less training time as compared to other algorithms.
- It predicts output with high accuracy, even for the large dataset it runs efficiently.
- It can also maintain accuracy when a large proportion of data is missing.

Applications of Random Forest

- 1.Banking:** Banking sector mostly uses this algorithm for the identification of loan risk.
- 2.Medicine:** With the help of this algorithm, disease trends and risks of the disease can be identified.
- 3.Land Use:** We can identify the areas of similar land use by this algorithm.
- 4.Marketing:** Marketing trends can be identified using this algorithm.

Advantages of Random Forest

- Random Forest is capable of performing both Classification and Regression tasks.
- It is capable of handling large datasets with high dimensionality.
- It enhances the accuracy of the model and prevents the overfitting issue.

Disadvantages of Random Forest

- While using a Random Forest Algorithm, more resources are required for computation.
- Consumes more time compared to the decision tree algorithm.
- Less intuitive when we have an extensive collection of decision trees.
- Extremely complex and requires more computational resources.

Confusion Matrix in Machine Learning

- The confusion matrix is a matrix used to determine the performance of the classification models for a given set of test data. It can only be determined if the true values for test data are known. The matrix itself can be easily understood, but the related terminologies may be confusing. Since it shows the errors in the model performance in the form of a matrix, hence also known as an **error matrix**.

- For the 2 prediction classes of classifiers, the matrix is of 2*2 table, for 3 classes, it is 3*3 table, and so on.
- The matrix is divided into two dimensions, that are **predicted values** and **actual values** along with the total number of predictions.
- Predicted values are those values, which are predicted by the model, and actual values are the true values for the given observations.

n = total predictions	Actual: No	Actual: Yes
Predicted: No	True Negative	False Positive
Predicted: Yes	False Negative	True Positive

- True Negative:** Model has given prediction No, and the real or actual value was also No.
- True Positive:** The model has predicted yes, and the actual value was also true.
- False Negative:** The model has predicted no, but the actual value was Yes, it is also called as **Type-II error**.
- False Positive:** The model has predicted Yes, but the actual value was No. It is also called a **Type-I error**.

Need for Confusion Matrix in Machine learning

- It evaluates the performance of the classification models, when they make predictions on test data, and tells how good our classification model is.
- It not only tells the error made by the classifiers but also the type of errors such as it is either type-I or type-II error.
- With the help of the confusion matrix, we can calculate the different parameters for the model, such as accuracy, precision, etc.

Confusion Matrix

n = 100	Actual: No	Actual: Yes	
Predicted: No	TN: 65	FP: 3	68
Predicted: Yes	FN: 8	TP: 24	32
	73	27	

- The table is given for the two-class classifier, which has two predictions "Yes" and "NO." Here, Yes defines that patient has the disease, and No defines that patient does not has that disease.
- The classifier has made a total of **100 predictions**. Out of 100 predictions, **89 are true predictions**, and **11 are incorrect predictions**.
- The model has given prediction "yes" for 32 times, and "No" for 68 times. Whereas the actual "Yes" was 27, and actual "No" was 73 times.

Calculations using Confusion Matrix:

- Classification Accuracy
- Misclassification rate
- Precision
- Recall
- F-measure

Classification Accuracy

- It is one of the important parameters to determine the accuracy of the classification problems.
- It defines how often the model predicts the correct output.
- It can be calculated as the ratio of the number of correct predictions made by the classifier to all number of predictions made by the classifiers.

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$$

Misclassification rate

- It is also termed as Error rate, and it defines how often the model gives the wrong predictions.
- The value of error rate can be calculated as the number of incorrect predictions to all number of the predictions made by the classifier.

$$\text{Error rate} = \frac{FP + FN}{TP + FP + FN + TN}$$

- **Precision:** It can be defined as the number of correct outputs provided by the model or out of all positive classes that have predicted correctly by the model, how many of them were actually true.

$$\text{Precision} = \frac{TP}{TP+FP}$$

- **Recall:** It is defined as the out of total positive classes, how our model predicted correctly.
- The recall must be as high as possible.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F-measure:** If two models have low precision and high recall or vice versa, it is difficult to compare these models. So, for this purpose, we can use F-score.
- This score helps us to evaluate the recall and precision at the same time.
- The F-score is maximum if the recall is equal to the precision.

$$\text{F-measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}}$$

Fundamental Deep Learning Methods

- **Artificial Neural Networks (ANN):** Perceptron, learning laws, layers, back propagation - **The scope of learning, popular architectures, An overview of Parallel and Distributed Processes (PDP), linear associative models, and stochastic networks;** - Convolutional Neural Networks (CNN): convolution, filters, pooling, stride, drop out, layers and applications; - Recurrent Neural Networks (RNN): unfolding, Backpropagation Through Time (BPTT), LSTM models, bidirectional networks, encoder, decoder and attention models.

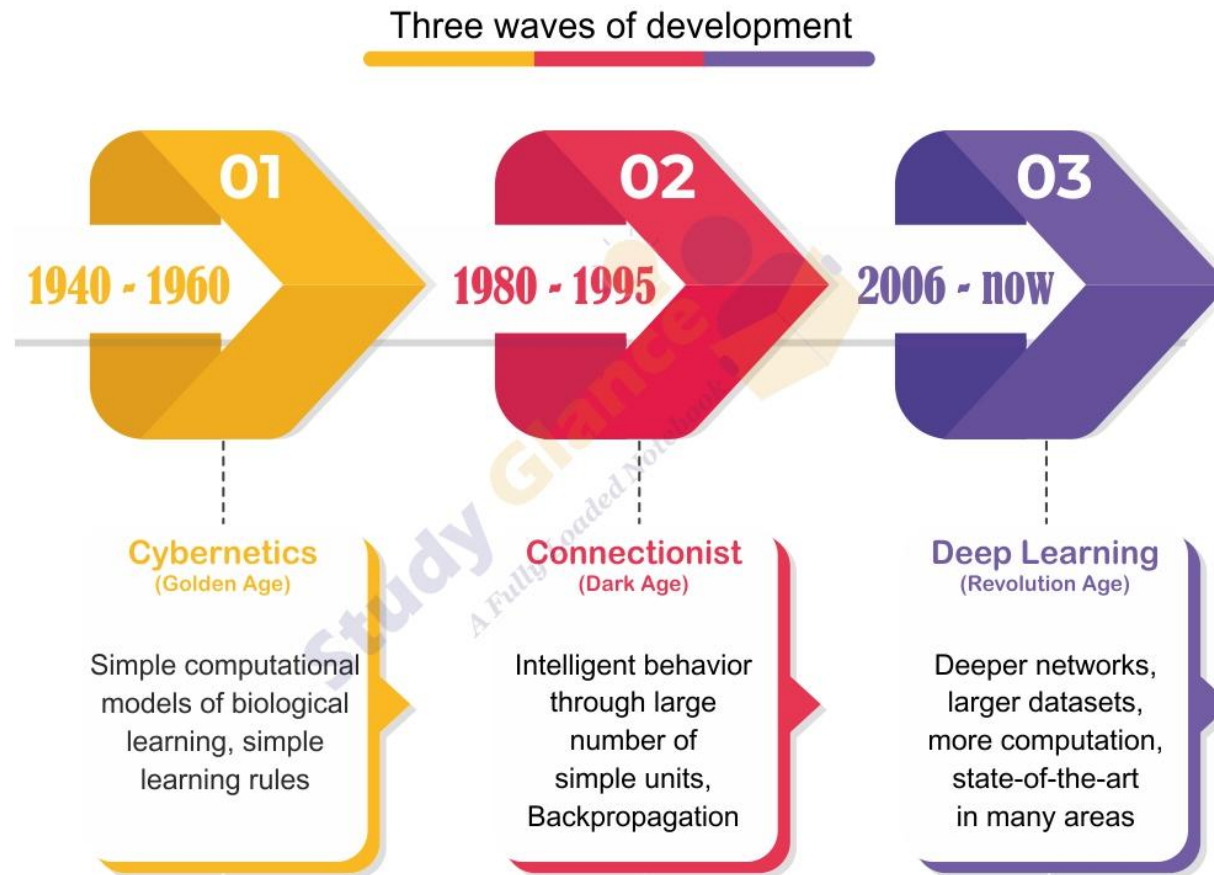
Artificial Neural Network (ANN)

- Artificial Neural Network (ANN) is a deep learning algorithm that emerged and evolved from the idea of Biological Neural Networks of human brains.
- An attempt to simulate the workings of the human brain culminated in the emergence of ANN.
- ANN works very similar to the biological neural networks but doesn't exactly resemble its workings.

Artificial Neural Network (ANN)

- ANN algorithm would accept only numeric and structured data as input.
- To accept unstructured and non-numeric data formats such as Image, Text, and Speech, **Convolutional Neural Networks (CNN)**, and **Recursive Neural Networks (RNN)** are used respectively.

The history of ANN can be divided into the following three eras



Comparison between ANN and BNN based on some criteria mentioned

Criteria	BNN	ANN
Processing	Massively parallel, slow but superior than ANN	Massively parallel, fast but inferior than BNN
Size	10^{11} neurons and 10^{15} interconnections	10^2 to 10^4 nodes (mainly depends on the type of application and network designer)
Learning	They can tolerate ambiguity	Very precise, structured and formatted data is required to tolerate ambiguity
Fault tolerance	Performance degrades with even partial damage	It is capable of robust performance, hence has the potential to be fault tolerant
Storage capacity	Stores the information in the synapse	Stores the information in continuous memory locations

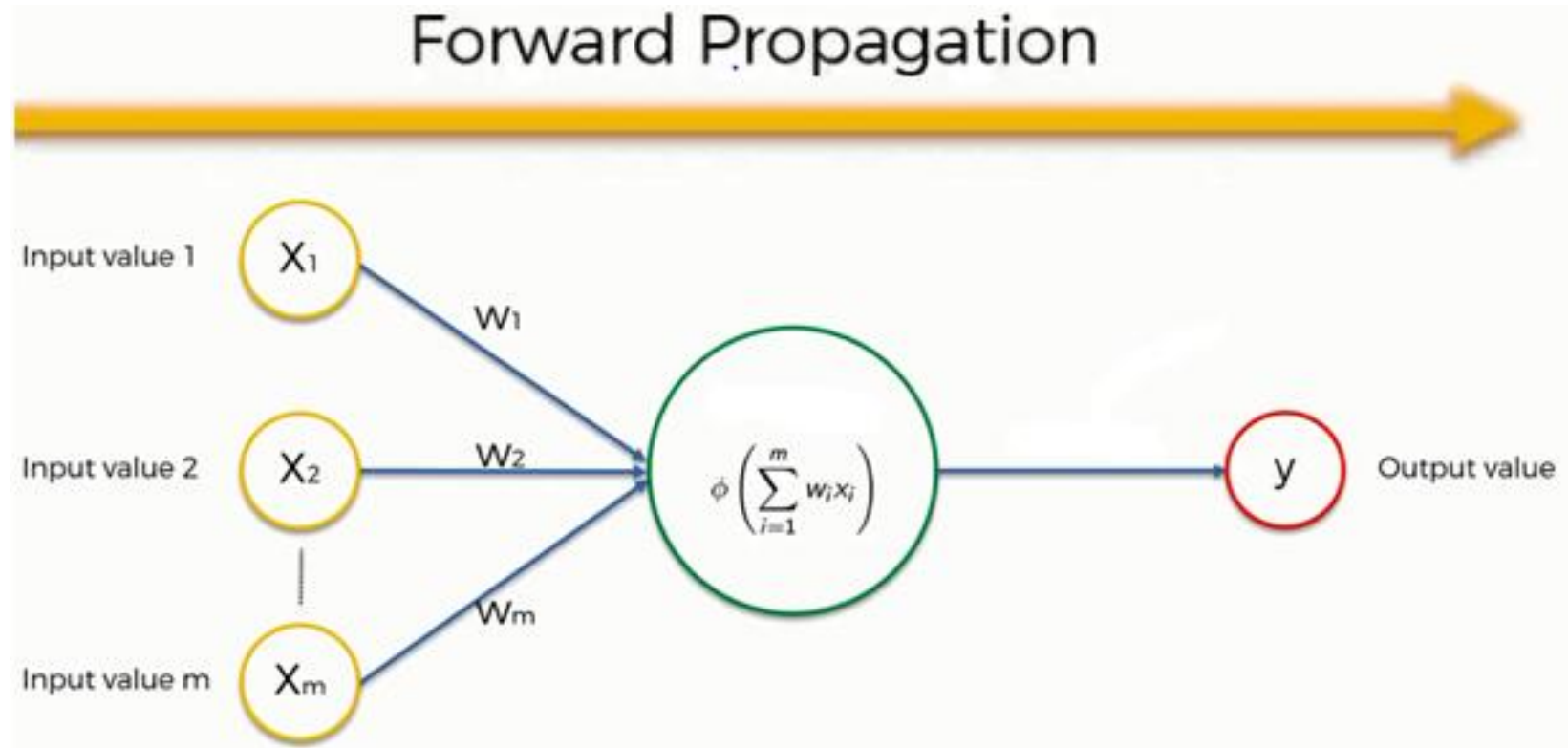
Structure of Artificial neurons and their functions

- A neural network with a single layer is called a perceptron.
- A multi-layer perceptron is called Artificial Neural Networks.
- A Neural network can possess any number of layers.
- Each layer can have one or more neurons or units.
- Each of the neurons is interconnected with each and every other neuron.
- Each layer could have different activation functions as well.

Structure of Artificial neurons and their functions

- ANN consists of two phases Forward propagation and Backpropagation.
- The forward propagation involves multiplying weights, adding bias, and applying activation function to the inputs and propagating it forward.

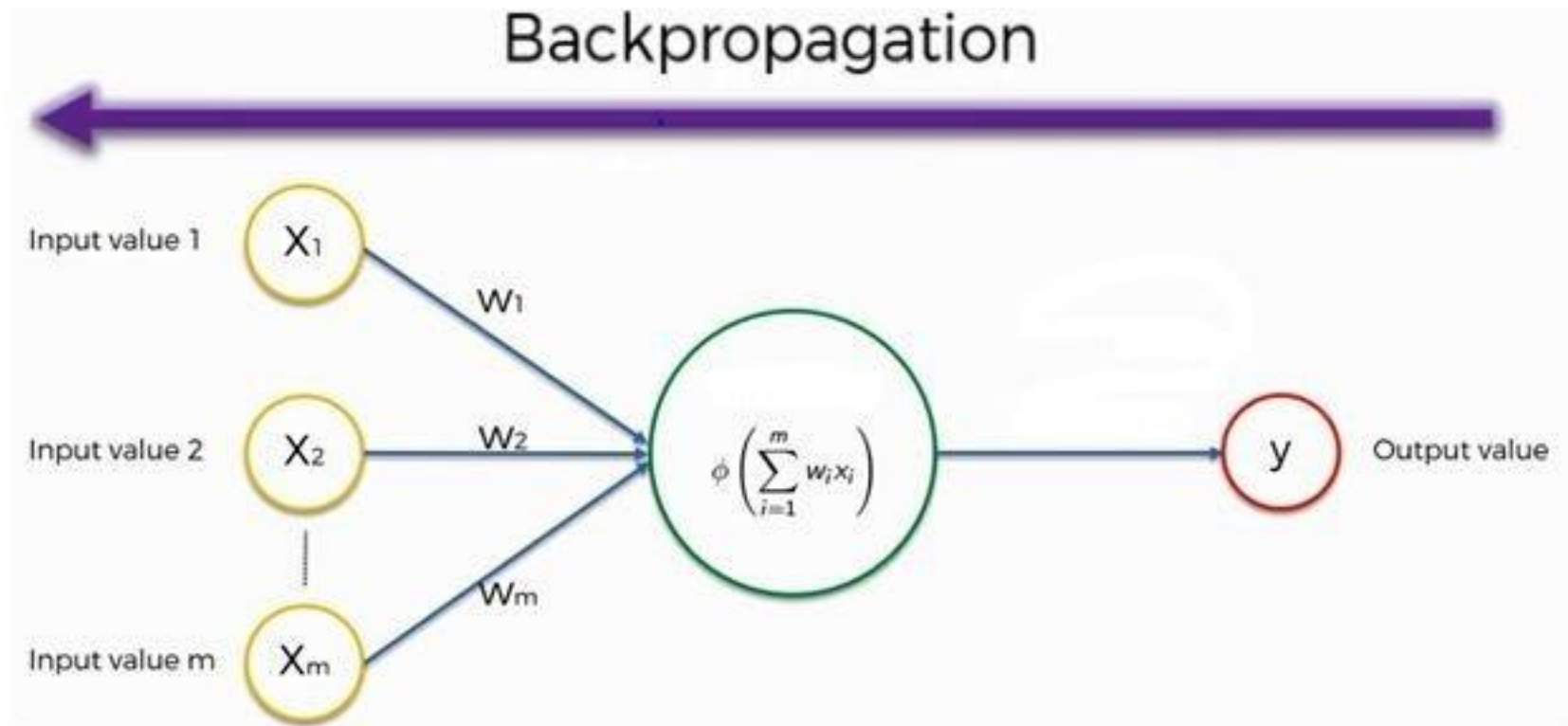
Forward propagation



Back Propagation

- The backpropagation step is the most important step which usually involves finding optimal parameters for the model by propagating in the backward direction of the Neural network layers.
- The backpropagation requires **optimization function** to find the optimal weights for the model.

Back Propagation



Perceptrons

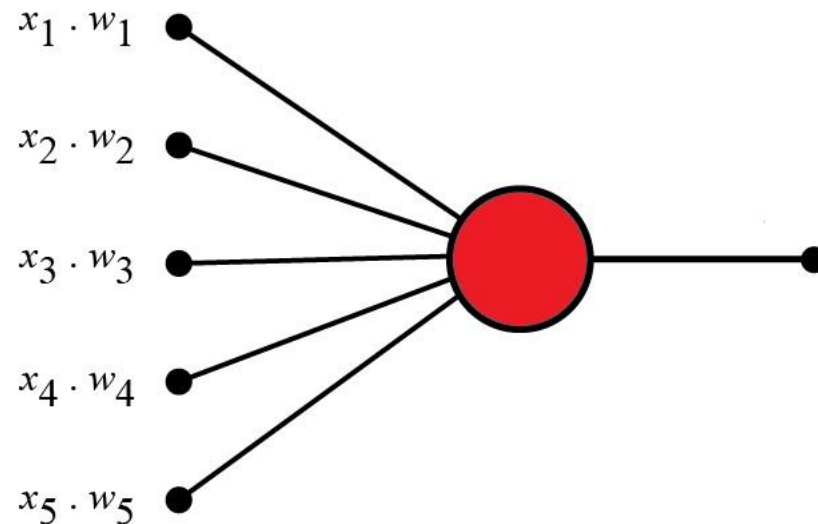
- A **Perceptron** is an **Artificial Neuron**
- It is the simplest possible **Neural Network**
- **Neural Networks** are the building blocks of **Machine Learning**.

Frank Rosenblatt

- **Frank Rosenblatt** (1928 – 1971) was an American psychologist notable in the field of Artificial Intelligence.
- In **1957** he started something really big. He "invented" a **Perceptron** program, on an IBM 704 computer at Cornell Aeronautical Laboratory.
- Scientists had discovered that brain cells (**Neurons**) receive input from our senses by electrical signals.
- The Neurons, then again, use electrical signals to store information, and to make decisions based on previous input.
- Frank had the idea that **Perceptrons** could simulate brain principles, with the ability to learn and make decisions.

The Perceptron

- The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).
- The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **yes** or **no** (true or false) (0 or 1).



Structure of Artificial neurons and their functions

- ANN can be applied to both **Regression and Classification tasks** by changing the activation functions of the output layers accordingly. (Sigmoid activation function for binary classification, Softmax activation function for multi-class classification and Linear activation function for Regression).

Terminologies

- **Metrics**
- A metric is used to gauge the performance of the model.
- Metric functions are similar to cost functions, except that the results from evaluating a metric are not used when training the model. Note that you may use any cost function as a metric.
- Have used Mean Squared Logarithmic Error as a metric and cost function.

Metrics

$$MSLE = \frac{1}{n} \sum_{i=1}^n ((\log y_i + 1) - (\log \hat{y} + 1))^2$$

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n ((\log y_i + 1) - (\log \hat{y} + 1))^2}$$

Mean Squared Logarithmic Error (MSLE) and Root Mean Squared Logarithmic Error(RMSLE)

Epoch

- A single pass through the training data is called an epoch.
- The training data is fed to the model in mini-batches and when all the mini-batches of the training data are fed to the model that constitutes an epoch.
-

Hyperparameters

- Hyperparameters are the **tunable parameters** that are not produced by a model which means the users must provide a value for these parameters.
- The values of hyperparameters that we provide affect the training process so hyperparameter optimization comes to the rescue.

The Hyperparameters used in this ANN model are

- Number of layers
- Number of units/ neurons in a layer
- Activation function
- Initialization of weights
- Loss function
- Metric
- Optimizer
- Number of epochs

Deep learning continues to push the field forward in many domains and on many core machine learning problems

- Text-to-speech synthesis (Fan et al., Microsoft, Interspeech 2014)
- Language identification (Gonzalez-Dominguez et al., Google, Interspeech 2014)
- Large vocabulary speech recognition (Sak et al., Google, Interspeech 2014)
- Prosody contour prediction (Fernandez et al., IBM, Interspeech 2014)
- Medium vocabulary speech recognition (Geiger et al., Interspeech 2014)
- English-to-French translation (Sutskever et al., Google, NIPS 2014)
- Audio onset detection (Marchi et al., ICASSP 2014)
- Social signal classification (Brueckner & Schuler, ICASSP 2014)

Deep learning continues to push the field forward in many domains and on many core machine learning problems

- Arabic handwriting recognition (Bluche et al., DAS 2014)
- TIMIT phoneme recognition (Graves et al., ICASSP 2013)
- Optical character recognition (Breuel et al., ICDAR 2013)
- Image caption generation (Vinyals et al., Google, 2014)
- Video-to-textual description (Donahue et al., 2014)
- Syntactic parsing for natural language processing (Vinyals et al., Google, 2014)
- Photo-real talking heads (Soong and Wang, Microsoft, 2014)

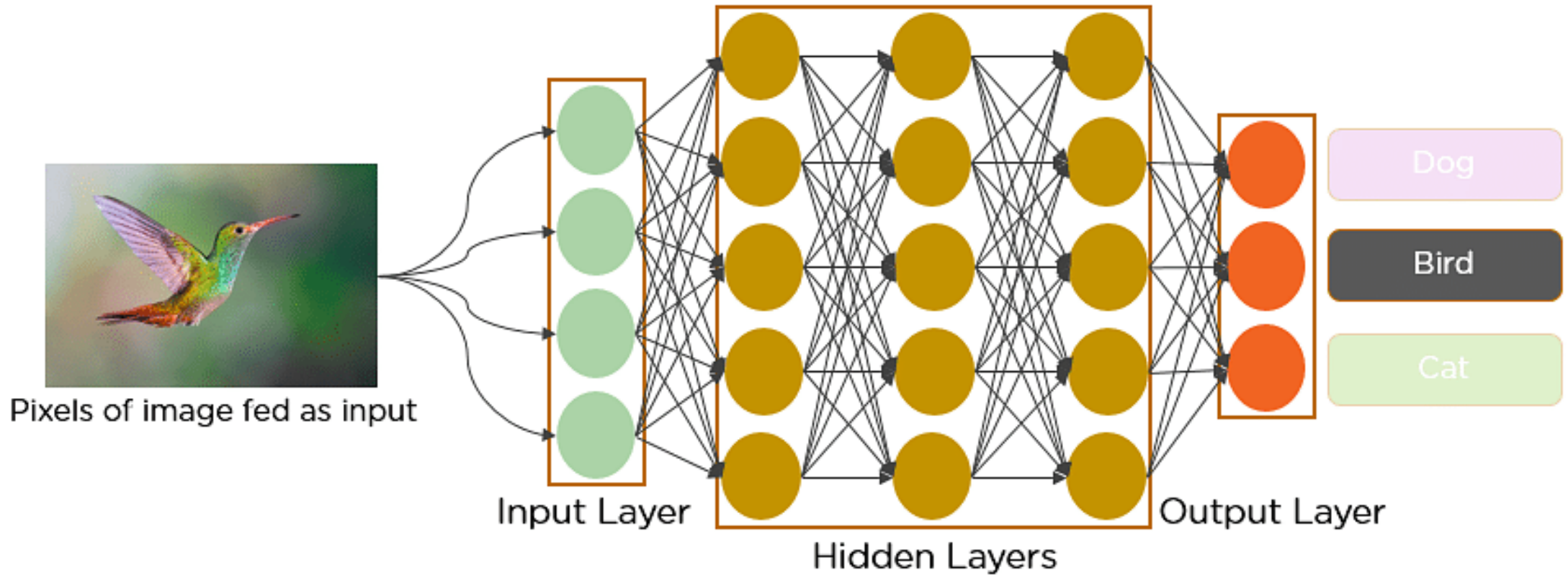
Convolutional Neural Networks

- **Yann LeCun**, director of Facebook's AI Research Group, is the pioneer of convolutional neural networks.
- He built the first convolutional neural network called LeNet in 1988.
- LeNet was used for character recognition tasks like reading zip codes and digits.
- Have you ever wondered how facial recognition works on social media, or how object detection helps in building self-driving cars, or how disease detection is done using visual imagery in healthcare? It's all possible thanks to convolutional neural networks (CNN).

Example of convolutional neural networks

- Imagine there's an image of a bird, and you want to identify whether it's really a bird or some other object.
- The first thing you do is feed the pixels of the image in the form of arrays to the input layer of the [neural network](#) (multi-layer networks used to classify things).
- The hidden layers carry out feature extraction by performing different calculations and manipulations.
- There are **multiple hidden layers** like the convolution layer, **the ReLU layer, and pooling layer**, that perform feature extraction from the image.
- Finally, there's a fully connected layer that identifies the object in the image.

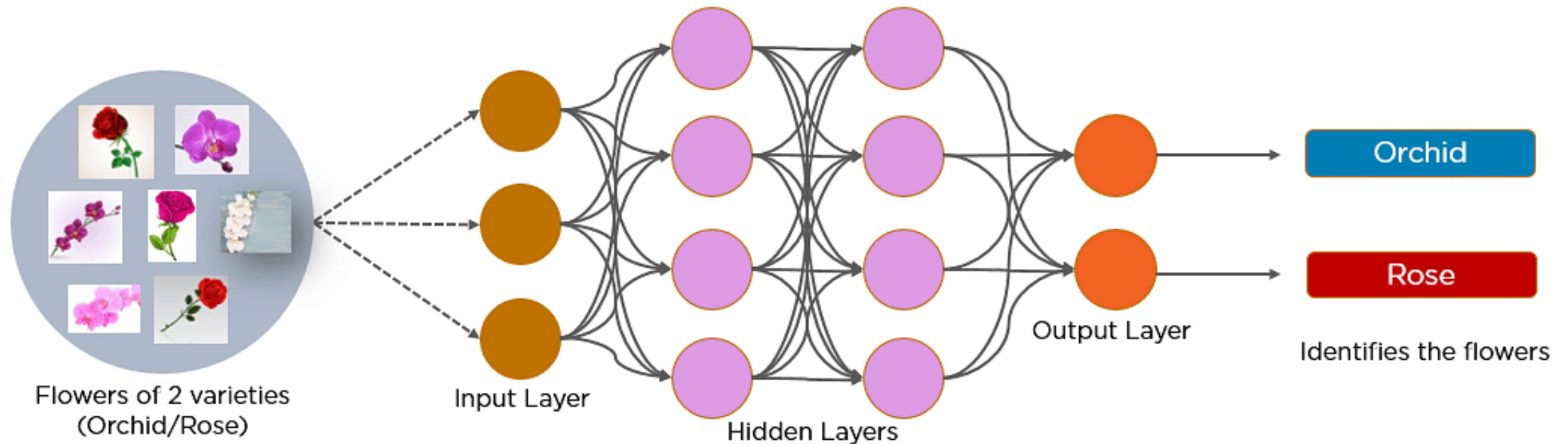
Convolutional Neural Network to identify the image of a bird



What is Convolutional Neural Network?

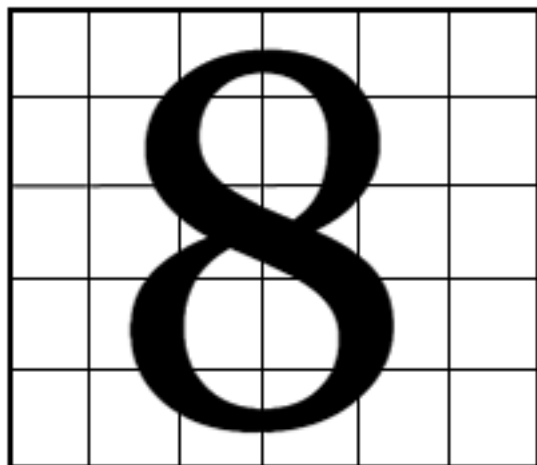
- A convolutional neural network is a feed-forward neural network that is generally used to analyze visual images by processing data with grid-like topology.
- It's also known as a ConvNet.
- A convolutional neural network is used to detect and classify objects in an image.

Below is a [neural network](#) that identifies two types of flowers:
Orchid and Rose





Real Image of the digit 8

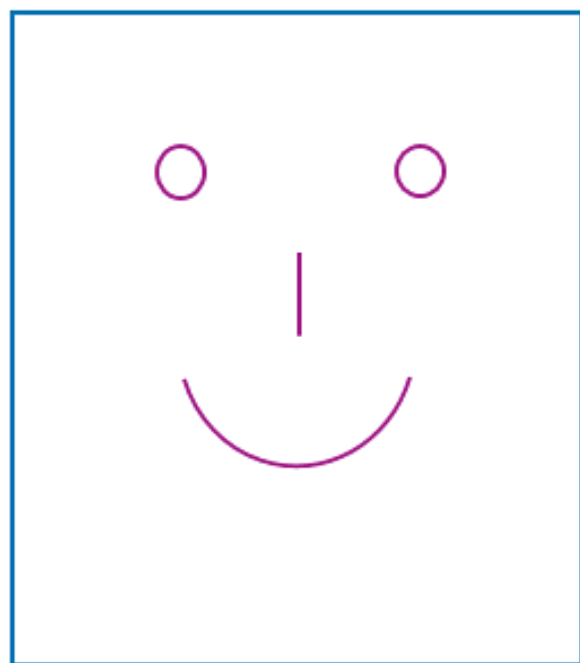


Represented in the form
of an array

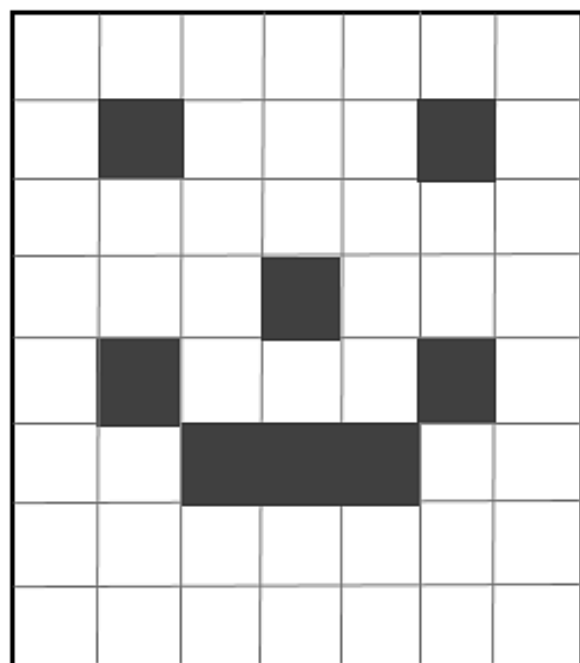


0	0	1	1	0	0
0	1	0	0	1	0
0	0	1	1	0	0
0	1	0	0	1	0
0	0	1	1	0	0

Digit 8 represented in the form
of pixels of 0's and 1's



Real Image



Represented in the form of
black and white pixels



0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0
0	0	0	0	0	0	0

Image represented in the
form of a matrix of numbers

Layers in a Convolutional Neural Network

- A convolution neural network has multiple hidden layers that help in extracting information from an image.
- The four important layers in CNN are:
 - 1.Convolution layer
 - 2.ReLU layer
 - 3.Pooling layer
 - 4.Fully connected layer

Convolution Layer

- This is the first step in the process of extracting valuable features from an image.
- A convolution layer has several filters that perform the convolution operation.
- Every image is considered as a matrix of pixel values.
- Consider the following 5x5 image whose pixel values are either 0 or 1.
- There's also a filter matrix with a dimension of 3x3.
- Slide the filter matrix over the image and compute the dot product to get the convolved feature matrix.

Image
pixels

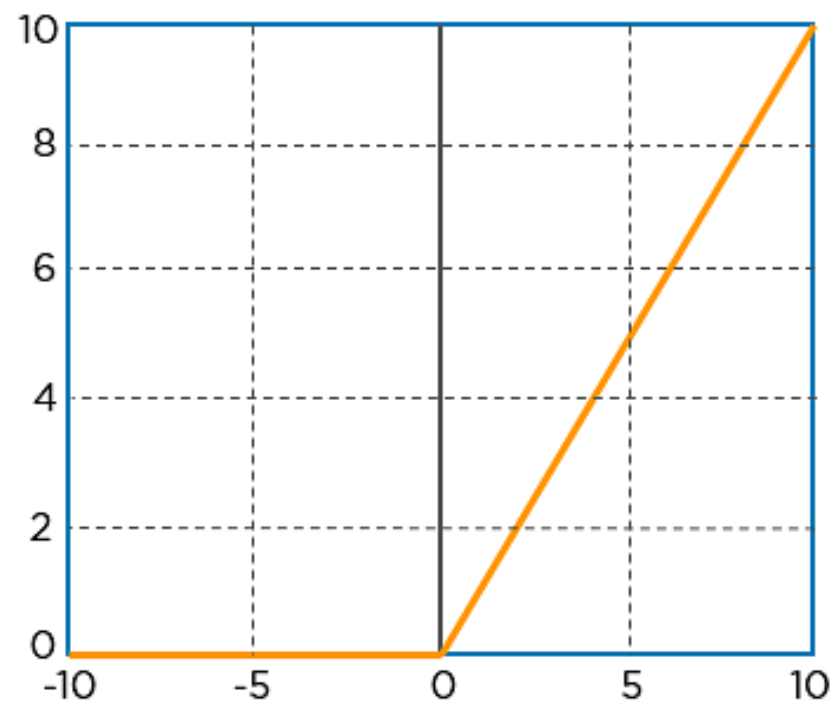
1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Filter

1	0	1
0	1	0
1	0	1

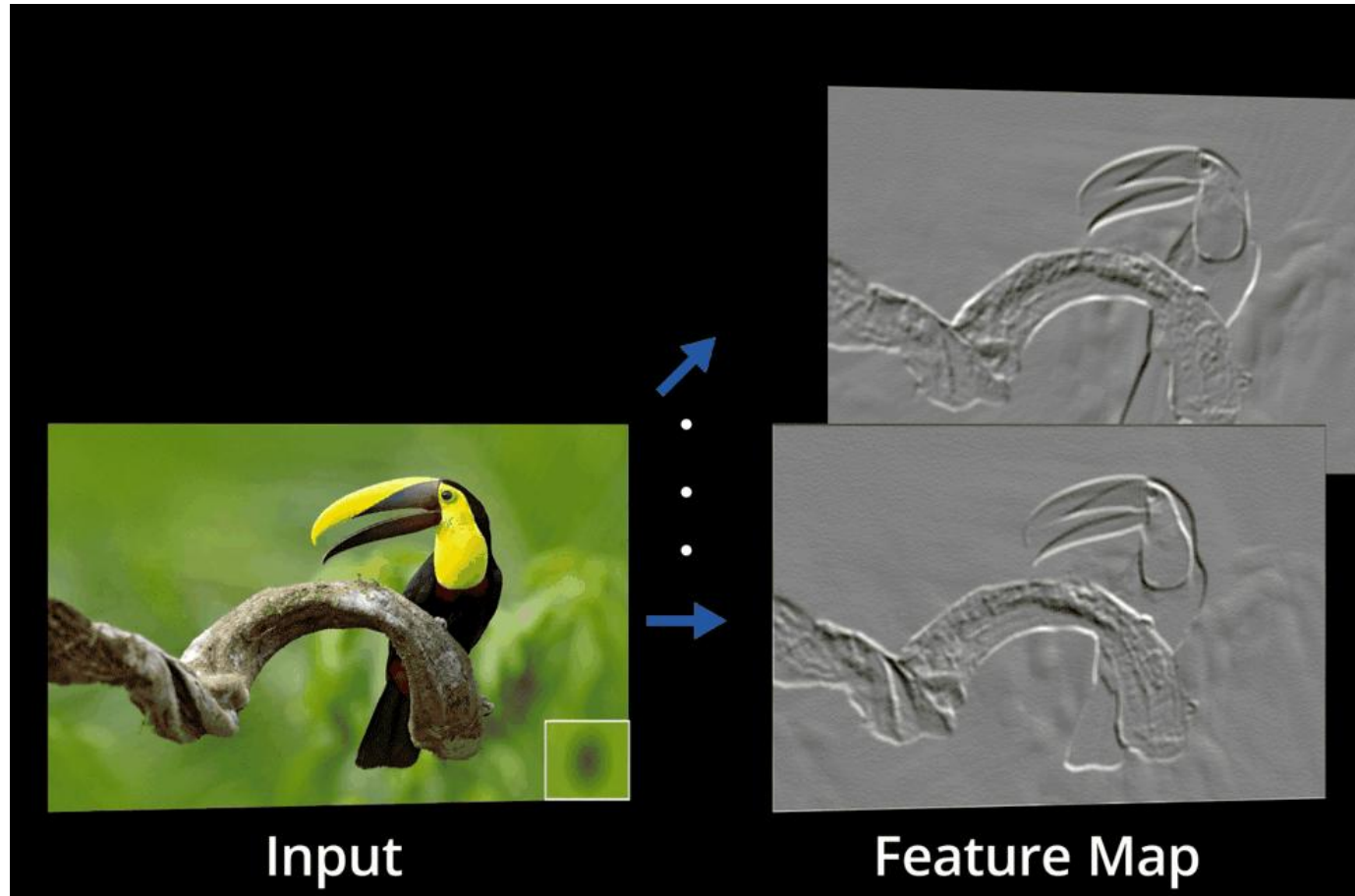
ReLU layer

- ReLU stands for the **rectified linear unit**.
- Once the feature maps are extracted, the next step is to move them to a ReLU layer.
- ReLU performs an element-wise operation and sets all the negative pixels to 0.
- It introduces non-linearity to the network, and the generated output is a rectified feature map.

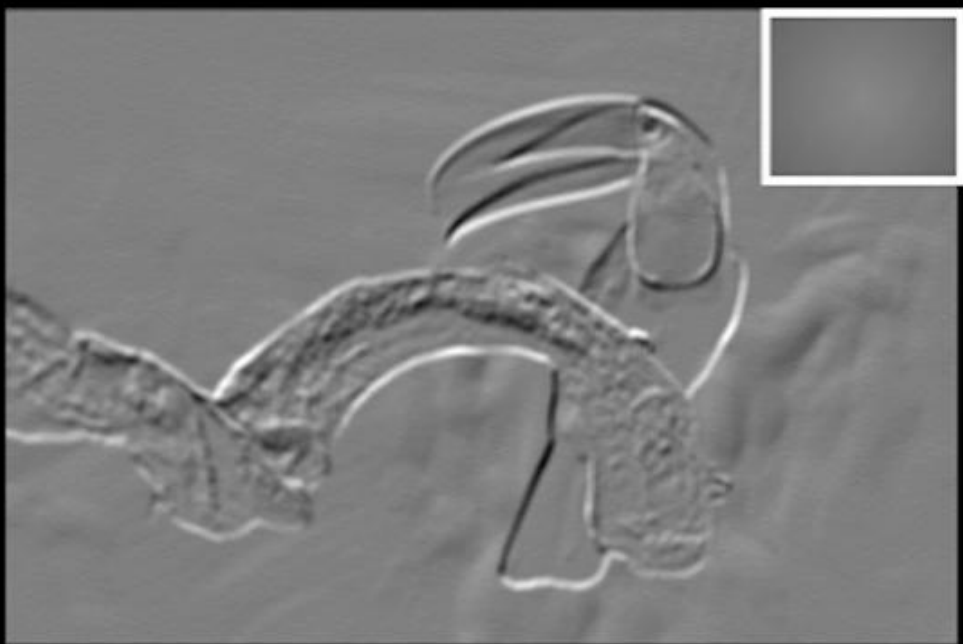


$$R(z) = \max(0, z)$$

The original image is scanned with multiple convolutions and ReLU layers for locating the features.

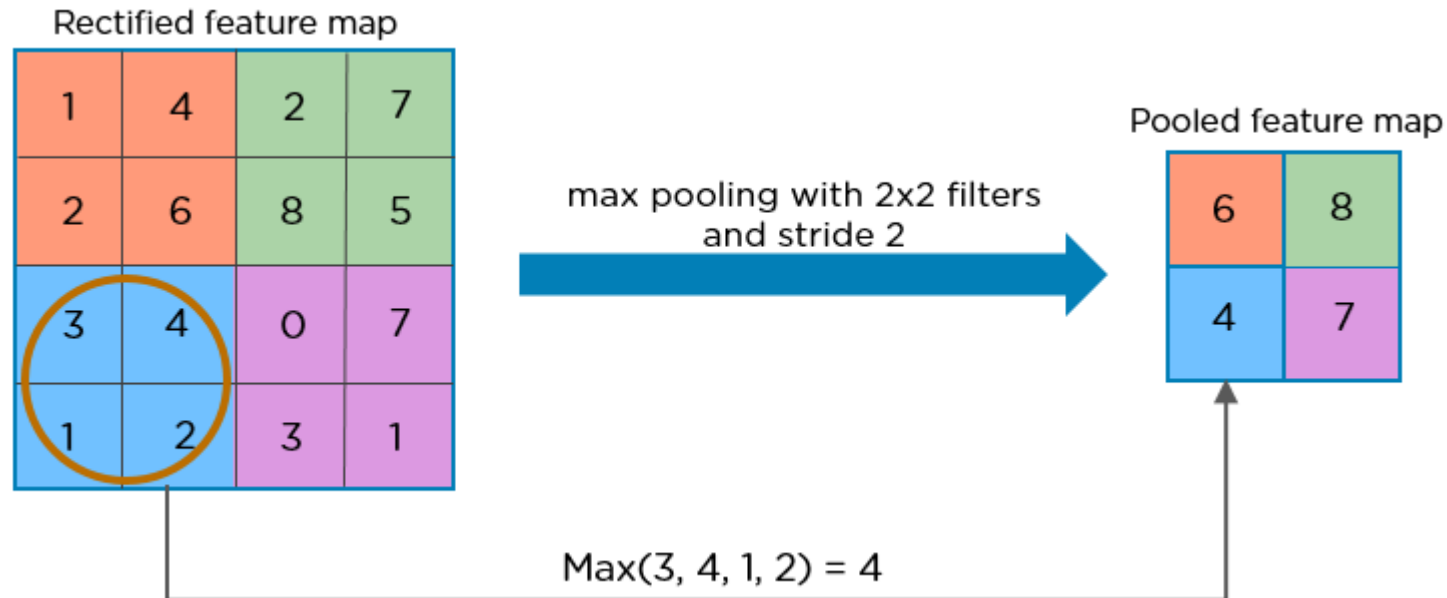


Input Feature Map

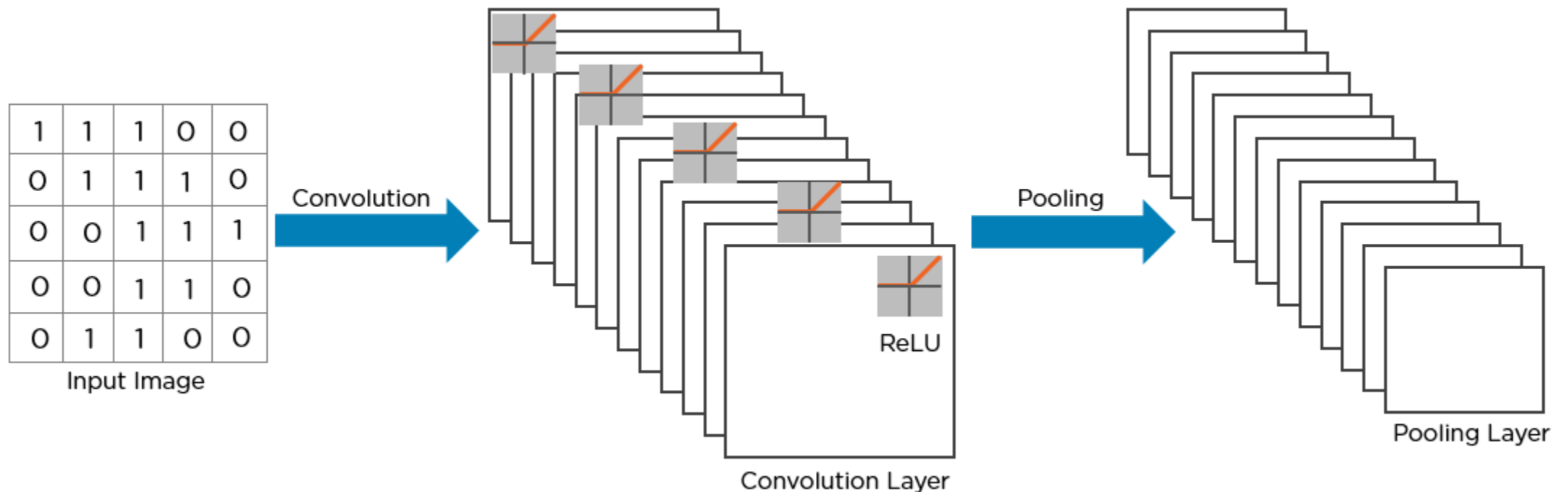


Pooling Layer

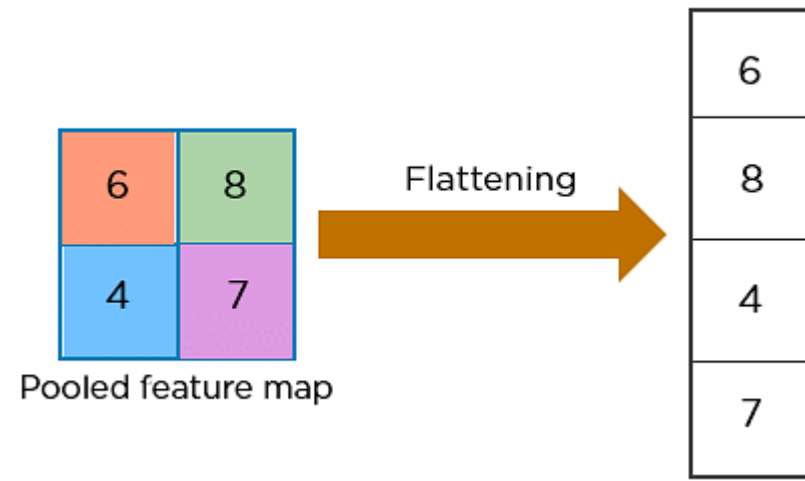
- Pooling is a down-sampling operation that reduces the dimensionality of the feature map.
- The rectified feature map now goes through a pooling layer to generate a pooled feature map.



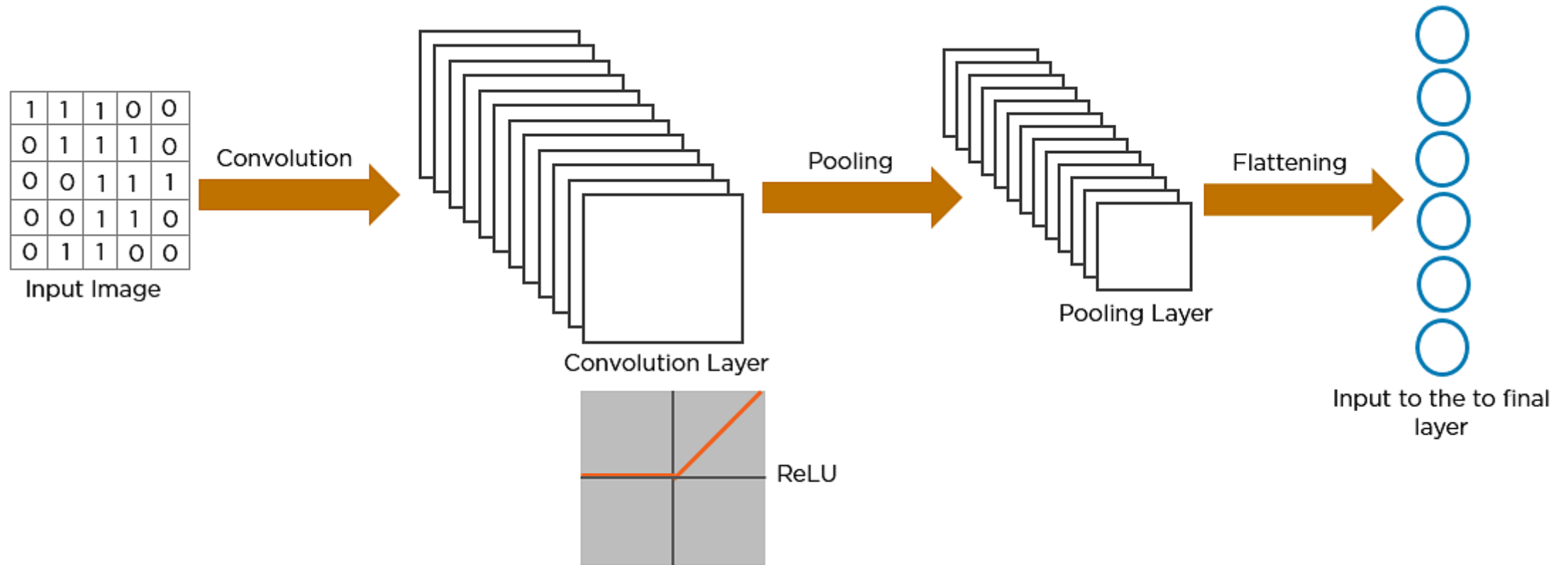
- The pooling layer uses various filters to identify different parts of the image like edges, corners, body, feathers, eyes, and beak.

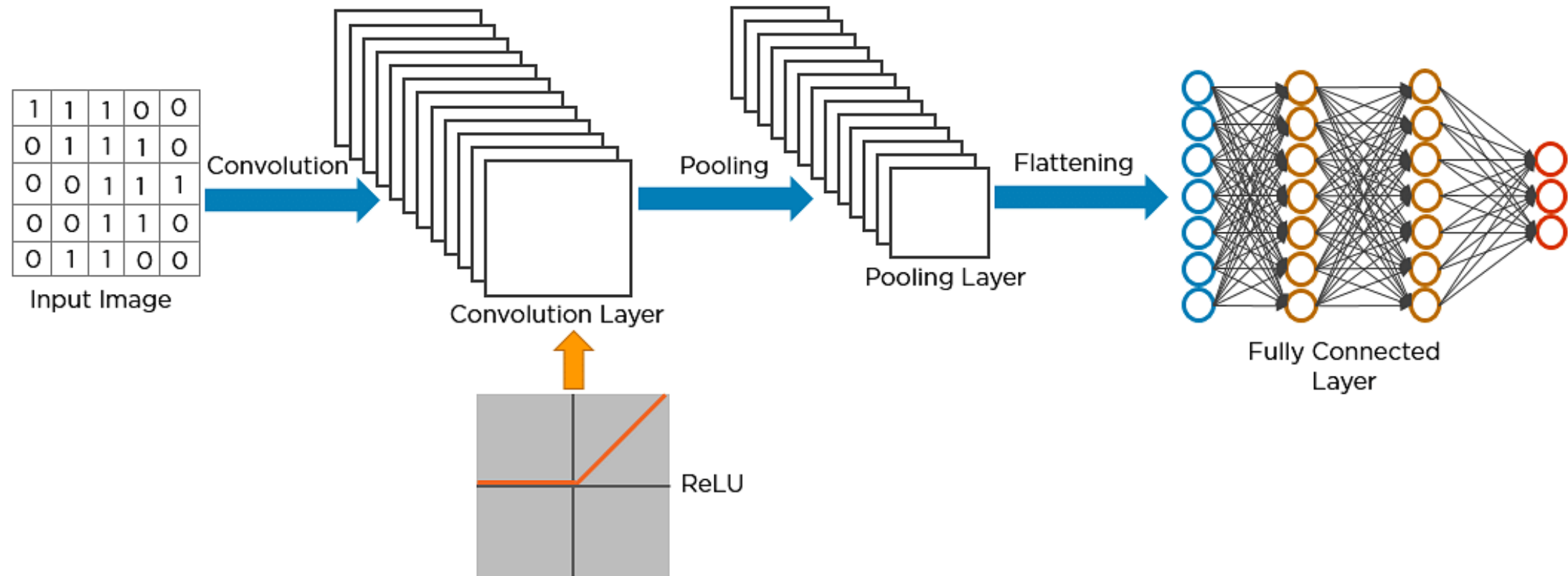


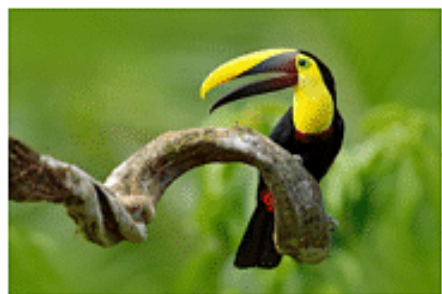
- The next step in the process is called flattening.
- Flattening is used to convert all the resultant 2-Dimensional arrays from pooled feature maps into a single long continuous linear vector.



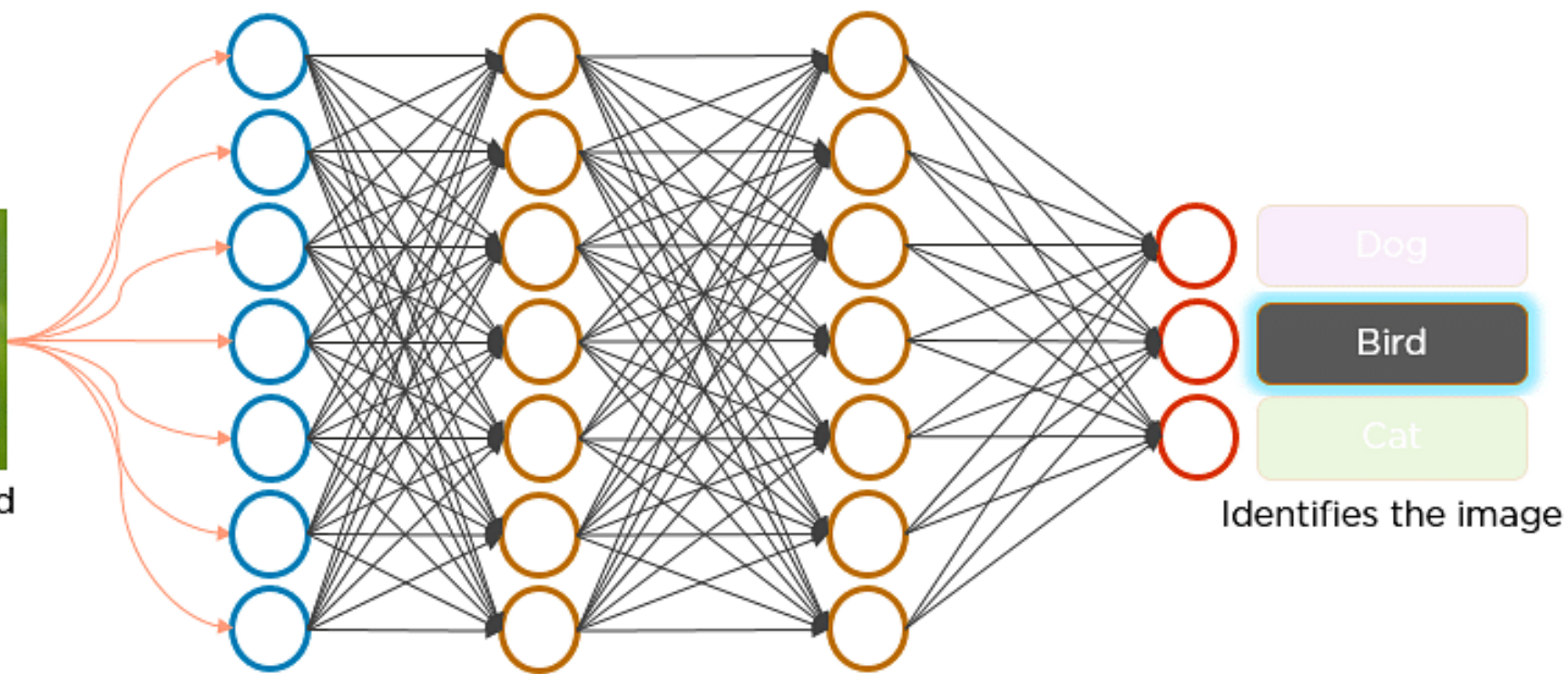
The flattened matrix is fed as input to the fully connected layer to classify the image.







Pixels from the flattened matrix fed as input

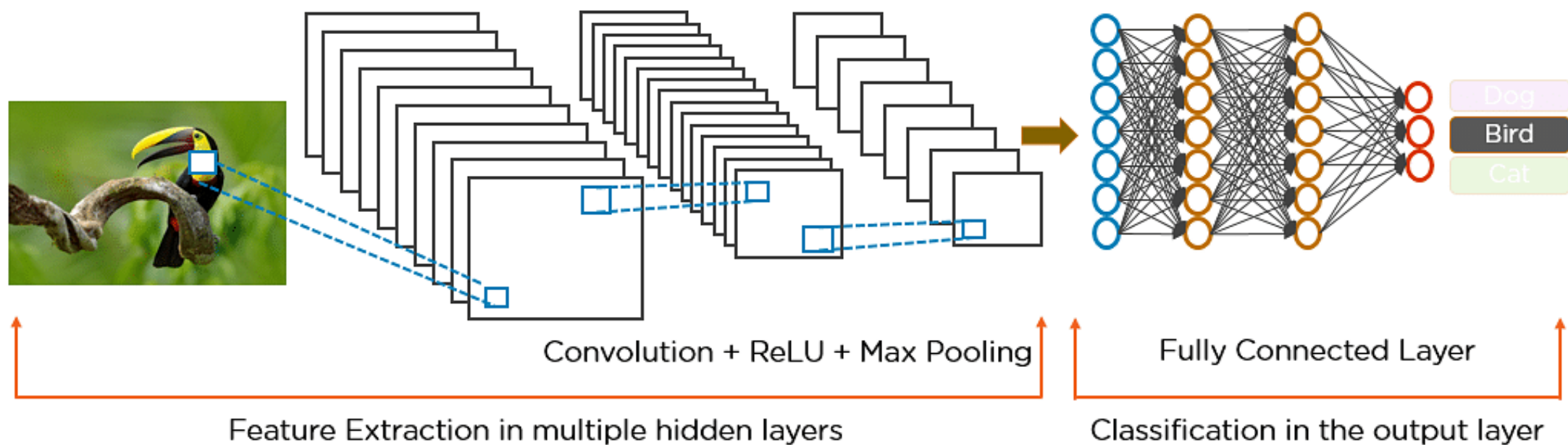


How exactly CNN recognizes a bird

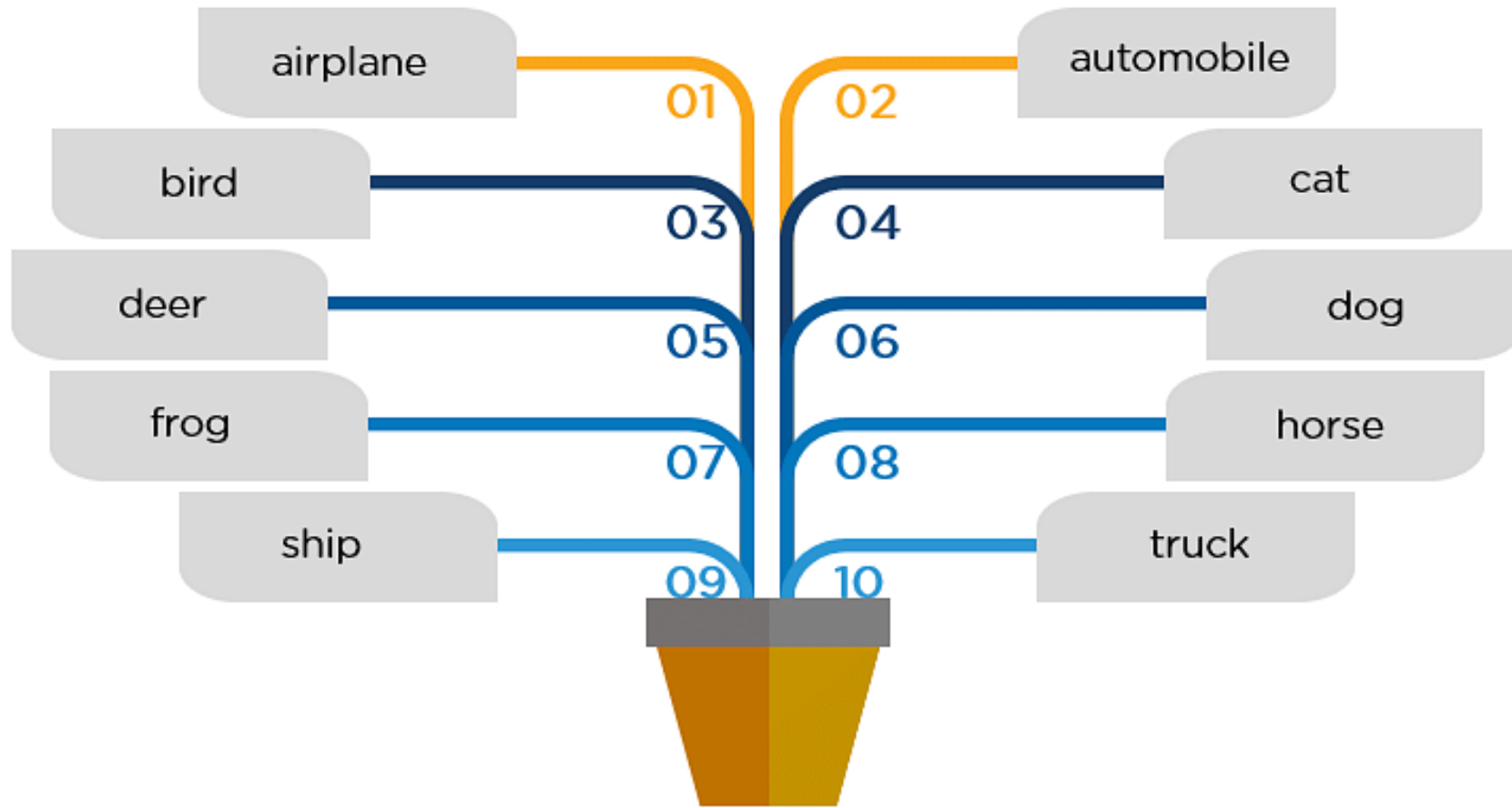
- The pixels from the image are fed to the convolutional layer that performs the convolution operation
- It results in a convolved map
- The convolved map is applied to a ReLU function to generate a rectified feature map
- The image is processed with multiple convolutions and ReLU layers for locating the features
- Different pooling layers with various filters are used to identify specific parts of the image
- The pooled feature map is flattened and fed to a fully connected layer to get the final output

•

How exactly CNN recognizes a bird:



Use case implementation using CNN



What Is a Neural Network?

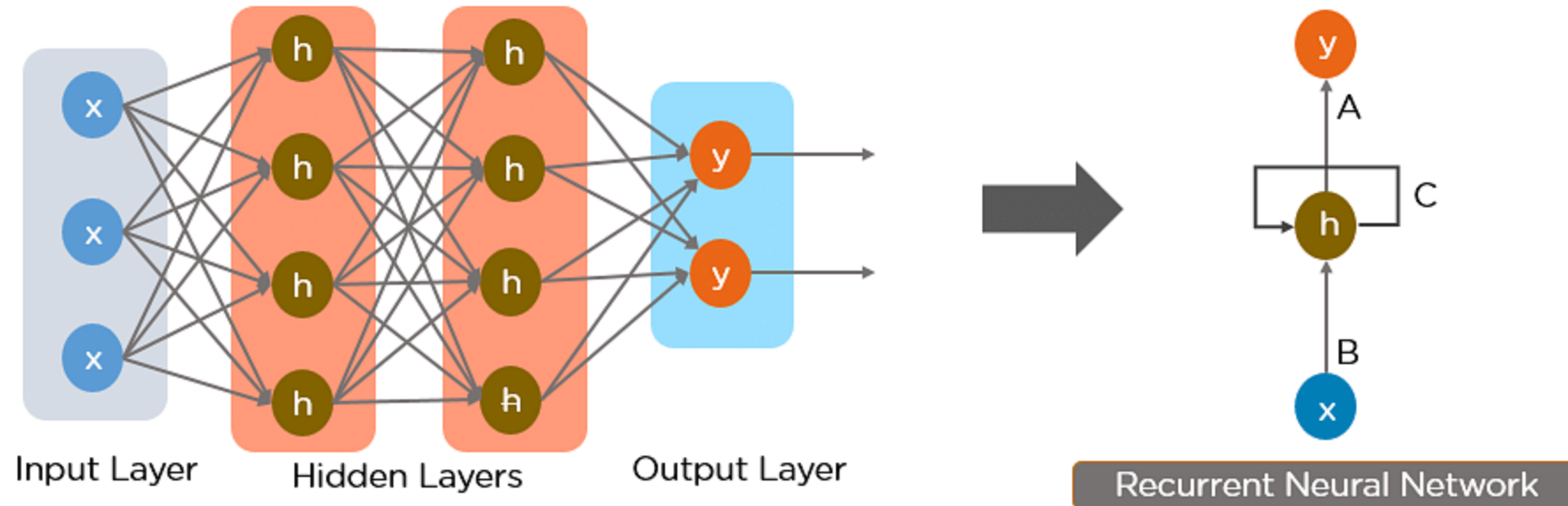
- A [Neural Network](#) consists of different layers connected to each other, working on the structure and function of a human brain.
- It learns from huge volumes of data and uses complex algorithms to train a neural net.

Several neural networks can help solve different business problems

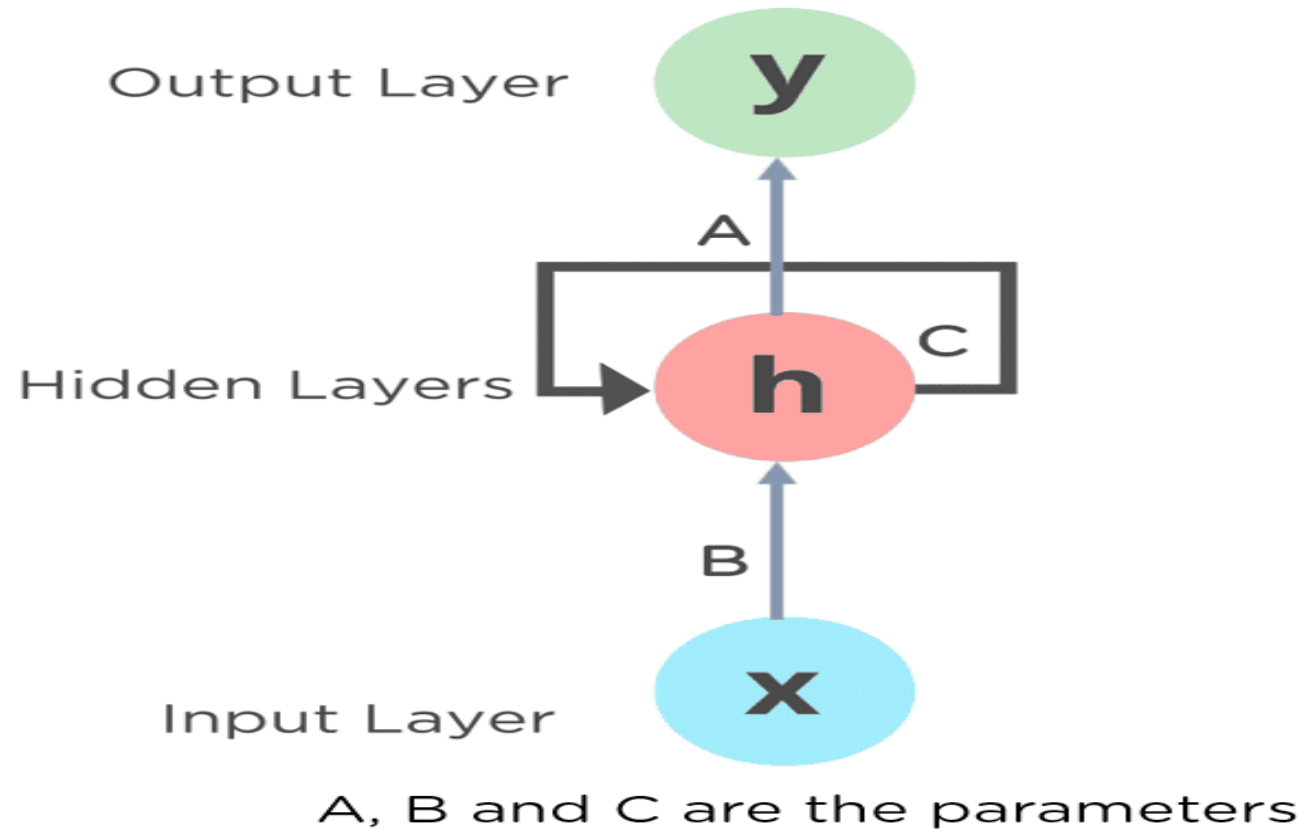
- **Feed-Forward Neural Network:** Used for general [Regression and Classification](#) problems.
- **Convolutional Neural Network:** Used for object detection and image classification.
- **Deep Belief Network:** Used in healthcare sectors for cancer detection.
- **RNN:** Used for speech recognition, voice recognition, time series prediction, and [natural language processing](#).

What Is a Recurrent Neural Network (RNN)?

- RNN works on the principle of saving the output of a particular layer and feeding this back to the input in order to predict the output of the layer.
- Below is how you can convert a Feed-Forward Neural Network into a Recurrent Neural Network

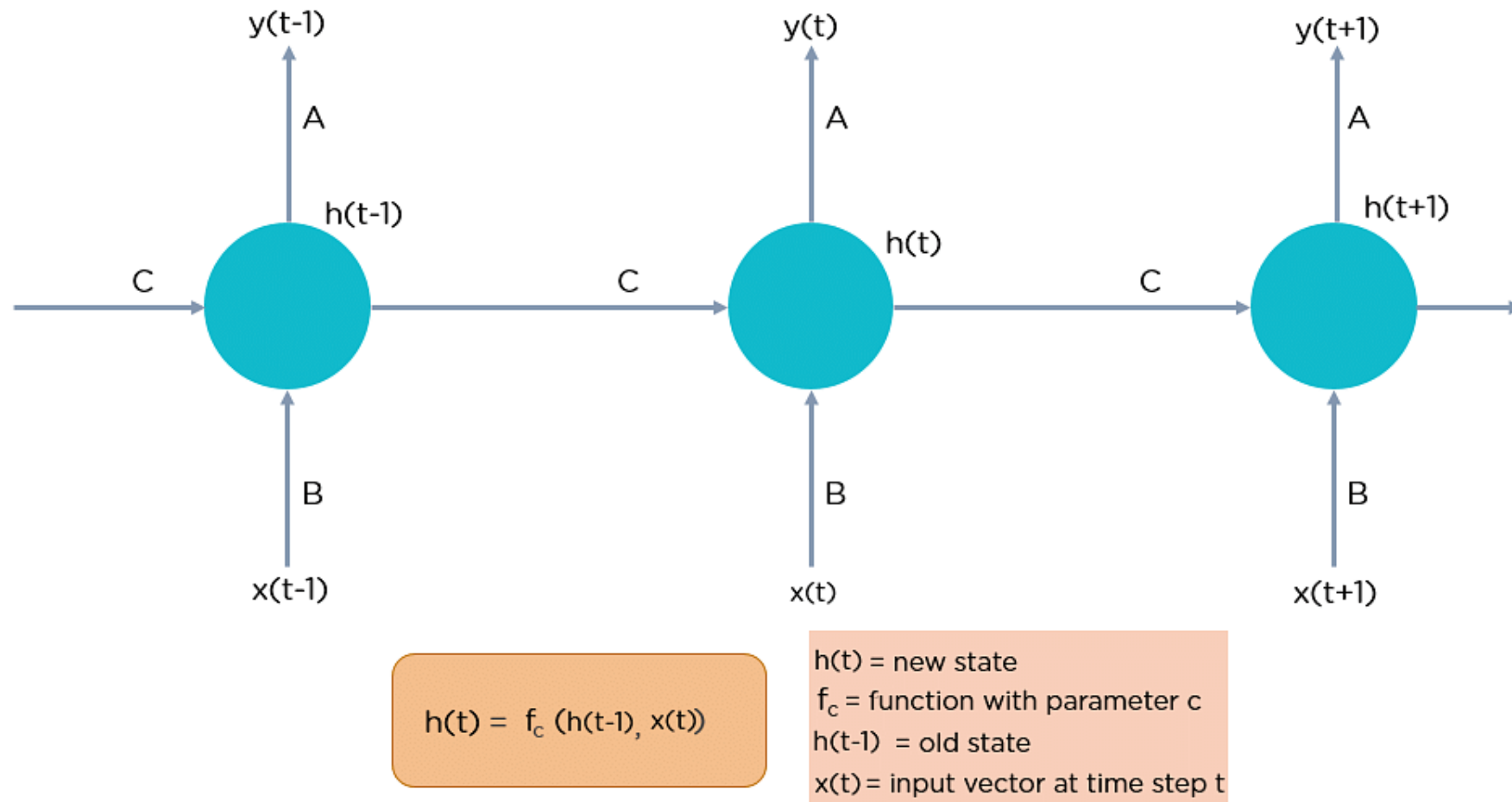


- The nodes in different layers of the neural network are compressed to form a single layer of recurrent neural networks.
- A, B, and C are the parameters of the network.



Fully connected Recurrent Neural Network

- Here, “x” is the input layer, “h” is the hidden layer, and “y” is the output layer.
- A, B, and C are the network parameters used to improve the output of the model.
- At any given time t, the current input is a combination of input at x(t) and x(t-1). The output at any given time is fetched back to the network to improve on the output.

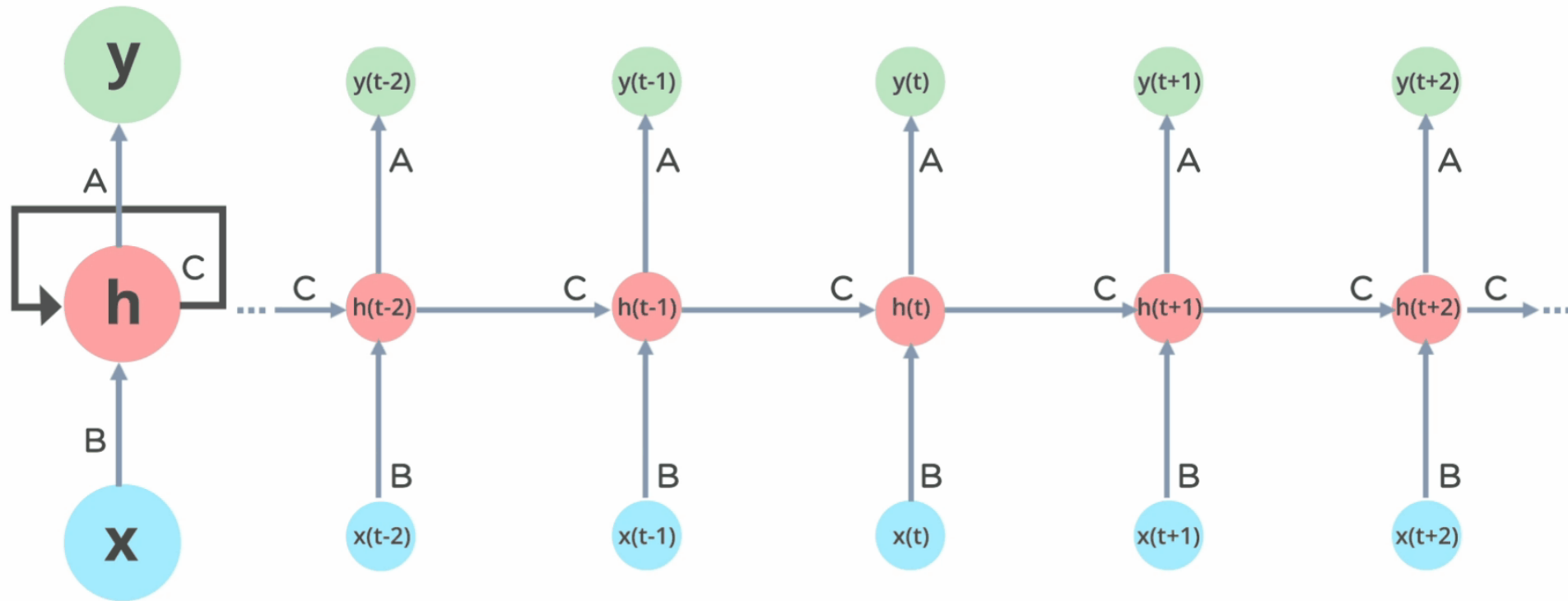


Why Recurrent Neural Networks?

- RNN were created because there were a few issues in the feed-forward neural network:
 - Cannot handle sequential data
 - Considers only the current input
 - Cannot memorize previous inputs
- The solution to these issues is the RNN.
- An RNN can handle sequential data, accepting the current input data, and previously received inputs.
- RNNs can memorize previous inputs due to their internal memory.

How Does Recurrent Neural Networks Work?

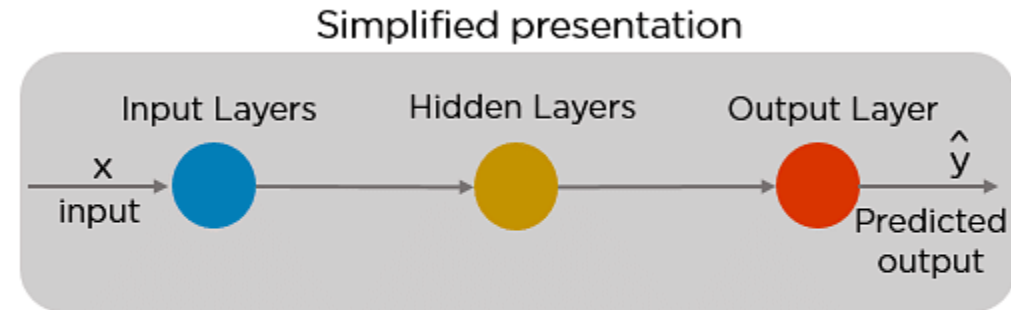
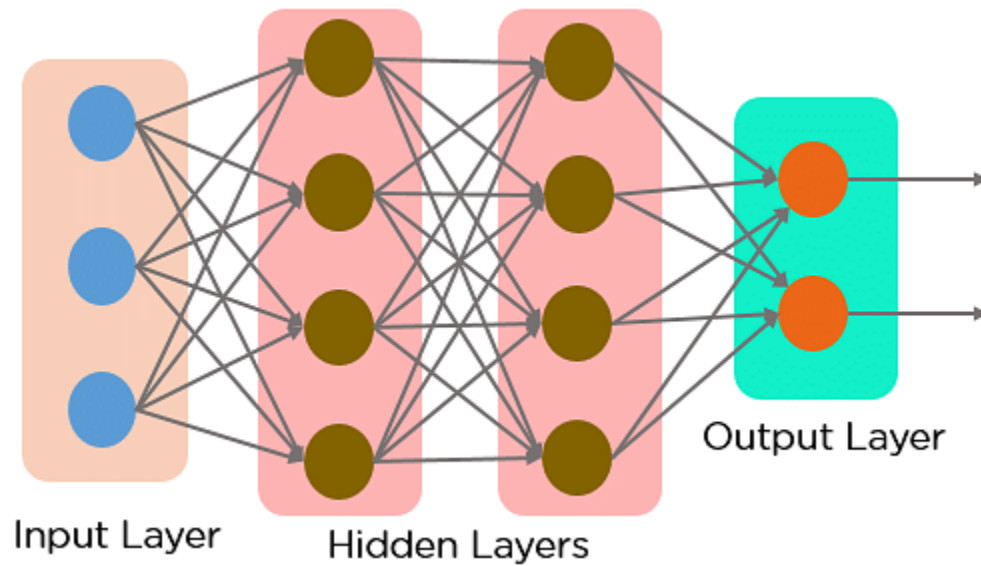
- In Recurrent Neural networks, the information cycles through a loop to the middle hidden layer.



- The input layer 'x' takes in the input to the neural network and processes it and passes it onto the middle layer.
- The middle layer 'h' can consist of multiple hidden layers, each with its own activation functions and weights and biases.
- If you have a neural network where the various parameters of different hidden layers are not affected by the previous layer, ie: the neural network does not have memory, then you can use a recurrent neural network.
- The Recurrent Neural Network will standardize the different activation functions and weights and biases so that each hidden layer has the same parameters.
- Then, instead of creating multiple hidden layers, it will create one and loop over it as many times as required.

Feed-Forward Neural Networks vs Recurrent Neural Networks

- A feed-forward neural network allows information to flow only in the forward direction, from the input nodes, through the hidden layers, and to the output nodes.
- There are no cycles or loops in the network.



Feed-forward neural networks

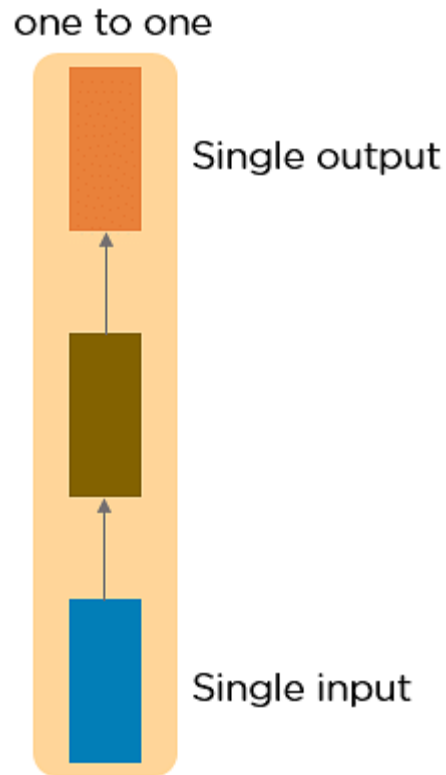
- In a feed-forward neural network, the decisions are based on the current input.
- It doesn't memorize the past data, and there's no future scope.
- Feed-forward neural networks are used in general regression and classification problems.

Types of Recurrent Neural Networks

- There are four types of Recurrent Neural Networks:
 1. One to One
 2. One to Many
 3. Many to One
 4. Many to Many

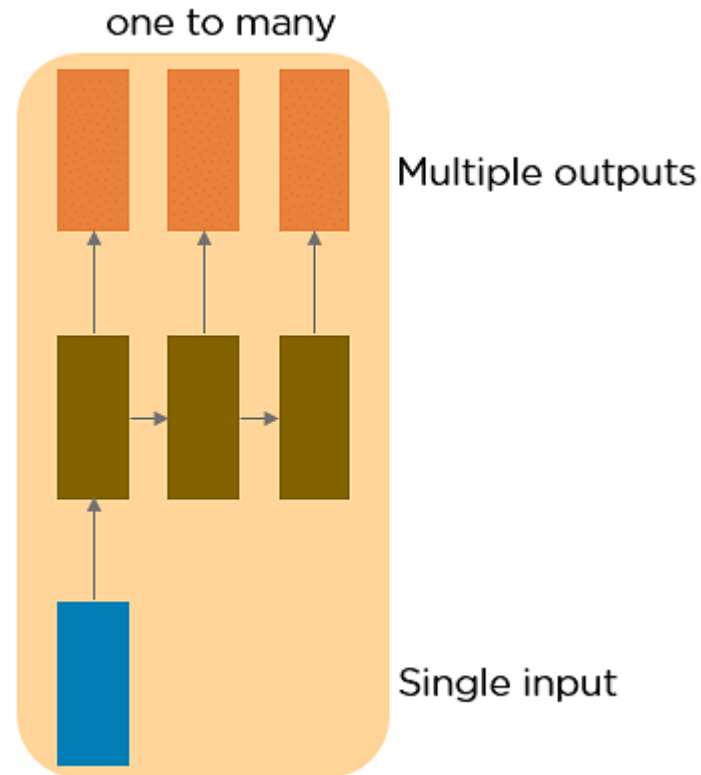
One to One RNN

- This type of neural network is known as the Vanilla Neural Network.
- It's used for general machine learning problems, which has a single input and a single output.



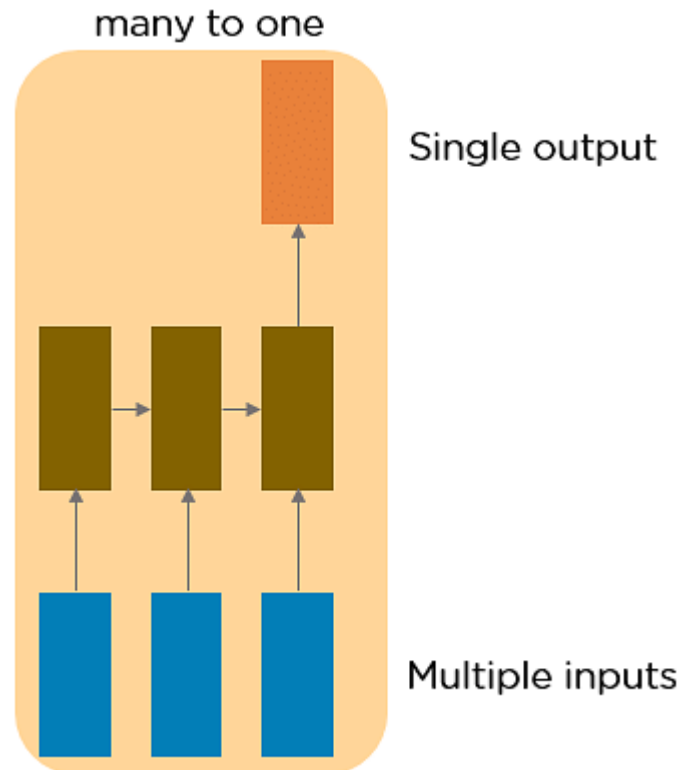
One to Many RNN

- This type of neural network has a single input and multiple outputs.
- An example of this is the image caption.



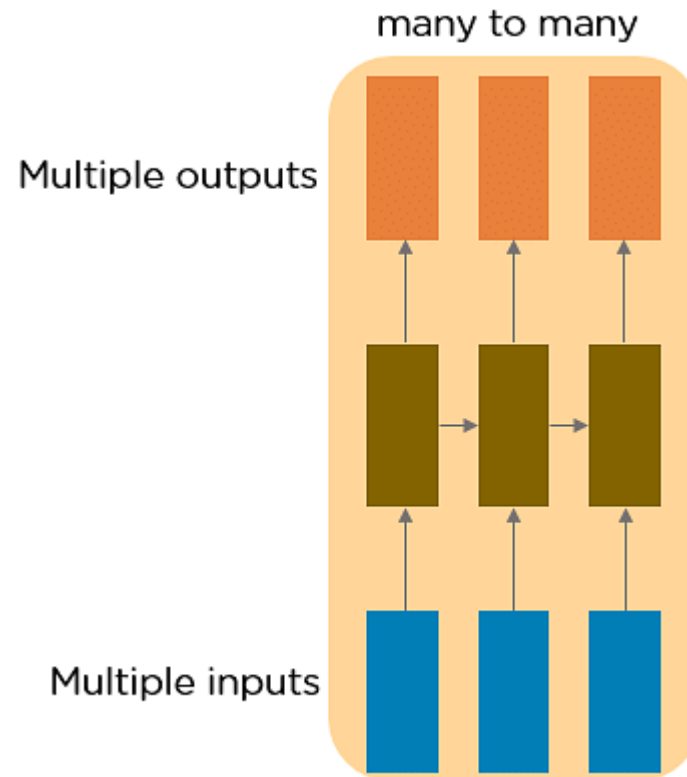
Many to One RNN

- This RNN takes a sequence of inputs and generates a single output.
- Sentiment analysis is a good example of this kind of network where a given sentence can be classified as expressing positive or negative sentiments.



Many to Many RNN

- This RNN takes a sequence of inputs and generates a sequence of outputs.
- Machine translation is one of the examples.

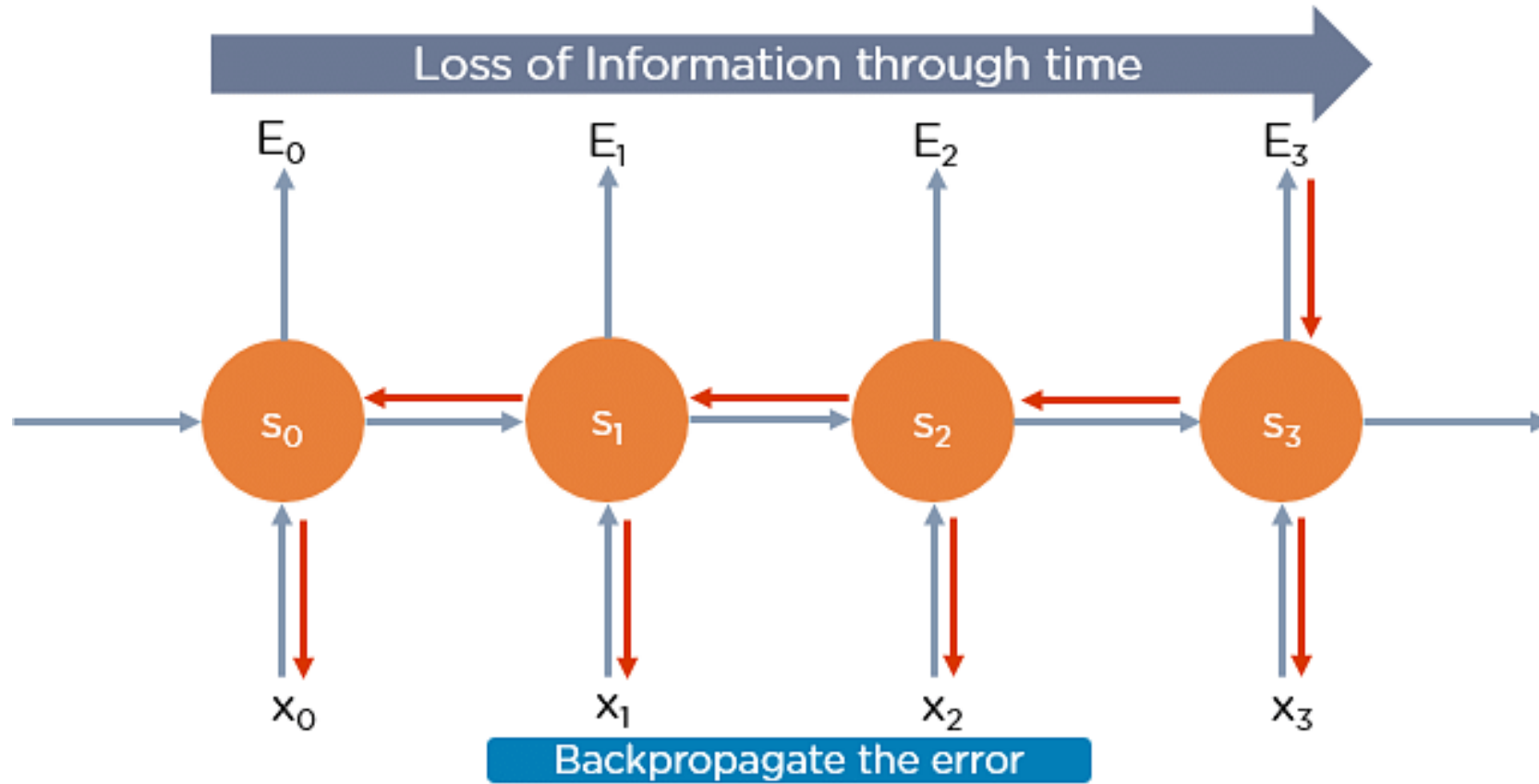


Two Issues of Standard RNNs

Vanishing Gradient Problem

- Recurrent Neural Networks enable you to model time-dependent and sequential data problems, such as stock market prediction, machine translation, and text generation.
- You will find, however, RNN is hard to train because of the gradient problem.
- RNNs suffer from the problem of vanishing gradients.
- The gradients carry information used in the RNN, and when the gradient becomes too small, the parameter updates become insignificant.
- This makes the learning of long data sequences difficult.

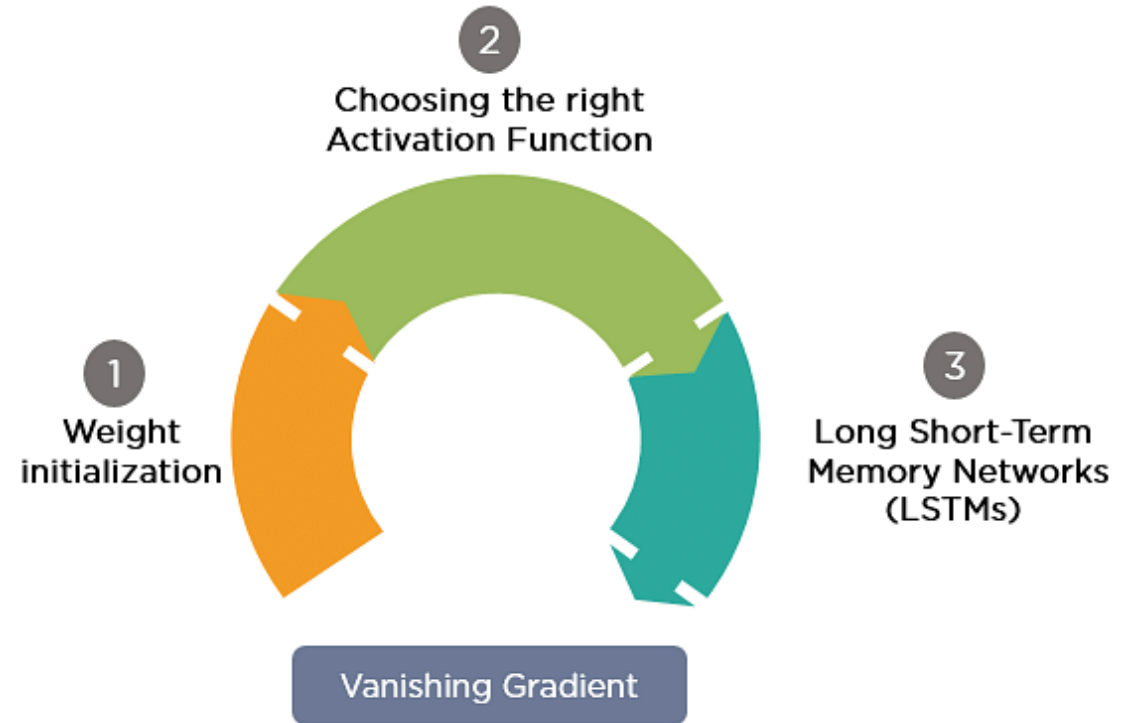
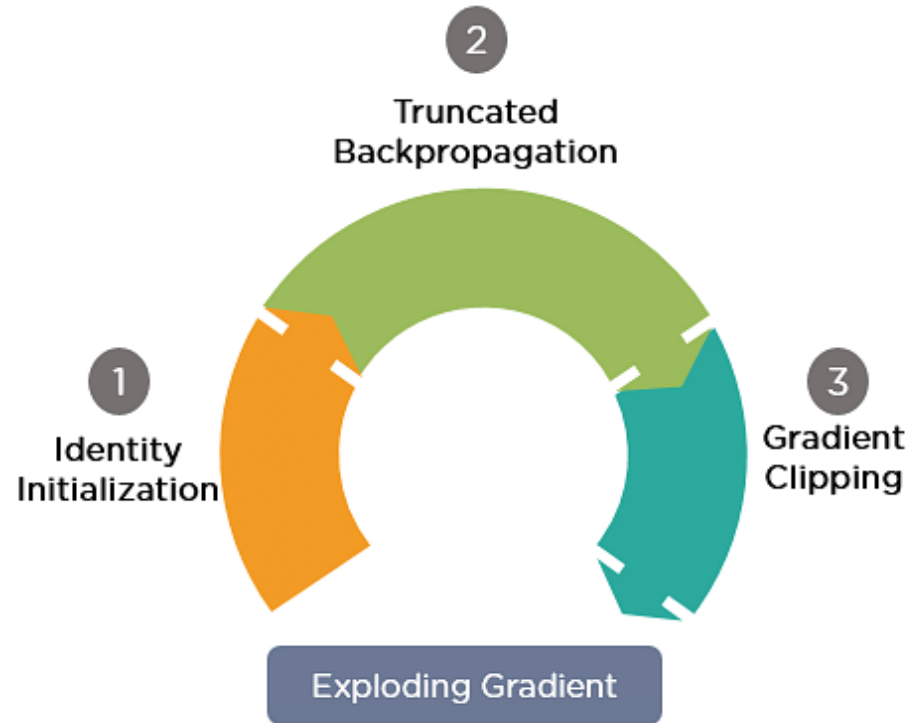
Vanishing Gradient Problem



Exploding Gradient Problem

- While training a neural network, if the slope tends to grow exponentially instead of decaying, this is called an **Exploding Gradient**.
- This problem arises when large error gradients accumulate, resulting in very large updates to the neural network model weights during the training process.
- Long training time, poor performance, and bad accuracy are the major issues in gradient problems.

Gradient Problem Solutions



- Now, let's discuss the most popular and efficient way to deal with gradient problems, i.e., Long Short-Term Memory Network (LSTMs).
- First, let's understand Long-Term Dependencies.
- Suppose you want to predict the last word in the text: "The clouds are in the _____."
- The most obvious answer to this is the "sky." We do not need any further context to predict the last word in the above sentence.
- Consider this sentence: "I have been staying in Spain for the last 10 years...I can speak fluent _____."
- The word you predict will depend on the previous few words in context.
- Here, you need the context of Spain to predict the last word in the text, and the most suitable answer to this sentence is "Spanish."
- The gap between the relevant information and the point where it's needed may have become very large. LSTMs help you solve this problem.

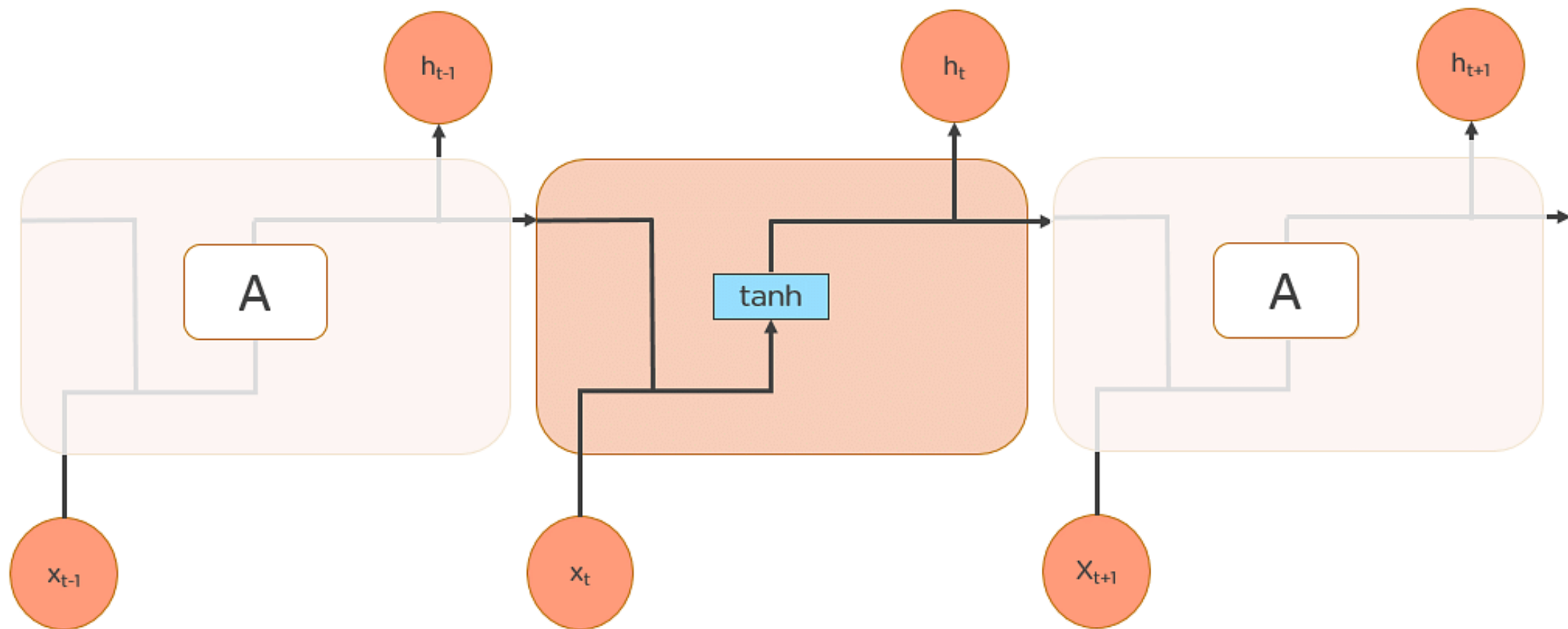
Backpropagation Through Time

- Backpropagation through time is when we apply a Backpropagation algorithm to a Recurrent Neural network that has time series data as its input.
- In a typical RNN, one input is fed into the network at a time, and a single output is obtained.
- But in backpropagation, you use the current as well as the previous inputs as input.
- This is called a timestep and one timestep will consist of many time series data points entering the RNN simultaneously.
- Once the neural network has trained on a timeset and given you an output, that output is used to calculate and accumulate the errors.
- After this, the network is rolled back up and weights are recalculated and updated keeping the errors in mind.

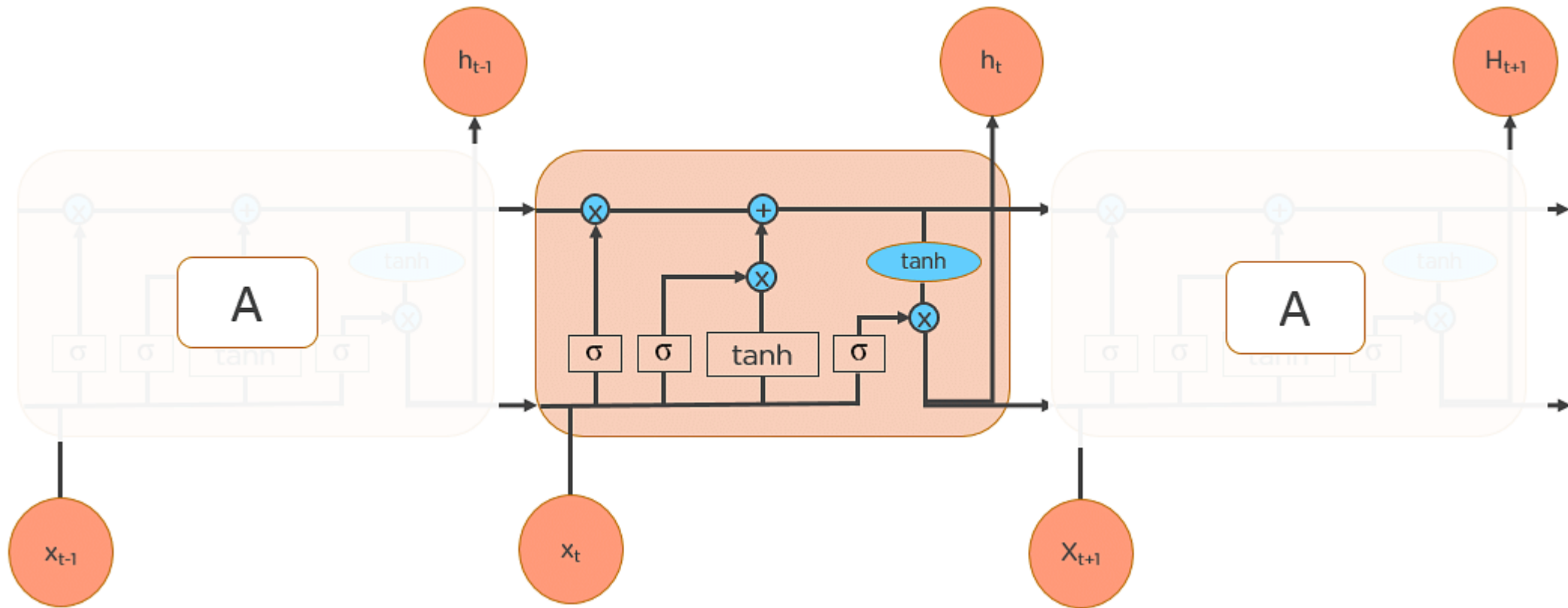
Long Short-Term Memory Networks

- LSTMs are a special kind of RNN — capable of learning long-term dependencies by remembering information for long periods is the default behavior.
- All RNN are in the form of a chain of repeating modules of a neural network.
- In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

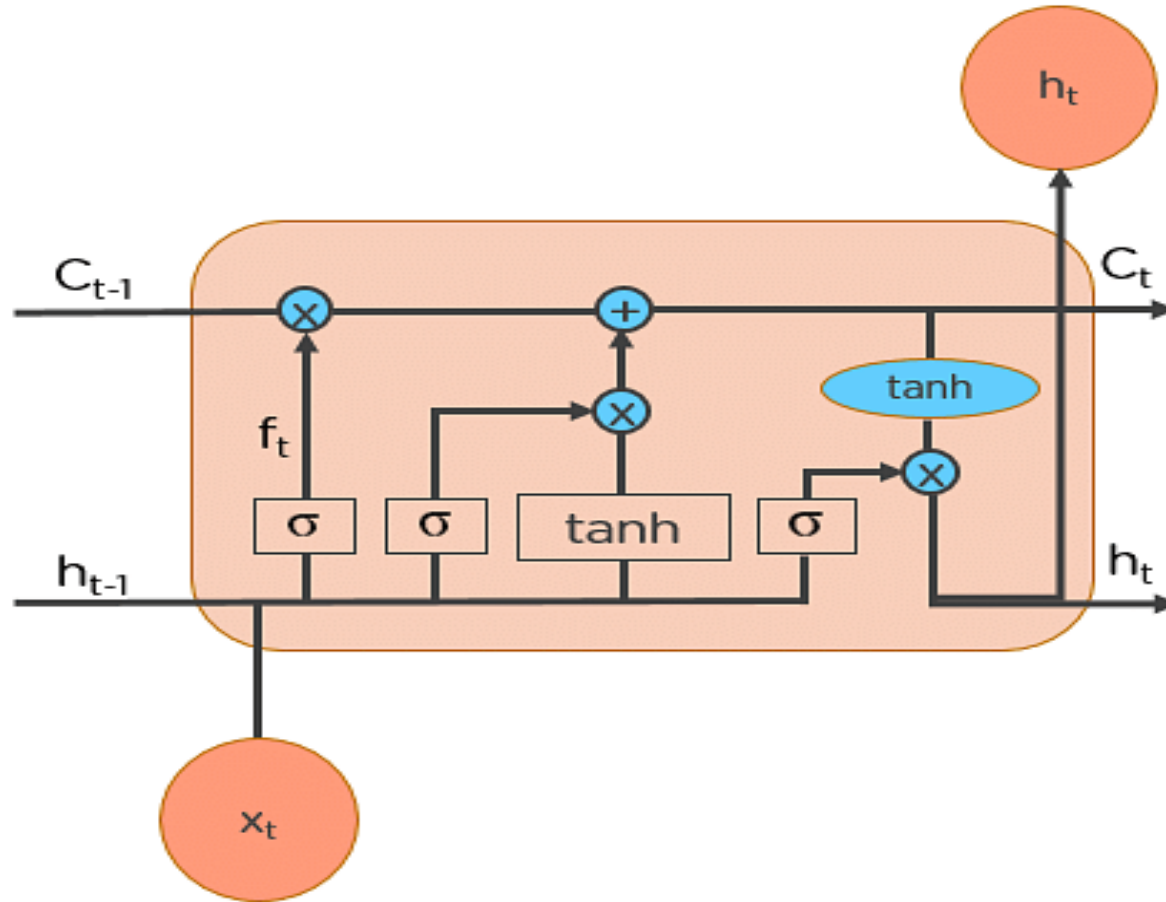
Long Short-Term Memory Networks



- LSTMs also have a chain-like structure, but the repeating module is a bit different structure.
- Instead of having a single neural network layer, four interacting layers are communicating extraordinarily.



Workings of LSTMs in RNN



LSTMs work in a 3-step process.

Step 1: Decide How Much Past Data It Should Remember

- The first step in the LSTM is to decide which information should be omitted from the cell in that particular time step.
- The sigmoid function determines this.
- It looks at the previous state (h_{t-1}) along with the current input x_t and computes the function.

$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

f_t = forget gate
Decides which information to
delete that is not important
from previous time step

Consider the following two sentences:

- Let the output of $h(t-1)$ be “Alice is good in Physics. John, on the other hand, is good at Chemistry.”
- Let the current input at $x(t)$ be “John plays football well. He told me yesterday over the phone that he had served as the captain of his college football team.”
- The forget gate realizes there might be a change in context after encountering the first full stop. It compares with the current input sentence at $x(t)$. The next sentence talks about John, so the information on Alice is deleted. The position of the subject is vacated and assigned to John.

Step 2: Decide How Much This Unit Adds to the Current State

- In the second layer, there are two parts. One is the sigmoid function, and the other is the tanh function.
- In the sigmoid function, it decides which values to let through (0 or 1).
- Tanh function gives weightage to the values which are passed, deciding their level of importance (-1 to 1).

$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

i_t = input gate
Determines which information to let through based on its significance in the current time step

With the current input at $x(t)$, the input gate analyzes the important information — John plays football, and the fact that he was the captain of his college team is important. “He told me yesterday over the phone” is less important; hence it's forgotten. This process of adding some new information can be done via the input gate.

Step 3: Decide What Part of the Current Cell State Makes It to the Output

- The third step is to decide what the output will be.
- First, we run a sigmoid layer, which decides what parts of the cell state make it to the output.
- Then, we put the cell state through tanh to push the values to be between -1 and 1 and multiply it by the output of the sigmoid gate.

$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

o_t = output gate
Allows the passed in information to
impact the output in the current
time step

- Let's consider this example to predict the next word in the sentence:
- “John played tremendously well against the opponent and won for his team.
- For his contributions, brave ____ was awarded player of the match.”
- There could be many choices for the empty space.
- The current input brave is an adjective, and adjectives describe a noun. So, “John” could be the best output after brave.

	ANN	CNN	RNN
Basics	One of the simplest types of neural networks.	One of the most popular types of neural networks.	The most advanced and complex neural network.
Structural Layout	Its simplicity comes from its feed forward nature — information flows in one direction only.	Its structure is based on multiple layers of nodes including one or more convolutional layers.	Information flows in different directions, which gives it its memory and self-learning features.
Data Type	Fed on tabular and text data.	Relies on image data.	Trained with sequence data.
Complexity	Simple in contrast with the other two models.	Considered more powerful than the other two.	Fewer features than CNN but powerful due to its self-learning & memory potential.
Commendable Feature	Ability to work with incomplete knowledge and high fault tolerance.	Accuracy in recognizing images.	Memory and self-learning.
Feature type: spatial recognition	No	Yes	No
Feature type: Recurrent connections	No	No	Yes
Main Drawback	Hardware dependence.	Large training data required.	Slow and complex training and gradient concerns.
Uses	Complex problem solving such as predictive analysis.	Computer vision including image recognition	Natural language processing including sentiment analysis and speed recognition.



	ANN	CNN	RNN
Type of Data	Tabular Data, Text Data	Image Data	Sequence data
Parameter Sharing	No	Yes	Yes
Fixed Length input	Yes	Yes	No
Recurrent Connections	No	No	Yes
Vanishing and Exploding Gradient	Yes	Yes	Yes
Spatial Relationship	No	Yes	No
Performance	ANN is considered to be less powerful than CNN, RNN.	CNN is considered to be more powerful than ANN, RNN.	RNN includes less feature compatibility when compared to CNN.
Application	Facial recognition and Computer vision.	Facial recognition, text digitization and Natural language processing.	Text-to-speech conversions.
Main advantages	Having fault tolerance, Ability to work with incomplete knowledge.	High accuracy in image recognition problems, Weight sharing.	Remembers each and every information, Time series prediction.
Disadvantages	Hardware dependence, Unexplained behavior of the network.	Large training data needed, don't encode the position and orientation of object.	Gradient vanishing, exploding gradient.

Bayes Rule

What is the Bayes' Theorem?

- In statistics and probability theory, the Bayes' theorem (also known as the Bayes' rule) is a mathematical formula used to determine the conditional probability of events.
- Essentially, the Bayes' theorem describes the [probability](#) of an event based on prior knowledge of the conditions that might be relevant to the event.
- The theorem is named after English statistician, Thomas Bayes, who discovered the formula in 1763.
- It is considered the foundation of the special statistical inference approach called the Bayes' inference.



The diagram illustrates Bayes' Theorem using colored boxes and mathematical symbols. On the left, an orange box contains the expression $P(A|B)$. To its right is a dark blue equals sign. Further right is a fraction. The numerator of the fraction consists of a teal box with $P(B|A)$ multiplied by another teal box with $P(A)$, indicated by a dark blue 'X' symbol. The denominator is a teal box with $P(B)$, positioned below a horizontal line.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

- Besides [statistics](#), the Bayes' theorem is also used in various disciplines, with medicine and pharmacology as the most notable examples.
- In addition, the theorem is commonly employed in different fields of finance.
- Some of the applications include but are not limited to, modeling the risk of lending money to borrowers or forecasting the probability of the success of an investment.

Formula for Bayes' Theorem

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- $P(A|B)$ – the probability of event A occurring, given event B has occurred
- $P(B|A)$ – the probability of event B occurring, given event A has occurred
- $P(A)$ – the probability of event A
- $P(B)$ – the probability of event B

A special case of the Bayes' theorem is when event A is a binary variable. In such a case, the theorem is expressed in the following way

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A^-)P(A^-) + P(B|A^+)P(A^+)}$$

- $P(B|A^-)$ – the probability of event B occurring given that event A^- has occurred
- $P(B|A^+)$ – the probability of event B occurring given that event A^+ has occurred

Example of Bayes' Theorem

- Imagine you are a financial analyst at an investment bank. According to your research of [publicly-traded companies](#), 60% of the companies that increased their share price by more than 5% in the last three years replaced their [CEOs](#) during the period.
- At the same time, only 35% of the companies that did not increase their share price by more than 5% in the same period replaced their CEOs. Knowing that the probability that the stock prices grow by more than 5% is 4%, find the probability that the shares of a company that fires its CEO will increase by more than 5%.

- Before finding the probabilities, you must first define the notation of the probabilities.
 - $P(A)$ – the probability that the stock price increases by 5%
 - $P(B)$ – the probability that the CEO is replaced
 - $P(A|B)$ – the probability of the stock price increases by 5% given that the CEO has been replaced
 - $P(B|A)$ – the probability of the CEO replacement given the stock price has increased by 5%

$$P(A|B) = \frac{0.60 \times 0.04}{0.60 \times 0.04 + 0.35 \times (1 - 0.04)} = 0.067 \text{ or } 6.67\%$$

Thus, the probability that the shares of a company that replaces its CEO will grow by more than 5% is 6.67%.

- Three persons A, B and C have applied for a job in a private company. The chance of their selections is in the ratio 1 : 2 : 4. The probabilities that A, B and C can introduce changes to improve the profits of the company are 0.8, 0.5 and 0.3, respectively. If the change does not take place, find the probability that it is due to the appointment of C.

- Let E_1 : person A get selected
- E_2 : person B get selected
- E_3 : person C get selected
- A: Changes introduced but profit not happened
- Now, $P(E_1) = 1/(1+2+4) = 1/7$
- $P(E_2) = 2/7$ and $P(E_3) = 4/7$
- $P(A|E_1) = P(\text{Profit not happened by the changes introduces by A}) = 1 - P(\text{Profit happened by the changes introduces by A}) = 1 - 0.8 = 0.2$
- $P(A|E_2) = P(\text{Profit not happened by the changes introduces by B}) = 1 - P(\text{Profit happened by the changes introduces by B}) = 1 - 0.5 = 0.5$
- $P(A|E_3) = P(\text{Profit not happened by the changes introduces by C}) = 1 - P(\text{Profit happened by the changes introduces by C}) = 1 - 0.3 = 0.7$

$$P(E_3|A) = \frac{P(A|E_3)P(E_3)}{P(A|E_1)P(E_1)+P(A|E_2)P(E_2)+P(A|E_3)P(E_3)}$$

$$P(E_3|A) = \frac{0.7 \times \frac{4}{7}}{0.2 \times \frac{1}{7} + 0.5 \times \frac{2}{7} + 0.7 \times \frac{4}{7}}$$

$$= 7/10.$$

$\therefore$ the required probability is 0.7.

UNIT 3

- **Advanced Deep Learning Methods: Variational Autoencoder (VAE), Deep Autoencoder (DAE). Generative Adversarial Network (GAN), Deep Boltzmann Machines, Deep Neural Network applications for multimedia, Sequence and Streaming data, Deep Belief Network**

What is an autoencoder?

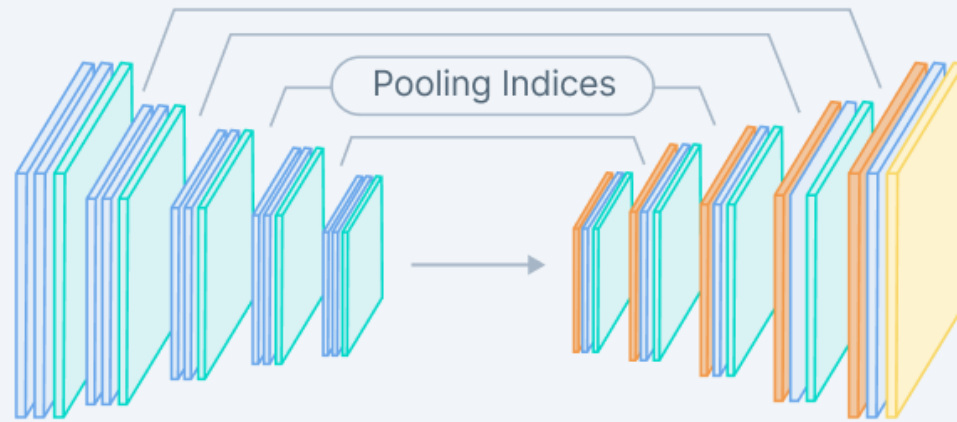
- An autoencoder is a type of [artificial neural network](#) used to learn data encodings in an unsupervised manner.
- The aim of an autoencoder is to learn a lower-dimensional representation (encoding) for a higher-dimensional data, typically for dimensionality reduction, by [training the network](#) to capture the most important parts of the input image.

Convolutional encoder-decoder

Input



RGB Image



Output



Segmentation

The architecture of autoencoders

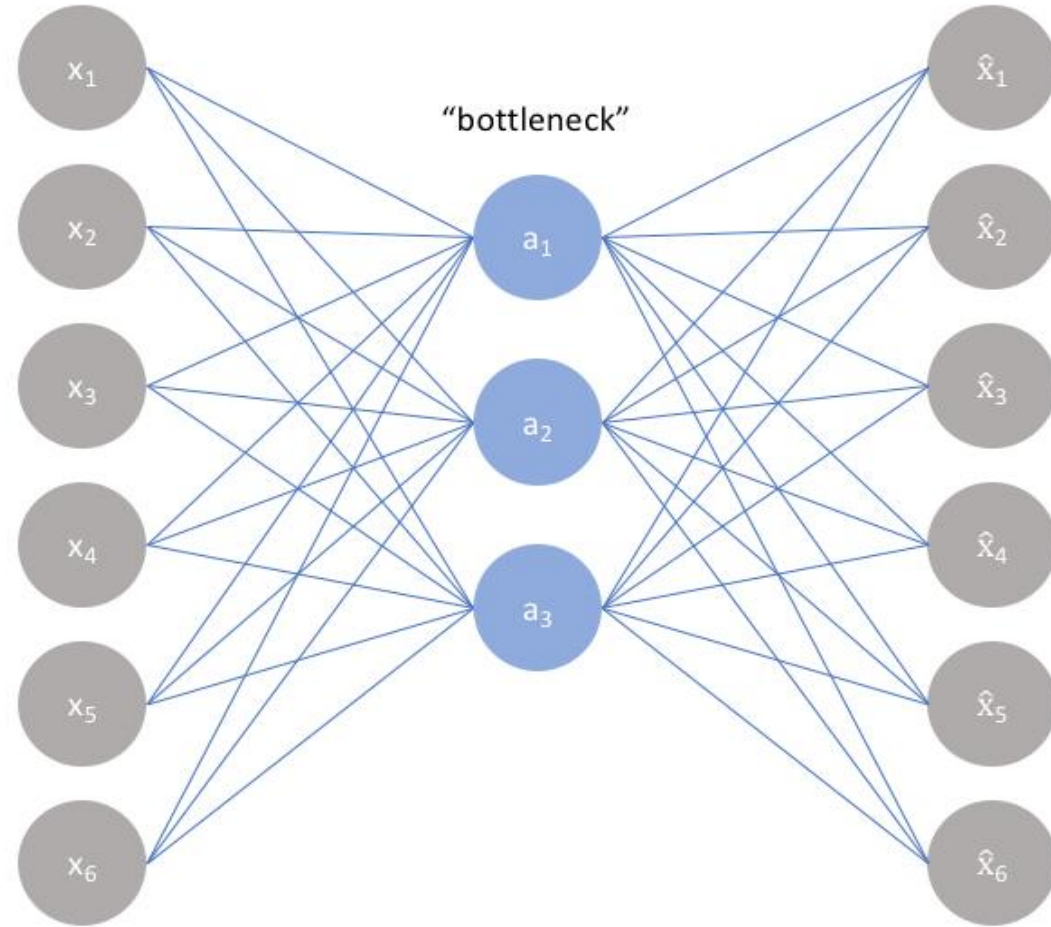
Autoencoders consist of 3 parts:

1. **Encoder:** A module that compresses [the train-validate-test set input data](#) into an encoded representation that is typically several orders of magnitude smaller than the input data.
2. **Bottleneck:** A module that contains the compressed knowledge representations and is therefore the most important part of the network.
3. **Decoder:** A module that helps the network “decompress” the knowledge representations and reconstructs the data back from its encoded form. The output is then compared with a ground truth.

Input layer

Hidden layer

Output layer



How to train autoencoders?

- You need to set 4 hyperparameters before *training* an autoencoder:
 - 1.Code size:** The code size or the size of the bottleneck is the most important hyperparameter used to tune the autoencoder. The bottleneck size decides how much the data has to be compressed. This can also act as a regularization term.
 - 2.Number of layers:** Like all neural networks, an important hyperparameter to tune autoencoders is the depth of the encoder and the decoder. While a higher depth increases model complexity, a lower depth is faster to process.

3.Number of nodes per layer: The number of nodes per layer defines the weights we use per layer. Typically, the number of nodes decreases with each subsequent layer in the autoencoder as the input to each of these layers becomes smaller across the layers.

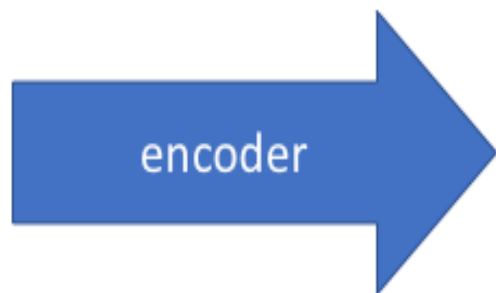
4.Reconstruction Loss: The [loss function](#) we use to train the autoencoder is highly dependent on the type of input and output we want the autoencoder to adapt to. If we are working with image data, the most popular loss functions for reconstruction are MSE Loss and L1 Loss. In case the inputs and outputs are within the range $[0,1]$, as in MNIST, we can also make use of Binary Cross Entropy as the reconstruction loss.

Five popular autoencoders

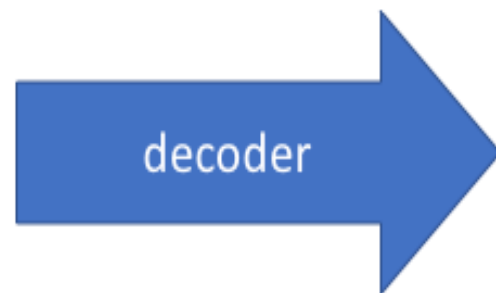
1. Undercomplete autoencoders
2. Sparse autoencoders
3. Contractive autoencoders
4. Denoising autoencoders
5. Variational Autoencoders (for generative modelling)

Variational autoencoder

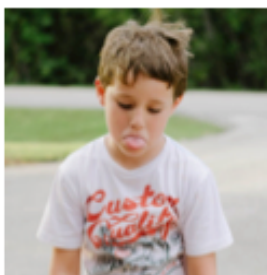
- Variational autoencoder was proposed in 2013 by Knigam and Welling at Google and Qualcomm.
- A variational autoencoder (VAE) provides a probabilistic manner for describing an observation in latent space.
- Thus, rather than building an encoder that outputs a single value to describe each latent state attribute, we'll formulate our encoder to describe a probability distribution for each latent attribute.
- It has many applications such as data compression, synthetic data creation etc.



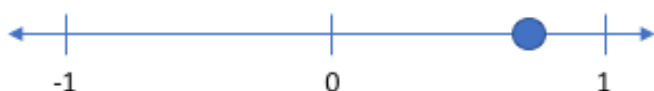
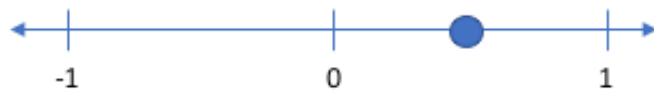
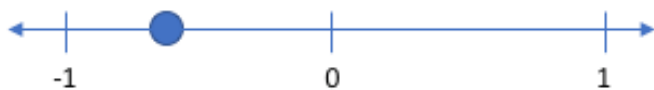
Smile: 0.99
Skin tone: 0.85
Gender: -0.73
Beard: 0.85
Glasses: 0.002
Hair color: 0.68



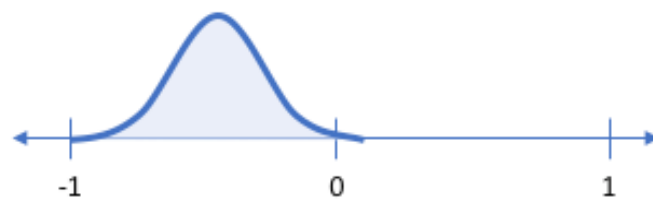
Latent attributes



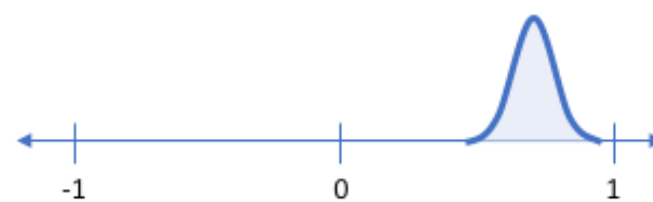
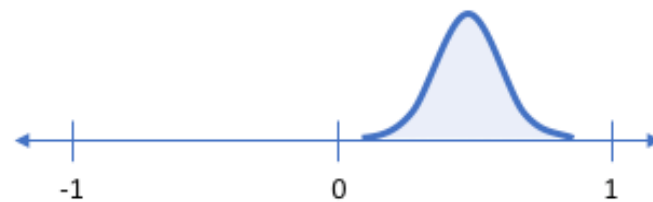
Smile (discrete value)

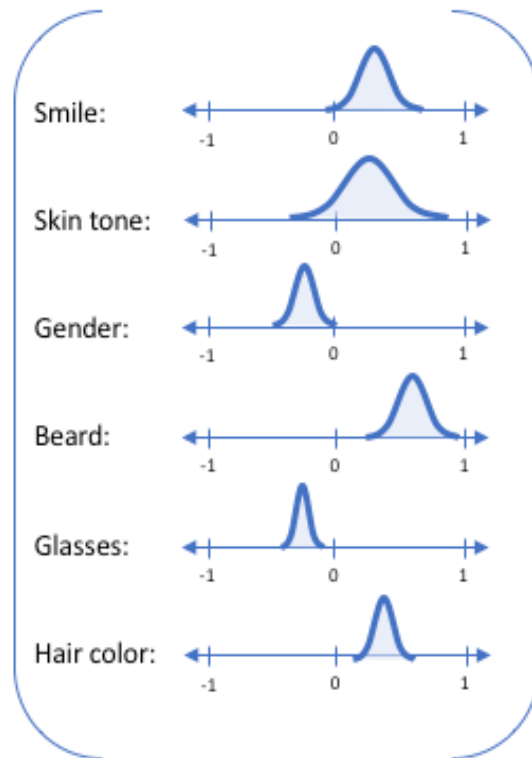
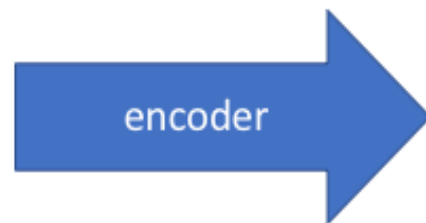


Smile (probability distribution)

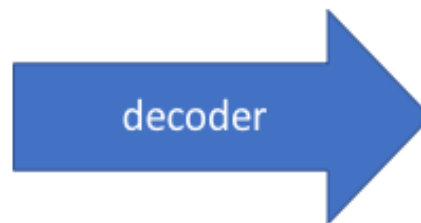


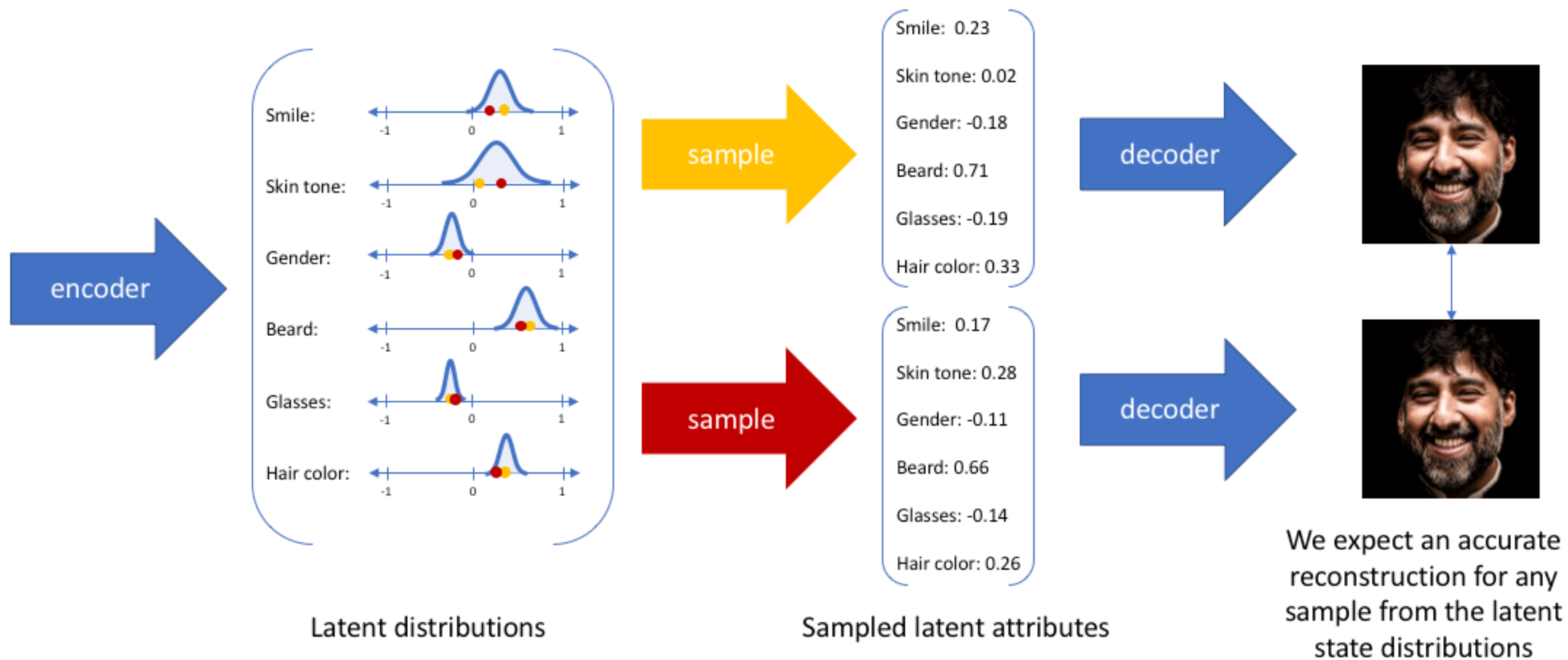
VS.

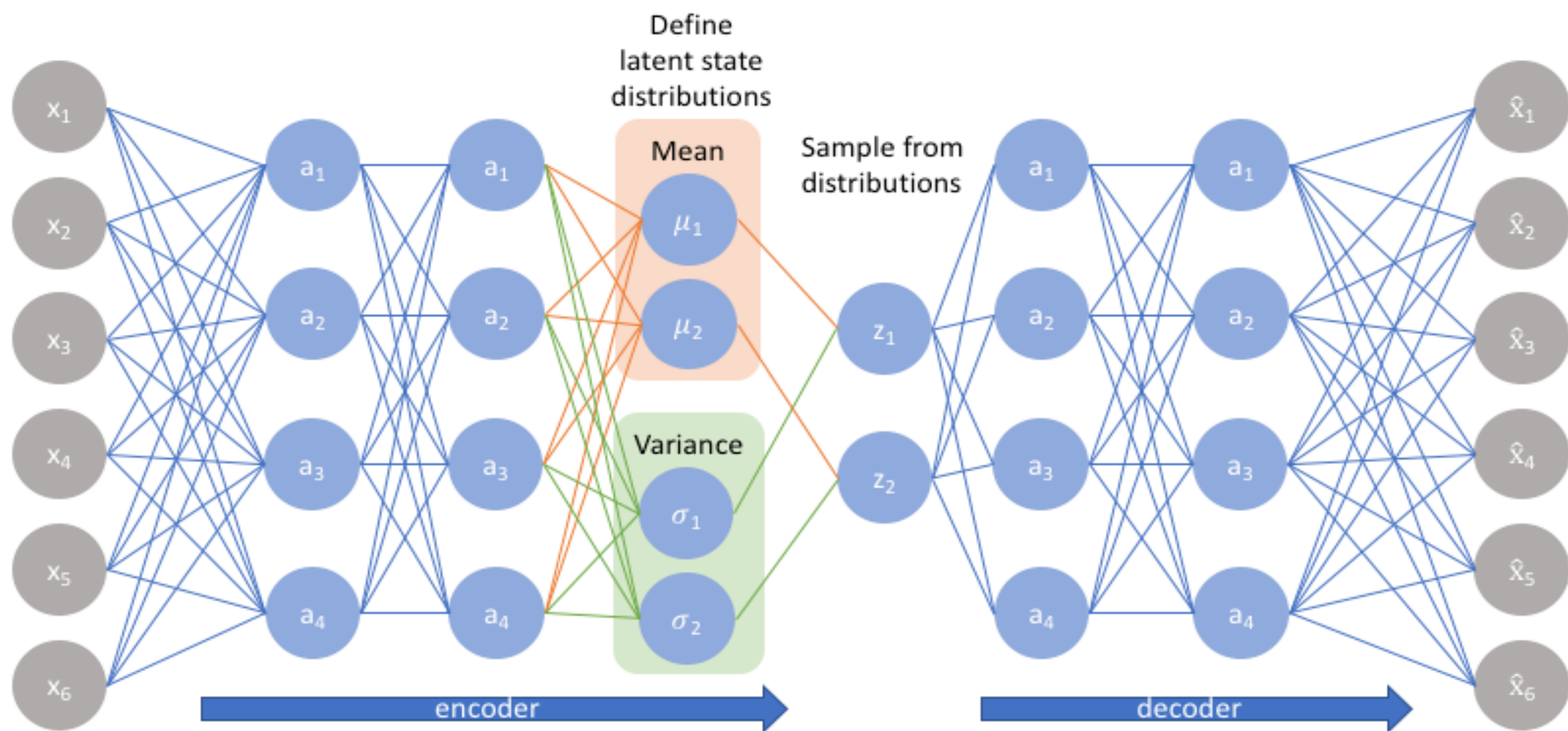


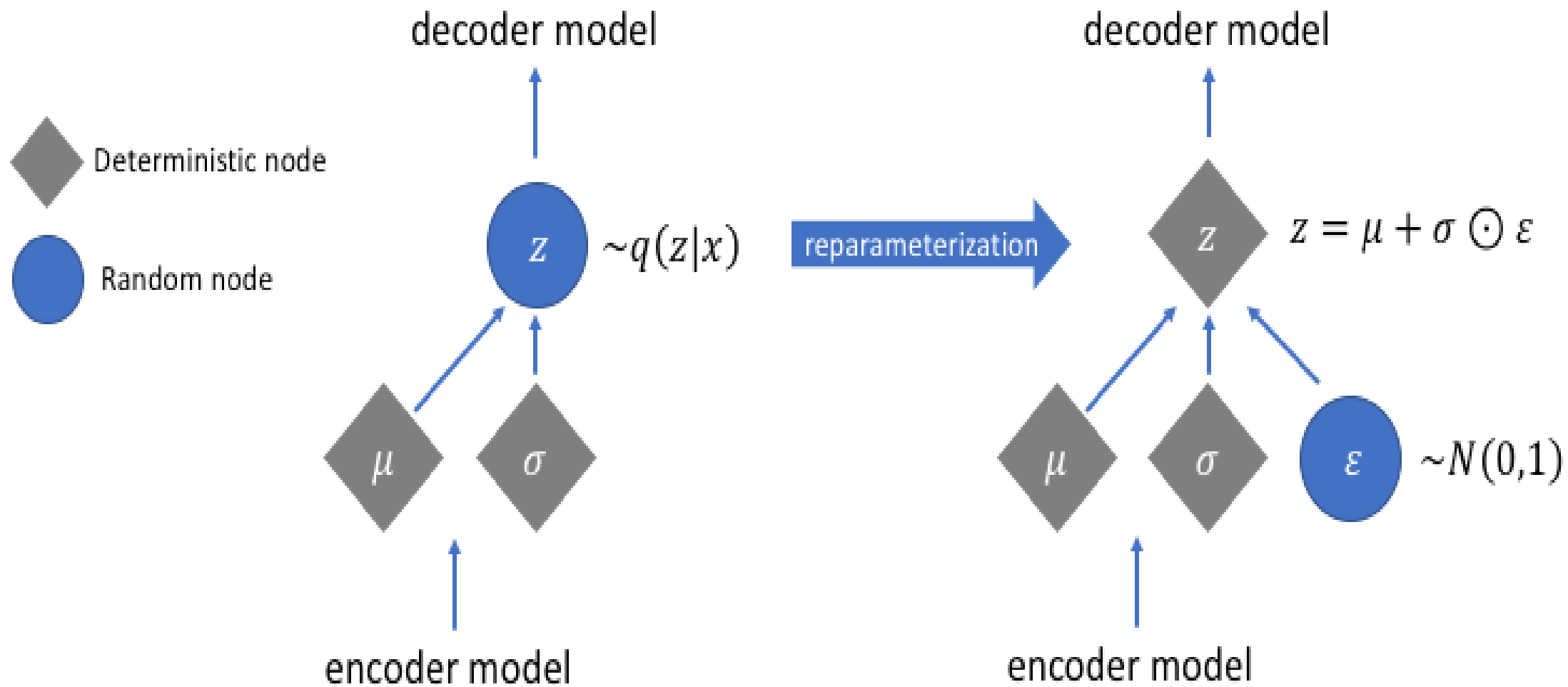


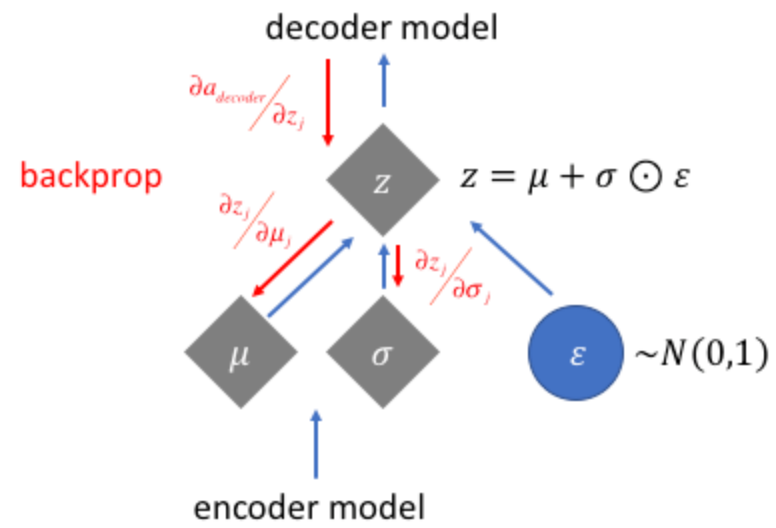
Latent attributes











Generative Adversarial Network

- Generative Adversarial Networks (GANs) are a powerful class of neural networks that are used for [unsupervised learning](#).
- It was developed and introduced by Ian J. Goodfellow in 2014.
- GANs are basically made up of a system of two competing neural network models which compete with each other and are able to analyze, capture and copy the variations within a dataset.

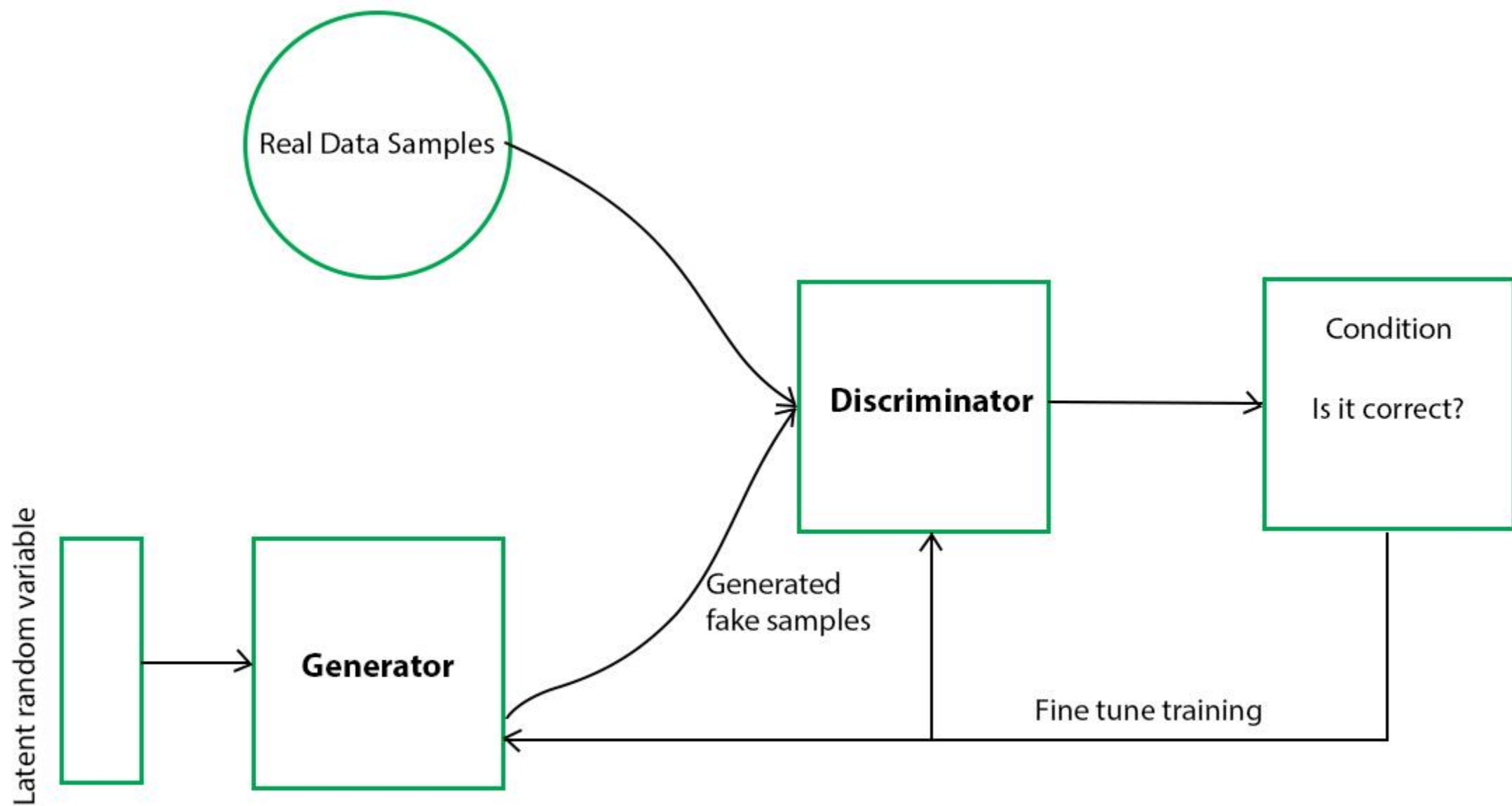
Why were GANs developed in the first place?

- It has been noticed most of the mainstream neural nets can be easily fooled into misclassifying things by adding only a small amount of noise into the original data.
- Surprisingly, the model after adding noise has higher confidence in the wrong prediction than when it predicted correctly.
- The reason for such an adversary is that most machine learning models learn from a limited amount of data, which is a huge drawback, as it is prone to [overfitting](#).
- Also, the mapping between the input and the output is almost linear.
- Although, it may seem that the boundaries of separation between the various classes are linear, but in reality, they are composed of linearities, and even a small change in a point in the feature space might lead to the misclassification of data.

How do GANs work?

- Generative Adversarial Networks (GANs) can be broken down into three parts:
- **Generative:** To learn a generative model, which describes how data is generated in terms of a probabilistic model.
- **Adversarial:** The training of a model is done in an adversarial setting.
- **Networks:** Use deep neural networks as artificial intelligence (AI) algorithms for training purposes.

- In GANs, there is a **Generator** and a **Discriminator**.
- The Generator generates fake samples of data (be it an image, audio, etc.) and tries to fool the Discriminator.
- The Discriminator, on the other hand, tries to distinguish between the real and fake samples.
- The Generator and the Discriminator are both [Neural Networks](#) and they both run in competition with each other in the training phase.
- The steps are repeated several times and in this, the Generator and Discriminator get better and better in their respective jobs after each repetition.



$$\min_G \max_D V(D, G)$$

$$V(D, G) = \mathbb{E}_{x \sim p_{data}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))]$$

where,

- G = Generator
- D = Discriminator
- $P_{data}(x)$ = distribution of real data
- $P(z)$ = distribution of generator
- x = sample from $P_{data}(x)$
- z = sample from $P(z)$
- $D(x)$ = Discriminator network
- $G(z)$ = Generator network

Generator Model

The Generator is trained while the Discriminator is idle.

After the Discriminator is trained by the generated fake data of the Generator, we can get its predictions and use the results for training the Generator and get better from the previous state to try and fool the Discriminator.

Discriminator Model

The Discriminator is trained while the Generator is idle.

In this phase, the network is only forward propagated and no back-propagation is done.

The Discriminator is trained on real data for n epochs and sees if it can correctly predict them as real.

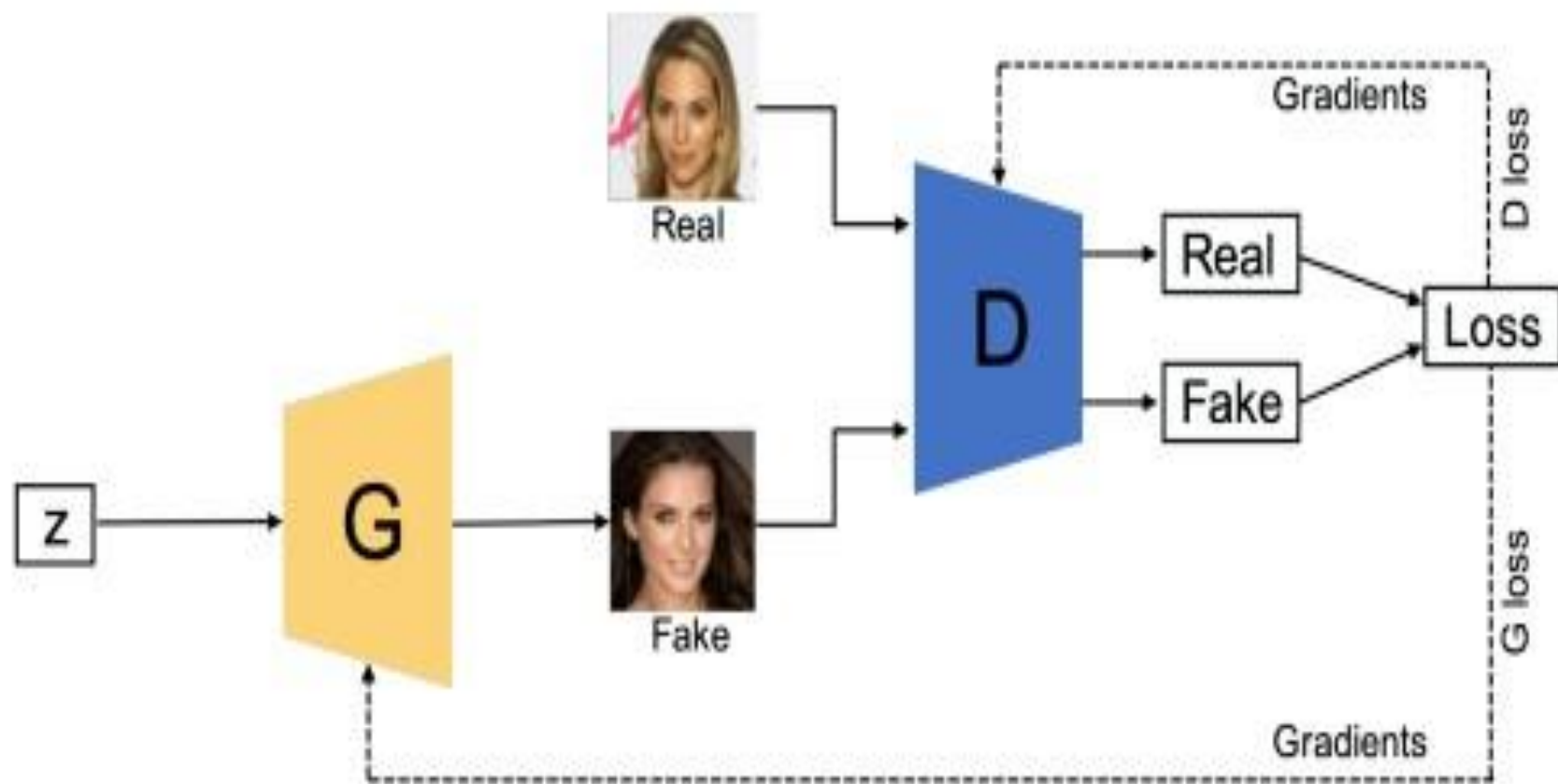
Also, in this phase, the Discriminator is also trained on the fake generated data from the Generator and see if it can correctly predict them as fake.

Different Types of GAN Models

- Vanilla GAN
- Conditional Gan (CGAN)
- Deep Convolutional GAN (DCGAN)
- CycleGAN
- Generative Adversarial Text to Image Synthesis
- Style GAN
- Super Resolution GAN (SRGAN)

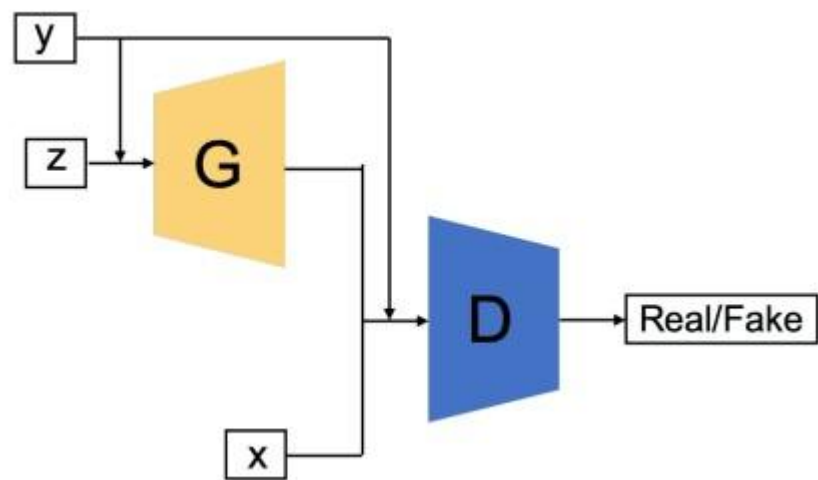
Vanilla GAN

- The Vanilla GAN is the simplest type of GAN made up of the generator and discriminator , where the classification and generation of images is done by the generator and discriminator internally with the use of multi layer perceptrons.
- The generator captures the data distribution meanwhile , the discriminator tries to find the probability of the input belonging to a certain class, finally the feedback is sent to both the generator and discriminator after calculating the loss function , and hence the effort to minimize the loss comes into picture.



Conditional Gan (CGAN)

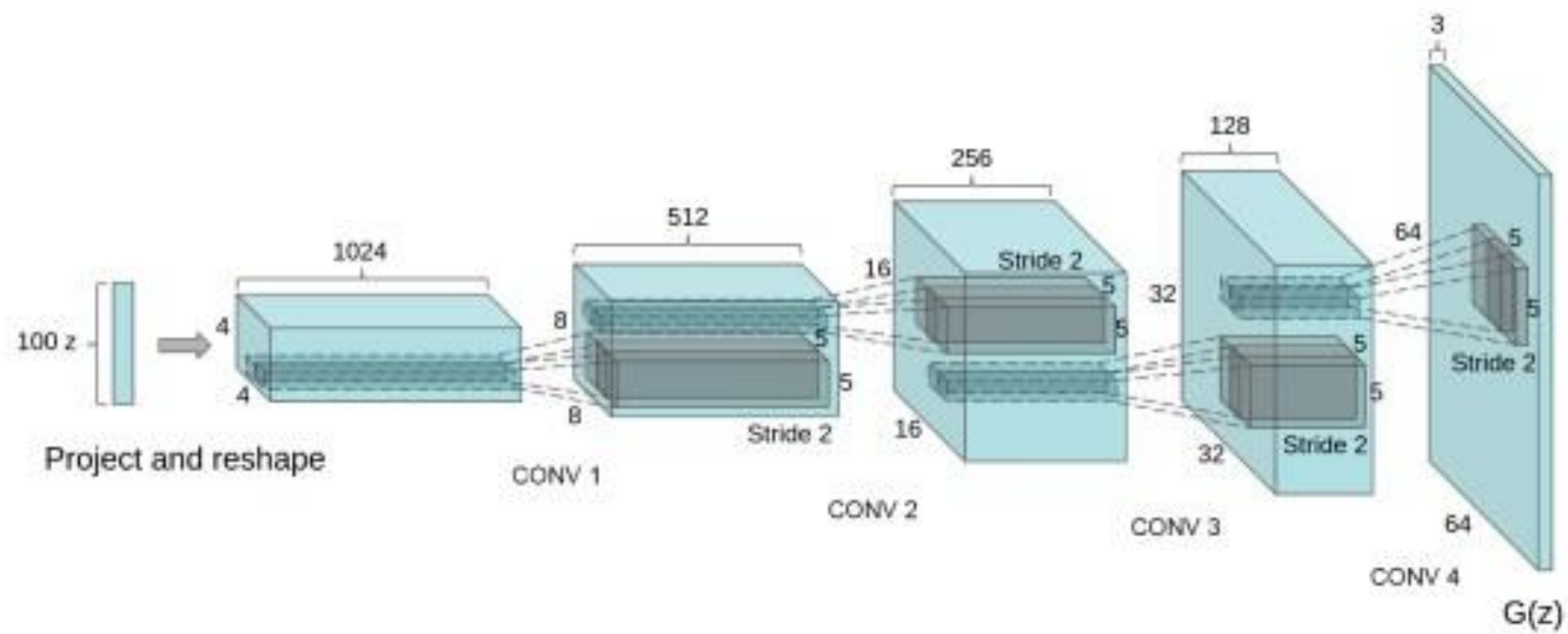
- In this GAN the generator and discriminator both are provided with additional information that could be a class label or any modal data.
- As the name suggests the additional information helps the discriminator in finding the conditional probability instead of the joint probability.



$$\min_G \max_D \mathbb{E}_{\mathbf{x} \sim p_r} \log [D(\mathbf{x}|\mathbf{y})] + \mathbb{E}_{\mathbf{z} \sim p_z} \log [1 - D(G(\mathbf{z}|\mathbf{y}))].$$

Deep Convolutional GAN (DCGAN)

- This is the first GAN where the generator used deep convolutional network , hence generating high resolution and quality images to be differentiated.
- ReLU activation is used in Generator all layers except last one where Tanh activation is used, meanwhile in Discriminator all layers use the Leaky-ReLu activation function.
- Adam optimizer is used with a learning rate of 0.0002.



CycleGAN

- This GAN is made for Image-to-Image translations, meaning one image to be mapped with another image.
- For example , if summer and winter are made to undergo the process of Image-Image translation we find a mapping function that could convert summer images into that of winter images and vice versa by adding or removing features according to the mapping function,such that the predicted output and actual output have minimized loss.



winter Yosemite → summer Yosemite



summer Yosemite → winter Yosemite

Generative Adversarial Text to Image Synthesis

- In this the GANs are capable of finding an image from the dataset that is closest to the text description and generate similar images.

this small bird has a pink breast and crown, and black primaries and secondaries.



this magnificent fellow is almost all black with a red crest, and white cheek patch.



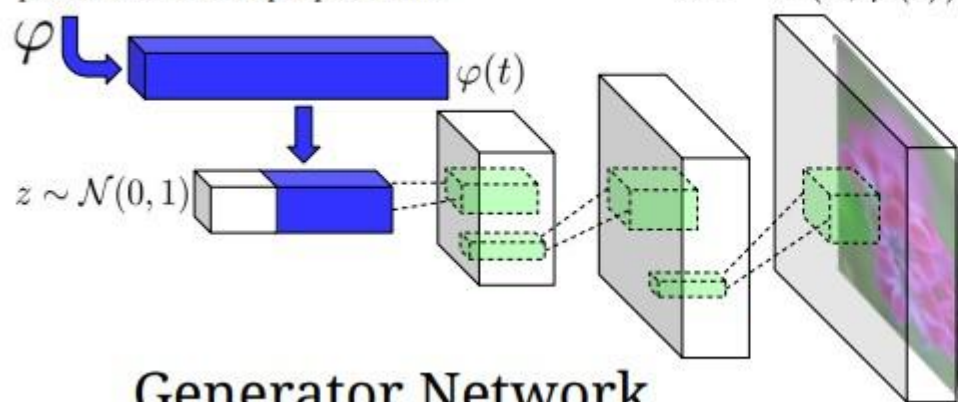
the flower has petals that are bright pinkish purple with white stigma



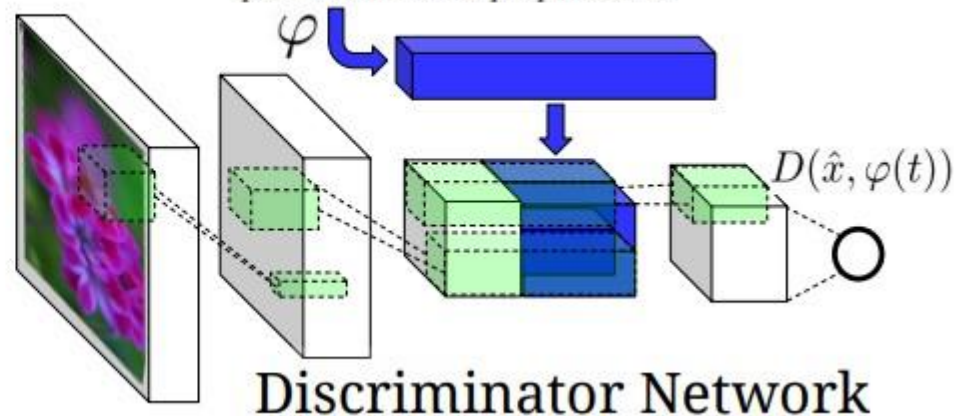
this white and yellow flower have thin white petals and a round yellow stamen



This flower has small, round violet petals with a dark purple center



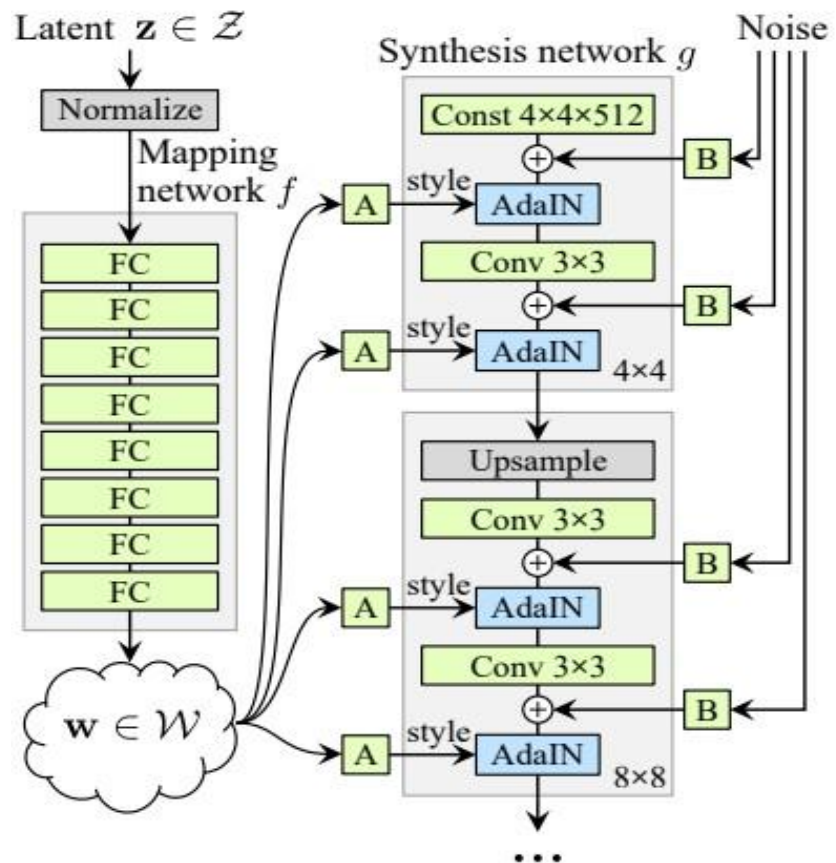
This flower has small, round violet petals with a dark purple center



Style GAN

- Other GANs focused on improving the discriminator in this case we improve the generator.
- This GAN generates by taking a reference picture.

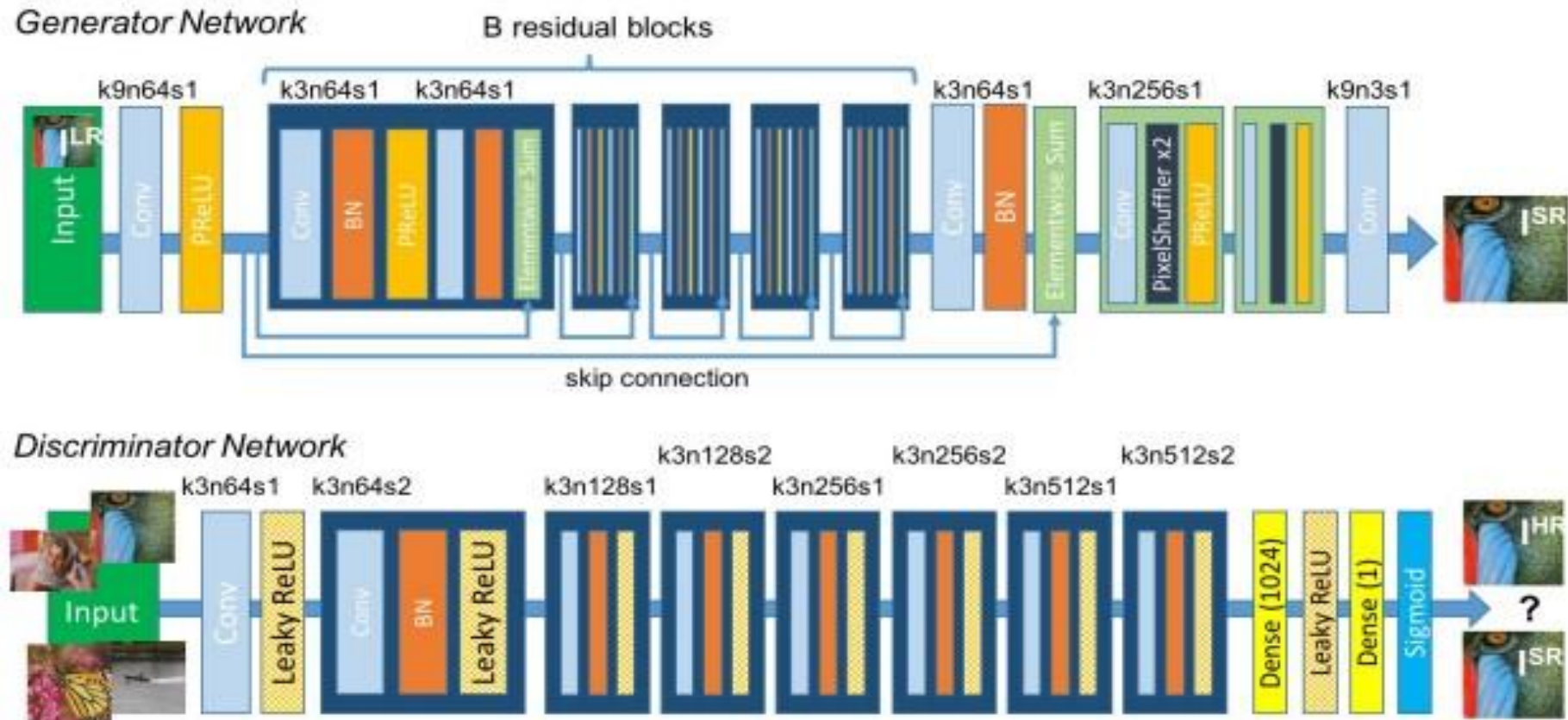




$$\text{AdaIN}(\mathbf{x}_i, \mathbf{y}) = y_{s,i} \frac{\mathbf{x}_i - \mu(\mathbf{x}_i)}{\sigma(\mathbf{x}_i)} + y_{b,i},$$

Super Resolution GAN (SRGAN)- The main purpose of this type of GAN is to make a low resolution picture into a more detailed picture.

This is one of the most researched problems in Computer vision. The architecture of the SRGAN is given below.



$$\hat{\theta}_G = \arg \min_{\theta_G} \frac{1}{N} \sum_{n=1}^N l^{SR}(G_{\theta_G}(I_n^{LR}), I_n^{HR})$$

$$l^{SR} = \underbrace{l_X^{SR}}_{\text{content loss}} + \underbrace{10^{-3} l_{Gen}^{SR}}_{\text{adversarial loss}}$$

perceptual loss (for VGG based content losses)

Applications:

1. Firstly, GANs can be used as data augmentation techniques where the generator generates new images taking the training dataset and producing multiple images by applying some changes.
2. GANs improve the classification techniques by training the discriminator on a very large scale of data that are real as well as fake.
3. GANs are also used to improve the resolution of any input image.
4. Just like filters used in Snapchat, a filter could be applied to see what a place might look like if in summer, winter, spring or autumn and many more conditions could be applied and hence that's where Deep ConvNets play their role.
5. GANs are used to convert semantics into images and better understand the visualizations done by the machine.

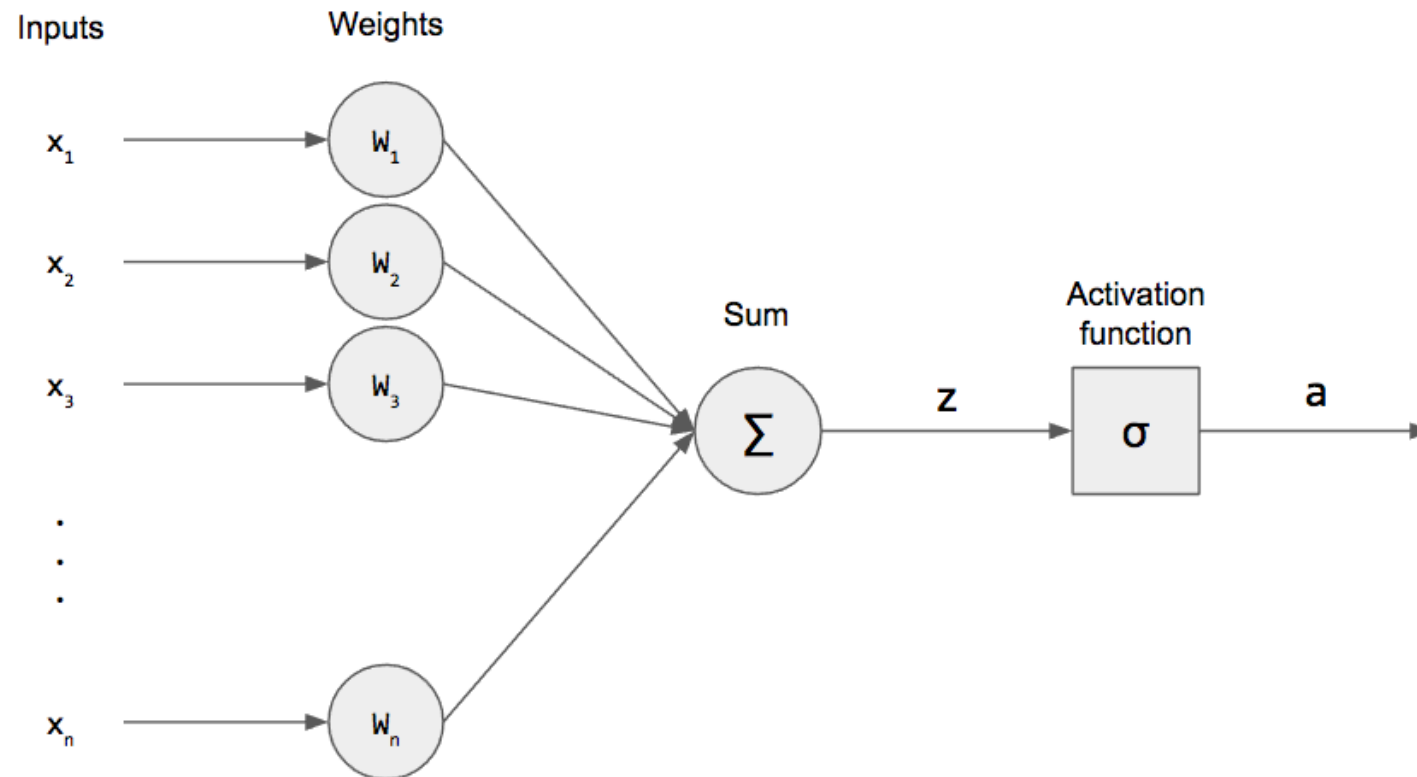
Advantages of Generative Adversarial Networks (GANs):

- 1.Synthetic data generation:** GANs can generate new, synthetic data that resembles some known data distribution, which can be useful for data augmentation, anomaly detection, or creative applications.
- 2.High-quality results:** GANs can produce high-quality, photorealistic results in image synthesis, video synthesis, music synthesis, and other tasks.
- 3.Unsupervised learning:** GANs can be trained without labeled data, making them suitable for unsupervised learning tasks, where labeled data is scarce or difficult to obtain.
- 4.Versatility:** GANs can be applied to a wide range of tasks, including image synthesis, text-to-image synthesis, image-to-image translation, [anomaly detection](#), [data augmentation](#), and others.

Disadvantages of Generative Adversarial Networks (GANs):

- 1.Training Instability:** GANs can be difficult to train, with the risk of instability, mode collapse, or failure to converge.
- 2.Computational Cost:** GANs can require a lot of computational resources and can be slow to train, especially for high-resolution images or large datasets.
- 3.Overfitting:** GANs can overfit the training data, producing synthetic data that is too similar to the training data and lacking diversity.
- 4.Bias and Fairness:** GANs can reflect the biases and unfairness present in the training data, leading to discriminatory or biased synthetic data.
- 5.Interpretability and Accountability:** GANs can be opaque and difficult to interpret or explain, making it challenging to ensure accountability, transparency, or fairness in their applications.

Neural Representation of AND, OR, NOT, XOR and XNOR Logic Gates (Perceptron Algorithm)



Prediction (y') = 1 if $Wx+b > 0$ and 0 if $Wx+b \leq 0$

Also, the steps in this method are very similar to how Neural Networks learn, which is as follows;

- Initialize weight values and bias
- Forward Propagate
- Check the error
- Backpropagate and Adjust weights and bias
- Repeat for all training examples

Truth Table of AND Logical GATE is,

A	B	$A \wedge B$
0	0	0
0	1	0
1	0	0
1	1	1

Weights $w_1 = 1.2$, $w_2 = 0.6$, Threshold = 1 and Learning Rate $n = 0.5$ are given

For Training Instance 1: $A=0$, $B=0$ and Target = 0

$$w_i.x_i = 0*1.2 + 0*0.6 = 0$$

This is not greater than the threshold of 1, so the output = 0, Here the target is same as calculated output.

For Training Instance 2: $A=0$, $B=1$ and Target = 0

$$w_i.x_i = 0*1.2 + 1*0.6 = 0.6$$

This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

For Training Instance 3: $A=1$, $B=0$ and Target = 0

$$w_i.x_i = 1*1.2 + 0*0.6 = 1.2$$

This is greater than the threshold of 1, so the output = 1. Here the target does not match with the calculated output. Hence we need to update the weights.

$$w_i = w_i + n(t - o)x_i$$

$$w_1 = 1.2 + 0.5(0 - 1)1 = 0.7$$

$$w_2 = 0.6 + 0.5(0 - 1)0 = 0.6$$

After updating weights are $w_1 = 0.7$, $w_2 = 0.6$ Threshold = 1 and Learning Rate $n = 0.5$

$w_1 = 0.7$, $w_2 = 0.6$ Threshold = 1 and Learning Rate $n = 0.5$

For Training Instance 1: $A=0$, $B=0$ and Target = 0

$$w_i.x_i = 0*0.7 + 0*0.6 = 0$$

This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

For Training Instance 2: $A=0$, $B=1$ and Target = 0

$$w_i.x_i = 0*0.7 + 1*0.6 = 0.6$$

This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

For Training Instance 3: $A=1$, $B=0$ and Target = 0

$$w_i.x_i = 1*0.7 + 0*0.6 = 0.7$$

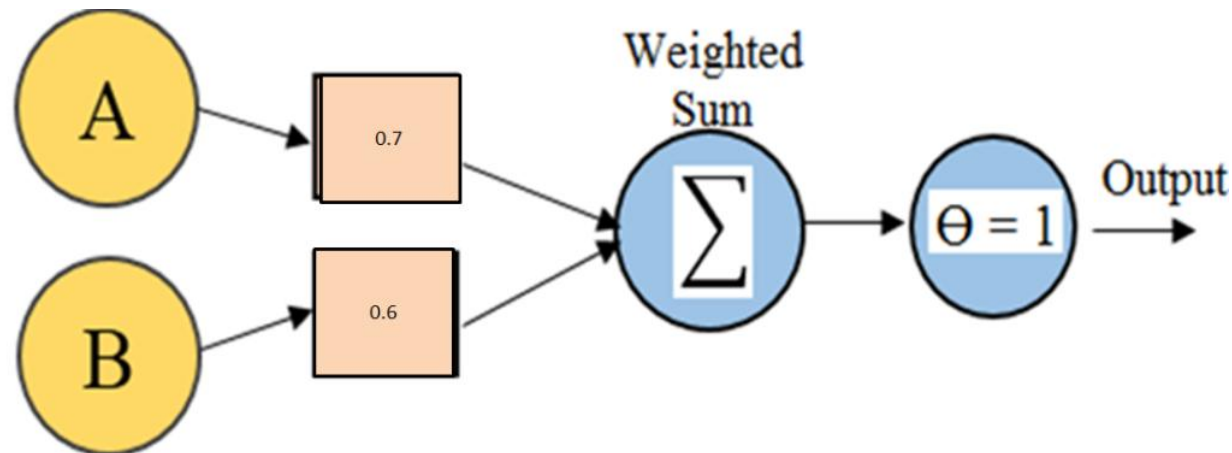
This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

For Training Instance 4: $A=1$, $B=1$ and Target = 1

$$w_i.x_i = 1*0.7 + 1*0.6 = 1.3$$

This is greater than the threshold of 1, so the output = 1. Here the target is same as calculated output.

Hence the final weights are $w_1 = 0.7$ and $w_2 = 0.6$, Threshold = 1 and Learning Rate $n = 0.5$.



Weights $w_1 = 0.6$, $w_2 = 1.1$, Threshold = 1 and Learning Rate $\eta = 0.5$ are given

For Training Instance 1: $A=0$, $B=0$ and Target = 0

$$w_i.x_i = 0*0.6 + 0*1.1 = 0$$

This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

For Training Instance 2: $A=0$, $B=1$ and Target = 1

$$w_i.x_i = 0*0.6 + 1*1.1 = 1.1$$

This is not greater than the threshold of 1, so the output = 0. Here the target is same as calculated output.

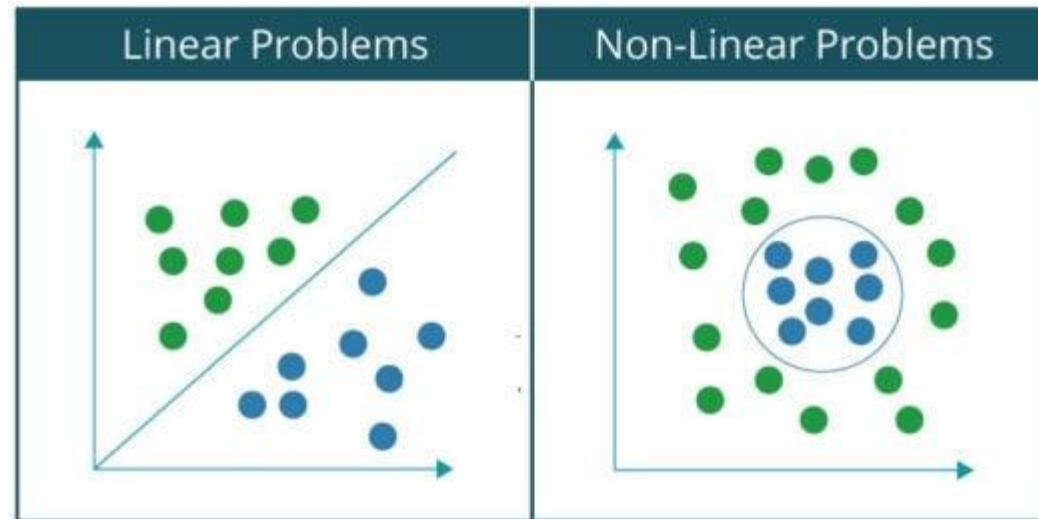
For Training Instance 3: $A=1$, $B=0$ and Target = 1

$$w_i.x_i = 1*0.6 + 0*1.1 = 0.6$$

This is not greater than the threshold of 1, so the output = 0. Here the target does not match with calculated output.

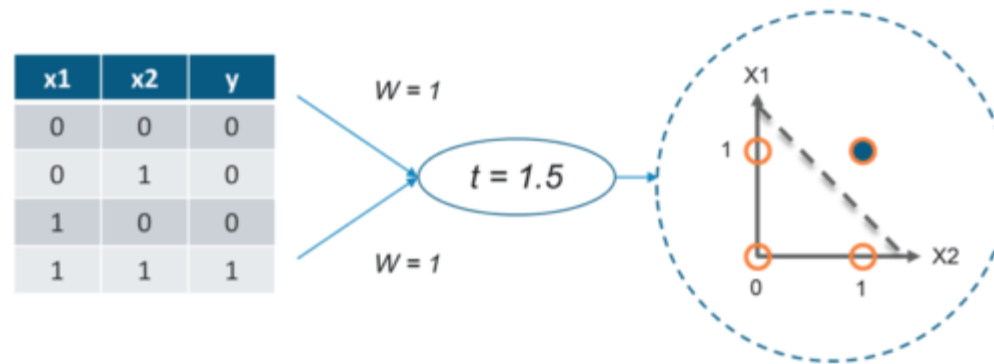
Deep Learning : Perceptron Learning Algorithm

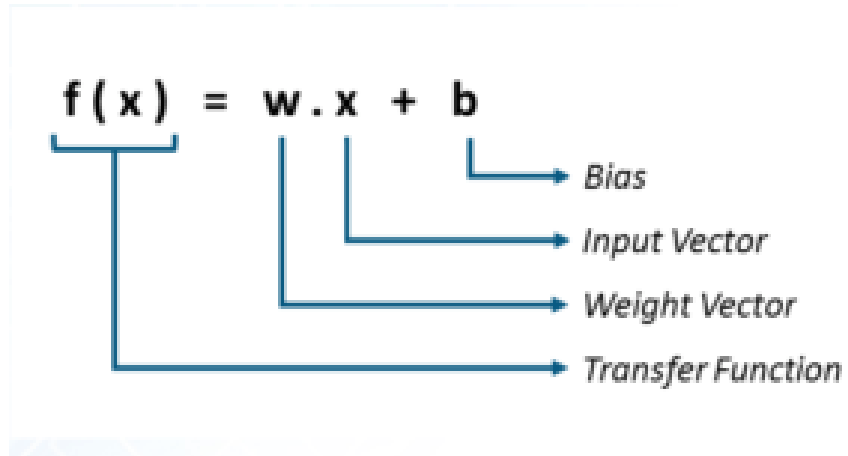
- **Types of Classification Problems**
 - **Linearly Separable Problems**
 - **Non-Linearly Separable Problems**



Perceptron as AND Gate

- AND gate produces an output as 1 if both the inputs are 1 and 0 in all other cases. Therefore, a perceptron can be used as a separator or a decision line that divides the input set of AND Gate, into two classes:
 - **Class 1:** Inputs having output as 0 that lies below the decision line.
 - **Class 2:** Inputs having output as 1 that lies above the decision line or separator.





- Each of the input received by the perceptron has been weighted based on the amount of its contribution for obtaining the final output.
- Bias allows us to shift the decision line so that it can best separate the inputs into two classes.

Prediction (y') = 1 if $Wx+b > 0$ and 0 if $Wx+b \leq 0$

First, we need to understand that the output of an AND gate is 1 only if both inputs (in this case, x_1 and x_2) are 1. So, following the steps listed above;

Row 1

- From $w_1*x_1+w_2*x_2+b$, initializing w_1 , w_2 , as 1 and b as -1 , we get;

$$x_1(1)+x_2(1)-1$$

- Passing the first row of the AND logic table ($x_1=0$, $x_2=0$), we get;

$$0+0-1 = -1$$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. Therefore, this row is correct, and no need for Backpropagation.

Row 2

- Passing ($x_1=0$ and $x_2=1$), we get;

$$0+1-1 = 0$$

- From the Perceptron rule, if $Wx+b \leq 0$, then $y'=0$. This row is correct, as the output is 0 for the AND gate.

- From the Perceptron rule, this works (for both row 1, row 2 and 3).

Row 4

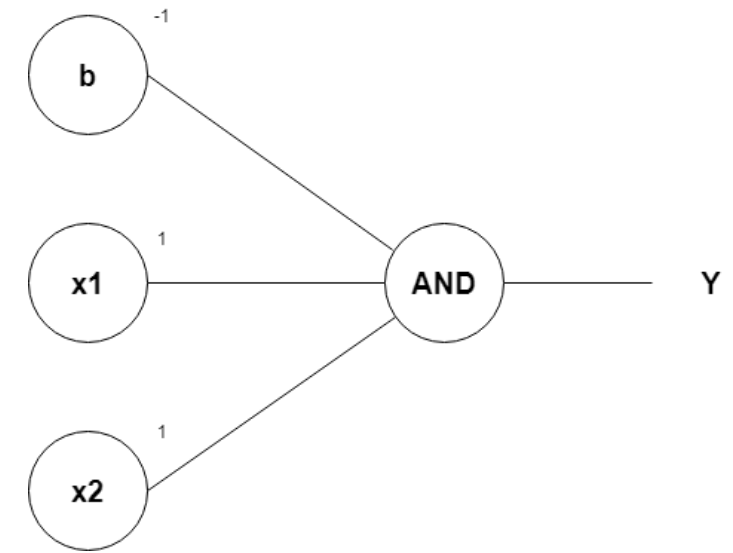
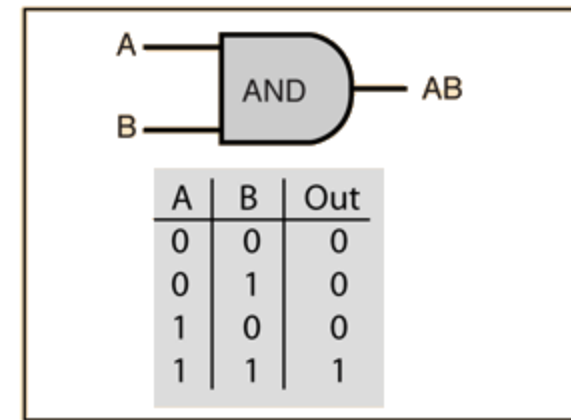
- Passing ($x_1=1$ and $x_2=1$), we get;

$$1+1-1 = 1$$

- Again, from the perceptron rule, this is still valid.

Therefore, we can conclude that the model to achieve an AND gate, using the Perceptron algorithm is;

$$x_1+x_2-1$$



Gradient Descent algorithm and its variants

- **Gradient descent** (GD) is an iterative first-order optimization algorithm used to find a local minimum/maximum of a given function.
- This method is commonly used in *machine learning* (ML) and *deep learning*(DL) to minimize a cost/loss function (e.g. in a linear regression).
- Due to its importance and ease of implementation, this algorithm is usually taught at the beginning of almost all machine learning courses.
- It works by iteratively adjusting the weights or parameters of the model in the direction of the negative gradient of the cost function, until the minimum of the cost function is reached.
- However, its use is not limited to ML/DL only, it's being widely used also in areas like:
 - control engineering (robotics, chemical, etc.)
 - computer games
 - mechanical engineering

Several variants of gradient descent

- 1. Stochastic Gradient Descent (SGD):** In SGD, only one training example is used to compute the gradient and update the parameters at each iteration. This can be faster than batch gradient descent but may lead to more noise in the updates.
- 2. Mini-batch Gradient Descent:** In mini-batch gradient descent, a small batch of training examples is used to compute the gradient and update the parameters at each iteration. This can be a good compromise between batch gradient descent and SGD, as it can be faster than batch gradient descent and less noisy than SGD.
- 3. Momentum-based Gradient Descent:** In momentum-based gradient descent, a momentum term is added to the gradient update to help accelerate convergence and smooth out the update process. This can help prevent the optimization process from getting stuck in local minima.

Several variants of gradient descent

4. Adaptive Learning Rate Methods: Adaptive learning rate methods like Adagrad, RMSProp, and Adam adjust the learning rate based on the past gradients to improve convergence and avoid oscillations. These methods are particularly useful in problems with sparse gradients or in deep learning where the gradients may vary significantly across different layers.
5. Conjugate Gradient Descent: Conjugate Gradient Descent (CGD) is a variant of GD that can be more efficient in solving large-scale optimization problems with a symmetric and positive definite matrix. CGD requires less memory and fewer iterations than other GD methods and can converge faster.

Advantages of gradient descent and its variants

- 1. Widely used:** Gradient descent and its variants are widely used in machine learning and optimization problems because they are effective and easy to implement.
- 2. Convergence:** Gradient descent and its variants can converge to a global minimum or a good local minimum of the cost function, depending on the problem and the variant used.
- 3. Scalability:** Many variants of gradient descent can be parallelized and are scalable to large datasets and high-dimensional models.
- 4. Flexibility:** Different variants of gradient descent offer a range of trade-offs between accuracy and speed, and can be adjusted to optimize the performance of a specific problem.

Disadvantages of gradient descent and its variants:

- 1.Choice of learning rate:** The choice of learning rate is crucial for the convergence of gradient descent and its variants. Choosing a learning rate that is too large can lead to oscillations or overshooting, while choosing a learning rate that is too small can lead to slow convergence or getting stuck in local minima.
- 2.Sensitivity to initialization:** Gradient descent and its variants can be sensitive to the initialization of the model's parameters, which can affect the convergence and the quality of the solution.

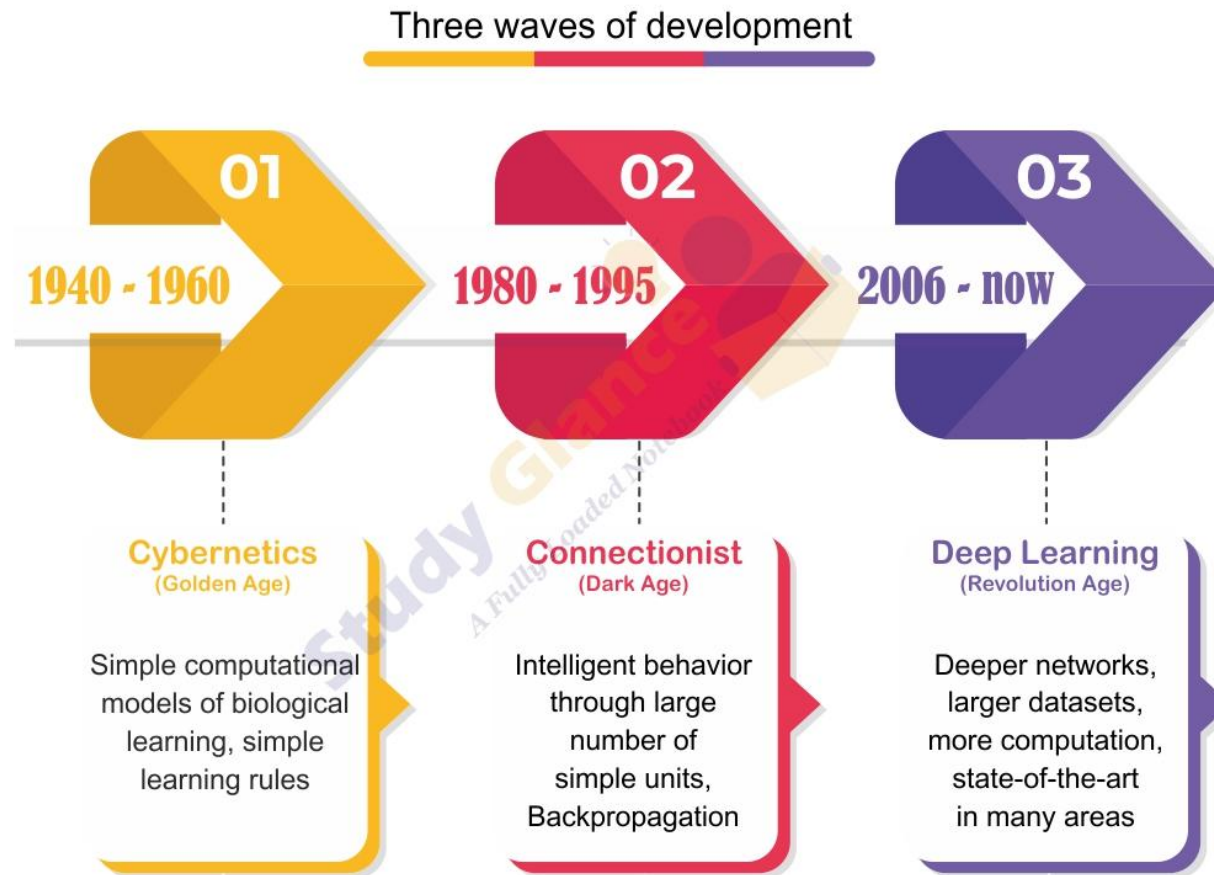
Disadvantages of gradient descent and its variants:

- 3. Time-consuming:** Gradient descent and its variants can be time-consuming, especially when dealing with large datasets and high-dimensional models. The convergence speed can also vary depending on the variant used and the specific problem.
- 4. Local optima:** Gradient descent and its variants can converge to a local minimum instead of the global minimum of the cost function, especially in non-convex problems. This can affect the quality of the solution, and techniques like random initialization and multiple restarts may be used to mitigate this issue.

An aircraft emergency locator transmitter (ELT) is a device designed to transmit a signal in the case of a crash. The Altigauge Manufacturing Company makes 80% of the ELTs, the Bryant Company makes 15% of them, and the Chartair Company makes the other 5%. The ELTs made by Altigauge have a 4% rate of defects, the Bryant ELTs have a 6% rate of defects, and the Chartair ELTs have a 9% rate of defects (which helps to explain why Chartair has the lowest market share).

- a. If an ELT is randomly selected from the general population of all ELTs, find the probability that it was made by the Altigauge Manufacturing Company.
- b. b. If a randomly selected ELT is then tested and is found to be defective, find the probability that it was made by the Altigauge Manufacturing Company.

3 Waves of development



Linear Algebra in Machine Learning and Deep Learning

- Vector and Matrix.
- System of Linear Equations.
- Vector Space.
- Basis

Linear Algebra in Machine Learning and Deep Learning

- Derivation of Regression Line.
- Linear Equation to predict the target value.
- Support Vector Machine Classification (SVM).
- Dimensionality Reduction.
- Mean Square Error or Loss function.
- Regularization.
- Covariance Matrix.
- Convolution.

A vector product

$$xy^T \in \mathbb{R}^{m \times n} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \begin{bmatrix} y_1 & y_2 & \cdots & y_n \end{bmatrix} = \begin{bmatrix} x_1 y_1 & x_1 y_2 & \cdots & x_1 y_n \\ x_2 y_1 & x_2 y_2 & \cdots & x_2 y_n \\ \vdots & \vdots & \ddots & \vdots \\ x_m y_1 & x_m y_2 & \cdots & x_m y_n \end{bmatrix}$$

Vector

- In linear algebra, a vector is an $n \times 1$ matrix. It has only one column.

$$\begin{bmatrix} 460 \\ 430 \\ 480 \\ \dots \\ \dots \end{bmatrix}$$

Matrix

- A matrix is an essential part of linear algebra.
- It stores $m*n$ elements of data, and we use it for the computation of the linear equation system or the linear mappings.
- It is an $m*n$ tuple of real-value elements.

$$\begin{pmatrix} a_1 & a_2 \\ a_3 & a_4 \end{pmatrix} \quad \text{Where } a_{ij} \in \mathbb{R}$$

The number of rows and columns is called the dimension of the matrix.

Matrix Multiplication

- Matrix multiplication is a dot product of rows and columns where the row of the matrix is multiplied and summed up with another matrix column.

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} * \begin{pmatrix} 7 & 8 \\ 9 & 10 \\ 9 & 12 \end{pmatrix} = \begin{pmatrix} 52 & 64 \\ 127 & 154 \end{pmatrix}$$

Where,

$$[1, 2, 3] * [8, 10, 12] = 1 * 8 + 2 * 10 + 3 * 12 = 64$$

Linear Regression

- There are multiple features to predict the price of a house. Below is the table of different houses with their features and the target value (price).

Where,

x_i = features of a house

y = target variable

No. of Rooms (x_0)	Size (Square feet) (x_1)	Year Built (x_2)	No. of Floors (x_3)	Price (y)
2	1434	2010	1	8500
3	1534	2019	2	9600
2	962	1996	3	25880
...	...	...	...	...
...	...	...	...	...

Scalar

Vector

Matrix

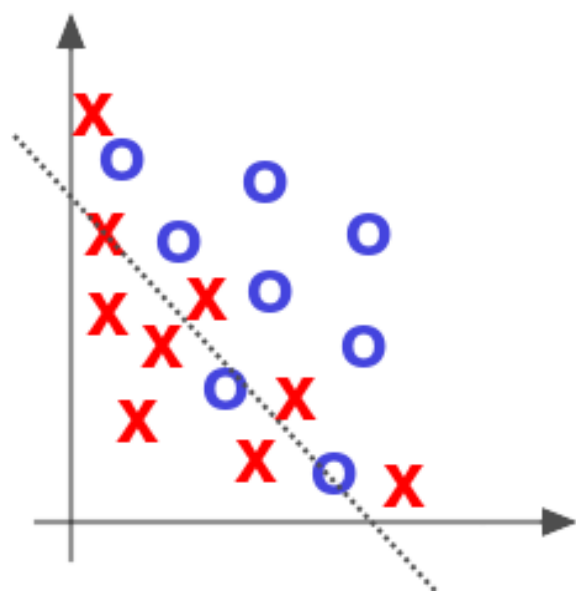
Tensor

1

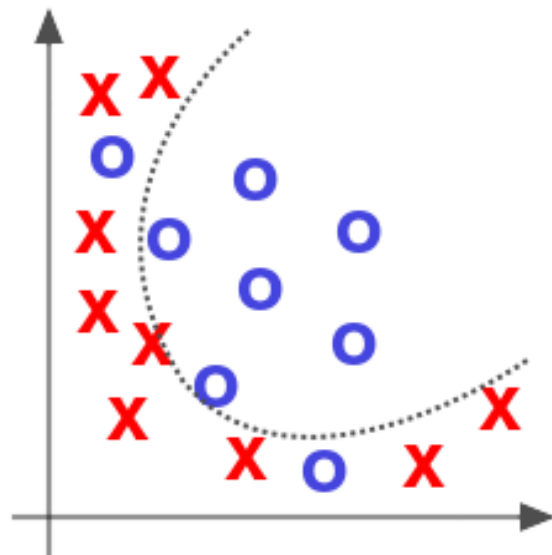
$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

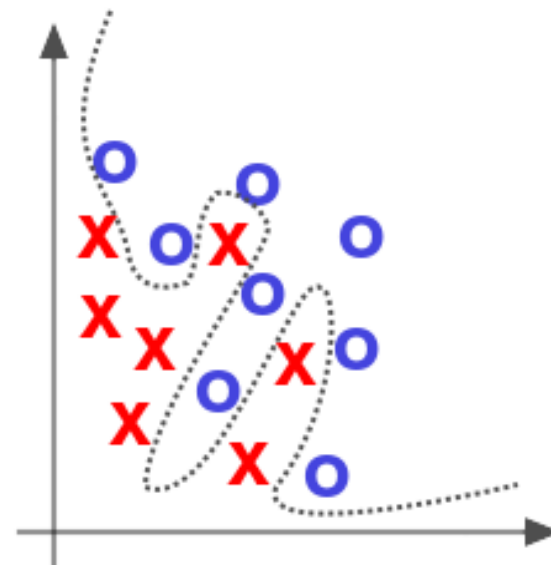
$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$



Under Fit



Appropriate

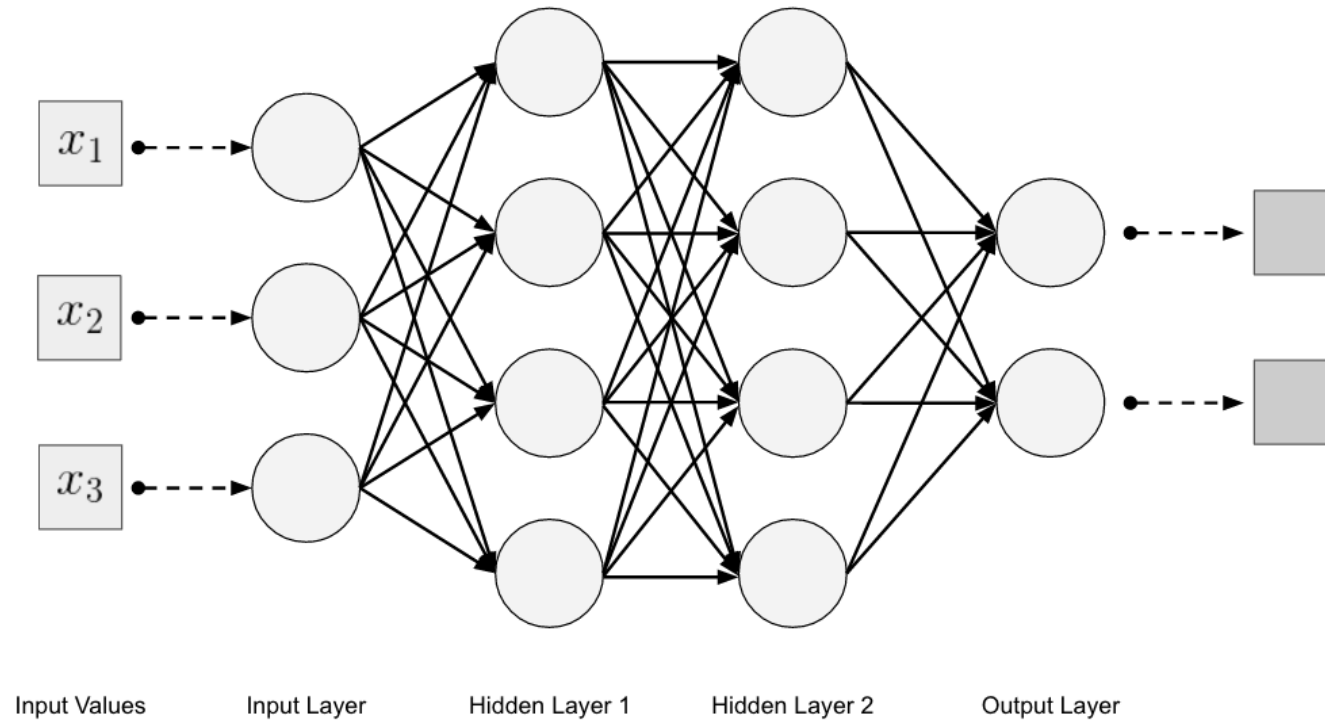


Over Fit

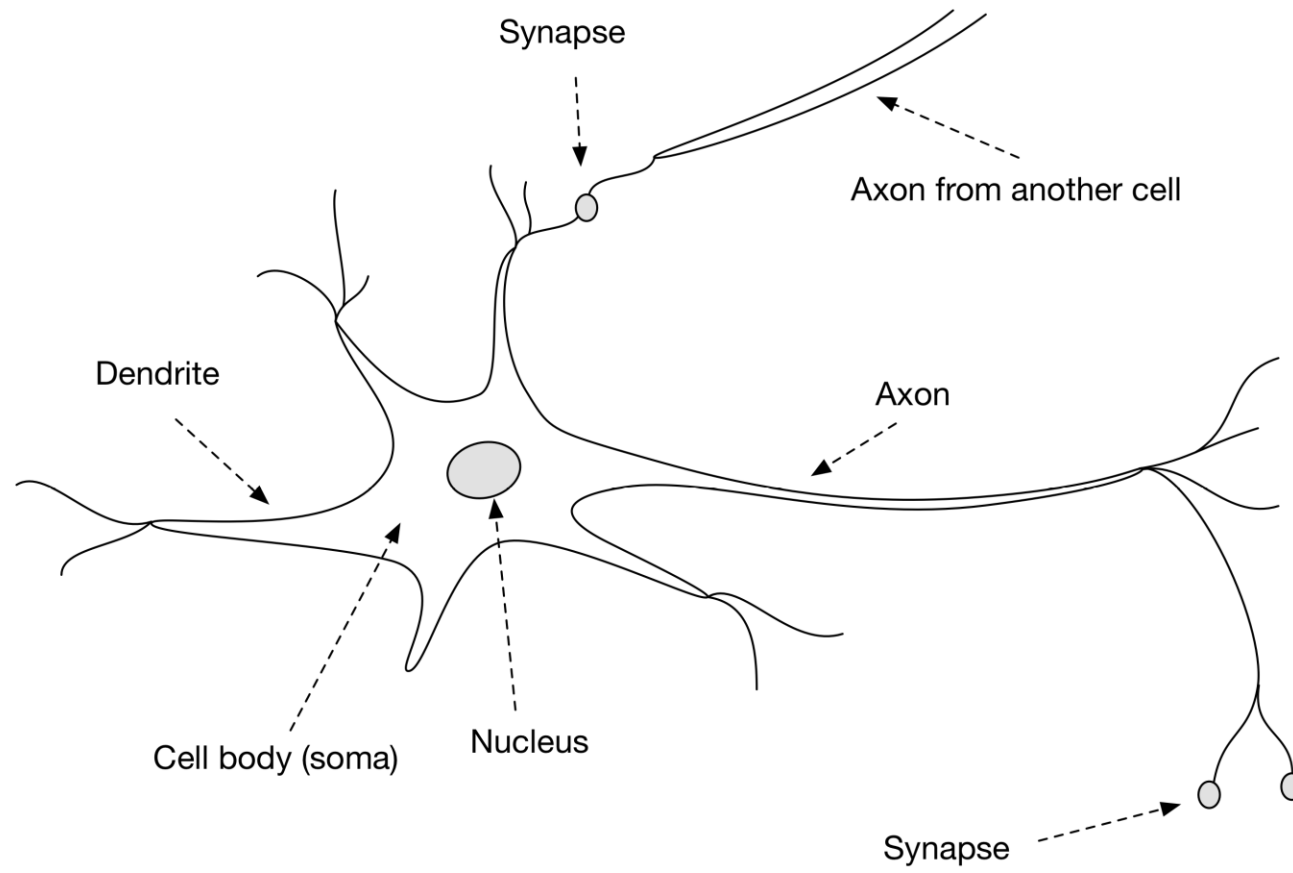
	P' (Predicted)	N' (Predicted)
P (Actual)	True Positive	False Negative
N (Actual)	False Positive	True Negative

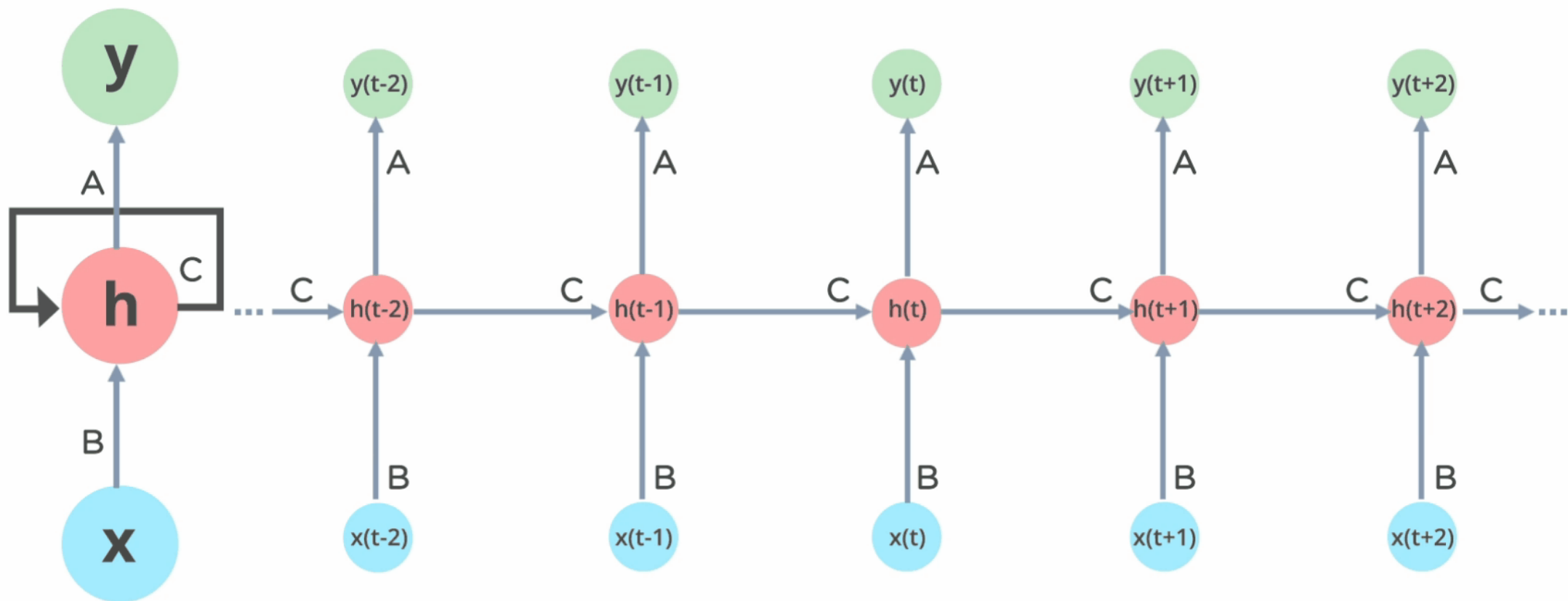
- True positives
 - — Positive prediction
 - — Label was positive
- False positives
 - — Positive prediction
 - — Label was negative
- True negatives
 - — Negative prediction
 - — Label was negative
- False negatives
 - — Negative prediction
 - — Label was positive

Multilayer neural network topology



The biological neuron





0	0	0	0	0	0	...
0	156	155	156	158	158	...
0	153	154	157	159	159	...
0	149	151	155	158	159	...
0	146	146	149	153	158	...
0	145	143	143	148	158	...
...	...	...	...	...	...	...

Input Channel #1 (Red)

0	0	0	0	0	0	...
0	167	166	167	169	169	...
0	164	165	168	170	170	...
0	160	162	166	169	170	...
0	156	156	159	163	168	...
0	155	153	153	158	168	...
...	...	...	...	...	...	...

Input Channel #2 (Green)

0	0	0	0	0	0	...
0	163	162	163	165	165	...
0	160	161	164	166	166	...
0	156	158	162	165	166	...
0	155	155	158	162	167	...
0	154	152	152	157	167	...
...	...	...	...	...	...	...

Input Channel #3 (Blue)

-1	-1	1
0	1	-1
0	1	1

Kernel Channel #1



308

+

1	0	0
1	-1	-1
1	0	-1

Kernel Channel #2



-498

+

0	1	1
0	1	0
1	-1	1

Kernel Channel #3



164

+ 1 = -25

Bias = 1

Output

-25				...
				...
				...
				...
...	...	...	...	...

Convolutional Neural Networks (CNN)

- Convolutional Neural Networks (CNNs) are a specialized class of deep neural networks designed primarily for processing grid-structured data such as images.
- They automatically learn spatial hierarchies of features through convolution operations. Below is a **detailed explanation of each topic** clearly and systematically.

Convolutional Neural Networks (CNN)

- **Definition**

- Convolution is the core mathematical operation in CNNs that extracts features from input images by applying filters (kernels).

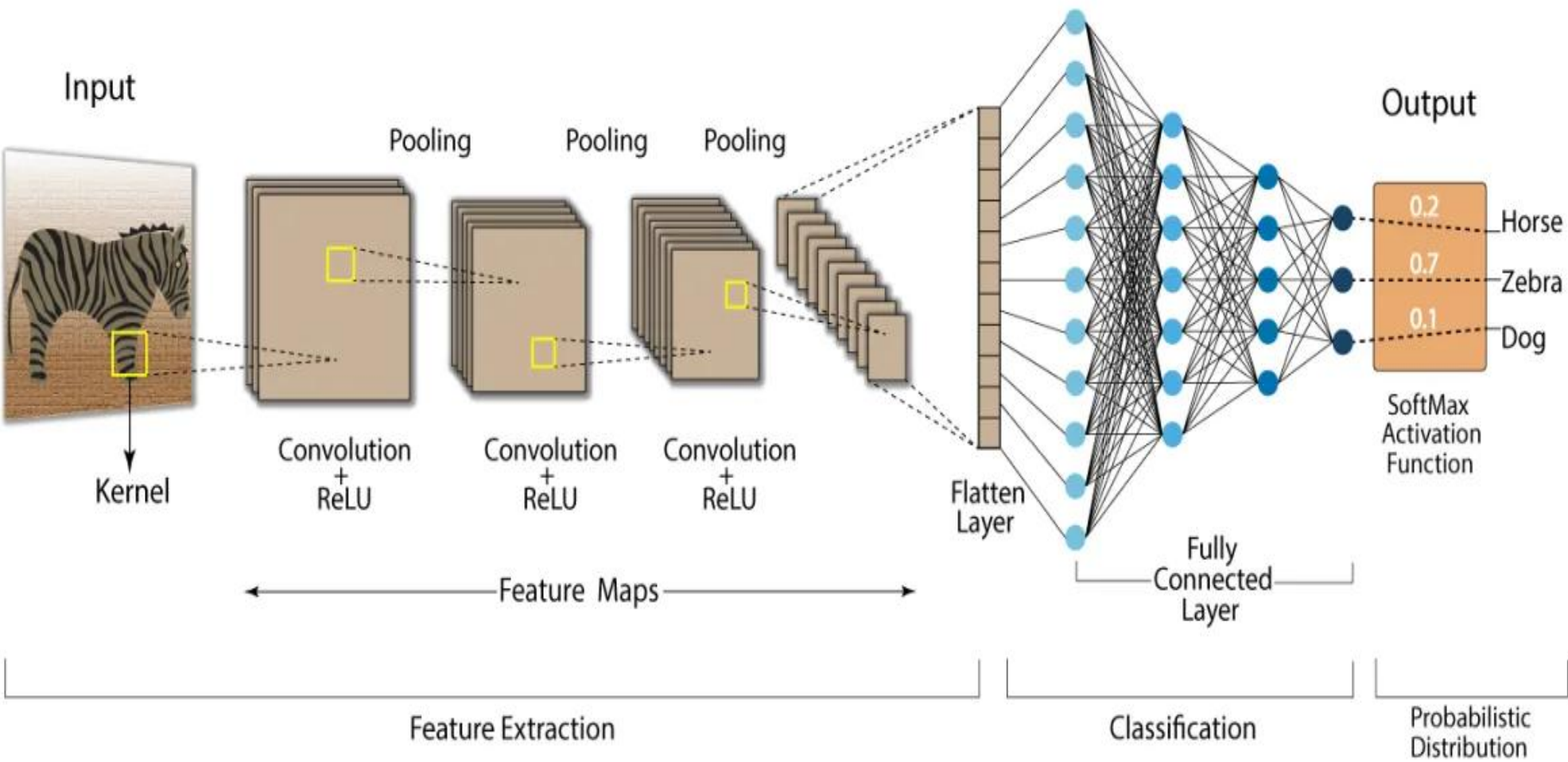
- **How It Works**

- A small matrix called a **kernel** slides over the input image.
- At each position, element-wise multiplication is performed.
- The multiplied values are summed to produce a single output value.
- This process generates a **feature map**.

Key Concepts

- **Local Receptive Field:** Each neuron looks at a small portion of the image.
- **Weight Sharing:** Same filter is applied across the entire image.
- **Translation Invariance:** Helps detect patterns regardless of location.
- **Feature Hierarchy:**
 - Early layers → edges & textures
 - Middle layers → shapes
 - Deep layers → objects
- **♦ Why Convolution is Important**
 - Reduces parameters compared to fully connected networks.
 - Preserves spatial relationships.
 - Improves computational efficiency.

Convolution Neural Network (CNN)



Convolution Operation

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Image Input

1	0	1
0	1	0
1	0	1

3x3 Kernel

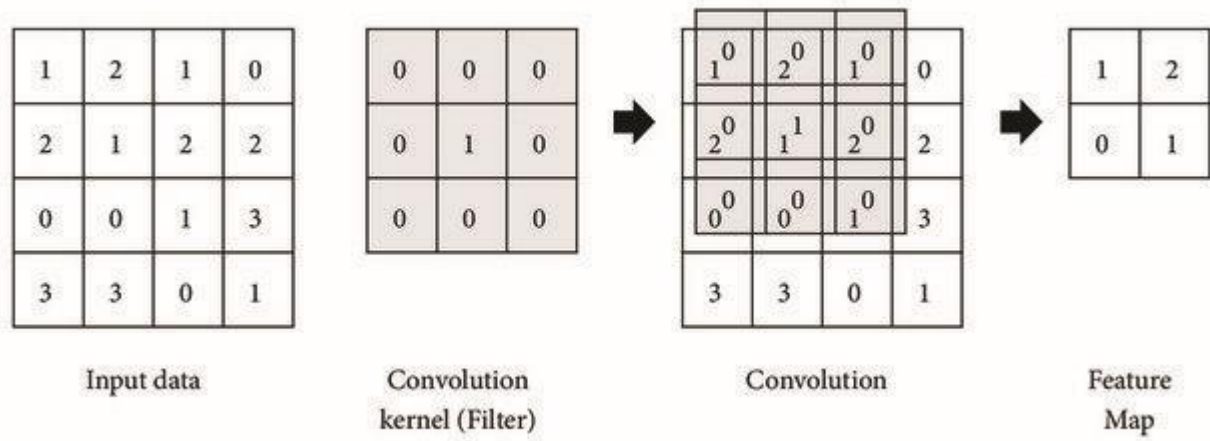
1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

1	1 _{x1}	1 _{x0}	0 _{x1}	0
0	1 _{x0}	1 _{x1}	1 _{x0}	0
0	0 _{x1}	1 _{x0}	1 _{x1}	1
0	0	1	1	0
0	1	1	0	0

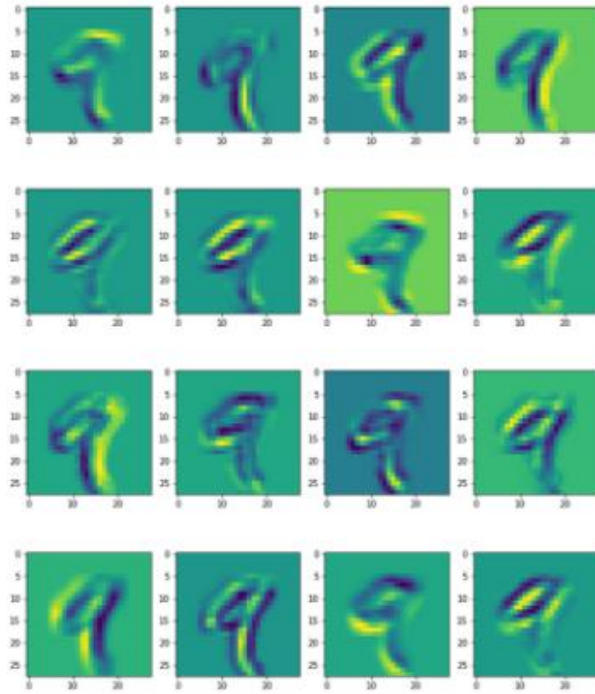
1	1	1	0	0
0 _{x1}	1 _{x0}	1 _{x1}	1	0
0 _{x0}	0 _{x1}	1 _{x0}	1	1
0 _{x1}	0 _{x0}	1 _{x1}	1	0
0	1	1	0	0

4	3	
2		

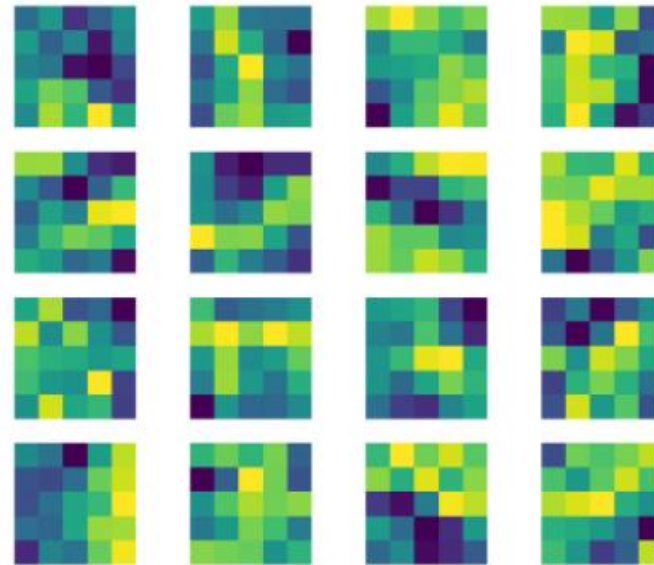
Feature Map

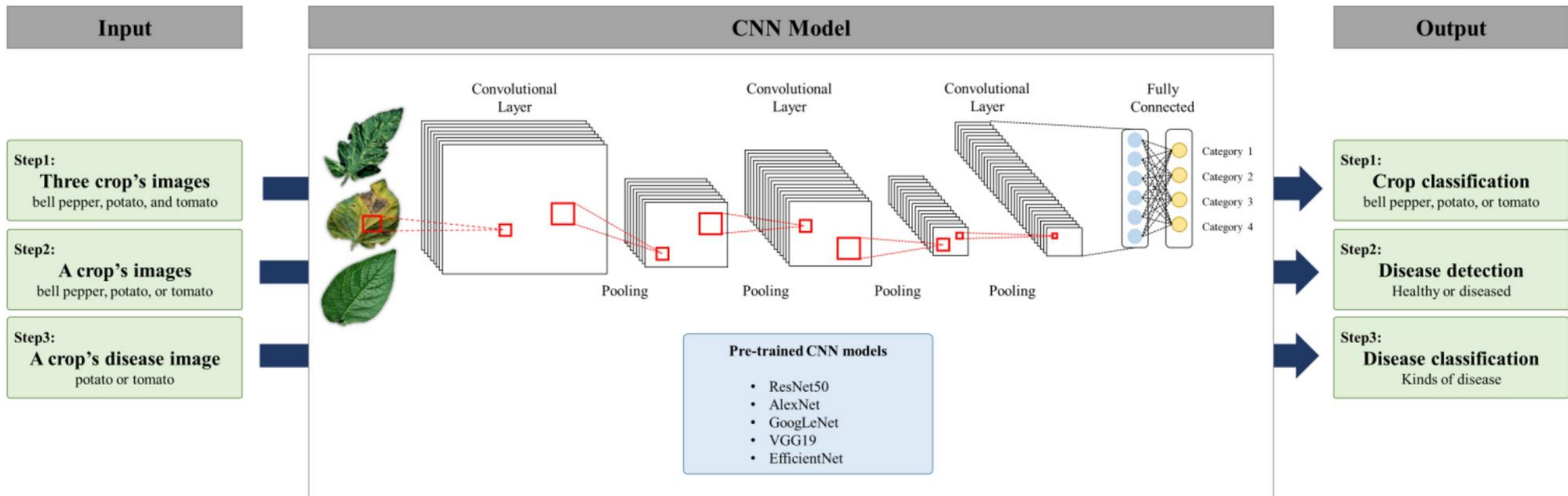


Feature map



Filters





Filters (Kernels)

- **Definition**
- A filter (or kernel) is a small matrix of learnable weights used to detect specific patterns in images.
- **♦ Characteristics**
- Common sizes: 3×3 , 5×5
- Depth equals number of input channels (e.g., 3 for RGB)
- Each filter produces one feature map
- **♦ What Filters Learn**
- Edge detectors
- Texture patterns
- Color gradients
- Object parts

- **Learning Process**

- Filters are randomly initialized.
- During training, backpropagation updates filter weights.
- Each filter specializes in recognizing specific features.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

0	1
-1	0

-1	1
-1	1

-1	0	1
-1	0	1
-1	0	1

-1	0	1
-2	0	2
-1	0	1

1	0
0	-1

-1	-1
1	1

-1	-1	-1
0	0	0
1	1	1

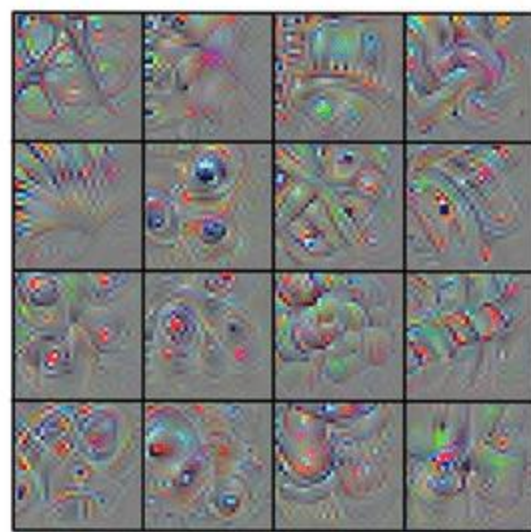
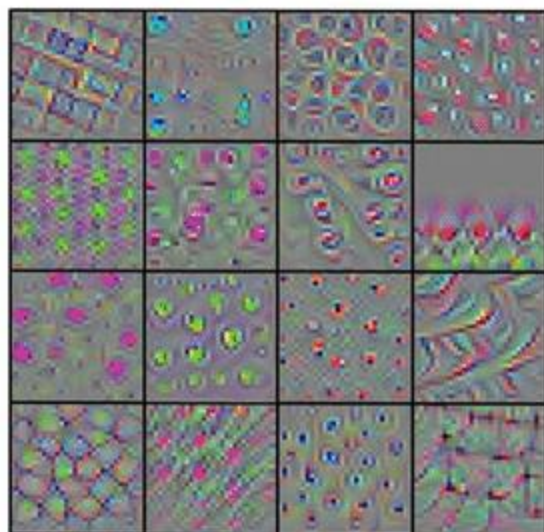
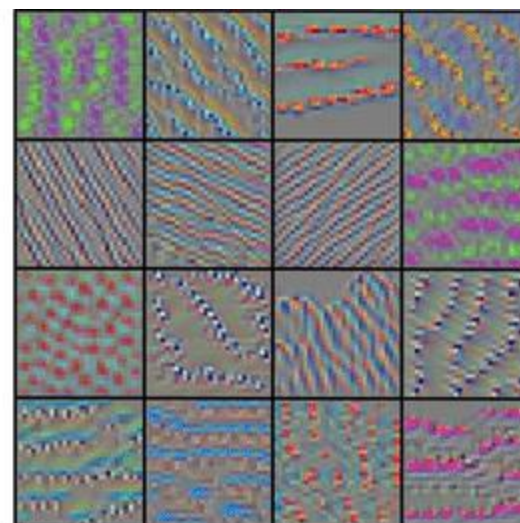
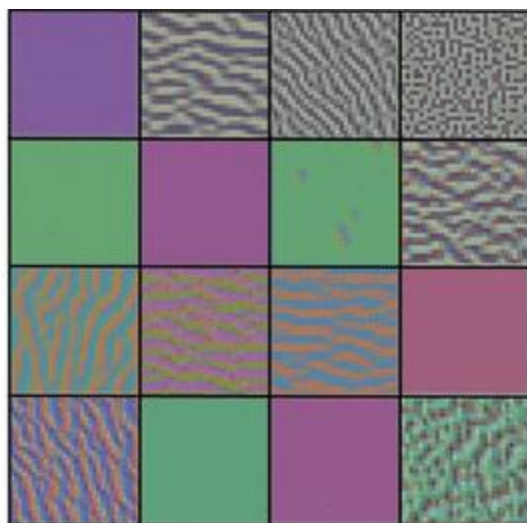
-1	-2	-1
0	0	0
1	2	1

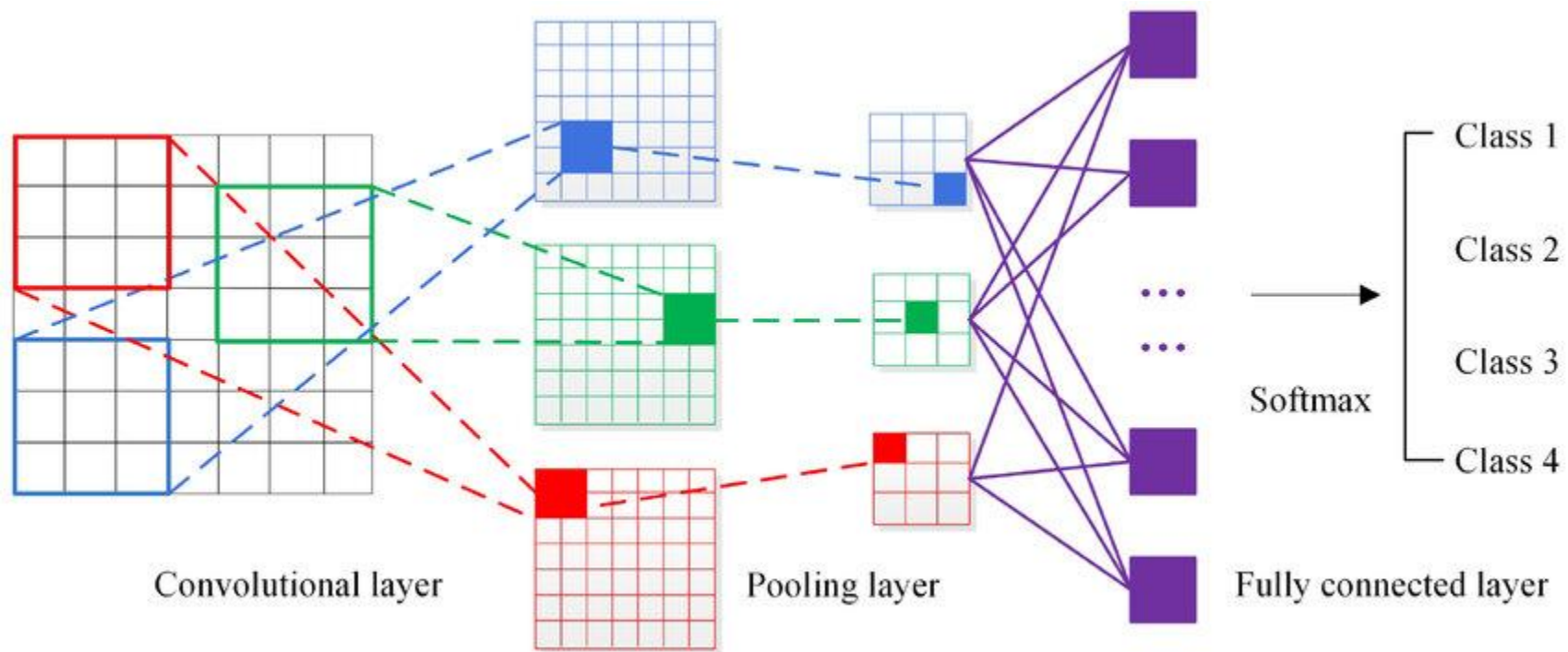
(A)

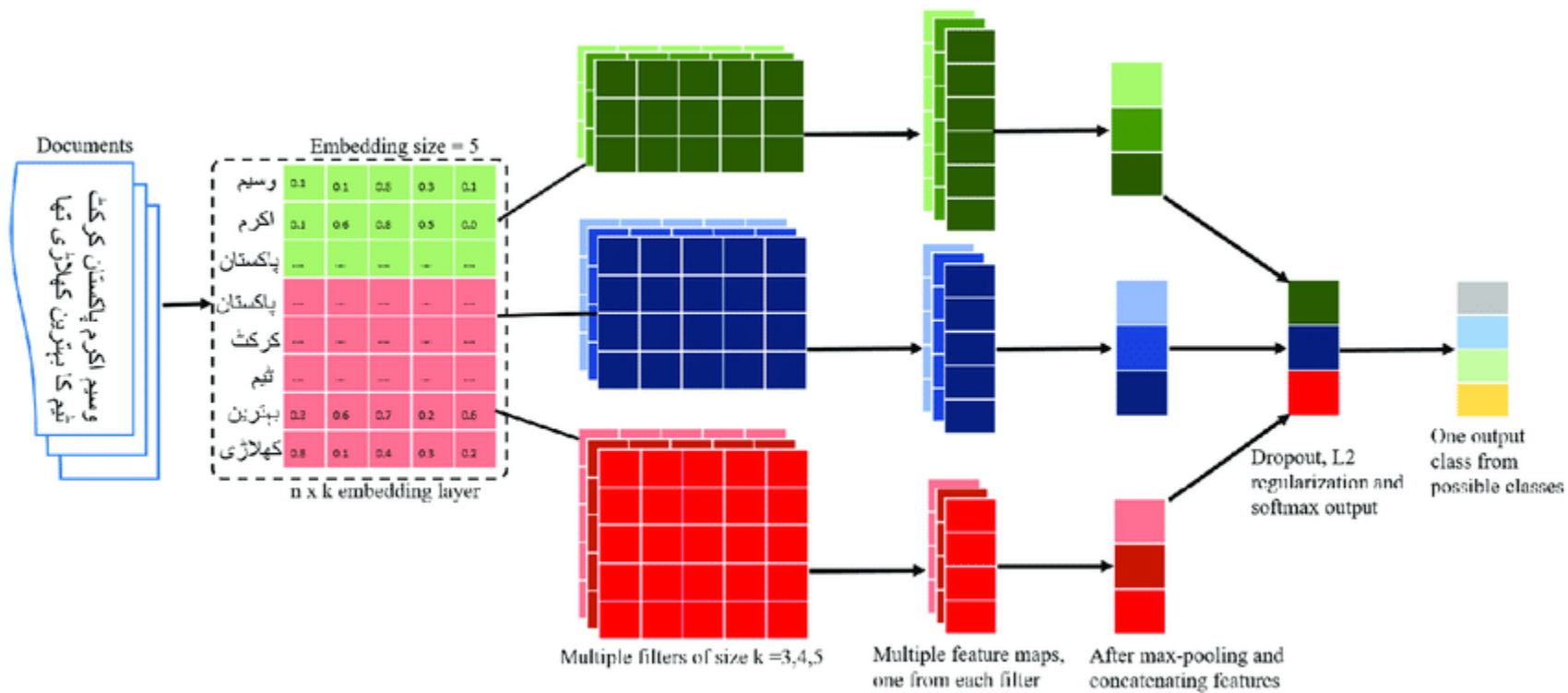
(B)

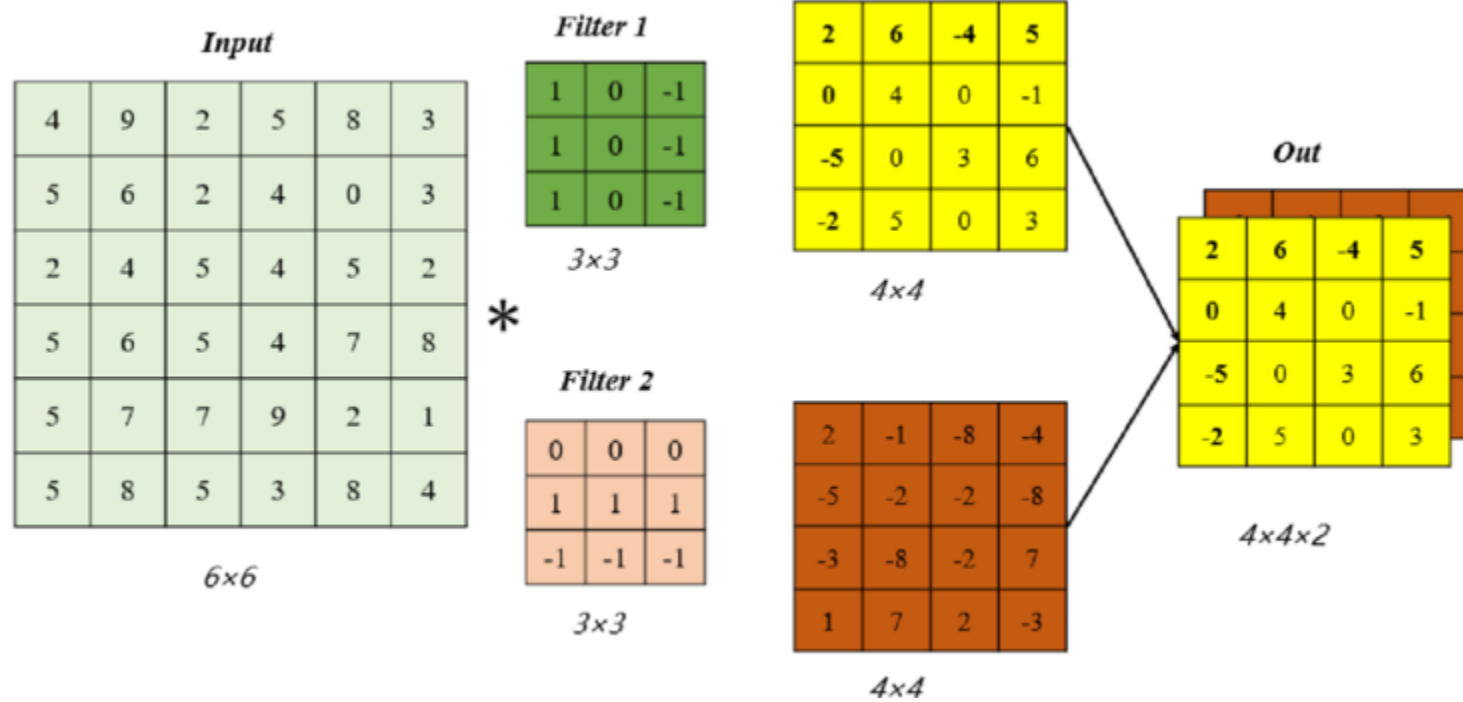
(C)

(D)









Pooling

- **Definition**
- Pooling is a downsampling operation used to reduce spatial dimensions of feature maps.
- ♦ **Types of Pooling**
- **Max Pooling** – Selects maximum value in region
- **Average Pooling** – Computes average value
- ♦ **Purpose**
- Reduces computational complexity
- Controls overfitting
- Makes model robust to small translations
- Retains important features
- ♦ **Example**
- A 2×2 max pooling with stride 2 reduces width and height by half.

1	5	8	0
4	6	6	6
2	2	5	5
8	0	4	1

Max Pooling with 2x2
Filter and Stride Value 2

6	8
8	5

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

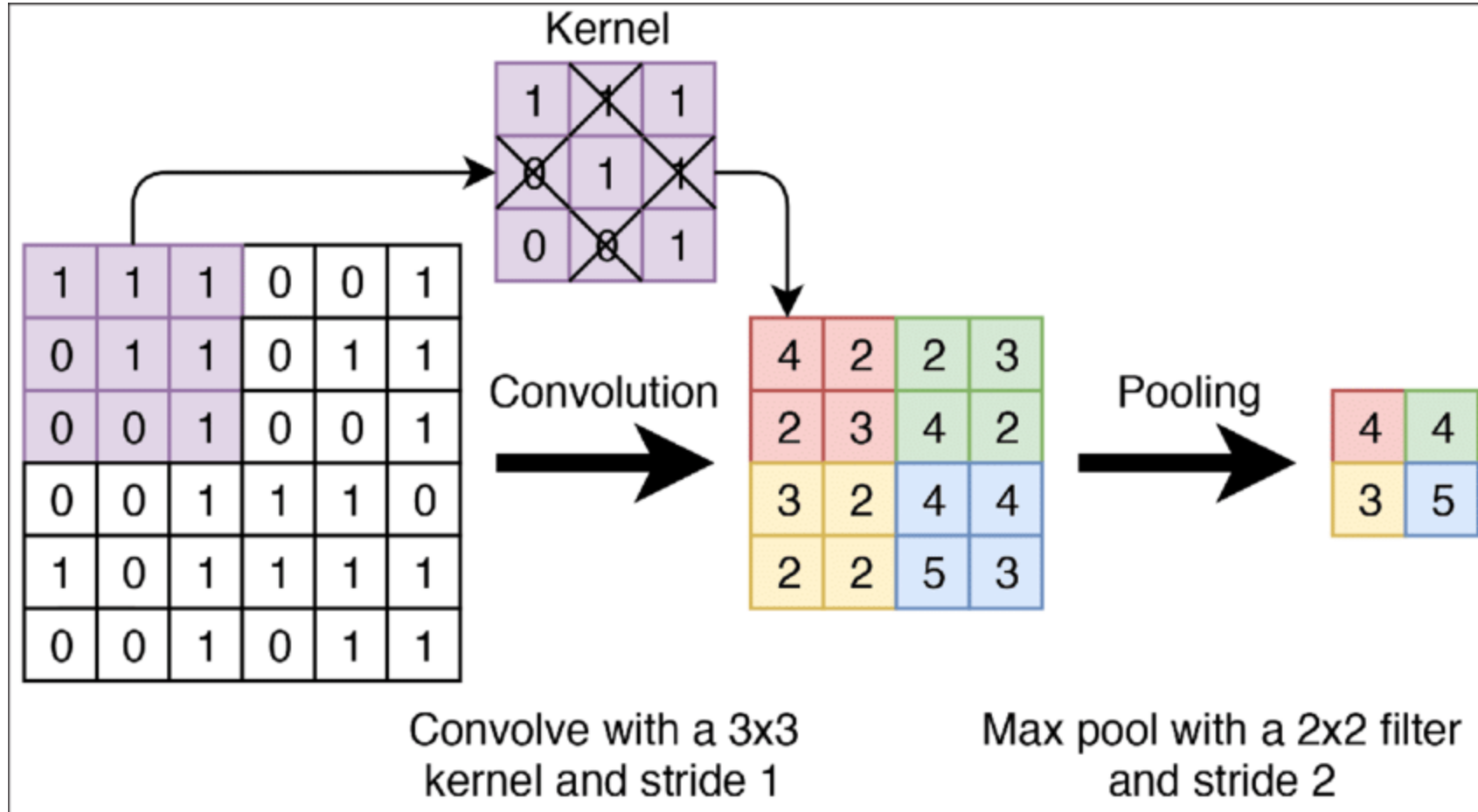
100	184
12	45

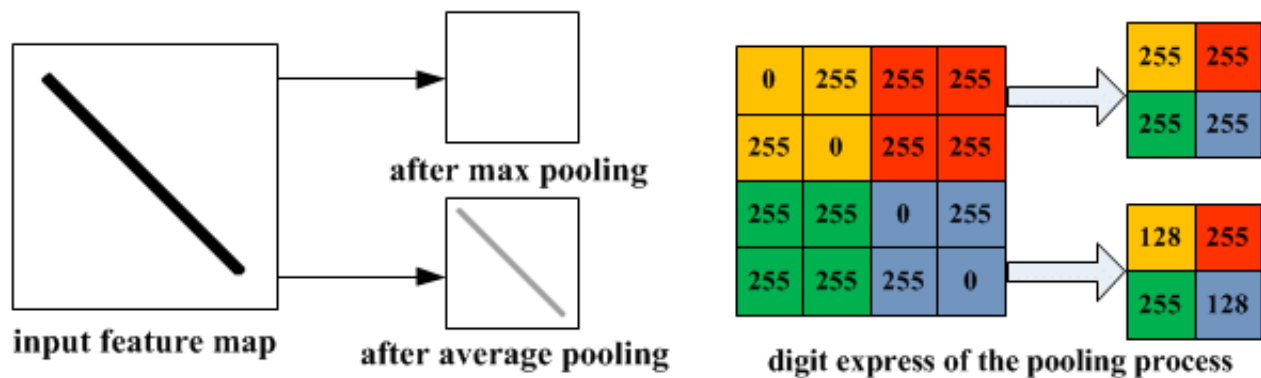
Average Pooling

31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

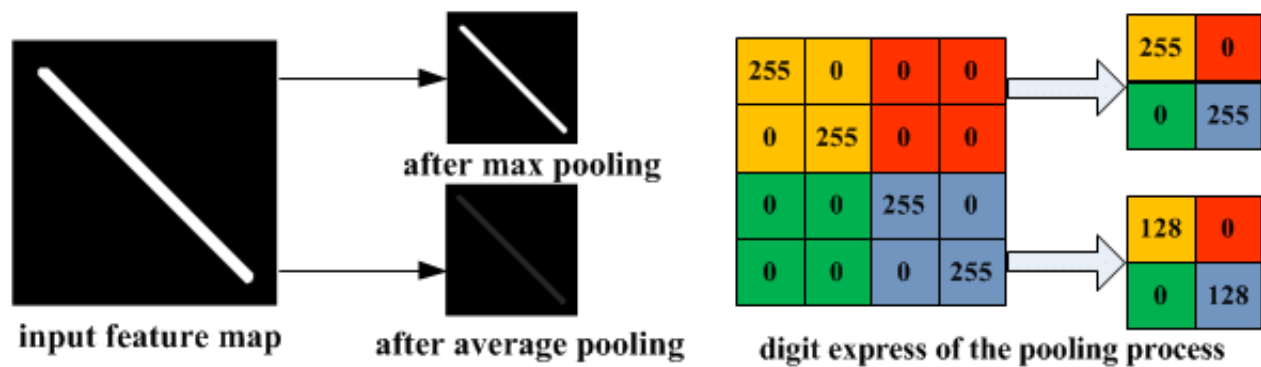
2 x 2
pool size

36	80
12	15





(a) Illustration of max pooling drawback

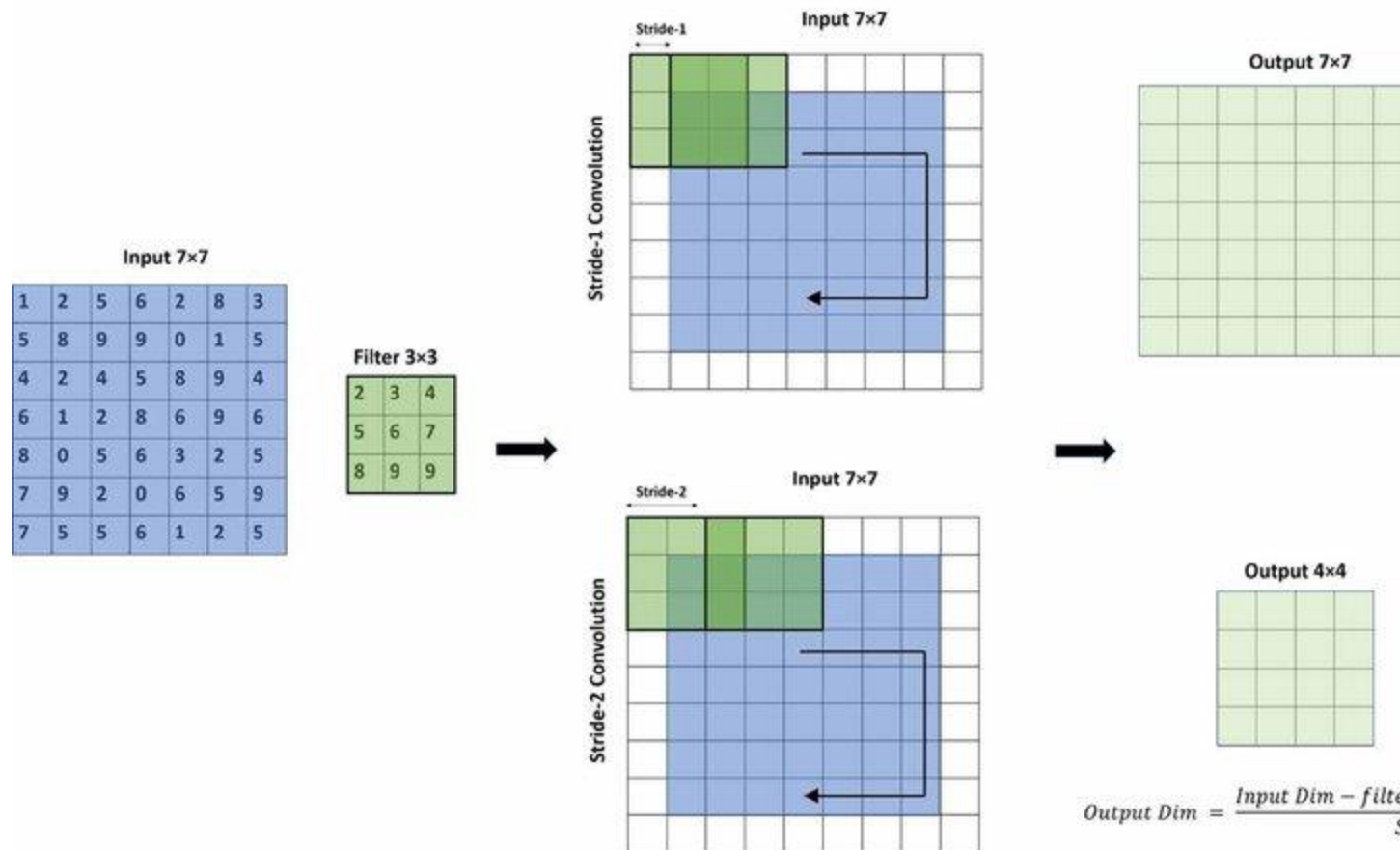


(b) Illustration of average pooling drawback

Stride

- **Definition**
- Stride determines how many pixels the filter moves at each step during convolution or pooling.
- **◆ Effects of Stride**
- $\text{Stride} = 1 \rightarrow$ Detailed feature extraction
- $\text{Stride} > 1 \rightarrow$ Reduced output size
- Larger stride $\rightarrow$ Faster computation but less detail
- **◆ Importance**
- Controls output dimension
- Acts as a downsampling mechanism
- Influences receptive field growth

Convolution with zero Padding



$$\text{Output Dim} = \frac{\text{Input Dim} - \text{filter size} + 2 \times \text{Padding}}{\text{Stride}} + 1$$

Input

0	0	0	0	0
0	0	1	2	0
0	3	4	5	0
0	6	7	8	0
0	0	0	0	0

Kernel

0	1
2	3

*

=

Output

0	8
6	8

Input Volume (+pad 1) (7x7x3)

x[:, :, 0]						
0	0	0	0	0	0	0
0	1	2	1	1	1	0
0	2	1	0	1	1	0
0	1	2	2	1	0	0
0	1	1	1	1	0	0
0	1	2	0	0	0	0
0	0	0	0	0	0	0
x[:, :, 1]						
0	0	0	0	0	0	0
0	1	2	2	0	2	0
0	2	1	1	1	2	0
0	1	1	2	2	2	0
0	1	2	0	0	1	0
0	2	0	0	2	1	0
0	0	0	0	0	0	0
x[:, :, 2]						
0	0	0	0	0	0	0
0	1	1	1	0	1	0
0	0	0	2	0	1	0
0	0	0	0	2	2	0
0	0	1	0	1	1	0
0	0	2	0	2	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:, :, 0]		
-1	0	1
0	1	-1
0	0	0
w0[:, :, 1]		
0	-1	-1
0	0	1
0	-1	0
w0[:, :, 2]		
0	-1	0
1	0	0
-1	0	0

Bias b0 (1x1x1)

b0[:, :, 0]		
1		

Filter W1 (3x3x3)

w1[:, :, 0]		
1	-1	1
1	1	-1
1	-1	0
w1[:, :, 1]		
0	-1	-1
-1	0	0
-1	1	0
w1[:, :, 2]		
1	-1	0
-1	0	-1
0	1	-1

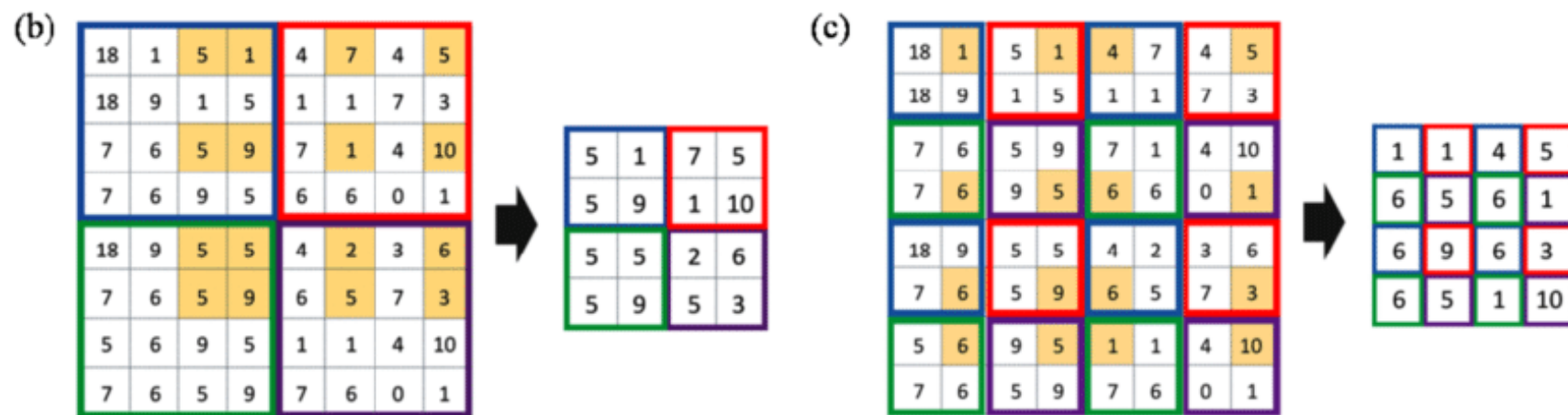
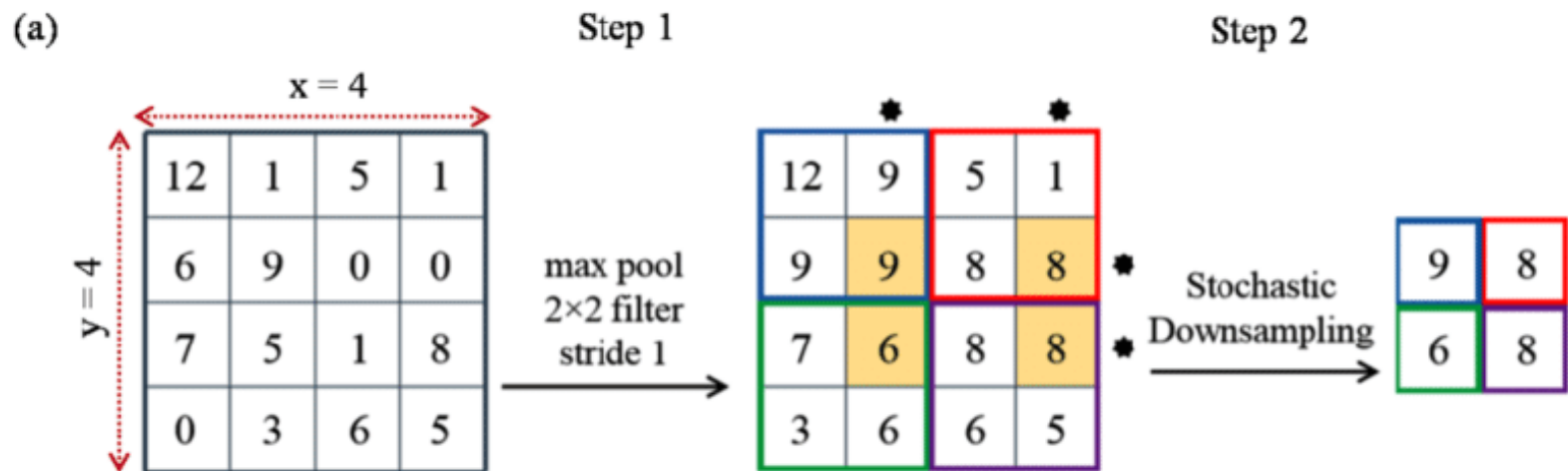
Bias b1 (1x1x1)

b1[:, :, 0]		
0		

Output Volume (3x3x2)

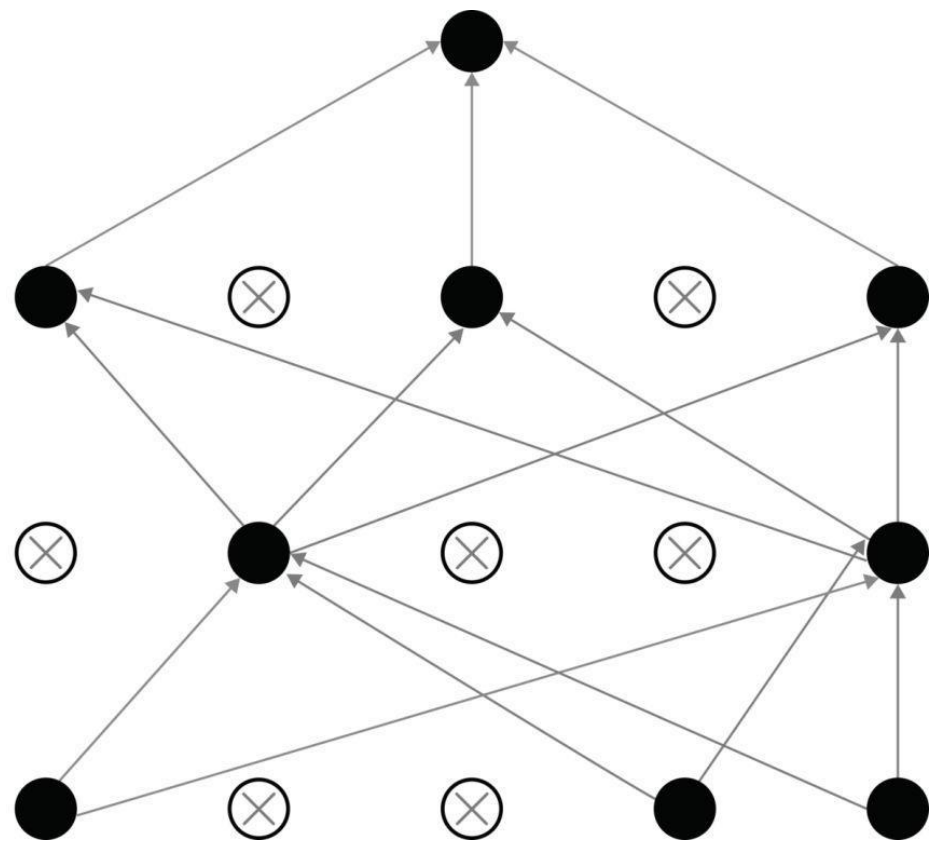
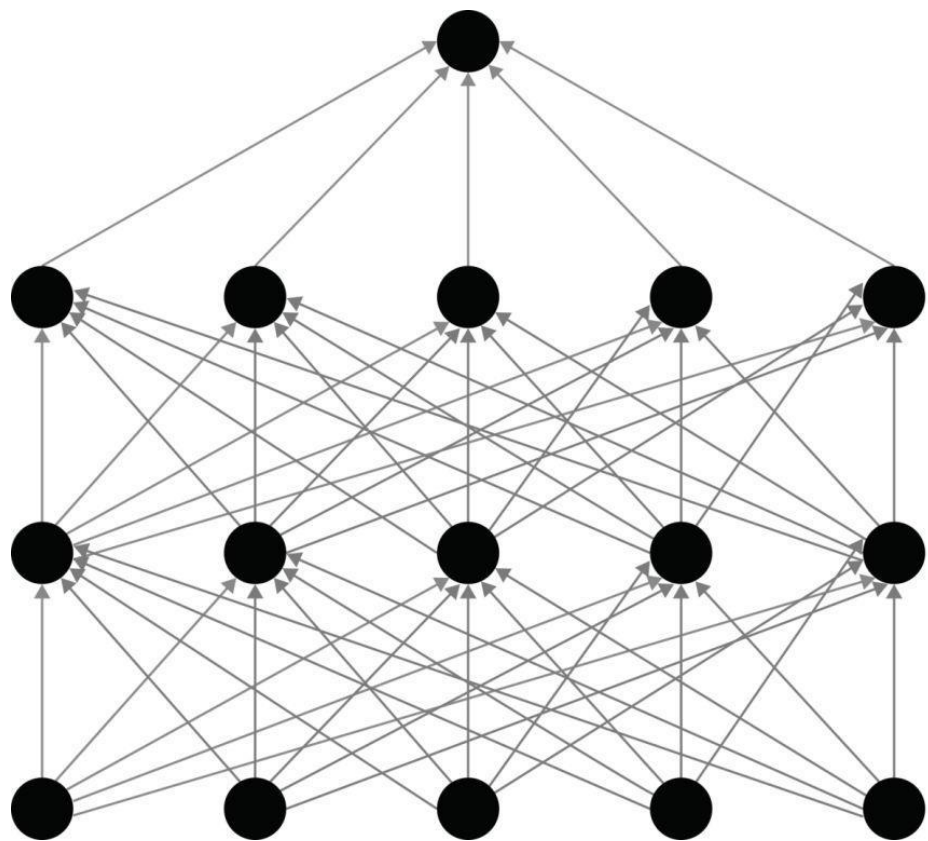
o[:, :, 0]		
0	1	0
-2	-1	-3
-2	5	0
o[:, :, 1]		
-2	2	4
-6	-5	-3
-6	0	-4

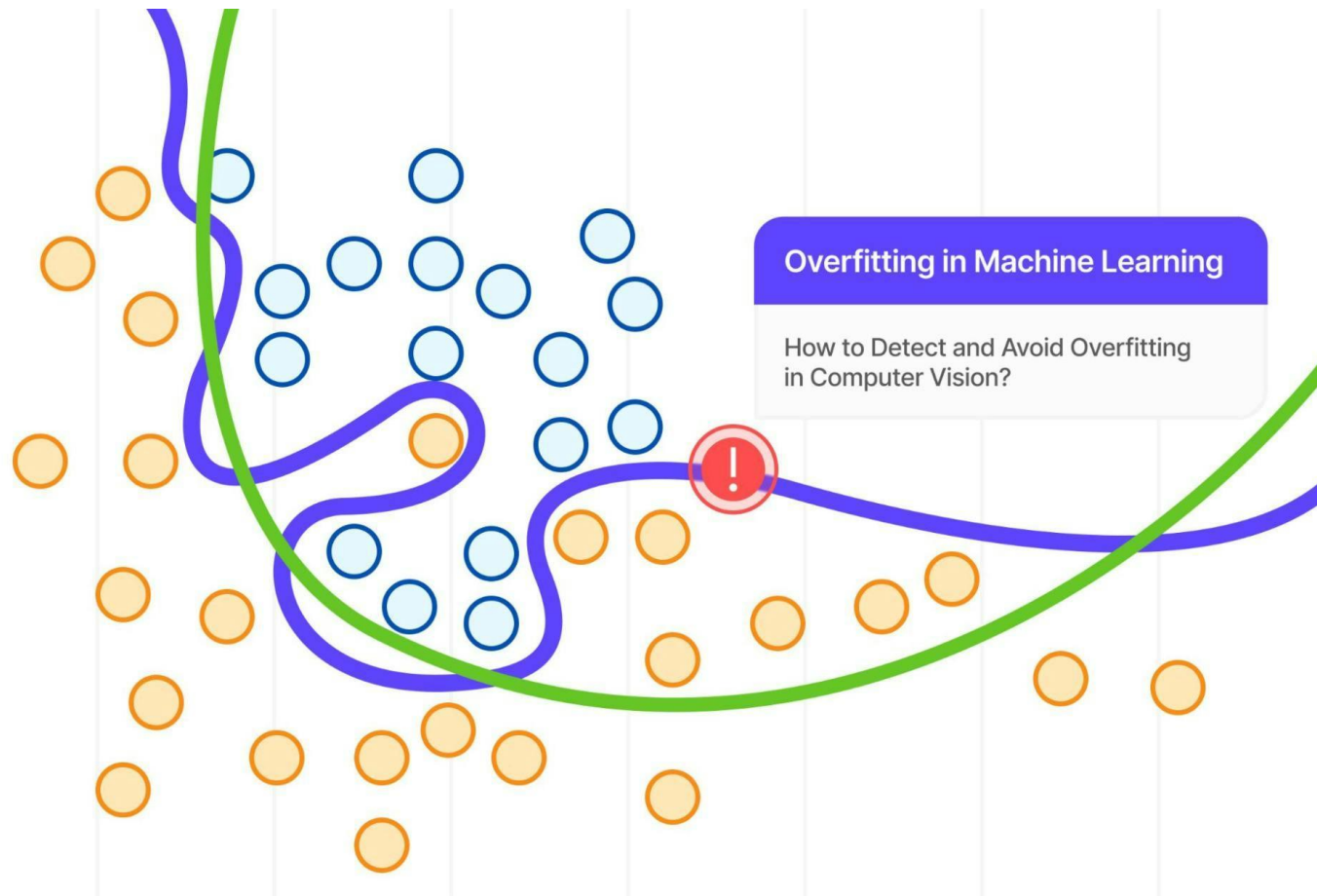
toggle movement



Dropout

- **Definition**
- Dropout is a regularization technique that randomly disables a fraction of neurons during training.
- **◆ Why It Is Used**
- Prevents overfitting
- Improves generalization
- Reduces neuron co-dependency
- **◆ How It Works**
- During training: Random neurons are dropped.
- During testing: All neurons are used (scaled appropriately).
- **◆ Typical Values**
- Dropout rate: 0.2 to 0.5

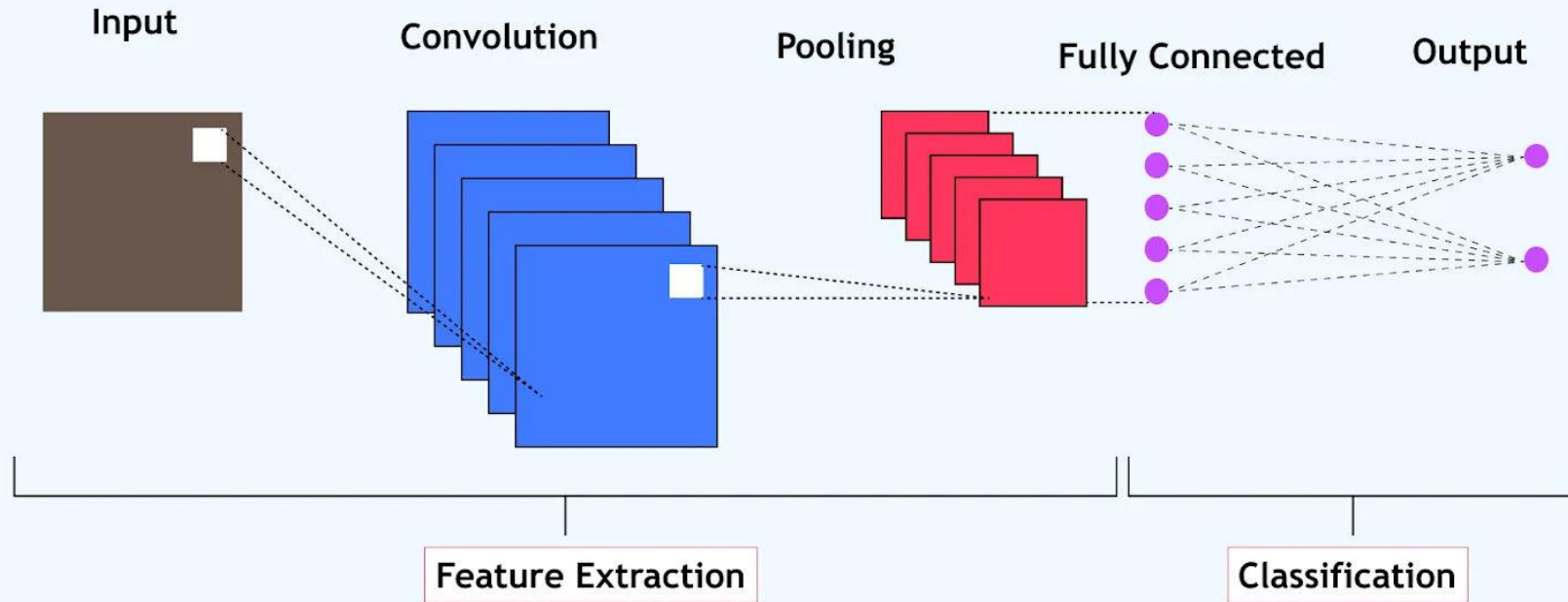


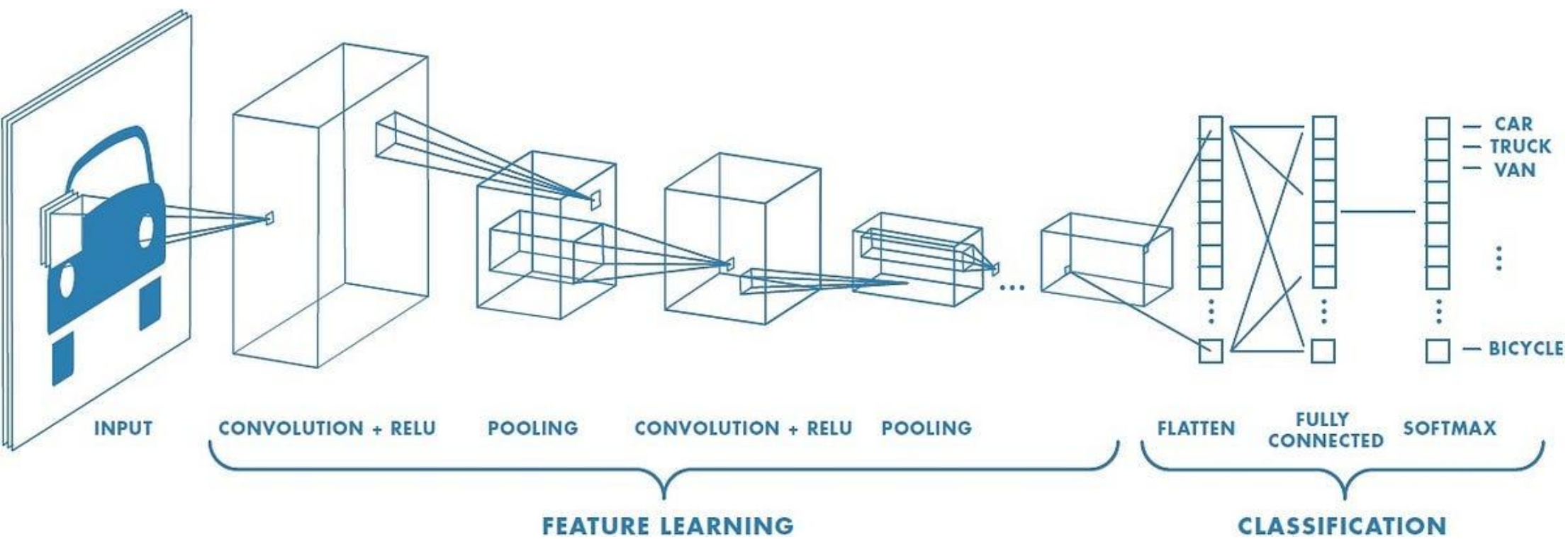


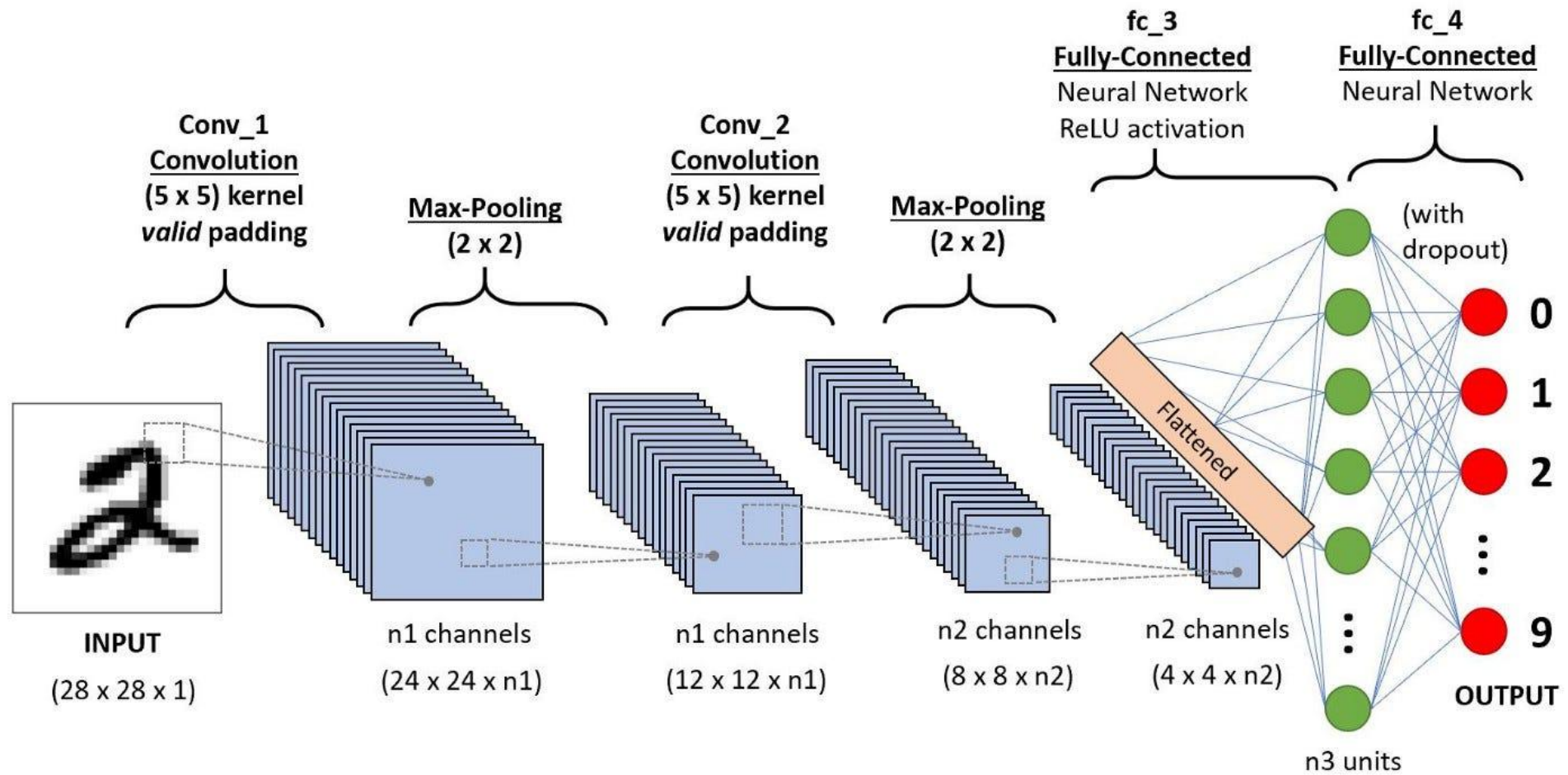
CNN Layers

- **Types of Layers**
- **Input Layer** – Receives image pixels
- **Convolution Layer** – Extracts features
- **Activation Layer (ReLU)** – Adds non-linearity
- **Pooling Layer** – Reduces dimensions
- **Batch Normalization** – Stabilizes learning
- **Dropout Layer** – Prevents overfitting
- **Fully Connected Layer** – Performs classification
- **Output Layer** – Produces class probabilities
- **◆ Layer Hierarchy**
- Shallow layers → Edges
- Middle layers → Shapes
- Deep layers → Objects

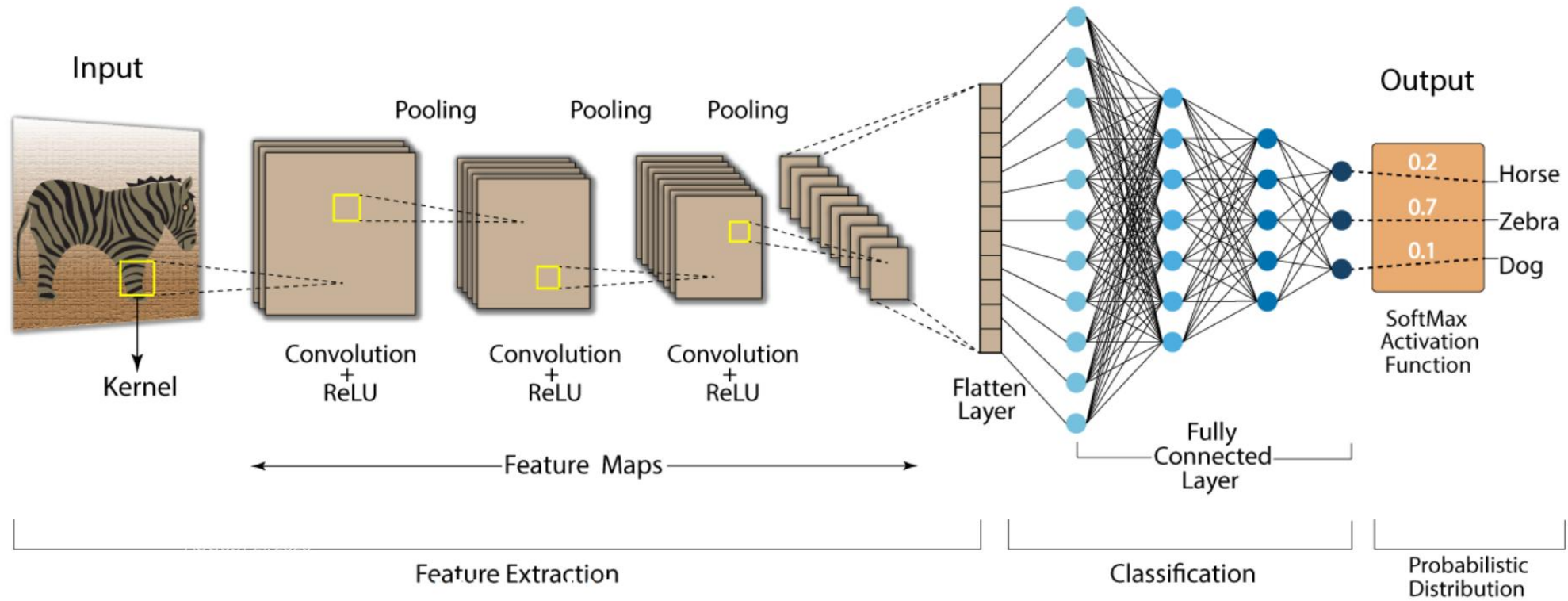
The Architecture of Convolutional Neural Networks



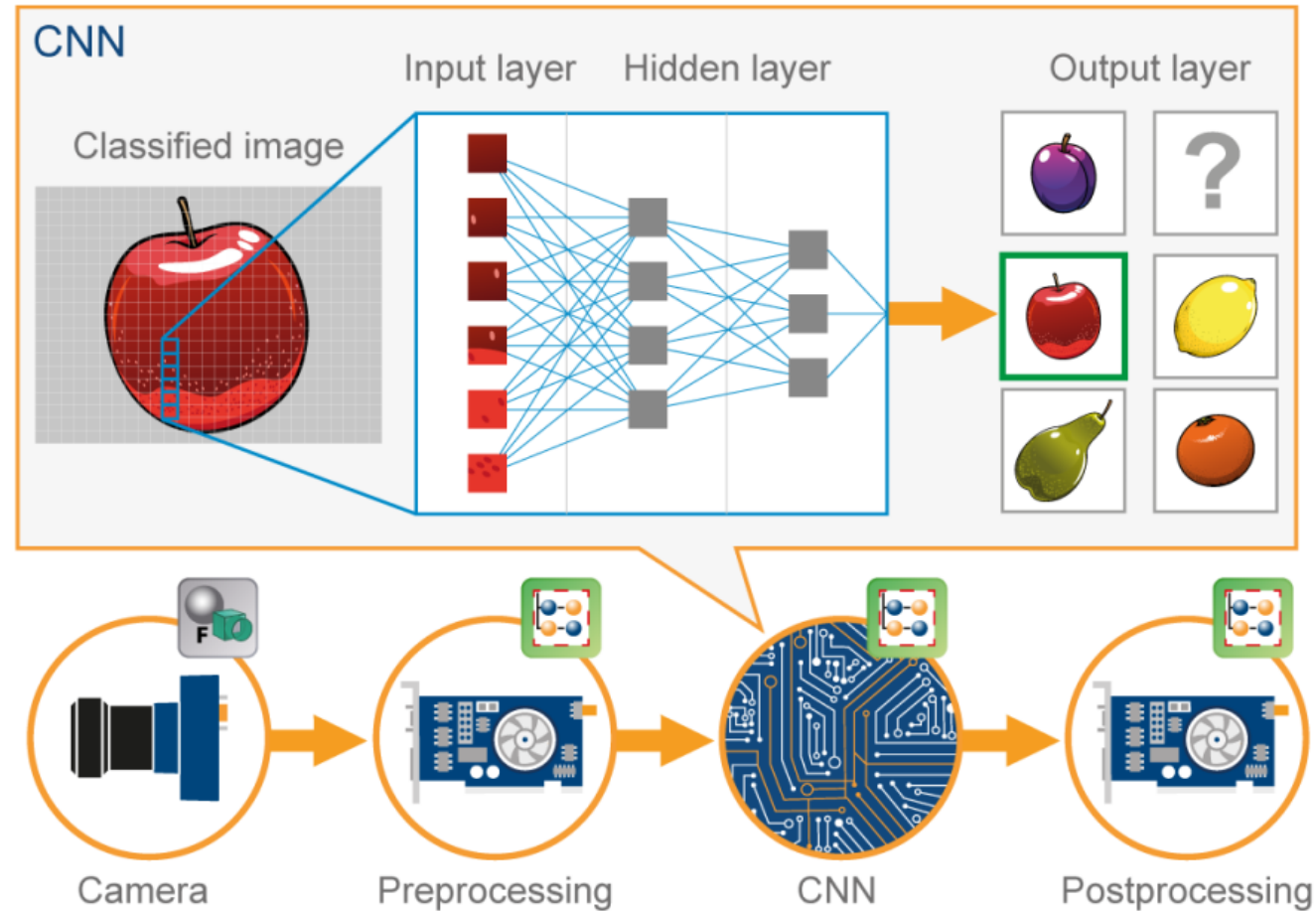


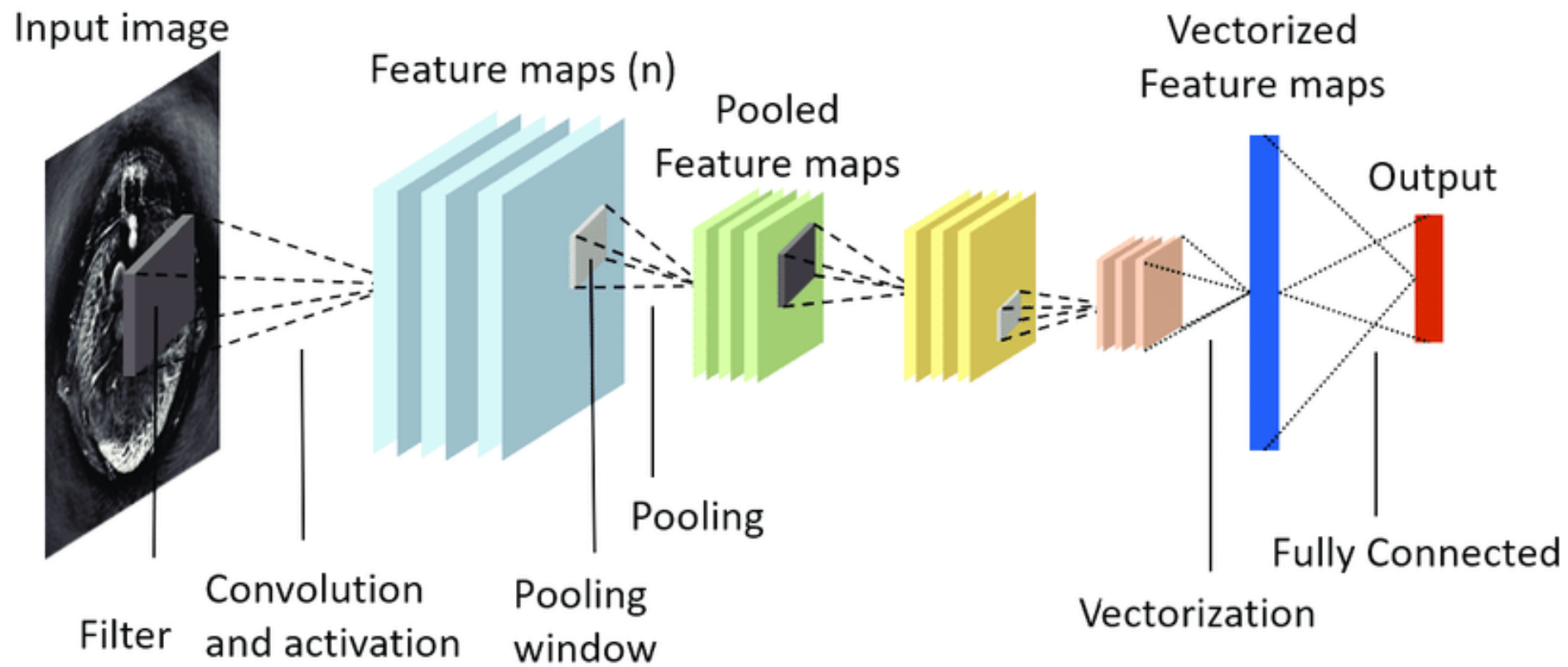


Convolution Neural Network (CNN)



Applications of CNN





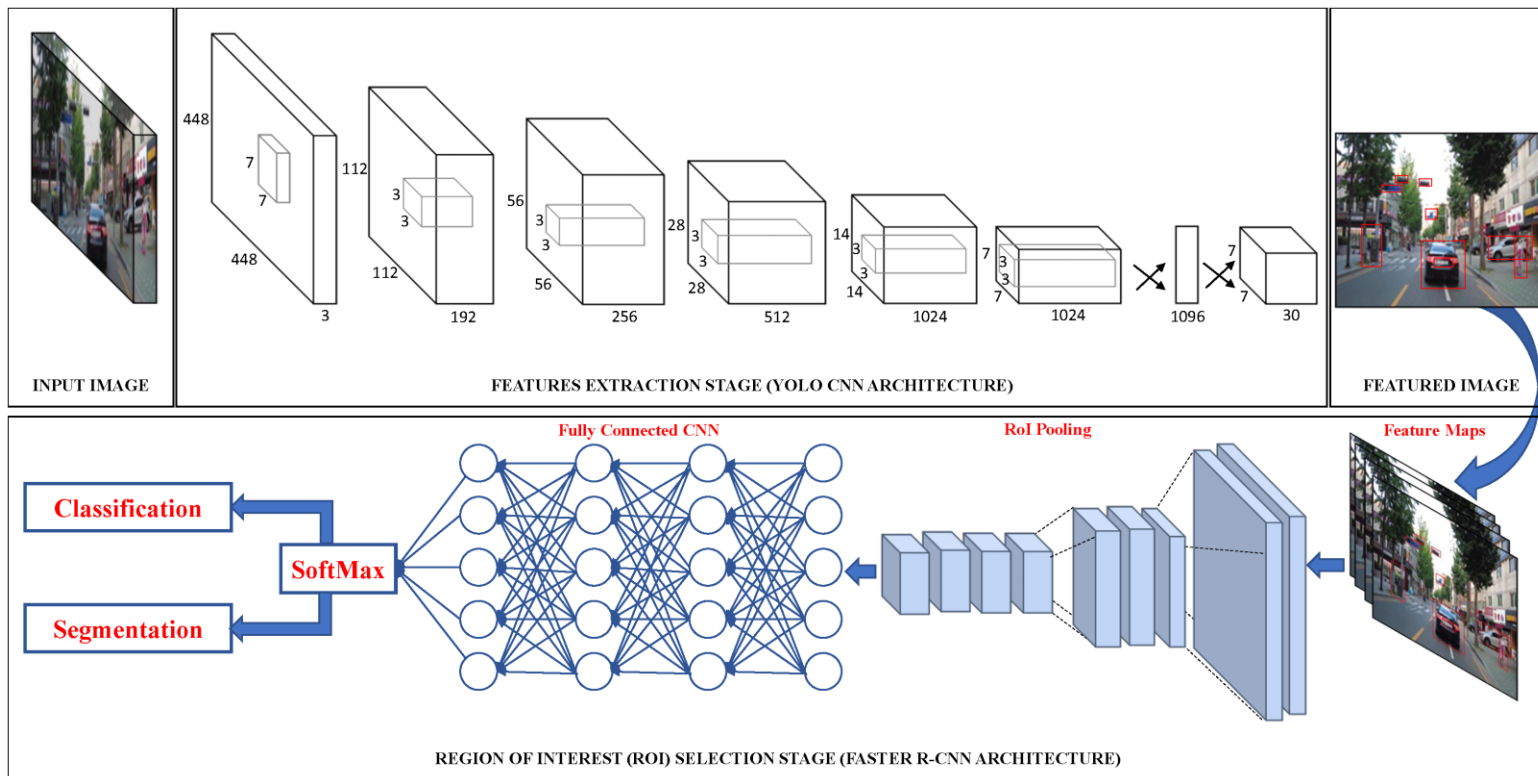




Fig. 1 Autonomous Vehicles [6]

CNNs are widely used in real-world applications:

- **◆ Computer Vision**

- Image classification
- Object detection
- Face recognition
- Image segmentation

- **◆ Medical Field**

- Tumor detection
- Disease diagnosis

- **Agriculture**
- Plant disease detection
- Crop health monitoring
- ♦ **Autonomous Systems**
- Self-driving cars
- Robotics
- ♦ **Other Applications**
- Handwriting recognition
- Video surveillance
- Biometric systems
- Satellite image analysis